

Ordo: A Practical Language for Governed Processes



Figure 1: Nebu — Ordo mascot

How to Turn Human Instructions into Clear, Executable Instructions for AI Models

A practical manuscript for the **Ordo** language.

Status: first reading edition.

Preface

This book grew out of a very practical problem: AI models can already help with analysis, documentation, code, tests, reviews, and decision-making, but complex instructions for them quickly turn into chaotic, oversized prompts.

At first, it seems enough to write more detail. Then exceptions have to be added. Then rules. Then examples. Then prohibitions. Then



Figure 2: Nebu — idea

checks. Then separate clarifications about what the model must not do. At some point, the prompt stops looking like an instruction and starts looking like a large document that the model reads a little differently every time.

Ordo emerged as a response to this problem.

The idea behind Ordo is simple: if we want an AI model to execute a complex process, that process should not be described as one continuous block of text. It should be described as a governed execution program. It needs a purpose, a contract, state, nodes, paths, checks, outputs, tracing, tests, and an improvement mechanism.

This book is neither an academic standard nor a final specification. It is a practical introduction, with examples, analogies, plain language, and a gradual transition from an ordinary prompt to a language for governing an AI model.

The main goal of the book is to help the reader understand not only Ordo syntax, but also the way of thinking behind it.

Table of Contents

Ordo: A Practical Language for Governed Processes	1
Preface	1
How a Problem Became an Idea: Why Prompts Were Not Enough	6
Who This Book Is For	9
How to Read This Book	9
Introduction: The Core Idea of Ordo	10
Chapter 1. Why Ordinary Prompts Are Not Enough	11
Chapter 2. What Ordo Is in Simple Terms	15
Chapter 3. Ordo as a Language for Governing a Model	24
Chapter 4. Intent, Contract, Context	32
Chapter 5. State: Process Memory	41
Chapter 6. Entry and Node: How the Model Asks Questions	51
Chapter 6.1. Atomicity of Steps in Processes with Language Models	60
Chapter 6.2. Micro-Prompts as Modular Process Architecture	83
Chapter 7. Output: What the Model Must Create	100
Chapter 8. Writing the First Ordo Source Program	109
Chapter 9. Compiling Ordo Source into Semantic JSON IR	121
Chapter 10. Why Semantic JSON IR Is the Best Choice Right Now	133
Chapter 11. A Gate Is Not Advice but a Control Point	144
Chapter 12. Negative Checks: ASSERT.NOT → ASSERTION	157
Chapter 13. Status Semantics	169
Chapter 14. Why Ordo Programs Need Debugging	181
Chapter 15. Debug Mode and Diagnostic Trace	190
Chapter 16. Tests, Fixtures, and Expected Behavior	206
Chapter 17. Regression Suite and Coverage	217
Chapter 18. Feedback & Improvement Loop	232
Chapter 19. Why Not Everything Should Be Formalized	243
Chapter 20. How to Use FREEFORM Correctly	253
Chapter 21. FREEFORM.COVERAGE	269
Chapter 22. Ordo Core	276
Chapter 23. Ordo Profiles	286
Chapter 24. Domain Packs	295

Chapter 25. Ordo Libraries: include, import, and Reusing Ready-Made Solutions	310
Chapter 26. Why All-in-One Is No Longer the Main Format .	322
Chapter 27. Documentation Runtime	329
Chapter 28. Rendered Artifact Validation	337
Chapter 29. The Old Playbook as a Markdown Knowledge Base	349
Chapter 30. Migrating a Playbook to Ordo	356
Chapter 31. Guided Intake as an Ordo Program	367
Chapter 32. Package Generation	377
Chapter 33. What Remains in FREEFORM	385
Chapter 34. Minimum Requirements for an Author	392
Chapter 35. How Not to Break a Playbook	400
Chapter 37. Ordo Without a Runtime	408
Chapter 37. Ordo Without a Runtime	413
Chapter 38. Ordo with a Helper Runner	419
Chapter 39. Ordo as a Native Model Language	429
Chapter 40. Process Rail: How Ordo Keeps AI on Track . . .	434
Chapter 41. Contract → Artifact Coverage and Go/No-Go . . .	435
Chapter 42. External Audit of the Pre-release Package	441
Chapter 43. Final Handoff: How to Transfer Ordo for External Review	444
Chapter 44. Receiving External Feedback After Review . . .	445
Chapter 45. Feedback Decision Planning	446
Chapter 46. Publication Readiness Decision	447
Chapter 47. Two-tier Rendering Model	448
Chapter 48. Runtime Source of Truth and CLI Honesty . . .	450
Chapter 49. Runtime Guided Intake Entry Protocol	451
Chapter 50. Runtime Mode Start Files Standard	453
Chapter 51. Package Build Profiles: dev / runtime / evidence	454
Chapter 52. Runtime Checkpoint Discipline	456
Chapter 53. CLI-enforced Runtime Package	457
Chapter 54. Multi-target Runtime Compilation Layer	459
Chapter 55. Ordo-code Runtime View	461
Chapter 56. Session-trace as a Proof Program	463
Chapter 57. Scenario Testing and PathWalk	465
Chapter 58. Restore-session: Safe Backtracking	471
Chapter 59. PathWalk Benchmark Readiness	473

Chapter 60. Model Benchmark Protocol	474
Chapter 61. Real Module Testcase Generation	477
Chapter 62. Human Review Scenario Cards	480
Chapter 63. Visual Graph Generator as a Companion Utility	482
Chapter 64. Shared Workflow for Companion Utilities	483
Chapter 65. Closing M61: A Stable Companion Utilities Layer	485
Chapter 66. APF as a Standard Applied Module	487
Chapter 67. APF Language Pattern Extraction	488
Chapter 68. Closing M62: Stable APF Integration	490
Chapter 69. M63: APF Release-candidate Integration	491
Chapter 70. APF rc.1 as a Release-candidate Standard Module	492
Chapter 71. APF rc.1 Validation Profile	493
Chapter 72. APF rc.1: Language Pattern Classification	493
Chapter 73. How APF Began Creating Itself	494
Chapter 74. EXECUTION_TRACE — the Complete History of Process Execution	502
Chapter 75. Validation, Replay, and Practical Use of EXECU- TION_TRACE	505
Chapter 76. Backlog, Maturity-state Synchronization, and the Purpose of PathWalk	506
Chapter 77. Optional Flow Reuse	507
Chapter 78. APF Replay, Prompt Registry, and Internal Mini- prompts	508
Chapter 79. Generic Template Tooling	510
Chapter 80. Session Package Load and Cache	511
Chapter 81. Deterministic ARF Runtime Control Model	513
Appendix A. Ordo Glossary	515
Ordo Capability Catalog	518
Appendix B. Opcode and Source-Construct Catalog	519
Ordo v0.12 Opcode / Construct Catalog	519
Ordo Source Construct Catalog	524
Appendix C. Verified Program Examples	525
Appendix D. Checklists	542
Appendix E. Anti-patterns	552
Appendix F. Practical YAML and Schema Reference	564
Appendix G. APF Session Package Cache	585

How a Problem Became an Idea: Why Prompts Were Not Enough



Figure 3: Nebu — thinking

The idea of Ordo did not appear all at once. At first, it might have seemed that we had simply decided to create a new language for describing processes. But an honest look at the path shows something else: the language did not arise from a desire to invent another format. It arose from a sequence of practical problems.

At the very beginning was a technological shift. The emergence of modern AI models opened an extremely attractive possibility: embedding artificial intelligence into real company processes and accelerating people’s work. This was especially relevant to roles in which much of the work consists of analysis, structuring, writing, task creation, documentation, requirements alignment, and keeping processes synchronized.

One such example was the work of an analyst in a software company. An analyst performs a great deal of complex but partly repetitive work: understanding a request, clarifying details, formalizing requirements, creating tasks, preparing documentation, checking logic, keeping track of open questions, and synchronizing changes with the team. The human remains the mind of the process, but a substantial amount of routine formalization could, in theory, be delegated to an AI model.

The initial idea looked simple:

there is a problem: analysts work slowly;
there is a new technology: AI models;
therefore, we can try to accelerate analysts with AI.

At that stage, the solution seemed obvious: write high-quality prompts and instructions. Those instructions could describe a standard analyst workflow: which questions to ask, what to check, how to format the result, when to return for clarification, and how to create documents or tasks.

The role of AI in this model was clear: the model performs the rou-

tine part of the work, while the analyst controls meaning, makes decisions, and confirms the result. Figuratively speaking, the analyst remains the head and AI becomes the hands.

But after many attempts, another problem became visible. AI really can help. It writes well, reformulates well, structures text, proposes alternatives, and produces drafts. Yet the more complex the process becomes, the worse an ordinary textual instruction performs.

When an instruction is short, the model can still keep track of it. But when a process contains many branches, conditions, backward transitions, exceptions, intermediate decisions, and checks, behavior becomes unstable. If the user distracts the model with an additional question, asks to return to a previous step, or changes a detail, the model may lose track of where it is. It starts skipping important parts, mixing stages, forgetting open questions, or behaving as if the process were already complete when it is not.

We tried to solve this problem with longer and more detailed instructions. It did not help. On the contrary, instructions grew to hundreds of kilobytes. They accumulated more rules, clarifications, exceptions, and special cases. Yet the more text there was, the harder it became for the model to execute the process consistently.

At some point, it became clear that the problem was not only prompt quality. The problem was the form of the instruction itself.

Human language is highly flexible. That is one of its strengths, but for an executable process it is also a weakness. The same action can be described in dozens of different ways. For example, a simple function that adds two numbers can be explained like this:

```
Return the sum of two numbers.  
Add the first argument to the second.  
The result must be a number equal to the sum of the supplied parameters.  
The function must take two values and return their addition.
```

All of these sentences mean approximately the same thing. For a human, that is normal. But in a complex process, this freedom creates a problem: the text must be interpreted every time. And when the process is large, interpretation begins to drift.

AI models often work better with code precisely because code has a stricter form. There is less room for ambiguity. A function, condition, variable, call, or return value has a concrete structure. A model can still make mistakes in code, but the format itself forces it to stay within certain rules.

That led to the key idea: perhaps processes for AI should not be described only in human language. Perhaps they need an intermediate

form—more technical, structured, and stable, while still understandable to a person.

Not a full programming language in the classical sense. And not merely a long prompt. Rather, a process description language in which we can explicitly say:

```
here is the intent;  
here is the contract;  
here is the state;  
here is the question;  
here are the possible branches;  
here are the transition conditions;  
here are the checks;  
here is the result;  
here is the template;  
here is the point where human confirmation is required.
```

This is how the idea of Ordo gradually emerged.

Its roots were not in a desire to make work more complicated. Quite the opposite: the goal was to reduce chaos. We wanted to preserve natural human reasoning while giving the AI model enough explicit structure not to lose its way in long processes.

The path to Ordo can therefore be described as a chain:

```
AI technological breakthrough  
→ desire to accelerate analysts' work  
→ attempt to describe processes through prompts  
→ discovery of instability in long textual instructions  
→ realization that a more formal form is needed  
→ idea of a governed process language
```

This is how Ordo began to take shape as a practical language for AI processes. Its purpose is not to replace the human or turn every kind of work into code. Its purpose is to make a complex process explicit enough for a person to control it and for an AI model to execute it consistently.

In this sense, Ordo did not arise as an abstract language. It arose as a response to a very specific problem: ordinary prompts work well for short tasks, but long governed processes require more form, memory, boundaries, and discipline.

Ordo became an attempt to give those processes a form.



Figure 4: Nebu — interesting

Who This Book Is For

This book is for people who already use AI models not only for simple answers, but for complex work processes.

It will be useful to analysts who describe playbooks, validation rules, data-collection scenarios, and document packages. It will also be useful to developers who want to turn AI instructions into something more stable, testable, and maintainable.

Another audience for this book includes authors of internal standards, QA specialists, product managers, technical writers, and anyone who has faced a situation where a model “understood almost everything” but skipped one critical step.

You do not need to be a programmer to read this book. However, a basic understanding of instructions, validation, process state, results, and execution errors will be useful.

This book is also for those who may want to publish Ordo programs, libraries, domain packs, or playbooks as open documentation on GitHub.

How to Read This Book



Figure 5: Nebu — thinking

You can read the book sequentially, from the first chapter through the appendices. This is the best approach if you are encountering Ordo for the first time.

The opening chapters explain why ordinary prompts are not enough and why Ordo should be understood as a language for governing model behavior. The book then introduces the basic concepts: intent, contract, context, state, node, gate, and output. After that, it moves on to debugging, testing, FREEFORM, Core, Profiles, Domain Packs, libraries, and large playbooks.

If you already work with complex instructions, you can read selectively. For example, the chapters on gates, debug mode, regression suites, or the feedback loop can serve as practical checklists for improving existing playbooks.

The book includes small examples, common mistakes, and mini-exercises. You do not have to complete them, but they are what help turn Ordo from theory into a practical tool.

Nebu mascot markers are used as visual cues for attention, ideas, points for reflection, or interesting observations.

Introduction: The Core Idea of Ordo



Figure 6: Nebu — idea

The core idea of Ordo is that a complex instruction for an AI model should not be merely text. It should be an executable contract.

An ordinary prompt often describes the desired result. Ordo describes the path to that result: what must be established, which decisions may be made, where execution must stop, which checks must be passed, what counts as a completed result, and how execution should be explained.

The basic Ordo formula looks like this:

intent → contract → context → state → path → steps → gates → result → handoff

This formula matters because it changes the entire approach to working with a model. We no longer simply ask the model to “do it well.” We define the process within which the model must operate.

Ordo does not try to turn an AI model into an ordinary deterministic program. The model remains a semantic executor: it understands

text, generalizes, formulates, compares, and proposes. But Ordo defines the boundaries of that freedom.

In short:

Ordo is a language for governed execution of instructions by AI models.

Or, even more simply:

Ordo turns human intent into a process that a model can execute, validate, explain,

M30 Clarification: Ordo as a Process Rail

The current core idea of Ordo is that it is neither a CLI-first runtime nor a rigid wizard. Ordo gives AI a Process Rail: a supporting structure that holds state, route, gates, backward transitions, and results while leaving the AI as an active, reasoning executor.

AI reasons and communicates.

The Process Rail holds the process.

Semantic JSON IR preserves the machine-readable form of the rail.

The CLI validates deterministic parts.

Chapter 1. Why Ordinary Prompts Are Not Enough



Figure 7: Nebu — attention

Why this matters

Prompts work well for simple tasks. If you need to translate a short sentence, invent a title, or quickly explain a concept, an ordinary prompt is often enough.

Problems begin when a task becomes a process.

For example, the task may be not merely to “prepare a document,” but first to collect data, clarify the contract, choose a scenario, ask

the right questions, avoid moving to the final result without confirmation, perform a self-check, and prepare a handoff. In such a task, the path to the result matters just as much as the result itself.

An ordinary prompt does not hold that path reliably.

A simple explanation

A prompt is a textual request to a model. It may be short or long, simple or detailed. But even a very long prompt remains a continuous block of text.

In continuous text, a model may:

- skip part of the rules;
- confuse an example with a mandatory requirement;
- execute a step too early;
- treat an unconfirmed assumption as a confirmed fact;
- mix statuses;
- create the final result before a gate is passed;
- fail to explain why it selected a particular path.

This does not mean that the model is “bad.” It means that we are trying to govern a complex process through a format that was not designed for that purpose.

A large prompt is not the same as a precise process

A common mistake is to assume that making a prompt longer automatically makes it more reliable.

In reality, long prompts often create new problems. They accumulate duplicates, exceptions, conflicting wording, examples with unclear normative force, and old instructions that no longer match the new logic.

The model reads all of this as text and tries to help. But it does not always clearly distinguish which part is a contract, which is an explanation, which is an example, and which is a prohibition.

Ordo proposes a different approach: do not make the prompt endless. Decompose the instruction into governed parts.

What actually breaks in ordinary prompts

In complex tasks, the following areas fail most often:

1. Contract

The model starts working before it understands what must actually be done.

2. Order of actions
The model moves to the final result before intermediate confirmations.
3. Branching
The model selects the wrong scenario and does not explain why.
4. Process state
The model forgets what has already been confirmed and what has not.
5. Checks
The model treats a gate as a recommendation rather than a blocking condition.
6. Prohibitions
The model does something that “looks useful” even though it was forbidden.
7. Explainability
After an error, it is unclear at which step the error appeared.
That is why complex instructions need more than better wording.
They need a different governance structure.

The Ordo shift in thinking

Ordo changes the main question.

Instead of:

What prompt should I give the model?

we ask:

What process should the model execute?

This is a small change in wording, but a major change in approach.

A prompt is oriented toward an answer.

Ordo is oriented toward execution.

In a prompt, we often describe the desired result. In Ordo, we describe the process in which that result is allowed to be created.

The basic formula

In its simplest form, an Ordo process can be described as:

intent → contract → state → path → steps → gates → result → handoff

This means:

intent – what the user wants;
contract – what the model is allowed to do;
state – what is already known and confirmed;
path – which execution scenario has been selected;
steps – which steps must be completed;
gates – which checks must be able to stop the process;
result – what may be delivered as the result;
handoff – what must be passed to a person or the next process.

This formula is the first answer to why ordinary prompts are not enough.

A typical mistake

A typical mistake is trying to repair an unstable instruction by adding one more paragraph at the end of the prompt.

For example:

And be sure not to forget to check everything before the final result.

That is better than nothing. But it is not enough for a complex process.

In Ordo, such a requirement should become a gate:

```
GATE.CHECK(method=self_verification) self_validation_passed  
ASSERTION(polarity=not) final_output_before_validation
```

Then it is no longer a request to “remember.” It is part of the process.

Mini-exercise

Take any long prompt that you actually use or could realistically use.

Try to identify:

- the primary goal;
- the expected result;
- the rules;
- the data that must be collected;
- decisions the model may make independently;
- decisions a human must confirm;
- points where the model must stop;
- checks required before the final result;
- prohibited actions.

If these things are scattered throughout the text and have no clear structure, the instruction is already a candidate for Ordo.

Short summary

Ordinary prompts are useful, but they are not a reliable way to govern complex processes.

When a task has many steps, branches, checks, statuses, and prohibitions, it needs a structure that describes not only the answer, but also the execution.

Ordo exists for exactly this purpose: to turn human instructions into a governed, explainable, and verifiable process for an AI model.

Chapter 2. What Ordo Is in Simple Terms

2.1. Ordo is not another “prompt format”

When people first hear about Ordo, it is easy to assume that it is simply another way to format prompts: some YAML, some JSON, and names such as intent, gate, state, and result.

That is not quite right.

A prompt is an address to a model.

Ordo is a description of a process that the model must execute.

A prompt often sounds like:

Create the result for me.

Ordo sounds different:

Here is the goal.

Here is the contract.

Here is what is already known.

Here are the questions you may ask.

Here is the order in which to proceed.

Here is what is forbidden.

Here is where you must stop.

Here are the checks you must pass.

Here is the result you may deliver.

Ordo therefore does not merely ask a model to do something. It defines the execution boundaries.

A prompt is a request.

Ordo is an instructional program.

Or even more simply:

A prompt tells the model: “do it.”

Ordo tells the model: “execute the process.”

2.2. An everyday analogy: a recipe and a kitchen

Imagine asking someone to cook borscht.

You can say:

Cook borscht.

An experienced person may do it well. But the result depends on their memory, habits, taste, and assumptions.

You can provide a recipe:

1. Prepare the ingredients.
2. Cook the stock.
3. Add the beetroot.
4. Add the cabbage.
5. Check the salt.
6. Serve with sour cream.

That is better. Yet even a recipe may be ambiguous. What happens if beetroot is unavailable? May an ingredient be substituted? When is the dish considered ready? Who confirms the taste?

Ordo is not merely a recipe. It is a recipe with execution rules:

- if an ingredient is missing, do not invent a substitute; ask;
- do not move to serving until readiness has been checked;
- if the dish is oversalted, do not mark it finished;
- if the user requested a vegetarian version, do not use meat stock;
- at the end, show which checks were passed.

That is how Ordo works with an AI model.

It describes not only the desired result, but also how the model must move toward it.

2.3. Ordo as a route map

Another simple analogy is navigation.

A prompt may resemble:

Drive to Lviv.

The model can choose the route itself. Sometimes that is fine. But if you must avoid toll roads, avoid a particular region, make a stop, check fuel, and not complete the route without arrival confirmation, one sentence is no longer enough.

Ordo is a route with checkpoints:

start
→ choose direction
→ check constraints
→ pass checkpoint
→ if the road is closed, switch to an alternative path
→ do not finish without arrival confirmation

In Ordo terms:

ENTRY.DEF
→ NODE.DEF
→ STATE.UPDATE
→ GATE.CHECK
→ PATH.SELECT
→ OUTPUT.DEF
→ HANDOFF

Ordo gives the model not only the destination, but the rules of movement.

2.4. The main parts of Ordo

At the simplest level, Ordo consists of several basic parts.

Intent

Intent is the goal.

```
intent:  
  goal: create_summary
```

or:

```
intent:  
  goal: guide_user_through_history_event_intake
```

Intent answers:

Why is this process being started?

Contract

Contract defines the result and its rules.

```
contract:  
  output:  
    format: bullet_list  
    max_items: 3  
  rules:
```

- do_not_invent_facts
- use_only_input_text

Contract answers:

What counts as a correct result?
 What is forbidden?
 Which constraints must be satisfied?

Context

Context contains input data, documents, sources, or previous decisions.

```
context:
  source_text: "$USER_INPUT.text"
  playbook: "history_event_playbook_ordo_v0_10"
```

Context answers:

Where does the model obtain the data it works with?

State

State is the current process state.

```
state:
  current_node: N1
  selected_path: null
  confirmed_contracts: []
  open_questions: []
```

State answers:

What is already known?
 Where are we now?
 What remains unconfirmed?

Path

Path is the selected execution route.

```
path:
  selected: Path_1
  reason: "internal source row change"
```

Path answers:

Which scenario are we following?

Step

Step is a concrete action.

```
step:  
  id: S1  
  do: read_input_text
```

Step answers:

What is the model doing now?

Gate

Gate is a blocking check.

A gate is not a suggestion. If its condition is not satisfied, the process must not move forward.

```
GATE.CHECK(method=self_verification) self_validation_passed
```

Gate answers:

What must be true before execution may continue?

Result

Result is the outcome that the process is allowed to produce after its requirements are satisfied.

It may be a document, answer, package, decision, structured record, or another process artifact.

Handoff

Handoff defines what is passed to a person or another process after the current process reaches the required state.

It answers:

What is being transferred?

To whom or to what?

In which status?

With which open questions, warnings, or evidence?

These concepts are simple individually. Their value appears when they are combined into one execution structure.

2.5. Ordo does not require everything to become machine code

A common misunderstanding is that a formal language must describe every sentence with a rigid operator.

Ordo does not require that.

Some parts of a process should be deterministic:

- allowed statuses;
- required fields;
- blocking conditions;
- path transitions;
- forbidden actions.

Other parts should remain semantic:

- explain the issue clearly;
- compare two variants;
- ask a natural follow-up question;
- write a useful summary;
- propose wording.

The model is still a model. Ordo does not remove its ability to understand language. It gives that ability a process rail.

2.6. Ordo has several reading levels

The same process may be represented at several levels.

Human level

A person reads the process as an explanation, guide, or playbook.

Ordo Source

The author describes the process in a source form designed for editing and review.

Semantic JSON IR

The compiler produces a machine-readable semantic representation in which entities, links, contracts, and execution rules are explicit.

Compact IR

Where useful, the process may also have a compact representation optimized for transport or runtime use.

These levels do not compete with one another. They serve different readers and different stages of the toolchain.



Figure 8: Nebu — attention

2.7. Ordo does not replace the model

Ordo is not an attempt to reduce an AI model to a traditional script.

The model still interprets meaning, works with incomplete language, writes, compares, summarizes, and communicates with the user.

Ordo governs where that freedom is allowed and where a hard rule must take priority.

The model reasons.

Ordo holds the process.

2.8. Ordo is not rigid bureaucracy

A small process should remain small.

If the task is simply:

Summarize the text in three bullets without inventing facts.

the Ordo program does not need fifty nodes and twenty gates.

The language should add structure where structure reduces risk. It should not create ceremony for its own sake.



Figure 9: Nebu — thinking

2.9. A small example: an ordinary task

Suppose the user wants a short summary.

A minimal process may be understood as:

```
intent: create a short summary
context: source text
contract: three bullets, no invented facts
step: read and summarize
gate: every bullet must be supported by the source
result: bullet list
```

The important difference is that the desired behavior has been separated into roles. The goal is not mixed with the data, and the check is not hidden in a final sentence.

2.10. A large example: a playbook

Now imagine a playbook for creating a complex domain artifact.

The process may contain:

```
entry;
guided intake;
answer registries;
state;
candidate paths;
path confirmation;
hard contracts;
assumption ledger;
blocking gates;
artifact generation;
QA;
handoff;
execution trace.
```

The principle is still the same as in the small example. The process is simply larger.

Ordo should scale by composition, not by turning one prompt into an ever-growing wall of text.

2.11. Ordo as a discipline of thought

Even before a compiler or runtime executes an Ordo program, writing a process in Ordo forces useful questions:

```
What is the actual intent?
What is the contract?
```



Figure 10: Nebu — interesting

What is data and what is a rule?
Which facts are confirmed?
Which assumptions remain?
Where may the process branch?
What blocks completion?
What is the final result?

This makes Ordo useful not only as syntax, but also as a method for designing processes.

2.12. Short chapter summary

Ordo is a language for describing governed processes executed with an AI model.

It is not simply a prompt format.

It separates goal, contract, context, state, route, actions, checks, result, and handoff.

It preserves the semantic strengths of the model while making the process more explicit, testable, and explainable.

Mini-exercise

Take a task you would normally describe with one prompt.

Write one line for each item:

Intent:
Contract:
Context:
State:
Path:
Step:
Gate:
Result:
Handoff:

If several of these concepts are mixed in the same sentence, try separating them.

Ordo after the Process Rail reframing

The preferred operating model of Ordo is the Process Rail.

The AI remains the active semantic executor. It communicates, interprets, proposes, and reasons. The rail holds the process structure: state, route, gates, transitions, and completion conditions.

AI reasons and communicates.

The Process Rail holds the process.

Semantic JSON IR preserves the machine-readable rail.

The CLI validates deterministic parts.

This is why Ordo is neither “just a prompt” nor “a replacement for the model.” It is the governed process layer between intent and execution.

Chapter 3. Ordo as a Language for Governing a Model



Figure 11: Nebu — idea

3.1. Not merely a format, but a way to govern behavior

It is tempting to look at Ordo syntax and focus on YAML, JSON, operators, and identifiers.

But the most important part of Ordo is not the brackets or field names. The important part is behavior governance.

When we write:

```
GATE.CHECK
```

we are not merely formatting a check. We are saying that execution must stop if the condition is not satisfied.

When we define:

`STATE.UPDATE`

we are saying that a process fact must become explicit state rather than remain an informal memory from the conversation.

When we define a node, we establish the scope of the current step.

The language therefore describes what the model may do, what it must do, and what it must not do at a given point in the process.

3.2. How Ordo differs from an ordinary instruction

An ordinary instruction may say:

First clarify the requirements, then prepare the document, and check it carefully.

A person understands the general meaning. A model probably does too.

But several questions remain:

What exactly counts as clarified?

Which requirements are mandatory?

May the model make assumptions?

When may the document be generated?

What does “check carefully” mean?

What happens if the check fails?

Ordo tries to make these execution questions explicit.

Instead of one broad instruction, the process can separate:

contract clarification;

state writes;

allowed assumptions;

generation conditions;

validation gates;

failure transitions.

The difference is not verbosity. The difference is that process semantics become visible.



Figure 12: Nebu — attention

3.3. Ordo defines an allowed action space

A useful way to understand Ordo is as a language that defines the model's allowed action space.

At a particular node, the model may be allowed to:

- ask one question;
- validate an answer;
- write a value to state;
- propose a candidate path.

At the same node, it may be forbidden to:

- confirm a hard contract;
- generate the final package;
- silently invent a missing value;
- skip a blocking gate.

This is stronger than saying "please be careful."

The process itself describes the boundary.

A well-designed Ordo program does not attempt to predict every sentence the model will produce. It defines the semantic freedom available at each stage.

3.4. The model as executor, not author of the process

In an unmanaged prompt, the model often becomes both executor and process designer.

The user gives a broad goal, and the model decides:

- what to ask;
- in which order;
- which assumptions to make;
- when enough information has been collected;
- when to produce the result.

Sometimes that works well. But the process may change from run to run.

In Ordo, the author of the process decides the important structure in advance.

The model executes that structure.

This does not mean the model is passive. It may still formulate natural questions, explain choices, summarize evidence, compare vari-

ants, or propose wording. But it should not silently rewrite the process while executing it.

The process author designs the rail.
The model travels the rail.

3.5. Human language remains important

Ordo is not based on the idea that natural language is bad.

Natural language is essential because many tasks are semantic.

For example:

Explain the difference in simple terms.

Summarize the user's concern.

Propose three clear names.

Compare the evidence.

Ask a natural follow-up question.

Trying to encode every such task as deterministic code would remove much of the value of an AI model.

Ordo therefore uses a hybrid approach.

Hard process structure is explicit.

Semantic work remains expressed in human language.

The language governs the boundary between them.

3.6. Ordo as an intermediate layer between person and model

A person usually thinks in goals and processes.

A model consumes language and context.

A traditional program executes formal instructions.

Ordo sits between these worlds.

human intent

↓

Ordo process

↓

semantic execution by AI

↓

validated result

The human does not need to write ordinary software for every workflow.

The model does not need to infer the entire process from an unstructured prompt.

Ordo provides an intermediate representation of process intent.

3.7. Ordo does not necessarily require a separate runtime today

A language may be useful before every part of its runtime is fully automated.

An Ordo source can already improve a process because it makes contracts, state, paths, and gates explicit.

A model can interpret the process directly.

A compiler can validate deterministic structure.

A CLI can check schemas, references, and invariants.

A future runtime may execute more of the rail automatically.

These stages are compatible.

The language contract should not depend on pretending that all execution is already deterministic.



Figure 13: Nebu — interesting

3.8. Ordo as “programming without code” for AI behavior

This phrase is only an analogy, but it is useful.

When designing an Ordo process, you think about familiar programming concepts:

```
state;  
branches;  
preconditions;  
postconditions;  
validation;  
errors;
```

outputs;
interfaces.

Yet the executor is a semantic model, and many actions remain natural-language tasks.

Ordo therefore resembles programming at the level of process behavior rather than machine instructions.

You are not programming CPU operations.

You are designing the governed behavior of an AI-assisted process.

3.9. Tasks that fit Ordo well

Ordo is especially useful when a task has several of these properties:

multiple steps;
branching scenarios;
required confirmations;
state that must survive a long dialogue;
blocking checks;
forbidden shortcuts;
structured outputs;
human approval points;
handoff to another process;
need for replay, audit, or explanation.

Examples include:

guided intake;
analytical playbooks;
document production workflows;
QA procedures;
requirements collection;
incident triage;
review and approval processes;
multi-stage research;
domain-specific AI assistants.

A one-line creative request usually does not need Ordo.

A process with contracts and consequences often does.

3.10. Typical misunderstandings about Ordo

Mistake 1. “Ordo is just JSON”

JSON may represent Ordo IR, but the language is the semantics of the process.



Figure 14: Nebu — thinking

Changing JSON formatting does not change the fact that a gate blocks, a state field records process memory, or a node limits the current action scope.

Mistake 2. “Ordo must formalize absolutely everything”

No.

Formalize what needs stable process semantics.

Leave semantic work to the model where flexibility is valuable.

Mistake 3. “Ordo is only for the History Event Playbook”

The History Event work was an important source of requirements, but Ordo is a general process language.

Its concepts apply to other domains.

Mistake 4. “Ordo replaces the human”

Ordo can explicitly require human confirmation.

The language is designed to distinguish model decisions from human-controlled decisions.

Mistake 5. “Ordo must always be complex”

A small process may use only a few concepts.

Complexity should follow the problem.

3.11. Short chapter summary

Ordo is a language for governing model behavior inside a process.

It defines the allowed action space, preserves process state, makes blocking conditions explicit, and separates semantic freedom from hard process rules.

The model remains an active executor.

The process author remains responsible for the process architecture.

Mini-exercise

Take one AI workflow you know.

Ask:

1. Which decisions may the model make?
2. Which decisions require a human?
3. Which actions are forbidden before confirmation?
4. What state must be preserved?
5. What condition blocks the final result?

These answers describe the beginning of an Ordo process.

Governing a model does not mean replacing the model

The Process Rail reframing makes this distinction especially important.

Ordo should not duplicate the model's semantic ability with a rigid pseudo-runtime. It should hold the parts that need continuity and control:

```
state;  
route;  
gates;  
transitions;  
contracts;  
completion conditions.
```

The model should continue to perform the semantic work:

```
understanding;  
conversation;  
comparison;  
explanation;  
proposal;  
reasoning.
```

The preferred architecture is therefore hybrid.

AI = active semantic executor

Ordo = governed process rail

CLI/compiler = deterministic validation and tooling

Chapter 4. Intent, Contract, Context



Figure 15: Nebu — attention

4.1. Why these three concepts matter

At the beginning of a complex process, three different questions are often mixed together:

What do we want?
What counts as a correct result?
What information are we working with?

In Ordo, these questions belong to three different concepts:

Intent
Contract
Context

The distinction is important because a model behaves more consistently when the goal, rules, and data are not hidden inside one paragraph.

A useful first approximation is:

Intent = why the process exists.
Contract = what the process must guarantee.
Context = what the process works with.

4.2. Intent: purpose or goal

Intent describes the purpose of the process.

A simple example:

```
intent:  
  goal: create_summary
```

A more specific example:

```
intent:  
  goal: guide_user_through_history_event_intake
```


Intent answers:

Why are we running this process?

A good intent is usually short and stable.

It should help distinguish this process from another process.

For example:

`create_summary`

is different from:

`validate_summary`

and both are different from:

`approve_summary_for_publication`

The output may look similar, but the purpose of the process is different.

4.3. Typical Intent mistakes

Mistake 1. Making intent too broad

Bad:

```
intent:
  goal: help_user
```

Almost any AI process can claim to “help the user.”

Better:

```
intent:
  goal: collect_requirements_for_jira_task
```

The intent should identify the process purpose.

Mistake 2. Putting rules inside intent

Bad:

```
intent:
  goal: create_summary_without_inventing_facts_and_using_three_bullets
```

The goal and the contract have been mixed together.

Better:

```
intent:
  goal: create_summary
```

```
contract:
  rules:
    - do_not_invent_facts
  output:
    max_items: 3
```

Mistake 3. Confusing intent and output

The intent explains why the process exists.

The output describes what the process produces.

For example:

Intent: clarify an incident.

Output: incident triage record.

These concepts are related but not identical.

4.4. Contract: what counts as a correct result

A contract defines the conditions that the process result must satisfy.

Example:

```
contract:
  output:
    format: bullet_list
    max_items: 3
  rules:
    - use_only_input_text
    - do_not_invent_facts
```

The contract answers:

What is the process required to guarantee?

What is forbidden?

Which conditions define an acceptable result?

A contract may define:

- required fields;
- output structure;
- allowed values;
- prohibitions;
- quality requirements;
- confirmation requirements;
- preconditions;
- postconditions.

The important point is that a contract is not merely a description of what would be nice.

It defines what the process treats as valid.

4.5. A Contract does not always have to be invented from scratch

In real systems, a contract may already exist.

It may come from:

- a schema;
- an API;
- a database model;
- a document template;
- a domain standard;
- an existing approved process;
- another Ordo program.

In that case, the Ordo process should reference or import the existing contract rather than silently reinterpret it.

For example:

```
contract:  
  source: jira_task_schema_v2
```

or conceptually:

```
CONTRACT.IMPORT jira_task_schema_v2
```

This reduces duplication and contract drift.



Figure 16: Nebu — thinking

4.6. Contract and confirmed

A particularly important process distinction is the difference between a proposed contract and a confirmed contract.

A model may infer or propose:

```
field path;  
event type;  
output structure;  
mapping;  
validation rule.
```

But a proposal is not automatically confirmed.

A process may preserve status explicitly:

```
contract:  
  status: proposed
```

and later:

```
contract:  
  status: confirmed
```

This distinction protects the process from turning a plausible model assumption into an authoritative fact.

A gate may require:

```
contract.status == confirmed
```

before the process moves to generation.

4.7. A Contract may contain more than output rules

It is easy to think of a contract only as an output schema.

But contracts may govern execution too.

For example:

```
contract:  
  input:  
    required:  
      - source_text  
  
  output:  
    format: bullet_list  
    max_items: 3  
  
  rules:  
    - do_not_invent_facts  
  
  confirmations:  
    required:  
      - user_approval
```

```
completion:
  require:
    - self_validation_passed
```

The contract may therefore describe the boundary between input, execution, confirmation, and completion.

4.8. Context: what the model works with

Context contains the information available to the process.

Example:

```
context:
  source_text: "$USER_INPUT.text"
  playbook: "history_event_playbook_ordo_v0_10"
```

Context may include:

- user input;
- documents;
- source records;
- reference data;
- previous decisions;
- domain packs;
- selected playbooks;
- external evidence.

Context answers:

What information may the process use?

This is different from a contract.

A document may be context.

A rule saying “do not invent facts” is a contract rule.

4.9. Why context should be bounded

A common mistake in AI systems is to give the model everything that might possibly be useful.

More context is not always better.

Large uncontrolled context may contain:

- obsolete instructions;
- conflicting versions;
- irrelevant examples;
- unconfirmed assumptions;
- documents from another scope.

A governed process should know which context belongs to the current execution.

For example:

```
context:
  primary_source: incident_report_17
  supporting_sources:
    - service_log_excerpt
  excluded_sources:
    - deprecated_runbook_v1
```

The purpose is not to starve the model of information. The purpose is to make information provenance visible.



Figure 17: Nebu — idea

4.10. Intent, Contract, and Context in one example

Suppose the user asks the model to summarize an incident report.

Intent

```
intent:
  goal: create_incident_summary
```

The process exists to create an incident summary.

Contract

```
contract:
  output:
    format: bullet_list
    max_items: 5
  rules:
    - do_not_invent_facts
    - preserve_severity
    - mention_unresolved_blockers
```

This defines an acceptable result.

Context

```
context:
  incident_report: "$INPUT.report"
  service_name: "$INPUT.service"
```

This defines the information available to the process.

The three concepts now have distinct roles:

```
Intent    → purpose
Contract  → validity
Context   → information
```

4.11. How these concepts relate to gates

Gates often validate contracts using state and context.

For example:

```
GATE.CHECK incident_report_present
GATE.CHECK severity_preserved
GATE.CHECK unresolved_blockers_reported
```

The gate is the execution mechanism that prevents the process from continuing when a required condition is not satisfied.

Intent says why.

Contract says what must be true.

Context provides information.

State records what has happened.

Gate checks whether execution may continue.

4.12. A small template for starting any Ordo program

A useful starting template is:

```
intent:
  goal: <process_goal>

contract:
  status: <proposed|confirmed>
  input:
    required: []
  output:
    required: []
```

```
rules: []

context:
  sources: []
  references: []
```

This is not a complete Ordo program.

It is a disciplined starting point.

Before designing nodes and paths, you should be able to explain the process purpose, validity conditions, and working information.

4.13. Typical mistakes

Mistake 1. Starting with output without intent

If you begin with a template before understanding the process goal, the process may optimize the wrong result.

Mistake 2. Writing rules in context

Bad:

```
context:
  do_not_invent_facts: true
```

This is a rule, not source information.

Put it in the contract or policy layer.

Mistake 3. Putting a data source in the contract

Bad:

```
contract:
  incident_report: report_17
```

The report is context.

The contract may require a report to be present, but the concrete report belongs to execution context.

Mistake 4. Not recording contract status

If a model proposes a contract and the process immediately treats it as confirmed, assumptions become invisible.

Use explicit statuses where confirmation matters.

4.14. Short chapter summary

Intent, Contract, and Context answer three different questions.

Intent – why does the process exist?

Contract – what must the process guarantee?

Context – what information does the process work with?

Keeping them separate makes the process easier to review, validate, and change.

Intent should be focused.

Contract should define validity and boundaries.

Context should expose the information available to execution.

Mini-exercise

Take any AI task and write:

Intent:

Contract:

Context:

Then inspect every sentence.

If a rule is in Context, move it to Contract.

If output structure is hidden in Intent, separate it.

If the process goal is still “help the user,” make it more specific.

Chapter 5. State: Process Memory



Figure 18: Nebu — idea

5.1. Why Ordo needs State

A long conversation is not the same thing as explicit process memory.

A model may see the conversation history, but that does not guarantee that every confirmed fact, open question, selected path, or blocker will remain semantically stable throughout execution.

A process therefore needs State.

State is the explicit memory of the current run.

It answers:

Where are we?
What is already known?
What has been confirmed?
What is still proposed?
What remains open?
What blocks completion?

Without State, the model must reconstruct the process from conversation text every time.

With State, the important execution facts are explicit.

5.2. How State differs from Context

Context is information available to the process.

State is information about the current execution.

For example:

```
context:  
  source_document: incident_report_17
```

This is a source.

But:

```
state:  
  source_document_reviewed: true  
  severity: high  
  severity_status: confirmed
```

describes what has happened in the current run.

A simple distinction is:

Context = what we work with.

State = what the process currently knows and where it is.

5.3. A simple State example

Consider a guided intake process.

```

state:
  current_node: N3
  selected_path: Path_1
  task_type: incident
  business_goal:
    value: restore_service
    status: confirmed
  acceptance_criteria:
    value: null
    status: missing
  open_questions:
    - acceptance_criteria
  blockers:
    - acceptance_criteria_missing

```

A model reading this state does not need to infer whether the business goal was confirmed.

It is explicit.

The model also sees that the process is at N3, follows Path_1, and cannot complete because acceptance criteria are missing.

5.4. STATE.SCHEMA

A process should define the shape of its state.

Conceptually:

```

state_schema:
  current_node:
    type: string
    required: true

  selected_path:
    type: string
    nullable: true

  confirmed_contracts:
    type: array

  assumptions:
    type: array

  open_questions:
    type: array

```

```
blockers:  
  type: array
```

In Ordo, STATE.SCHEMA exists to make process memory explicit and validatable.

The schema answers:

Which state fields exist?
Which values are allowed?
Which fields are required?
Which statuses are valid?

This is important because uncontrolled state can become as chaotic as an uncontrolled prompt.

5.5. State as protection against repeated questions

Imagine that the user has already confirmed:

The request is an incident.

If the process writes:

```
state:  
  request_type:  
    value: incident  
    status: confirmed
```

the next node can see that the question has already been answered.

The process should not ask it again unless a specific transition invalidates or reopens that field.

This improves user experience and process consistency.

State is therefore not only technical memory. It is also a mechanism for respecting what the user has already told the process.

5.6. State as protection against premature results

Suppose the process must not generate a final Jira task until the business goal and acceptance criteria are confirmed.

State may contain:

```
state:  
  business_goal:  
    status: confirmed  
  acceptance_criteria:  
    status: missing
```

```
handoff:
  allowed: false
```

A gate can check:

```
business_goal.status == confirmed
AND
acceptance_criteria.status == confirmed
```

Until the condition is satisfied, the final handoff remains blocked.

Without State, the model may decide that it has “enough information.”

With State, completion conditions can be evaluated explicitly.



Figure 19: Nebu — thinking

5.7. State and statuses

Values in a process often need status, not only content.

For example:

```
state:
  event_type:
    value: address_change
    status: proposed
```

Later:

```
state:
  event_type:
    value: address_change
    status: confirmed
```

Useful status categories may include:

- missing
- proposed
- confirmed
- rejected
- deprecated
- blocked

The exact status set depends on the contract.

The important principle is that a value and the authority of that value are different things.

A plausible value is not automatically a confirmed value.

5.8. Assumption ledger as part of State

Models are good at filling gaps. In governed processes, invisible gap-filling is dangerous.

An assumption ledger makes assumptions explicit.

Example:

```
state:
  assumptions:
    - id: A1
      statement: "The change affects only the primary address."
      status: proposed
      evidence_refs:
        - user_message_12
```

The process can later:

- confirm the assumption;
- reject it;
- replace it with a fact;
- block completion until it is resolved.

The main rule is:

An assumption may help the process move,
but it must not silently become a confirmed contract.

5.9. State and open questions

Open questions should also be explicit.

Example:

```
state:
  open_questions:
    - id: Q1
      field: acceptance_criteria
      blocking: true
    - id: Q2
      field: optional_label
      blocking: false
```

This allows the process to distinguish between:

- a question that must be resolved before completion;
- a question that may remain open;
- a question that has already been answered.

A final result can then report unresolved non-blocking questions while refusing completion when blocking questions remain.

5.10. State and traceability

State tells us the current situation.

Trace tells us how we arrived there.

For example:

```
State:
event_type = address_change
status = confirmed
```

The execution trace may show:

```
event 12: user proposed address_change
event 13: validation passed
event 14: user confirmed
event 15: state changed from proposed to confirmed
```

These concepts should not be confused.

State = current process memory.

EXECUTION_TRACE = history of process execution.

Together, they make the run both operable and explainable.

5.11. State in compiled IR

In compiled IR, state structure should remain machine-readable.

For example:

```
{
  "state": {
    "current_node": "N3",
    "selected_path": "Path_1",
    "fields": {
      "business_goal": {
        "value": "restore_service",
        "status": "confirmed"
      },
      "acceptance_criteria": {
```

```

        "value": null,
        "status": "missing"
    },
    "open_questions": [
        "acceptance_criteria"
    ],
    "blockers": [
        "acceptance_criteria_missing"
    ]
}

```

The runtime or model should not need to parse a narrative paragraph to determine whether handoff is allowed.



Figure 20: Nebu — attention

5.12. Typical State mistakes

Mistake 1. Not maintaining State explicitly

If important facts exist only in chat history, the process has no stable execution memory.

Mistake 2. Mixing Context and State

A source document is Context.

The fact that the document has been reviewed and accepted is State.

Mistake 3. Not distinguishing proposed and confirmed

This is one of the most dangerous mistakes in AI-assisted processes.

A model suggestion may be reasonable and still require confirmation.

Mistake 4. Not preserving blockers

If a blocker is only mentioned in prose, the process may later forget it.

Blockers should be explicit.

Mistake 5. Not clearing assumptions before the final result

A process should know which assumptions remain unresolved.

Where the contract requires confirmed facts, unresolved assumptions must block completion.

5.13. A practical State template for an Ordo process

A useful starting structure is:

```
state:
  current_node: START
  selected_path: null

  fields: {}

  confirmed_contracts: []

  assumptions: []

  open_questions: []

  blockers: []

  approvals: []

  outputs:
    status: not_started

  handoff:
    status: draft
    allowed: false
```

A real process will extend this structure.

The important thing is that process memory has an explicit home.

5.14. Short chapter summary

State is the explicit memory of an Ordo run.

It records:

current position;
selected path;

known values;
value statuses;
confirmed contracts;
assumptions;
open questions;
blockers;
approvals;
output and handoff readiness.

State differs from Context.

Context contains information available to the process.

State describes the current execution.

State also differs from EXECUTION_TRACE.

State tells us where the process is now.

Trace tells us how it got there.

Mini-exercise

Take any process in which the model must ask several questions before producing a result.

For example:

Prepare a Jira task from a user's description.

Try to describe State:

```
state:
  current_node: START
  task_type: unknown
  business_goal: missing
  acceptance_criteria: missing
  out_of_scope: missing
  assumptions: []
  open_questions: []
  blockers: []
  handoff:
    status: draft
    allowed: false
```

Then answer:

1. Which fields must be confirmed before the final task?
2. Which fields may be proposed?
3. Which open questions block handoff?
4. Which gate prevents the final document from being created too early?

5.11. Revision-bound evidence and invalidation

State must not merely say that an artifact was validated or confirmed. It must record exactly which artifact revision was validated or confirmed.

```
artifact_state:
  artifact_id: "playbook_package"
  revision: 7
  sha256: "..."
```

```
validation_evidence:
  target_revision: 7
  target_sha256: "...
  status: passed
```

When an upstream artifact changes, downstream evidence that depended on the previous revision becomes stale. Ordo must invalidate it explicitly rather than silently carrying it forward.

```
upstream revision changes
→ dependent confirmation becomes invalid
→ dependent validation becomes invalid
→ completion is blocked
→ new evidence is required
```

A safe runtime keeps an append-only invalidation log and increases `state_version` after every state mutation. Old evidence is historical information, not authorization for a new revision.

Chapter 6. Entry and Node: How the Model Asks Questions

6.1. Why Ordo needs Entry and Node

In an ordinary conversation with a model, the user often expects the model to understand where to begin.

For example, the user writes:

```
I want to create a new historical event.
```

The model may react in different ways. It may immediately propose an alias. It may ask for a source row. It may start building a passport.

It may move to QA. It may ask ten questions at once.

For a simple conversation, this is not always critical. For a playbook, that freedom is dangerous.

If a process has a correct order, the model should not decide on its own where to begin. The starting point must be explicit.



Figure 21: Nebu — idea: Entry as the process entry point

Ordo uses ENTRY.DEF for this.

ENTRY.DEF answers:

Where does this process begin?

And NODE.DEF answers:

Which concrete step or decision node is being executed now?

Together, they turn a conversation with a model into a governed route.

6.2. Entry is the door into the process

You can think of ENTRY.DEF as a door.

When the user says, “Let’s begin,” the model should not run across the entire playbook. It should enter through the defined door.

Example:

```
{
  "op": "ENTRY.DEF",
  "id": "START_HISTORY_EVENT_INTAKE",
  "description": "Start guided intake for a new History Event",
  "first_node": "N1_INPUT_TYPE"
}
```

This means:

The historical-event creation process always starts at N1_INPUT_TYPE.

Not with the alias. Not with display names. Not with QA. Not with automation. Not with the complete passport template.

It starts with the first question in the tree.

That is the value of ENTRY.DEF: it prevents the model from starting with whatever happens to look useful.

6.3. Node is one governed step

NODE.DEF describes one process node.

A node may be a question, a choice, a check, or an intermediate decision. In the introductory model, the easiest way to think about a node is as one question that the model asks the user.

Example:

```
{
  "op": "NODE.DEF",
  "id": "N1_INPUT_TYPE",
  "kind": "question",
  "question": "How should the system understand that the historical event occurred?",
  "answers": ["A", "B", "C", "D", "E", "F"],
  "state_write": "entry_input_type"
}
```

Such a node clearly defines:

- which question to ask;
- which answers are allowed;
- where to write the answer;
- what to do after the answer.

This matters because the model no longer has to invent the dialogue format while executing it.

6.4. Why you should not ask every question at once

When a model receives a complex task, it often tries to be helpful and asks everything at once:

Provide the alias, Ukrainian and English names, source row, type/sub_type, field path, values contract, QA fixture, automation scope, and runner status.

This looks efficient. But it overloads the user and creates process risk.



Figure 22: Nebu — attention: do not ask every question at once

Why?

Because some questions do not make sense until the path has been selected.

For example, until we know whether this is Path 1, Path 3, Path 4, or Path 5, we cannot correctly ask about a specific source contract. Until the source or ChangeRecord is confirmed, asking about `HistoryEvent.item.values` is unsafe. Until the contract is confirmed, it is too early to move to QA automation.

Ordo introduces the rule:

one node – one primary question

This does not mean that the process has few questions. It means that questions arrive in the correct order.

6.5. Answer Registry: allowed answers

In a normal conversation, the user may answer in any form. That is good for natural language, but risky for a governed process.

If a node expects A, B, C, D, E, or F, and the user writes 5, the model should not guess what the user meant. It should validate the answer.

This is what `ANSWER.REGISTRY` is for.

Example:

```
{
  "op": "ANSWER.REGISTRY",
  "node": "N1_INPUT_TYPE",
  "allowed": [
    {
      "key": "A",
      "meaning": "internal_mongo_source_row_change"
    },
    {
```

```

        "key": "B",
        "meaning": "new_source_or_data_block_extension"
    },
    {
        "key": "C",
        "meaning": "existing_internal_changerecord"
    },
    {
        "key": "D",
        "meaning": "external_history_fact_or_change_payload"
    },
    {
        "key": "E",
        "meaning": "state_comparison_or_data_block_comparison"
    },
    {
        "key": "F",
        "meaning": "unknown_or_other"
    }
],
"on_invalid": "ASK_VALID_OPTION_AGAIN"
}

```

This means that the user's answer must be validated before execution moves on.

If the answer is not in the allowed list, the model must not proceed to the next node.

6.6. State update after an answer

A node does not exist in isolation. Its result must enter state.

For example, the user answered:

A

The model should record:

```

state:
  entry_input_type: internal_mongo_source_row_change
  path_candidate: Path 1 or Path 2

```

Only then can execution move to the next node.

Without state, the model may “remember” the answer from the conversation text, but that is unreliable. In a long process, the answer may be lost or reinterpreted.

Ordo forces the model to update state explicitly.

6.7. Node as a way to prevent jumping ahead

In a correctly described Ordo process, every node has boundaries.

For example, `N1_INPUT_TYPE` is allowed only to identify the input type. It is not allowed to confirm an alias, invent type/sub_type, or create a QA package.

This can be described as:

```
{
  "op": "NODE.DEF",
  "id": "N1_INPUT_TYPE",
  "allowed_actions": [
    "ASK_QUESTION",
    "VALIDATE_ANSWER",
    "STATE.UPDATE",
    "PATH.CANDIDATE.SET"
  ],
  "forbidden_actions": [
    "ALIAS.CONFIRM",
    "SOURCE_ROW.REQUEST_FULL",
    "VALUES.CONTRACT.DEFINE",
    "PACKAGE.GENERATE"
  ]
}
```

This is an important idea.



Figure 23: Nebu — thinking: the boundaries of one node

A node does not only say what to do. It also says what must not be done at this step.

6.8. Example of a small tree

Imagine a simple support process.

The user writes:

I have a problem with the service.

The Ordo process might begin like this:

```
entry:
  id: START_SUPPORT_TRIAGE
  first_node: N1_REQUEST_TYPE

nodes:
  N1_REQUEST_TYPE:
    question: "What kind of request is this?"
    answers:
      incident: "incident / outage"
      change_request: "change request"
    write_to: request_type

  N2_INCIDENT_SEVERITY:
    when: request_type == incident
    question: "How critical is the problem?"
    answers:
      high: "blocks work"
      normal: "does not fully block work"
    write_to: severity

  N3_INCIDENT_EVIDENCE:
    when: request_type == incident
    question: "Do you have reproduction steps or an example?"
    answers:
      present: "yes"
      missing: "no"
    write_to: reproduction_evidence
```

This is already a decision tree. The model does not improvise the route; it executes it.

6.9. Node and user experience

It may seem that this approach makes the dialogue slower. In practice, it often makes it easier.

The user does not receive a huge questionnaire. They receive one question at a time.

The model does not burden the user with technical details too early. It guides them along the route.

This is especially important when the user does not know the system's internal terminology. An analyst does not need to know what Path 1 or Path 5 means. They can simply answer:

How should the system understand that the event occurred?

The model maps the answer to a candidate path.

6.10. Typical Entry and Node mistakes

The first mistake is having no explicit entry.

Then the model starts differently every time.

The second mistake is mixing several decisions in one node.

For example:

Select the Path, alias, source row, and QA fixture.

This is a bad node. It combines several different hard contracts.

The third mistake is having no allowed answers.

If answers are undefined, the model starts guessing.

The fourth mistake is failing to write the answer to state.

Then the answer exists in the chat but not in execution state.

The fifth mistake is allowing a node to do more than it should.

For example, the starting node must not be allowed to create the final package.

6.11. Short chapter summary

ENTRY.DEF and NODE.DEF prevent the model from starting a process arbitrarily or jumping between steps.

ENTRY.DEF defines the entry point.

NODE.DEF defines the concrete process node.

ANSWER.REGISTRY defines allowed answers.

STATE.UPDATE preserves the result of the answer.

The main principle is:

one node – one primary decision

This makes the dialogue governed, verifiable, and convenient for the user.

Mini-exercise

Take any complex process in which the model must ask the user questions.

Describe:

1. Which ENTRY starts the process?
2. What is the first NODE?
3. What one primary question does it ask?
4. Which answers are allowed?
5. Where is the answer written in STATE?
6. Which actions are forbidden at this node?

If you cannot answer these questions, the process is not yet ready for stable model execution.

6.12. Deterministic node contracts

For strict ARF execution, every executable node has one explicit profile and one atomic responsibility. A node contract should define:

```
node_contract:  
  profile: execution_node  
  prerequisites: []  
  allowed_inputs: []  
  allowed_actions: []  
  forbidden_actions: []  
  required_outputs: []  
  validation_gates: []  
  explicit_transitions: []  
  invalidation_effects: []  
  authorization_boundary: null
```

An action absent from allowed_actions is blocked in closed-world execution. A node cannot combine routing, execution, validation, and

authorization responsibilities merely for convenience. Such a node must be split.

The standard profiles are:

- `routing_node` — selects an explicit transition but performs no domain work;
- `capture_node` — obtains and records bounded input;
- `execution_node` — performs the declared atomic action;
- `validator_node` — evaluates a target without changing it;
- `authorization_node` — records explicit authorization for an exact scope;
- `terminal_node` — computes a final status from mandatory evidence.

The model may discuss related topics without changing the active node. Execution leaves the current node only through an explicit transition.

Chapter 6.1. Atomicity of Steps in Processes with Language Models

One step — one responsibility, one result, and one point of validation or confirmation.

A process executed by a person, a program, or a language model often looks simple only at the level of the step name. For example, the phrase “prepare the document” may hide fact collection, recovery of missing data, decision-making, writing, validation, approval, and a state change. For a person, such a phrase may sometimes be a convenient shorthand. For a language model, it is dangerous: the model may perform some actions, infer others, skip still others, and nevertheless declare the step complete.

Atomicity is not needed to split a process into artificial fragments. It is needed so that every step has a clear responsibility boundary, an observable result, and a local control point. Then an error can be found, fixed, and repeated exactly where it occurred instead of restarting the whole process.

1. Statement of the Principle

An **atomic step** is a complete operation with defined inputs, one primary responsibility, one observable result, and one set of completion criteria.

“One step” does not necessarily mean one short command or one sentence. A step may contain several mechanical actions if together they create one result and share one readiness criterion. For example, sorting keys in a structured document, formatting indentation, saving the file, and calculating a checksum may be one technical operation. But the same operation must not silently add missing requirements or approve a new version.

One Step

One complete operation that can be started, completed, repeated, or rejected as a whole.

One Responsibility

A step is responsible for one type of change: collect facts, form a decision, create an artifact, validate a result, or apply a state transition. If a step both determines facts and makes a decision, the two tasks begin to influence each other.

One Result

A step must have one primary observable output. It may be an evidence artifact, a design decision, a completed document, a validation report, or a new state. Several files may count as one result only when they form an indivisible technical set and cannot have different readiness statuses.

One Validation Point

A step must be checked against criteria that correspond specifically to its result. Factual-data validation differs from document-completeness validation, and document validation differs from user confirmation.

One Confirmation Point

Confirmation is required where a decision, agreement, or confirmed state changes. It must not be replaced by a model assumption. At the same time, confirmation should not be required after every mechanical action that does not change meaning.

2. Why Mixing Responsibilities Is Dangerous

Data Loss Between Internal Subtasks

When one large step contains several hidden operations, intermediate results often never materialize. The model may discover a contradiction but fail to record it before moving to generation. Later, it becomes impossible to determine whether the contradiction was lost, ignored, or resolved by assumption.

Hidden Assumptions

If a missing value is needed for the next action, a model tends to insert a plausible value. Without a separate reconstruction step, that value quickly starts to look like a confirmed fact.

Premature Transition

In a compound step, the first successful subcondition may incorrectly open the next stage. For example, data has been received but has not passed validation; nevertheless, the process moves to generation because `input_received = true`.

Inability to Localize an Error

If a step simultaneously collects data, makes a decision, and creates a document, the status `failed` does not explain where the problem occurred. A status of `completed` may also hide a weak result from one of the internal operations.

Formal Validation Instead of Semantic Validation

A large step is easy to check only for the presence of files, sections, or identifiers. This creates a false green: the structure exists, but the content is not executable.

Inability to Repeat One Operation

If the final review finds a missing rollback, the correct action is to return to creation of the rollback section. In a monolithic step, the whole process must be repeated, including facts that were already confirmed.

Mixing Facts, Assumptions, and Decisions

Facts must come from evidence. Assumptions must be marked. Decisions must rely on a recorded evidence artifact. If all of this happens in one context without intermediate outputs, the boundaries disappear.

Excessive Compression of the Result

A language model tends to compress repetitive or similar actions. Several concrete scenarios may turn into the phrase “check the relevant cases,” even though the concrete detail was the main value.

Self-Validation in the Same Context

The model may evaluate not the actual text but the intention it still holds in context. Therefore, post-render review must work with the materialized artifact rather than with the plan for creating it.

3. The Basic Pattern

- obtain or prepare input data
- create one defined result
- validate the result separately
- show the result or status
- obtain confirmation if required
- move to the next operation

Obtain or Prepare Input Data

Responsibility: collect confirmed data and explicitly mark gaps.

Allowed inputs: sources, previously confirmed artifacts, user answers.

Expected output: a structured input artifact.

Completion criterion: it is known what is confirmed, what is contradictory, and what is missing.

Must not do: make business decisions or silently reconstruct critical values.

Create One Defined Result

Responsibility: materialize one artifact or decision result.

Allowed inputs: only recorded and permitted data.

Expected output: a concrete document, record, decision, or state proposal.

Completion criterion: the result exists in a form suitable for separate validation.

Must not do: independently declare the result validated or confirmed.

Validate the Result Separately

Responsibility: check the materialized output against explicit criteria.

Allowed inputs: the completed artifact and validation rules.

Expected output: evidence with status passed, failed, or needs_review.

Completion criterion: every mandatory criterion has evidence.

Must not do: silently rewrite the result and then set passed.

Show the Result or Status

Responsibility: make process state visible to a person or the next node.

Expected output: artifact reference, diagnostic summary, or stage status.

Must not do: hide partial success behind a general completed.

Obtain Confirmation

Responsibility: record a decision where confirmed state changes.

Expected output: approved, rejected, or revision_requested.

Must not do: treat silence or plausibility as consent.

Move to the Next Operation

Responsibility: apply an explicit state transition after all preconditions are satisfied.

Completion criterion: the previous step's completion criteria are fully satisfied.

Must not do: transition on one partial positive signal.

4. Atomic Node Pattern

```
step:
  purpose:
  inputs:
  action:
  output:
  completion_criteria:
  validation:
  confirmation_required:
  next_step:
  failure_route:
```

purpose

One sentence explaining why the step exists. If the purpose contains several independent verbs, that is a signal to decompose the step.

inputs

An exact list of allowed inputs. It is important to distinguish confirmed data, proposed data, and open gaps.

action

The operation that creates the result. The action is what changes an artifact or forms a new output.

output

One primary observable result that can be passed to validation.

completion_criteria

Conditions under which the action is genuinely complete. They must not be limited to checking that a file exists.

validation

A separate check of the result. It must not silently fix or regenerate the output. If a fix is needed, validation returns a failure route.

confirmation_required

An explicit rule stating whether human or system confirmation is required. Missing confirmation must not be replaced by an assumption.

next_step

The next step is allowed only after completion criteria, validation, and, where required, confirmation have been satisfied.

failure_route

A route back to the nearest step capable of fixing the specific error. It must not automatically restart the entire process.

5. Correct Decomposition Patterns

Pattern Card: Collect → Resolve → Confirm

When to use: when sources contain gaps, contradictions, or incomplete values.

Problem it removes: mixing facts with reconstruction and confirmation.

Minimal structure:

- Collect confirmed facts and gaps
- Resolve one gap as proposal
- Confirm or reject proposal

Abstract example — a mixed document-preparation step.

Incorrect:

- Analyze the request, recover missing information, define processing rules, create the instruction, validate it, and mark it ready.

The model reads the request, notices that `delivery_mode` is missing, assumes scheduled, forms the rules, creates the document, validates it itself, and moves the process to approved.

Problem: it is impossible to determine whether scheduled was a fact, a reconstruction, or an assumption.

Correct:

- Step 1 – Extract confirmed data from the request
- Result: list of facts and list of gaps

Step 2 – Process the delivery_mode gap
Result: proposed value or explicit open gap

Step 3 – Show the reconstruction for confirmation
Result: confirmed value or return to clarification

Step 4 – Form processing rules
Result: separate design artifact

Step 5 – Generate the instruction
Result: completed document

Step 6 – Validate the document independently
Result: passed or failed validation

Step 7 – Confirm the document
Result: approved state

Success criterion: every reconstructed value has a separate status and is not used as confirmed before confirmation.

If a missing value affects rules, reconstruction must be a separate step. If the value does not affect the result, it may remain an open gap.

Pattern Card: Design → Materialize → Review

When to use: when a structure or contract must be defined before creating the actual artifact.

Problem it removes: treating a plan or coverage list as the completed result.

Minimal structure:

Design artifact
→ Materialized artifact
→ Post-render review evidence

Abstract example — a test plan is incorrectly treated as a completed instruction.

Incorrect: a positive scenario, empty value, no-change case, and error case are identified, after which MANUAL_CHECKS.md is created:

1. Prepare positive data.
2. Perform the update.
3. Check the expected result.
4. Roll back if necessary.

The file exists and formally has sections, but the executor must invent the query, values, checks, and rollback.

Correct: first create an execution specification:

```
manual_case:
  id: CASE-01
  fixture_lookup:
    source: sample_records
    filter:
      external_key: "EXAMPLE-1042"
  projection:
    external_key: 1
    current_value: 1
  action:
    method: PATCH
    path: /api/example-records/EXAMPLE-1042
    body:
      current_value: "new-example-value"
  expected_result:
    current_value: "new-example-value"
    audit_status: "recorded"
  rollback:
    method: PATCH
    path: /api/example-records/EXAMPLE-1042
    body:
      current_value: "original-example-value"
  post_rollback:
    expected_value: "original-example-value"
```

Only after a completeness gate may the specification be transformed into a document for a person.

Success criterion: the executor can perform the instruction without constructing material commands or expectations.

If, after reading the document, the executor still has to invent material commands or criteria, the document is not an executable instruction.

Pattern Card: Generate → Validate → Approve

When to use: for any result that must be checked and confirmed.

Problem it removes: generate-and-self-approve and automatic transition to confirmed state.

Minimal structure:

```
Generate draft
→ Validate final artifact
→ Request approval
→ Apply state transition
```

Abstract example — explicit state transition.

Incorrect:

```
draft → confirmed
```

immediately after document generation.

Correct:

```
GENERATE_DOCUMENT
output: draft_document
```

```
VALIDATE_DOCUMENT
output: validation_evidence
```

```
REQUEST_CONFIRMATION
output: approved | rejected | revision_requested
```

```
APPLY_STATE_TRANSITION
precondition:
  validation_status == passed
  approval_status == approved
output:
  state == confirmed
```

Success criterion: confirmed state changes only through a separate operation with explicit preconditions.

Pattern Card: One Artifact at a Time

When to use: when a package contains documents with different purposes and quality criteria.

Problem it removes: a general completed status that hides a weak file.

Abstract example — one step generates several documents.

Incorrect:

```
Based on confirmed requirements, create the specification,
user instruction, validation checklist, and final manifest.
```

Three files are meaningful, but the checklist consists of generic phrases. The overall step status is completed.

Correct:

Artifact A: specification

→ generate

→ validate

→ approve

Artifact B: user instruction

→ generate

→ validate

→ approve

Artifact C: validation checklist

→ generate

→ validate

→ approve

Artifact D: manifest

→ assemble from approved artifacts

→ cross-document validation

Success criterion: the package is assembled only from artifacts that have an individual ready status.

If results may have different readiness statuses, they must not be created as one indivisible result.

Pattern Card: Reconstruction as a Separate Branch

When to use: when the process must create something that the normal branch only reads.

Problem it removes: hidden reconstruction and mixing operations with different risk levels.

Abstract example — reconstruction of a record inside ordinary processing.

Incorrect: the step “Process the record and move to result generation” silently reconstructs a missing record from two sources, sets valid, and continues without evidence.

Correct:

Check whether the intermediate record exists

└ record exists → standard processing branch

└ record missing → separate reconstruction branch

↓

reconstruction evidence

↓
validation of reconstruction
↓
confirmation or rejection
↓
return to standard branch

Success criterion: the reconstructed object has evidence, validation, and an explicit status before it returns to the main branch.

If a process creates what it normally only reads, that is a different operation type and a separate branch.

Pattern Card: Explicit State Transition

When to use: when draft, approved, confirmed, released, or another meaningful state changes.

Problem it removes: a state transition occurring as a side effect of generation.

Success criterion: the transition has preconditions, a separate action, and a separate output state.

Pattern Card: Independent Post-Render Review

When to use: after materializing a document, file, table, PDF, or other final representation.

Problem it removes: checking intention instead of the actual result.

Abstract example — self-validation against the plan.

Incorrect: the model had the requirement “every section must contain a concrete example,” saw examples in its own plan, and returned `validation_status: passed`, even though one section of the completed document has no example.

Correct:

Generation:
create final_document.md

Post-render review:
open final_document.md
enumerate required sections
find a concrete example in every section
record missing examples
return passed or failed

Success criterion: validation evidence refers to the actual artifact, not to the plan or prompt.

Validate the artifact after materialization. The presence of a requirement in the plan does not prove its presence in the result.

6. Antipatterns

Antipattern Card: Combined Node

Signs: one node collects data, makes a decision, generates a result, and moves on.

Why it appears: the author wants to shorten the flow or reduce the number of nodes.

Consequences: hidden subtasks have different success levels but one overall status.

How to detect: purpose or action contains more than one independent verb; there are several outputs; different parts need different gates.

Replace with: Collect → Resolve → Confirm, Design → Materialize → Review.

Antipattern Card: Hidden Reconstruction

Signs: missing data is reconstructed and immediately used as confirmed.

Why it appears: the model tries to complete the task without leaving an open gap.

Consequences: downstream logic depends on an unconfirmed value.

How to detect: the trace contains no reconstruction event, but the result contains a value absent from the sources.

Replace with: Reconstruction as a Separate Branch.

Antipattern Card: Design Equals Artifact

Signs: a plan, scenario list, or coverage list is incorrectly treated as a completed instruction.

Why it appears: the structure looks complete and the file formally exists.

Consequences: the executor reconstructs concrete actions, inputs, and expected results.

How to detect: the document contains many words such as “appropriate,” “required,” or “expected,” but few concrete values.

Replace with: Design → Materialize → Review.

Antipattern Card: Generate and Self-Approve

Signs: the same step creates the result and declares it ready.

Why it appears: an attempt to save a separate review step.

Consequences: the model evaluates its intention rather than the artifact.

How to detect: there is no validation evidence between generation and approved state.

Replace with: Generate → Validate → Approve.

Antipattern Card: Validation by Presence

Signs: validation checks only whether a file, section, or ID exists.

Why it appears: such checks are easy to automate.

Consequences: generic phrases pass the gate even though the result is not executable.

How to detect: validation does not check specificity, completeness, or executability.

Replace with: artifact-level completeness and post-render review.

Antipattern Card: Automatic Transition on Partial Success

Abstract example — partial success starts the next stage.

Incorrect: a step must receive data, validate it, and determine a mode. The data is received, but an attribute fails validation; the process moves to generation because `input_received = true`.

Correct:

```
input_status: received
validation_status: failed
decision_status: not_started
generation_status: blocked
overall_status: blocked_by_input_validation
```

Transition is allowed only when:

```
input_status == received
AND validation_status == passed
AND required_gaps == 0
```

Replace with: NO_PARTIAL_SUCCESS_TRANSITION_GATE.

A transition depends on full completion criteria, not on the first positive signal.

Antipattern Card: Package-Level Success Hides Artifact Failure

Signs: the overall package is passed although one artifact failed or was not validated.

Consequences: the user receives a set in which a weak component is hidden by the aggregate status.

Replace with: One Artifact at a Time and ARTIFACT_LEVEL_STATUS_GATE.

Antipattern Card: Generic Instruction as Completed Work

Abstract example — semantic placeholders.

Incorrect:

```
Find the appropriate record.
Apply the required change.
Make sure the result is correct.
Perform a rollback.
```

The words appropriate, required, and correct hide unresolved decisions.

Correct:

```
Find the record in the fictional sample_items dataset
using the filter { "sample_key": "ITEM-204" }.
```

```
Change the display_label field
from "Old sample label"
to "New sample label".
```

```
After the update, verify:
- display_label == "New sample label";
- change_status == "applied".
```

To roll back, restore:

```
display_label = "Old sample label".
```

After rollback, repeat the lookup
and verify the original value.

Replace with: explicit input/output contract and executable specification.

If an adjective or pronoun replaces a concrete object, value,
or criterion, it is a semantic placeholder.

Antipattern Card: Evidence Collection Mixed with Decision

Abstract example — fact collection is mixed with decision-making.

Incorrect:

Collect information about the request and determine
whether it can be approved automatically.

While reading, the model immediately classifies ambiguous values as
positive in order to complete the decision.

Correct:

Step A – Evidence collection

Output:

- confirmed facts;
- source references;
- contradictions;
- open gaps.

Step B – Eligibility evaluation

Input:

- only the recorded evidence artifact.

Output:

- eligible;
- not eligible;
- needs review.

Step C – Approval

Input:

- evaluation result.

Output:

- confirmed decision.

If an action determines what counts as a fact, it must not
simultaneously decide which outcome is desirable.

7. When a Step Must Be Split

Split a step if at least one of the following is true:

- it creates more than one independent result;
- different parts need different quality criteria;
- one part can succeed while another fails;
- one part requires confirmation and another does not;
- an error must be fixed without repeating the whole process;
- facts, assumptions, and decisions are mixed;
- one context creates and validates the same result;
- different activity types are combined: analysis, reconstruction, generation, validation;
- the step can hide uncertainty;
- the result of one internal part becomes the input of another without explicit recording.

A practical test: if the failure route says “repeat the step,” but in reality only one part of the step must be repeated, the step is not atomic.

8. When Combining Actions Is Acceptable

The atomicity principle does not require every tiny mechanical action to become a separate node.

Combining actions is acceptable if they:

- have one responsibility;
- create one result;
- share one completion criterion;
- contain no new business decision;
- do not change confirmed state;
- do not hide reconstruction;
- can be safely repeated as one operation;
- are a mechanical transformation of a confirmed source.

Abstract Example of Acceptable Combination

After a structured document is confirmed, the process must:

- sort keys;
- format indentation;
- save the file;
- calculate a checksum.

These may be performed as one technical step: meaning does not change, no decisions are added, and the output is one release artifact.

Unacceptable Combination

The same step must not:

- add missing requirements;
- rewrite ambiguous rules;
- remove sections as “unnecessary”;
- automatically confirm the new version.

Mechanical transformations may be grouped. Semantic decisions, reconstruction, and confirmation may not.

9. Validating Process Atomicity

The following are methodological gates. They help a process author inspect structure, but by themselves they do not change the normative language contract.

SINGLE_RESPONSIBILITY_GATE

Checks: whether the step has one type of responsibility.

Passes: purpose and action describe one operation.

Blocks: the step simultaneously collects, reconstructs, decides, and generates.

Diagnostic: STEP_ATOMICITY_001: step mixes evidence collection and decision making.

SINGLE_OUTPUT_GATE

Checks: whether there is one primary result.

Passes: output has one artifact/status contract.

Blocks: independent results may have different readiness.

Diagnostic: STEP_ATOMICITY_002: multiple independently validatable outputs detected.

EXPLICIT_INPUT_OUTPUT_GATE

Checks: whether inputs and output are defined.

Blocks: semantic placeholders or unrecorded intermediate data are used.

Diagnostic: STEP_ATOMICITY_003: output depends on implicit input.

NO_HIDDEN_RECONSTRUCTION_GATE

Checks: whether recovery of missing data is modeled as a separate branch.

Blocks: a reconstructed value is used as confirmed without evidence.

Diagnostic: STEP_ATOMICITY_004: reconstructed value has no proposal/confirmation state.

SEPARATE_GENERATION_VALIDATION_GATE

Checks: whether validation works with a completed result separately from generation.

Blocks: the same step generates, fixes, and sets passed.

Diagnostic: STEP_ATOMICITY_005: generation and validation share one uncontrolled action.

EXPLICIT_CONFIRMATION_GATE

Checks: whether confirmation of required decisions is recorded explicitly.

Blocks: confirmed state is reached without approval evidence.

Diagnostic: STEP_ATOMICITY_006: confirmed transition has no explicit approval.

NO_PARTIAL_SUCCESS_TRANSITION_GATE

Checks: whether the next step depends on complete completion criteria.

Blocks: a transition starts after one successful subcondition.

Diagnostic: STEP_ATOMICITY_007: transition triggered by partial success.

FAILURE_LOCALIZATION_GATE

Checks: whether the failure route leads to the nearest operation capable of fixing the error.

Blocks: a local error restarts the entire process.

Diagnostic: STEP_ATOMICITY_008: failure route is broader than affected responsibility.

ARTIFACT_LEVEL_STATUS_GATE

Checks: whether every independent artifact has its own status.

Blocks: package-level passed hides a failed or unvalidated artifact.

Diagnostic: STEP_ATOMICITY_009: aggregate status masks artifact failure.

10. Statuses and Errors

One general status is insufficient for a process in which different stages may have different states.

```
input_status:
decision_status:
generation_status:
validation_status:
approval_status:
execution_status:
overall_status:
```

Abstract Example of Different Statuses

Incorrect:

```
status: passed
```

while the document has been created, quality review has not been performed, execution is blocked, and confirmation is missing.

Correct:

```
input_status: confirmed
design_status: completed
generation_status: completed
validation_status: not_started
approval_status: pending
execution_status: blocked
overall_status: awaiting_validation
```

Another variant:

```
generation_status: completed
validation_status: failed
approval_status: blocked
execution_status: not_started
overall_status: failed_quality_gate
```

Status shows exactly where the process is. File created does not mean file is high quality, and file is high quality does not mean file is confirmed.

Partial success must remain visible. `overall_status` aggregates but does not replace detailed statuses.

11. Relationship to Language-Model Behavior

Language models have several properties that make atomicity especially important.

Tendency to Compress Repetitive Steps

When many scenarios share a similar structure, a model may combine them into a general phrase. Intermediate artifacts and separate gates preserve concrete detail.

Turning a Specification into a Description

A model may replace exact values with words such as “appropriate,” “relevant,” or “expected.” An atomic output contract makes it possible to check whether safe specificity is present.

Inferring Missing Information

A model often tries to finish the task. A separate reconstruction branch makes assumptions visible and prevents them from becoming facts without confirmation.

Plausible Does Not Mean Confirmed

A logically probable value may still be wrong for the specific process. Evidence collection is therefore separated from decision evaluation.

Self-Evaluation Against Intention

The model remembers that it intended to add a section and may consider the section present. Independent post-render review forces validation of the actual text or file.

Silent Transition to the Next Task

After generation, a model may begin the next stage without an explicit state transition. Completion criteria and a next-step gate stop such transitions.

One Large Prompt Loses Local Constraints

The more responsibilities a prompt contains, the more likely a local condition is to disappear among global instructions. Decomposition reduces the active context of each step.

12. Practical Process-Design Template

1. Name the final results.
2. Define a separate responsibility for every result.
3. Separate fact collection from reconstruction.
4. Separate decision-making from materialization.
5. Separate generation from validation.
6. Add completion criteria.
7. Add a local failure route.
8. Add a confirmation point only where a decision or state changes.
9. Forbid transition on partial success.
10. Assemble the aggregate result only from individually ready parts.

Abstract Example of a Local Failure Route

Incorrect: final validation finds a missing rollback, and the process returns to the beginning, collecting already confirmed data again.

Correct:

```
POST_RENDER_REVIEW
  ↓ failed: rollback section missing
ROLLBACK_SPEC_RESOLUTION
  ↓
DOCUMENT_REGENERATION_FOR_AFFECTED_SECTION
  ↓
POST_RENDER_REVIEW
```

The following are not repeated:

- fact collection;
- contract confirmation;
- scenario selection;
- independent artifacts that already passed.

A failure route returns to the nearest step capable of fixing the error, not to the beginning of the process.

13. Final Comparison Table

Situation	What Is Mixed	Primary Risk	Correct Separation	Gate
Preparing a document with a missing field	facts, reconstruction, design, generation, approval	assumption becomes fact	Collect → Resolve → Confirm → Generate → Validate → Approve	NO_HIDDEN_RECONSTRUCTION_GATE
A test plan is treated as a runbook	design and materialization	non-executable document	Design → execution specification → rendered guide → review	EXPLICIT_INPUT_OUTPUT_GATE
A missing record is reconstructed during processing	read and create	no evidence	separate reconstruction branch	NO_HIDDEN_RECONSTRUCTION_GATE
One step creates several documents	independent outputs	weak file hidden by package	One Artifact at a Time	SINGLE_OUTPUT_GATE
The model validates its own plan	intention and result	false green	Independent Post-Render Review	SEPARATE_GENERATION_VALIDATION_GATE
Data is received but invalid	partial success and transition	premature generation	full completion criteria	NO_PARTIAL_SUCCESS_TRANSITION_GATE
Generic phrases replace specifics	design and execution	semantic placeholders	executable specification	EXPLICIT_INPUT_OUTPUT_GATE
Fact collection immediately determines eligibility	evidence and decision	biased evidence	Evidence → Evaluation → Approval	SINGLE_RESPONSIBILITY_GATE

Situation	What Is Mixed	Primary Risk	Correct Separation	Gate
Formatting, saving, checksum	mechanical actions for one output	low risk	acceptable combination	SINGLE_OUTPUT_GATE
A local error restarts everything	failure route and global restart	lost time and drift	nearest corrective step	FAILURE_LOCALIZATION_GATE
One passed for everything	detailed statuses and aggregate status	hidden incompleteness	separate stage statuses	ARTIFACT_LEVEL_STATUS_GATE
Generation automatically confirms	generation and state transition	confirmed without approval	Generate → Validate → Approve → Transition	EXPLICIT_CONFIRMATION_GATE

Final Rule

Do not combine in one step actions that may have different results, different quality criteria, different failure routes, or different confirmation needs.

Practical formula:

First create one observable result. Then validate it separately. Only after that move forward.

Chapter 6.2. Micro-Prompts as Modular Process Architecture

A micro-prompt is often understood as a shorter version of a large prompt. In this book, it is treated differently: as a **local unit of behavior** that can be evolved, tested, versioned, replaced, and rolled back independently without rebuilding the entire process.

The core idea is simple:

A process should evolve as a composition of independent behavioral modules, not as one large instruction.

This is not a normative language requirement and not a mandatory syntactic element. It is a practical architectural recommendation for processes in which a language model performs many different actions.

1. From a Large Instruction to a Composition of Behavior

Monolithic Approach

In a monolithic approach, the entire process is described by one large text:

One large instruction

↓

Any local change

↓

Almost the entire process must be reread and retested

↓

The risk of accidentally changing other behavior increases

Such text may contain rules for fact collection, reconstruction, generation, validation, confirmation, failure routes, and final response formatting. Even when only one rule changes, it remains inside a shared context with every other rule.

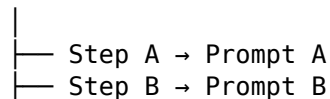
A language model does not execute the text as an isolated set of functions. It interprets the text as one context. Because of this, a local edit may:

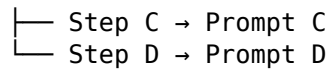
- change the priority of neighboring instructions;
- weaken a constraint in another step;
- add a new assumption where none existed;
- change the style or completeness of outputs from other nodes;
- create a contradiction between the beginning and end of the prompt;
- increase the risk that some rules disappear inside a long context.

Modular Approach

In a modular approach, the process tree remains a separate structure and every node references its own behavioral instruction:

Decision Tree





If Step C behavior must be improved, Prompt C changes. Prompt A, Prompt B, and Prompt D are not rewritten without a separate reason.

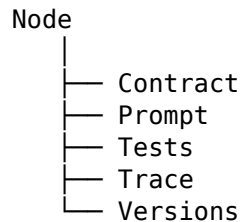
This does not mean other parts of the process are automatically guaranteed to remain unchanged. But the area of potential impact becomes smaller, more visible, and easier to validate.

2. What a Micro-Prompt Is in Process Architecture

A micro-prompt is not merely a short text. It is a local behavioral module that:

- serves one defined responsibility;
- accepts concrete inputs;
- creates a concrete output;
- operates within the node contract;
- has its own quality criteria;
- may have its own tests;
- may have an independent version;
- leaves an observable execution trace.

It is useful to separate the node itself from the micro-prompt that implements its behavior:



The node defines **what must be done**. The micro-prompt describes **how the model should perform that local behavior** within the allowed contract.

Therefore, changing micro-prompt text must not silently change:

- the purpose of the node;
- its input type;
- its output form;
- completion criteria;
- allowed transitions;
- confirmed state;
- failure route.

If at least one of these changes, the work is no longer only a local micro-prompt refactor. It is a contract or process-architecture change.

3. Analogy with Software Engineering

Micro-prompts are closer to modules in a software system than to separate paragraphs of documentation.

Function

A function has parameters, performs one operation, and returns a result. A micro-prompt should likewise receive a defined context and create one expected result.

Module or Library

A module hides internal implementation behind a stable interface. In the same way, micro-prompt text may change while the node contract remains stable.

Class

A class combines behavior around a defined responsibility. A micro-prompt should also be attached to a concrete role rather than trying to govern the whole process at once.

Dependency Injection

A process may reference a concrete version of a prompt module instead of embedding the text directly. This makes it possible to substitute another implementation without changing the process tree.

Interface and ABI/API Contract

The most important analogy is a stable interface. Safe local evolution is possible only when a new micro-prompt version does not break the interaction contract with neighboring nodes.

For example:

```
node_contract:
  id: reconstruction_step
  input:
    confirmed_facts: list
    open_gaps: list
```

```

output:
  proposed_values: list
  unresolved_gaps: list
  evidence_links: list
completion_criteria:
  - every proposal has evidence
  - every gap is resolved or explicitly open

```

The prompt implementation may change. But if a new version stops returning `evidence_links`, that is an interface violation.

Local Refactoring

Refactoring improves internal implementation without changing external behavior. Similarly, you may:

- make an instruction more precise;
- remove ambiguity;
- add a better analysis order;
- clarify the local response format;
- add protection against a common error;

while preserving the node contract unless a separate decision changes it.

4. Main Advantages

4.1. Locality of Changes

Improving one step happens in one place. The author does not need to find every mention of the behavior inside a large prompt.

Locality criterion:

Changing the behavior of one node does not require editing the texts of other nodes if their contracts have not changed.

4.2. Smaller Regression Scope

A monolithic instruction creates a broad regression scope: even a small edit may affect the entire context.

Monolith

↓

one local change

↓

almost complete process retesting

Micro-prompts make it possible to narrow the validation area:

Micro-prompt of node C

↓
local change
↓
node C tests
↓
contract tests for C → downstream nodes
↓
limited end-to-end regression

A numeric percentage is not universal. The principle matters: the more stable the contracts between nodes, the smaller the part of the process that must be retested after a local change.

4.3. Independent Versioning

Every micro-prompt may have its own history:

```
node:
  id: document_review
  contract_version: 1.0.0
  prompt:
    id: review_prompt
    version: 1.4.2
  approved_regression_suite: review-suite-7
  rollback_version: 1.4.1
```

It is important to distinguish:

1. the node contract version;
2. the prompt text version;
3. the test version;
4. the version of the process using the prompt.

A prompt change does not always require a contract change. A contract change always requires broader dependency validation.

4.4. Targeted Experiments

A new version can be tested on one node only:

- A/B comparison of two versions;
- replay of the same input;
- output comparison;
- evaluation of one concrete metric;
- validation on a local edge-case set.

4.5. Simpler Review

A reviewer can see:

- which node changed;
- why it changed;
- which contract was preserved;
- which tests were added;
- which regression scope was checked.

This is much more reliable than reviewing a diff in a large instruction where a local change disappears among dozens of other rules.

4.6. Simple Rollback

If a new version makes the result worse, the previous version of one prompt module can be restored without rolling back the entire process.

Rollback is safe only when the previous version remains compatible with the current node contract.

4.7. Better Execution Trace

Trace should record not only the node name but the actual behavior version:

```
trace_event:
  node_id: reconstruction_step
  contract_version: 1.0.0
  prompt_id: reconstruction_prompt
  prompt_version: 1.4.2
  input_artifact: evidence-104
  output_artifact: proposal-105
  validation_status: passed
```

This makes it possible to:

- repeat a run with the same version;
- compare results from different versions;
- localize a regression;
- determine which behavioral implementation affected the result;
- reproduce a historical decision.

4.8. Local Knowledge Capture

In a monolith, new knowledge is often added to the general text. Over time, it becomes difficult to understand which step a particular rule belongs to.

In a modular architecture, experience is stored next to the node:

```
Node C
├─ prompt.md
├─ contract.yaml
├─ examples/
├─ tests/
├─ regressions/
├─ known_failures.md
└─ changelog.md
```

An error in a concrete node produces:

- a local prompt change;
- a local regression test;
- a local note about the known problem;
- a local explanation of the decision.

Knowledge does not spread across one huge document.

5. Abstract Example: Reconstruction as a Local Module

Imagine a four-step process:

```
Fact collection
→ Reconstruction
→ Document generation
→ Review
```

Variant 1. Monolithic Instruction

All behavior is described in one prompt. Practical use reveals that reconstruction rules too easily treat a plausible value as a confirmed fact.

To fix the problem, the author edits the large text. Afterward, the author must check:

- whether fact collection changed;
- whether reconstruction output format changed;
- whether document generation began to skip open gaps;
- whether review logic changed;
- whether transition order remained intact.

A local problem created an almost complete regression scope.

Variant 2. Separate Reconstruction Micro-Prompt

The process keeps a stable structure:

FACT_COLLECTION → RECONSTRUCTION → DOCUMENT_GENERATION → REVIEW

The RECONSTRUCTION node has a separate prompt. Only these rules change:

- every proposed value must reference evidence;
- plausibility does not equal confirmed;
- an uncertain value remains an open gap;
- confirmed facts must not be changed;
- output preserves provenance for every reconstruction.

After the change, run:

1. unit-like tests for the reconstruction micro-prompt;
2. a contract test of its output;
3. validation of the RECONSTRUCTION → DOCUMENT_GENERATION transition;
4. a short end-to-end regression for the whole process.

Other prompt modules are not rewritten.

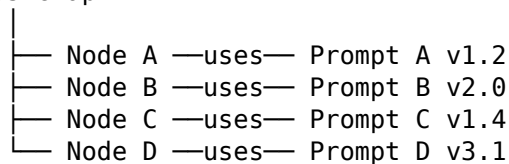
Success criterion:

The new version reduces reconstruction errors, preserves the node contract, and does not change neighboring-node behavior outside the defined regression scope.

6. Prompt Dependency Graph

Separate prompts do not exist in a vacuum. They form a dependency graph.

Process Graph



But node → prompt links are not the only important relationships. Contract dependencies must also be visible:

Prompt A output contract



Prompt B input contract



Prompt C precondition

A micro-prompt registry should therefore answer at least:

- which node uses the prompt;
- which version is active;
- which contract it implements;
- which nodes depend on its output;
- which tests protect a change;
- which version is allowed for rollback;
- in which processes it is reused.

An abstract registry:

prompt_registry:

- prompt_id: reconstruction_prompt
- version: 1.4.2
- implements_contract: reconstruction_contract@1.0.0
- used_by:
 - process_a.reconstruction
 - process_b.missing_data_resolution
- tests:
 - reconstruction_unit_suite
 - reconstruction_contract_suite
- rollback_to: 1.4.1

7. Contract Compatibility and the Boundary of a Local Change

Micro-prompts reduce risk but do not remove it.

A local update is sufficient if:

- the node purpose does not change;
- input schema does not change;
- output schema does not change;
- status semantics do not change;
- completion criteria do not change;
- failure route does not change;
- the prompt receives no new right to change confirmed state.

A local update is not sufficient if:

- the node begins making a new business decision;
- the output type changes;
- the meaning of a status changes;
- a new mandatory dependency appears;
- the prompt begins doing the work of a neighboring node;
- process order changes;
- the contract between nodes changes.

In that case, the work requires:

- a new contract version;
- Prompt Dependency Graph analysis;
- broader regression scope;
- updated trace semantics;
- possible migration of processes using the old contract.

8. Micro-Prompt Architecture Patterns

Pattern: One Node — One Behavioral Module

When to use: when a node has one clear responsibility.

Structure:

Node contract
→ one micro-prompt
→ one observable output
→ local validation

Advantage: behavior changes are easy to localize.

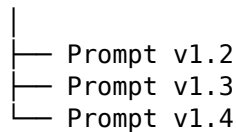
Risk: the prompt may silently grow and begin performing several tasks.

Control: review it together with step-atomicity criteria.

Pattern: Prompt Behind a Stable Contract

When to use: when behavior improves frequently but the interface must remain stable.

Stable node contract



Criterion: every version passes the same contract suite.

Pattern: Prompt Registry

When to use: when prompts are reused or have several versions.

The registry separates a behavior reference from process text. The process specifies `prompt_id` and version policy, while the prompt itself is stored separately.

Pattern: Contract Test Before End-to-End Test

Validate local contract compliance first and the entire process afterward.

```
prompt test
→ output schema test
→ semantic contract test
→ neighboring-node integration
→ selected end-to-end regression
```

Pattern: Traceable Prompt Resolution

Before execution, the runtime or model must record which prompt version was actually selected.

This protects against a process referencing latest when the historical run cannot later be reproduced.

Pattern: Local Failure Knowledge

Every discovered error is linked to a concrete prompt module, test, and fix version.

```
issue:
  id: ISSUE-17
  affected_prompt: reconstruction_prompt@1.4.1
  failure_class: unsupported_inference
  regression_test: REC-CASE-09
  fixed_in: 1.4.2
```

Pattern: Approved Promotion

A new version does not become active immediately after editing.

```
draft
→ local tests passed
→ contract tests passed
→ regression passed
→ review approved
→ promoted
```

9. Risks and Antipatterns

Antipattern: Micro-Prompt in Name Only

The prompt is called a micro-prompt but contains data collection, decision-making, generation, review, and a state transition.

Sign: changing one part requires retesting everything inside the prompt.

Fix: first separate responsibilities at the process level.

Antipattern: Hidden Contract in Prompt Text

The contract exists only as several sentences inside prompt text.

Risk: a new version accidentally changes output or statuses.

Fix: move the contract into an explicit structure and test it separately.

Antipattern: Floating Latest

The process references the “latest” version without recording actual resolution.

Risk: two identical runs use different behavior and replay becomes unreliable.

Fix: record the resolved version in the execution trace.

Antipattern: Shared Prompt with Divergent Responsibilities

One prompt is reused by nodes with similar names but different contracts.

Risk: an improvement for one process makes another worse.

Fix: use a shared core with separate adapters or separate prompts.

Antipattern: Prompt Change Without Regression Artifact

The text changes, but there is no example, test, or trace showing which problem was fixed.

Fix: every meaningful improvement should add or update a local regression case.

Antipattern: Prompt Owns the State Transition

The micro-prompt not only creates output but also decides that the process is now confirmed.

Risk: local behavior gains the right to change global state.

Fix: keep state transition as a separate operation with preconditions.

Antipattern: Over-Fragmentation

Every sentence is moved into a separate prompt even though the actions form one mechanical operation.

Risk: too many dependencies, difficult version management, and unnecessary orchestration overhead.

Fix: the prompt-module boundary should match a real responsibility boundary, not text length.

10. Versioning and Lifecycle

Recommended minimum version record:

```
micro_prompt:
  id: document_review_prompt
  version: 2.3.0
  implements_contract: document_review@1.1.0
  status: approved
  created_from: 2.2.1
  change_reason: detect missing concrete examples
  regression_suite: review-regression-12
  rollback_to: 2.2.1
```

A practical version-change rule:

- **patch** — wording clarification without a contract change;
- **minor** — new compatible behavior or additional local protection;
- **major** — incompatible change requiring a new contract or dependency migration.

This is not a rigid standard but a useful discipline. The important thing is that history can answer:

- what changed;
- why;
- which problem it fixes;
- which tests prove the improvement;
- how to return to the previous version;
- which processes use this version.

11. Testing Micro-Prompts

Local Examples

A set of short inputs that checks the primary behavior of the node.

Edge Cases

Cases with gaps, contradictions, ambiguity, missing evidence, or an invalid format.

Contract Tests

They check interface compliance rather than the beauty of the text:

- all mandatory fields are present;
- statuses use allowed values;
- output contains no forbidden state transitions;
- every decision has evidence;
- open gaps are not hidden.

Metamorphic Tests

They check invariants. For example, changing the order of independent facts should not change the decision.

Replay Tests

The same input is executed on old and new versions. Results are compared using defined criteria.

Neighbor Integration Tests

They check whether the next node can correctly consume the new version's output.

Selected End-to-End Regression

Even with a stable contract, a limited end-to-end test is needed. Language models may create side effects that a formal schema does not detect.

12. Relationship to Other Principles in This Book

Step Atomicity

A micro-prompt makes sense only when the node itself is atomic. If the node mixes several responsibilities, a separate prompt merely hides a monolith under a new name.

Single Responsibility

One prompt module should own one behavior type and one primary result.

Explicit Contracts

A contract separates stable external behavior from changing internal implementation.

Local Failure Routes

An error in a concrete prompt should return to the nearest node capable of fixing it, not restart the whole process.

Execution Trace

Trace records node, prompt ID, prompt version, contract version, input, output, and validation evidence.

Independent Validation

A prompt must not declare its own result correct. Validation remains a separate operation or separate node.

Artifact-Oriented Approach

Micro-prompt input and output are better represented as observable intermediate artifacts. This makes behavior testable and reproducible.

13. Practical Checklist

Before extracting a separate micro-prompt, check:

1. Does the node have one responsibility?
2. Is there one primary observable output?
3. Are input and output contracts defined?
4. Can the internal instruction change without changing the process?
5. Does the prompt have its own tests?
6. Is its version recorded in trace?
7. Is there a local rollback?
8. Is the regression scope clear?
9. Does the prompt avoid changing confirmed state by itself?
10. Does it avoid hidden work belonging to a neighboring node?

11. Is one prompt avoided across incompatible contracts?
12. Is the reason for a new prompt stronger than a desire to shorten text?

Before a local update, check:

- the node contract has not changed;
- dependent inputs/outputs remain compatible;
- a regression case was added;
- local tests passed;
- contract tests passed;
- selected end-to-end regression passed;
- the resolved version will be recorded in the execution trace;
- a rollback version is defined.

14. Final Table

Situation	Risk	Correct Action	Validation Scope
Wording of one local rule was clarified	Side effect on node behavior	New micro-prompt patch version	Local and contract tests
Compatible edge-case protection was added	Regression on old inputs	New minor version and regression case	Node, neighbor integration, selected end-to-end
Output schema changed	Downstream nodes break	New contract version	Dependency graph and broad integration validation
One prompt is used by different contracts	Unpredictable side effects	Split implementations or add adapters	All consuming processes
A new version worsened the result	Defect propagation	Local rollback	Rollback-version compatibility validation
Trace lacks prompt version	Reliable replay is impossible	Record resolved prompt version	Trace validation

Situation	Risk	Correct Action	Validation Scope
Prompt began performing several responsibilities	Hidden monolith	Decompose the node	Atomicity review

15. Final Rule

The smaller and clearer the responsibility scope of a micro-prompt, the easier it is to improve, test, roll back, and reuse.

And one more architectural generalization:

A stable process should depend on behavioral-module contracts, not on the accidental shape of one large instruction.

Micro-prompts do not eliminate the need for system testing. They make changes more local, dependencies more visible, regression scope smaller, and process evolution governable.

Chapter 7. Output: What the Model Must Create

7.1. Why Ordo Describes Output Separately

A process can be executed correctly and still end with the wrong artifact.

For example, the model may correctly identify the path, collect the required contracts, and pass the checks, but then return a short chat summary instead of the document package expected by the next process.

That is why Ordo describes output explicitly.

OUTPUT.DEF answers:

What exactly must be created?

In what form?

When is creation allowed?

How can the result be checked?

Output is not simply “the model’s answer.” It is a process artifact with a contract.



Figure 24: Nebu — idea: Output is an explicit process artifact

7.2. Output Is Not Only a Text Reply

A model may create many kinds of output:

- a chat response;
- a document;
- a set of documents;
- a JSON report;
- a YAML specification;
- an archive;
- a blocked handoff status.

For example:

```
{  
  "op": "OUTPUT.DEF",  
  "id": "OUT_HISTORY_EVENT_PACKAGE",  
  "kind": "artifact_set",  
  "required": [  
    "01_HISTORY_EVENT_PASSPORT_<ALIAS>.md",  
    "02_JIRA_TASK_<ALIAS>.md",  
    "05_QA_PACKAGE_<ALIAS>.md"  
  ]  
}
```

If the process only says “prepare the result,” the model may decide that one short walkthrough is sufficient.

The output contract says that a defined set of files is required.

7.3. Output Must Be Connected to a Terminal Path

In a decision-tree process, different terminal paths may have different outputs.

For example, in a support process:

- incident ready → INCIDENT_REPORT.md + CUSTOMER_REPLY.md
- incident needs triage → INCIDENT_TRIAGE_NOTE.md

change ready → CHANGE_BRIEF.md + ACCEPTANCE_CHECKLIST.md
change unclear → CHANGE_CLARIFICATION_NOTE.md

This means the output is not always the same. It depends on the selected path.

In Ordo, this can be described as:

```
terminal_outputs:
  T_INCIDENT_READY:
    outputs:
      - INCIDENT_REPORT.md
      - CUSTOMER_REPLY.md

  T_INCIDENT_NEEDS_TRIAGE:
    outputs:
      - INCIDENT_TRIAGE_NOTE.md

  T_CHANGE_READY:
    outputs:
      - CHANGE_BRIEF.md
      - ACCEPTANCE_CHECKLIST.md

  T_CHANGE_NEEDS_CLARIFICATION:
    outputs:
      - CHANGE_CLARIFICATION_NOTE.md
```

This protects the process from creating the wrong document for the correct path.

7.4. Output May Be Allowed or Blocked

Not every output may be created at any time.

For example, a final archive must not be created while contracts remain unconfirmed.

Output therefore needs readiness conditions.

Example:

```
{
  "op": "OUTPUT.DEF",
  "id": "OUT_FINAL_ARCHIVE",
  "kind": "zip_archive",
  "allowed_when": [
    "path_confirmed",
    "mandatory_contracts_confirmed",
```

```

    "required_documents_approved",
    "validation_status_go"
  ],
  "blocked_when": [
    "unresolved_mandatory_contract",
    "proposed_contract_used_as_confirmed",
    "approval_missing",
    "validation_status_no_go"
  ]
}

```



Figure 25: Nebu — attention: Output is allowed only after gates

This is important: output defines not only “what to create,” but also “when creation is allowed.”

7.5. Draft Output and Final Output

In real work, the model often needs to create a draft first.

For example:

```

first a draft passport in chat;
then user approval;
then a draft Jira task;
then approval;
then a QA package;
then an automation spec;
and only after approvals – the final archive.

```

This means there are different output levels:

```

draft output;
review output;
approved output;
final output.

```

In Ordo, these levels should be explicit.

For example:

```

outputs:
  passport_draft:
    file: "01_HISTORY_EVENT_PASSPORT_<ALIAS>.md"
    status: draft
    requires_approval: true

  passport_final:
    file: "01_HISTORY_EVENT_PASSPORT_<ALIAS>.md"
    status: approved
    allowed_when:
      - passport_approval_passed

```

Without this distinction, the model may confuse a draft with a final document.

7.6. Output and Template

In many Ordo processes, output is created from a template rather than from scratch.

For example:

A Jira task must contain Summary, Context, Acceptance criteria, Out of scope, and QA

Then OUTPUT.DEF alone is not enough. The output must be connected to a template.

This is a profile-level construct:

TEMPLATE.BIND

Example:

```

{
  "op": "TEMPLATE.BIND",
  "output": "02_JIRA_TASK_<ALIAS>.md",
  "template": "templates/TEMPLATE_JIRA_TASK.md",
  "required_sections": [
    "Summary",
    "Context",
    "Acceptance criteria",
    "Out of scope",
    "QA reference data",
    "Test coverage"
  ]
}

```

OUTPUT.DEF says that the file must exist.

TEMPLATE.BIND says which structure must be used to create it.

7.7. Output Must Be Verifiable

If output cannot be checked, the model may report completion even when the artifact is incomplete.

A weak output contract:

Create a good QA package.

What does “good” mean? For the model, it may mean a short summary. For a tester, it may mean a complete step-by-step runbook.

A better output contract:

```
output:
  file: "05_QA_PACKAGE_<ALIAS>.md"
  required_for_each_executable_tc:
    - goal
    - preconditions
    - source_lookup_before_action
    - preflight_restore
    - rest_action
    - source_lookup_after_action
    - changerecord_lookup_or_expected_absence
    - history_processing_step
    - history_event_lookup_or_expected_absence
    - change_errors_lookup
    - rollback
    - post_rollback_source_lookup
    - expected_result
    - diagnostics
```

Now the output can be checked.

The model cannot replace the required structure with “see the general flow.”

7.8. Output May Be Absent — and That Is Also a Result

Sometimes the correct result is not to create a document.

If mandatory contracts are not confirmed, the correct behavior is not to invent a package but to stop.

The output may then be:

```
{
  "op": "OUTPUT.DEF",
  "id": "OUT_BLOCKED_STATUS",
  "kind": "handoff_status",
  "status": "blocked_requires_confirmation",
  "include": [
    "missing_contracts",
    "open_questions",
    "next_required_action"
  ]
}
```



Figure 26: Nebu — thinking: a blocked handoff is also a result

This is an important idea.

In Ordo, a result may be not only a completed artifact but also an honest blocked handoff.

It is better to say:

The package cannot be finalized because `HistoryEvent.item.values` has not been confirmed than to create a polished but incorrect document.

7.9. Output and Handoff

Output is what is created.

Handoff is how the process passes the result onward.

For example, the output may be a set of files, while the handoff is a short status message:

```
Status: ready_for_review
Created: passport draft, Jira draft, QA draft
Blockers: automation runner contract not confirmed
Next action: approve the QA package
```

In Ordo, these concepts should remain separate.

OUTPUT.DEF describes the artifact.

HANDOFF.EMIT describes the transfer of status to the user or the next process.

Example:

```
{
  "op": "HANDOFF.EMIT",
  "include": [
    "status",
    "created_outputs",
    "gate_report",
    "open_questions",
    "next_action"
  ]
}
```

7.10. Typical Output Mistakes

The first mistake is describing output too broadly.

Create the package.

The second is failing to say how many files must exist.

The third is failing to distinguish draft from final.

The fourth is failing to bind output to a template.

The fifth is failing to define readiness conditions.

The sixth is assuming that because the model wrote something, the output is already valid.

The seventh is failing to provide a blocked output.

In complex processes, a correct “I cannot finalize because...” is often more valuable than an incorrect “Done.”

7.11. Short Chapter Summary

OUTPUT.DEF defines exactly what the model must create.

Output may be:

- a chat response;
- a document;
- a set of documents;

a JSON report;
a YAML specification;
an archive;
a blocked handoff status.

A good output contract answers:

what is being created;
in which format;
from which template;
when creation is allowed;
what blocks creation;
how the result is checked;
whether the result is draft, review, or final.

The main principle is:

A result must be not merely polished, but allowed, structured, and verifiable.

Mini-Exercise

Take the task:

The model must prepare a document for handoff to a developer.

Describe OUTPUT.DEF:

```
output:
  id: OUT_DEVELOPER_HANDOFF
  kind: document
  file_name: "IMPLEMENTATION_PROMPT.md"
  required_sections:
    - goal
    - context
    - files_to_check
    - required_changes
    - what_not_to_change
    - tests
    - acceptance_criteria
  status: draft
  allowed_when:
    - business_contract_confirmed
  blocked_when:
    - missing_acceptance_criteria
    - unresolved_scope
```

Then answer:

1. Which sections are mandatory?
 2. What blocks creation of the final version?
 3. Which template is required?
 4. How do we verify that the rendered document is not empty?
-

Chapter 8. Writing the First Ordo Source Program

8.1. Why Start with Ordo Source

So far, we have discussed Ordo as a way of thinking: there is a goal, a contract, state, nodes, answers, results, and checks. Sooner or later, this must be written in a form that a model or a future runtime can work with.

That is what Ordo Source is for.



Figure 27: Nebu — idea: Ordo Source as the first program

Ordo Source is a human-readable description of a model behavior program. It should be understandable to the person designing the process. It is not necessarily the best execution format for a model, but it is convenient for writing, discussion, review, and change.

An analogy with conventional programming is useful:

- a person writes source code
- a compiler transforms it into an executable form
- a machine executes the compiled form

The Ordo idea is similar:

- a person writes Ordo Source
- the Ordo compiler / translator transforms it into Ordo IR
- the model or runtime executes Ordo IR

At an early stage, we may not yet have a complete compiler. Even then, writing Ordo Source manually already structures an instruction much better than an ordinary prompt.

8.2. What the First Program Should Be Like

The first Ordo program should be very simple.

Do not begin with a large playbook containing dozens of gates and templates. Start with a task that is easy to understand:

Summarize text in Ukrainian in no more than 3 bullet points.
Do not add facts that are absent from the input text.
If the text is insufficient, say so.

This is a good first example because it already contains the basic Ordo elements:

```
goal;  
input data;  
result contract;  
rules;  
checks;  
fallback behavior;  
result.
```

An ordinary prompt might look like this:

Summarize the provided text in Ukrainian in no more than 3 bullet points. Do not add

That is perfectly acceptable for simple use. But we want to see how the same meaning looks as Ordo Source.

8.3. Minimal Ordo Source Structure

A minimal Ordo Source program may contain these parts:

```
ordo  
program  
intent  
contract  
context  
state  
path  
result  
handoff
```

Not every program always needs every part in a fully expanded form, but a teaching example benefits from showing the complete structure.

General skeleton:

```
ordo: "0.12-draft"  
program: "minimal.summary.uk"
```

```
intent:  
  goal: "summarize_text"
```

```
contract:  
  input: {}  
  output: {}  
  rules: []
```

```
context: {}
```

```
state: {}
```

```
path:  
  steps: []  
  gates: []
```

```
result: {}
```

```
handoff: {}
```

This is not yet a program. It is only an empty skeleton. Now we will fill it in.

8.4. Describing Intent

intent answers the question: "What do we want to do?"

For our example:

```
intent:  
  goal: "summarize_text"  
  language: "uk"
```

We do not write a long explanation here. We record the primary action: summarize the text.

Importantly, intent should not contain every rule. Do not put "do not invent facts," "maximum 3 items," and "if the text is insufficient" into the intent. Those belong to the contract and gates.

A typical mistake:



Figure 28: Nebu — attention: do not put all rules into intent

```
intent:
  goal: "summarize_text_in_ukrainian_max_3_items_without_facts_and_with_fallback"
```

Avoid this. It turns intent into an uncontrolled phrase. Separate the concerns:

```
intent:
  goal: "summarize_text"
```

and place the rest in contract.

8.5. Describing the Contract

contract defines what counts as a correct result.

For our example:

```
contract:
  input:
    text: required

  output:
    format: "bullet_list"
    max_items: 3
    language: "uk"

  rules:
    - id: R1
      rule: "use_only_input_text"
    - id: R2
      rule: "do_not_invent_facts"
    - id: R3
      rule: "if_input_insufficient_return_insufficient_data_status"
```

Here we can already see how Ordo differs from a prompt. We do not merely ask, “please do not invent.” We create an explicit contract rule.

The contract should be as concrete as possible. If the result must be a list, say so. If it may contain at most three items, say so. If the language must be Ukrainian, say so. If the model may not add external facts, say that too.

The contract is not a recommendation. It is a validity condition for the result.

8.6. Describing Context

context defines where the model gets its data.

In our example, there is one source: the user's text.

```
context:
  source:
    text: "$USER_INPUT.text"
```

This means: do not take text from memory, do not search for additional facts, and do not add external knowledge. The primary source is the user's input text.

In more complex programs, context may contain:

```
documents;
tables;
a source row;
previous state;
examples;
project rules;
confirmed contracts;
environment constraints.
```

For the first program, one source is enough.

8.7. Describing State

Even a simple program may have state.

For the summary task, this is sufficient:

```
state:
  status: "draft"
  assumptions: []
  unsupported_facts: []
```

Why is this useful?

status shows that the result is not final until the gates pass.

assumptions records assumptions if the model is forced to make any.

unsupported_facts records facts that the model cannot support from the input text.

This may look excessive for a simple summary, but the principle matters: an Ordo program should know what it has collected, what it has checked, and what it cannot confirm.

8.8. Describing the Path and Steps

path describes how the model reaches the result.

Our path is simple:

```
path:
  id: "summarize_or_report_insufficient_input"
  steps:
    - id: S1
      do: "read_input_text"
    - id: S2
      do: "extract_supported_points"
    - id: S3
      do: "select_up_to_3_key_points"
    - id: S4
      do: "render_ukrainian_bullet_summary"
```

The important point is that we do not ask the model to immediately “write a summary.” We define the order:

```
first read;
then extract supported points;
then select at most 3;
then render them in Ukrainian.
```

This reduces the risk that the model immediately starts generating polished but unverified prose.

8.9. Adding Gates

Now add control points:

```

path:
  gates:
    - id: G1
      check: "input_text_present"
      on_fail: "return_insufficient_data"

    - id: G2
      check: "no_unsupported_facts"
      on_fail: "remove_or_mark_unsupported"

    - id: G3
      check: "max_3_items"
      on_fail: "compress_to_3_items"

    - id: G4
      check: "output_language_uk"
      on_fail: "rewrite_in_ukrainian"

```



Figure 29: Nebu — thinking: a gate as a check point

A gate is a place where the model must stop and check a condition.

G1 checks whether input text exists.

G2 checks for invented facts.

G3 checks that there are no more than three items.

G4 checks the output language.

Importantly, `on_fail` does not always mean “error and terminate.” Sometimes it is a repair action. If there are five items, the model can compress them to three. If the output is not in Ukrainian, the model can rewrite it. But if there is no input text, the process must return the fallback.

8.10. Describing Result and Handoff

`result` defines what is returned:

```
result:
  primary: "summary"
  fallback: "insufficient_data_message"
```

handoff defines what the model shows at the end:

```
handoff:
  include:
    - "status"
    - "result"
    - "gate_report"
```

This means the user receives not only the summary but also the status and, where needed, a short gate report.

A simple UI may hide the gate report. For learning and complex tasks, however, it is very useful.

8.11. The Complete First Ordo Source Program

Now put everything together:

```
ordo: "0.12-draft"
program: "minimal.summary.uk"

intent:
  goal: "summarize_text"
  language: "uk"

contract:
  input:
    text: required
  output:
    format: "bullet_list"
    max_items: 3
    language: "uk"
  rules:
    - id: R1
      rule: "use_only_input_text"
    - id: R2
      rule: "do_not_invent_facts"
    - id: R3
      rule: "if_input_insufficient_return_insufficient_data_status"

context:
  source:
```

```

    text: "$USER_INPUT.text"

state:
  status: "draft"
  assumptions: []
  unsupported_facts: []

path:
  id: "summarize_or_report_insufficient_input"
  steps:
    - id: S1
      do: "read_input_text"
    - id: S2
      do: "extract_supported_points"
    - id: S3
      do: "select_up_to_3_key_points"
    - id: S4
      do: "render_ukrainian_bullet_summary"

  gates:
    - id: G1
      check: "input_text_present"
      on_fail: "return_insufficient_data"
    - id: G2
      check: "no_unsupported_facts"
      on_fail: "remove_or_mark_unsupported"
    - id: G3
      check: "max_3_items"
      on_fail: "compress_to_3_items"
    - id: G4
      check: "output_language_uk"
      on_fail: "rewrite_in_ukrainian"

result:
  primary: "summary"
  fallback: "insufficient_data_message"

handoff:
  include:
    - "status"
    - "result"
    - "gate_report"

```

This is already a complete small Ordo Source program.

8.12. What Is Important to Notice

First, the program is not very long, but it already has structure.

Second, the rules are not hidden inside one sentence. They are separated into the contract and gates.

Third, the model has fallback behavior when the input text is insufficient.

Fourth, the result has constraints: format, item count, and language.

Fifth, there is a handoff that defines what is returned to the user.

All of this could also be written in an ordinary prompt, but Ordo makes the structure explicit.

8.13. How a Person Can Execute Such a Program Through a Model

If we do not yet have an Ordo runtime, we can provide this program to a model as an instruction and ask it to execute the program literally:

Use the provided Ordo Source program as the execution contract.
Do not execute the task as an ordinary prompt.
Check the gates first, then return the result and gate_report.

This is not ideal because the model is still interpreting Ordo Source itself. Even so, behavior becomes more disciplined.

In the future, a runtime can read Ordo Source or compiled IR and control execution order itself.

8.14. Typical Mistakes When Writing the First Ordo Program

Mistake 1. Put Everything into Intent

Bad:

```
intent:  
  goal: "summarize text in 3 bullets without unsupported facts and fallback if insuf
```

Better:

```
intent:  
  goal: "summarize_text"
```

```
contract:
  output:
    max_items: 3
  rules:
    - do_not_invent_facts
```

Mistake 2. Do Not Describe Gates

Bad:

```
rules:
  - do_not_invent_facts
```

but there is no gate that checks the rule.

Better:

```
gates:
  - id: G_NO_UNSUPPORTED_FACTS
    check: "no_unsupported_facts"
    on_fail: "remove_or_mark_unsupported"
```

Mistake 3. Do Not Describe a Fallback

If there is no input data, the model must not invent content. The fallback must therefore be explicit.

Mistake 4. Mix Source and Result

context says where the data comes from.

result says what is returned.

Do not mix them.

Mistake 5. Treat Ordo Source as the Final Machine Format

Ordo Source is a convenient human form. Compiled IR is preferable for execution. The next chapter covers this.

8.15. Short Chapter Summary

Ordo Source is the human-readable representation of an Ordo program.

It allows an author to describe a process not as one continuous prompt, but as a structured execution contract.

A first simple Ordo program should contain:

```
intent;  
contract;  
context;  
state;  
path;  
steps;  
gates;  
result;  
handoff.
```

The main principle is:

Do not write “model, please do this correctly.”
Explicitly describe what “correctly” means.

Mini-Exercise

Try rewriting an ordinary prompt as Ordo Source.

Prompt:

Write a short email to a client in Ukrainian. Explain that we received the request,

Describe:

1. intent;
2. contract.output;
3. contract.rules;
4. context;
5. gates;
6. result;
7. handoff.

Hint: create a separate gate that checks that the email does not contain a specific response deadline.

Chapter 9. Compiling Ordo Source into Semantic JSON IR

9.1. Why Compile Anything at All

Ordo Source is designed for people. It should be readable, discussable, and convenient to edit.

Execution has different needs.

A model or runtime benefits from explicit operations, stable identifiers, ordered steps, visible gates, state updates, and defined failure behavior. If all of this remains only in a human-oriented document, the executor must repeatedly interpret the author's intent.

Compilation reduces that interpretive burden.



Figure 30: Nebu — idea: compilation creates an execution form

The basic flow is:

```
Ordo Source
  ↓
compiler / translator
  ↓
Semantic JSON IR
  ↓
model or runtime execution
```

Compilation does not mean turning Ordo into a conventional programming language. It means projecting human-readable process semantics into a more explicit execution representation.

9.2. What Semantic JSON IR Is

Semantic JSON IR is Ordo's structured intermediate representation.

It is **semantic** because every operation carries execution meaning.

It is **JSON** because the representation is explicit, portable, and easy to inspect or process.

It is **IR** because it sits between authoring and execution.

A small operation may look like this:

```
{
  "op": "INTENT.SET",
  "id": "I_SUMMARIZE_TEXT",
  "goal": "summarize_text",
  "language": "uk"
}
```

The important part is not the braces. The important part is that the executor no longer has to infer whether the text describes a goal, a rule, a gate, or an output. The op makes the operation class explicit.

9.3. Why JSON

JSON is not chosen because it is the most pleasant format for authors. It is not.

It is useful because:

- objects and arrays are explicit;
- field names are visible;
- nesting is deterministic;
- the format is widely supported;
- schemas can validate it;
- runtime tools can consume it;
- models are already familiar with it.

A person should normally write Ordo Source. The compiler should produce JSON IR.

The authoring format and the execution format do not need to be identical.

9.4. Ordo Source and IR in One Example

Consider this source fragment:

```
intent:
  goal: summarize_text
  language: uk

contract:
  output:
    format: bullet_list
    max_items: 3
  rules:
    - do_not_invent_facts
```

```

context:
  source:
    text: "$USER_INPUT.text"

path:
  steps:
    - read_input_text
    - extract_supported_points
    - render_summary

  gates:
    - id: no_unsupported_facts
      check: no_unsupported_facts
      on_fail: remove_or_mark_unsupported

```

A semantic IR projection may be:

```

[
  {
    "op": "INTENT.SET",
    "id": "I_SUMMARIZE_TEXT",
    "goal": "summarize_text",
    "language": "uk"
  },
  {
    "op": "CONTRACT.DEF",
    "id": "C_SUMMARY_OUTPUT",
    "output": {
      "format": "bullet_list",
      "max_items": 3
    }
  },
  {
    "op": "RULE.ADD",
    "id": "R_NO_INVENTED_FACTS",
    "rule": "do_not_invent_facts"
  },
  {
    "op": "CONTEXT.LOAD",
    "id": "CTX_USER_TEXT",
    "source": "$USER_INPUT.text",
    "bind": "STATE.input_text"
  },
  {
    "op": "STEP.RUN",
    "id": "S_READ_INPUT",
    "fn": "READ_INPUT_TEXT"
  }
]

```

```

},
{
  "op": "STEP.RUN",
  "id": "S_EXTRACT_POINTS",
  "fn": "EXTRACT_SUPPORTED_POINTS"
},
{
  "op": "STEP.RUN",
  "id": "S_RENDER_SUMMARY",
  "fn": "RENDER_SUMMARY"
},
{
  "op": "GATE.CHECK",
  "id": "G_NO_UNUNSUPPORTED_FACTS",
  "method": "self_verification",
  "trust_class": "model_judgment",
  "assert": "NO_UNUNSUPPORTED_FACTS",
  "source": "RESULT.summary",
  "on_fail": "REPAIR.REMOVE_OR_MARK_UNUNSUPPORTED"
},
{
  "op": "HANDOFF.EMIT",
  "id": "H_FINAL",
  "include": ["status", "result", "gate_report"]
}
]

```

The source is easier to author.

The IR is easier to execute step by step.

9.5. What the Compiler Actually Does

A compiler should not merely convert YAML syntax into JSON syntax.

That would be serialization, not semantic compilation.

The compiler must recognize the role of each construct and project it into execution operations.

For example:

intent	→ INTENT.SET
contract	→ CONTRACT.DEF
rule	→ RULE.ADD
context	→ CONTEXT.LOAD
state	→ STATE.INIT / STATE.UPDATE
step	→ STEP.RUN

gate	→ GATE.CHECK
approval	→ APPROVAL.REQUIRE
output	→ OUTPUT.DEF
handoff	→ HANDOFF.EMIT

The compiler may also:

- normalize identifiers;
- resolve references;
- expand shorthand;
- attach source-map information;
- validate operation names;
- detect missing required fields;
- preserve controlled FREEFORM blocks;
- emit diagnostics for ambiguous constructs.

A compiler must not invent domain rules merely because they look plausible.

9.6. Compilation Must Not Distort Meaning

The most important compiler rule is semantic preservation.

If the source says:

```
final archive is blocked until mandatory contracts are confirmed
the IR must not weaken it to:
prefer confirmation before creating the archive
```

If the source distinguishes:

```
candidate
proposed
confirmed
blocked
ready_for_first_run
```

the IR must preserve those distinctions.

Compilation may make semantics more explicit. It must not silently change them.

A useful principle is:

```
Source meaning in
  ↓
explicit execution meaning out
not:
```



Figure 31: Nebu — attention: compilation must preserve meaning

Source meaning in
↓
compiler interpretation
↓
new policy out

When the compiler cannot safely formalize a fragment, it should preserve the fragment in a controlled FREEFORM representation or emit a diagnostic.

9.7. Source Map: How Not to Lose the Link to Human Text

Compilation creates another representation of the program. We therefore need to know where each IR operation came from.

That is the role of a source map.

Example:

```
{
  "ir_id": "G_NO_UNSUPPORTED_FACTS",
  "source": {
    "file": "minimal_summary.ordo.yaml",
    "path": "path.gates[1]",
    "line_start": 31,
    "line_end": 34
  }
}
```

A source map helps answer:

Which source construct created this operation?
Where should the author edit the rule?
Did two source fragments compile into one op?
Was any source fragment lost?

Without a source map, IR becomes detached from the human source of truth.

This is especially dangerous during review and debugging.

9.8. Execution Trace: What the Model Actually Executed

A source map describes the relation between source and IR.

An execution trace describes what happened during a concrete run.

For example:

```
{
  "run_id": "RUN-2026-0017",
  "events": [
    {
      "op_id": "S_READ_INPUT",
      "status": "completed"
    },
    {
      "op_id": "S_EXTRACT_POINTS",
      "status": "completed",
      "output_ref": "ARTIFACT.points.1"
    },
    {
      "op_id": "G_NO_UNSUPPORTED_FACTS",
      "status": "failed",
      "reason": "one point has no source support"
    },
    {
      "op_id": "REPAIR.REMOVE_OR_MARK_UNSUPPORTED",
      "status": "completed"
    },
    {
      "op_id": "G_NO_UNSUPPORTED_FACTS",
      "status": "passed"
    }
  ]
}
```

The trace makes the run reproducible and inspectable.

It can show:

- which operations executed;
- in which order;
- which gates passed or failed;
- which repairs ran;
- which state fields changed;

- which outputs were created;
- which prompt or behavior version was used.

The trace is not the same as the source program. It is evidence of one execution of that program.



Figure 32: Nebu — thinking: source map and execution trace answer different questions

9.9. IR Does Not Have to Be Comfortable for Humans

This is an important point: Ordo Source and Semantic JSON IR have different goals.

Ordo Source should be understandable to a person.

IR should be convenient for execution.

Therefore, IR does not need to be beautiful to read. It may be longer, drier, and more formal.

That is normal.

In the future, an even more compact format may exist:

```
[
  ["I", "summarize_text", "uk"],
  ["C", {"out": {"type": "bullets", "max": 3, "lang": "uk"}}],
  ["G", "NO_UNsupported_FACTS", "summary", "REPAIR.REMOVE_UNsupported"],
  ["H", ["status", "result", "gate_report"]]
]
```

This is hardly convenient for a person, but it may be useful for a runtime or a model specifically trained on Ordo.

The current Semantic JSON IR is an intermediate practical format. It remains readable enough while already being structured enough.

9.10. What Good IR Must Contain

Good Ordo IR should follow several rules.

1. Every Operation Must Have op

Bad:

```
{
  "goal": "summarize_text"
}
```

Better:

```
{
  "op": "INTENT.SET",
  "goal": "summarize_text"
}
```

op shows what must be done.

2. Every Important Operation Must Have id

```
{
  "op": "GATE.CHECK",
  "id": "G_NO_UNsupported_FACTS",
  "method": "self_verification",
  "trust_class": "model_judgment"
}
```

Without an id, trace, source maps, and validation reports become difficult.

3. Gates Must Be Explicit

Bad:

```
{
  "rules": ["do not invent facts"]
}
```

Better:

```
{
  "op": "GATE.CHECK",
  "method": "self_verification",
  "trust_class": "model_judgment",
  "assert": "NO_UNsupported_FACTS",
  "on_fail": "REPAIR.REMOVE_UNsupported"
}
```

4. State Must Be Explicit

If the model must remember a user answer, that information belongs in state.

```
{
  "op": "STATE.UPDATE",
  "key": "request_type",
  "value": "incident"
}
```

5. Handoff Must Be Described

The model must know what to return.

```
{
  "op": "HANDOFF.EMIT",
  "include": ["status", "result", "gate_report"]
}
```

9.11. Typical Compilation Mistakes

Mistake 1. Leaving an Important Gate in Ordinary Text

If a rule blocks the final result, it should compile into `GATE.CHECK`, not remain only a sentence.

Mistake 2. Turning a Domain Explanation into a Universal Rule

For example, `HistoryEvent.item.values` is a domain contract, not an Ordo Core rule.

Mistake 3. Losing Status Semantics

If the source contains `candidate`, `confirmed`, `blocked`, and `ready_for_first_run`, the IR must preserve the differences.

Mistake 4. Failing to Preserve Non-Formalized Content in FREEFORM

If a fragment cannot be safely structured, it must not simply disappear.

Mistake 5. Omitting the Source Map

Without a source map, it is difficult to prove that the IR corresponds to the source program.

Mistake 6. Making IR Too Human-Oriented

IR is not an essay. If it contains too much free prose, the model is once again working from a prompt rather than an execution contract.

9.12. What This Looks Like in a Real Playbook

A simple example may contain 10-20 operations.

A real playbook may contain dozens or hundreds.

For example, an Ordo-native History Event Playbook contains operation groups such as:

```
PROGRAM.DEF
PROFILE.USE
DOMAIN_PACK.LOAD
ENTRY.DEF
NODE.DEF
ANSWER.REGISTRY
STATE.SCHEMA
OUTPUT.DEF
TEMPLATE.BIND
GATE.DEF
APPROVAL.REQUIRE
DOC.SPLIT
DOC.CATALOG
DOC.SELECT
RENDER.VALIDATE
FREEFORM.ADD
HANDOFF.DEF
```

This does not mean the model must show every operation to the user. The user may only need to see the current node, selected documents, state changes, and the next question.

Internally, however, the process should remain controlled.

9.13. Short Chapter Summary

Ordo Source is for people.

Semantic JSON IR is for execution.

Compiling Ordo Source into IR makes the process more explicit:

```
intent becomes INTENT.SET;
contract becomes CONTRACT.DEF;
context becomes CONTEXT.LOAD;
state becomes STATE.INIT / STATE.UPDATE;
steps become STEP.RUN;
gates become GATE.CHECK;
result and handoff become HANDOFF.EMIT.
```

Good IR has:

```
explicit op values;
stable IDs;
gates;
state;
a source map;
an execution trace;
controlled FREEFORM for content that cannot be safely formalized.
```

The main idea is:

```
compilation should not make an instruction prettier;
it should make it more executable.
```

Mini-Exercise

Take this simple Ordo Source program:

```
intent:
  goal: write_customer_reply

contract:
  output:
    type: email
    language: uk
  rules:
    - polite_tone
    - no_specific_deadline
    - acknowledge_request

context:
  customer_message: "$USER_INPUT.message"

result:
  primary: customer_reply
```

Try to transform it manually into Semantic JSON IR.

At minimum, you need these operations:

```
INTENT.SET
CONTRACT.DEF
RULE.ADD
CONTEXT.LOAD
STATE.INIT
STEP.RUN
GATE.CHECK
HANDOFF.EMIT
```

Then consider separately: which rule should become GATE.CHECK, and which may remain style guidance?

Chapter 10. Why Semantic JSON IR Is the Best Choice Right Now

10.1. Why an IR Is Needed at All

In the previous chapter, we established that Ordo Source is a human-friendly form of a program. It can be read, discussed, edited, and reviewed with an analyst or developer. It resembles a well-structured YAML document in which intent, contract, state, gates, outputs, and handoff are visible.

But Ordo Source is not yet the best form for model execution.

A person can comfortably read:

```
intent:
  goal: summarize_text

contract:
  output:
    format: bullet_list
    max_items: 3
    language: uk

rules:
  - use_only_input_text
  - do_not_invent_facts
```

A model can understand this too. But when a process grows to dozens of nodes, gates, statuses, libraries, templates, domain rules, and FREEFORM blocks, human-oriented YAML becomes too flexible. Fields may be nested differently. Some rules may be written as prose. Some links may remain implicit.

That is why an intermediate layer is needed: IR.



Figure 33: Nebu — idea: IR as an execution form

IR means Internal Representation.

In Ordo, IR is a form of the program designed primarily for more precise execution by a model or a future Ordo runtime rather than for comfortable human reading.

Ordo Source answers:

How can a person conveniently describe the process?

Ordo IR answers:

How can a model or runtime execute the process more reliably?

This distinction is fundamental.

10.2. Why Not Keep Only YAML

YAML may seem sufficient. It is readable, structured, and far better than one continuous prompt. For many simple tasks, it is sufficient.

But YAML remains an authoring format. It is good for writing and discussing an instruction, while execution exposes several weaknesses.

First, YAML does not always define strict execution order.

For example:

```
steps:
  - read_input
  - extract_points
  - render_summary

gates:
  - input_present
  - no_unsupported_facts
```

A person understands that the gates should be applied at the correct moments. A model may not always know whether `input_present` runs before `extract_points`, after it, or only before handoff.

Second, YAML often allows levels to be mixed. Machine-readable fields and long human explanations may appear side by side.

Third, YAML is readable but not always easy to execute or validate. A runtime or weaker model benefits from a list of concrete operations:

```
[
  {"op": "INTENT.SET", "goal": "summarize_text"},
  {"op": "CONTRACT.DEF", "output": {"format": "bullet_list", "max_items": 3}},
  {"op": "STEP.RUN", "fn": "READ_INPUT_TEXT"},
  {
    "op": "GATE.CHECK",
    "method": "mechanical",
    "trust_class": "deterministic",
    "assert": "INPUT_TEXT_PRESENT"
  }
]
```

Now the representation shows not only what exists in the program, but what must be done.

That is why Ordo Source and IR should remain separate layers.

10.3. Why Semantic JSON IR

At the current stage of AI-model development, the best primary IR for Ordo is Semantic JSON IR.

This means a JSON structure in which each operation has an explicit op, id, parameters, inputs, outputs, bindings, and expected behavior.

For example:

```
{
  "op": "GATE.CHECK",
  "id": "G_NO_UNsupported_FACTS",
  "method": "self_verification",
  "trust_class": "model_judgment",
  "assert": "NO_UNsupported_FACTS",
  "source": "RESULT.summary",
  "on_fail": "REPAIR.REMOVE_UNsupported"
}
```

This is not JSON for the sake of JSON. The important word is Semantic.

Semantic JSON IR means that the structure directly carries execution meaning.

op says which action to perform.

id provides a stable identifier.

assert says what is checked.

source says what the check applies to.

on_fail says what happens after failure.

This form works well for models because it is simultaneously:

- structured enough;
- understandable enough;
- not excessively compressed;
- independent of a special binary or native runtime format;
- easy for a person to inspect;
- easy to map to a future runtime.

10.4. Why Not Compact Opcode IR Immediately

We have already seen a compact form:

```
[
  ["I", "summarize_text", "uk"],
  ["C", {"out": {"type": "bullets", "max": 3}}],
  ["G", "NO_UNSUPPORTED_FACTS", "summary", "REPAIR.REMOVE_UNSUPPORTED"]
]
```

It is short and looks attractive as a future machine-native representation. But it is risky for current models.

Compactness reduces obviousness.



Figure 34: Nebu — attention: compactness does not always help

A person or model must remember what I, C, and G mean, the order of arguments, allowed values, and failure behavior. That is normal for a specially trained runtime. It is not yet ideal for a general language model.

Compact Opcode IR may become a good future format when:

the model is specifically trained on Ordo;
or an Ordo execution layer exists;

or a compiler/runtime executes the opcodes itself;
or Compact IR is used only as an internal format rather than as the primary LLM inst
At the practical stage, there is no need to rush.

Semantic JSON IR is a compromise between human intelligibility and machine precision.

10.5. Why Not Just Natural Language

Natural language is powerful. It lets us explain complex ideas without formal schemas. But it is often ambiguous.

For example:

Do not create the final package if the contracts are not confirmed.

A person understands the intention. Execution still raises questions:

- which contracts are mandatory;
- what confirmed means;
- where the condition is checked;
- which status is set on failure;
- whether a draft package is allowed;
- whether the user must be asked;
- whether this is a hard stop;
- whether the rule applies to all outputs or only the archive.

Semantic JSON IR can describe the rule more precisely:

```
{
  "op": "GATE.CHECK",
  "id": "G_NO_FINAL_ARCHIVE_BEFORE_CONTRACTS",
  "method": "mechanical",
  "trust_class": "deterministic",
  "assert": "ALL_MANDATORY_CONTRACTS_CONFIRMED",
  "scope": "final_archive",
  "on_fail": "BLOCK_FINAL_ARCHIVE",
  "failed_status": "no_go_requires_confirmation"
}
```

There is less room for accidental interpretation.

Natural language remains important in FREEFORM, explanations, descriptions, and templates. Hard rules, however, are better represented as structured operations.

10.6. The Same Instruction in Three Forms

Consider a simple rule:

Before the final result, check that there are no invented facts.

Natural language is understandable but not very formal.

In Ordo Source:

```
gates:
- id: no_unsupported_facts
  check: no_unsupported_facts
  source: result
  on_fail: remove_or_mark_unsupported
```

In Semantic JSON IR:

```
{
  "op": "GATE.CHECK",
  "id": "G_NO_UNUNSUPPORTED_FACTS",
  "method": "self_verification",
  "trust_class": "model_judgment",
  "assert": "NO_UNUNSUPPORTED_FACTS",
  "source": "RESULT.primary",
  "on_fail": "REPAIR.REMOVE_OR_MARK_UNUNSUPPORTED"
}
```

In Compact Opcode IR:

```
["G", "NO_UNUNSUPPORTED_FACTS", "RESULT.primary", "REPAIR.REMOVE_OR_MARK_UNUNSUPPORTED"]
```

All three forms describe the same idea, but they serve different roles.



Figure 35: Nebu — thinking: one instruction in three forms

Natural language is for explanation.

Ordo Source is for the author.

Semantic JSON IR is for execution by a current model or runtime.

Compact Opcode IR is for future optimized execution.

10.7. Main Advantages of Semantic JSON IR

Semantic JSON IR has several strong advantages.

1. Explicit Execution Order

IR can be a list of operations. The model receives a sequence rather than merely a set of rules:

```
first load documentation;  
then determine entry;  
then ask the node question;  
then update state;  
then check the gate;  
then move to the next node.
```

This is especially important for playbooks.

2. Stable Identifiers

Every operation has an id.

This allows explicit references:

```
{  
  "op": "APPROVAL.REQUIRE",  
  "id": "A_PASSPORT_APPROVAL",  
  "target": "OUTPUT.01_HISTORY_EVENT_PASSPORT"  
}
```

A validation report can later say:

```
{  
  "approval_id": "A_PASSPORT_APPROVAL",  
  "status": "passed"  
}
```

This creates traceability.

3. Simple Logging

When a program executes step by step, the runtime or model can maintain a trace:

```
{  
  "executed_op": "G_CONTRACTS_CONFIRMED",  
  "status": "failed",  
  "reason": "source_field_paths missing",  
  "next_action": "ASK_CONFIRMATION"  
}
```

This is much more informative than “some data needs clarification.”

4. Easier Gate Validation

A structured gate is easier to inspect and potentially automate:

```
{
  "op": "ASSERT.NOT",
  "id": "A_NO_ITEM_PREFIX_IN_DELTA_FIELD",
  "target": "ChangeRecord.delta.field",
  "forbidden_pattern": "^item\\.\""
}
```

This is already close to an executable check.

5. Easier Portability Between Models

The same Semantic JSON IR can be given to different models. Their response style may differ, but the task structure remains stable.

This matters for Ordo as a future standard.

10.8. Disadvantages of Semantic JSON IR

Semantic JSON IR is not perfect.

Its main disadvantage is length. One gate may occupy a single line in compact form and ten lines in Semantic JSON IR.

That length buys clarity.

The second disadvantage is the need for discipline. If every author invents new operation names, the IR quickly becomes chaotic. Ordo therefore needs a standard operation catalog.

The third disadvantage is that JSON is not pleasant to write manually. That is why a person writes Ordo Source and a compiler or model assistant translates it into IR.

Semantic JSON IR is not necessarily what an analyst should write by hand. It is what the model should receive for execution.

10.9. The Role of the Compiler

In a mature Ordo architecture, a compiler or translator should sit between the person and the model.

A person writes or says:

I want the model to guide the analyst through a tree, collect contracts, avoid creat

The compiler turns this into Ordo Source:

```

entry:
  id: guided_intake

state:
  contracts_confirmed: false
  approvals: pending

gates:
  - no_final_archive_before_contracts
  - rendered_artifacts_validated

```

Then it compiles the source into Semantic JSON IR:

```

[
  {"op": "ENTRY.DEF", "id": "guided_intake"},
  {"op": "STATE.SCHEMA", "fields": {"contracts_confirmed": "boolean"}},
  {
    "op": "GATE.CHECK",
    "id": "G_NO_FINAL_ARCHIVE",
    "method": "mechanical",
    "trust_class": "deterministic",
    "assert": "ALL_CONTRACTS_CONFIRMED"
  }
]

```

The model no longer has to extract the entire process from prose. It receives a structured execution map.

10.10. Why This Matters for Weaker Models

A strong model can often infer what the author intended. A weaker model may not.

Ordo should not be useful only for the strongest model. One of its purposes is to make complex instructions executable even by models that hold long context less reliably.

Semantic JSON IR helps weaker models because it:

- reduces ambiguity;
- makes order explicit;
- separates operations;
- provides stable statuses;
- shows what to do on failure;
- reduces the need to guess.

It is easier for a weaker model to execute:

```

{"op": "NODE.ASK", "id": "N1", "allowed_answers": ["A", "B", "C"]}

```

than a long instruction explaining that the first tree question should be asked only if the path is not yet selected.

10.11. Why This Matters for a Future Runtime

Semantic JSON IR is also a bridge to a future Ordo runtime.

A runtime may perform part of the work without a model:

- maintain state;
- determine the current node;
- validate allowed answers;
- run gates;
- check status transitions;
- determine which documents to load;
- block the final archive;
- generate validation reports.

The model then does what it does best:

- explains questions to the user;
- interprets answers;
- forms document content;
- works with domain prose;
- writes understandable text.

The runtime controls the process; the model fills the semantic content.

This is an important strategic idea in Ordo.

10.12. Practical Format-Selection Rule

For the current Ordo version, the rule can be stated as follows:

Ordo Source – primary format for people.

Semantic JSON IR – primary format for model execution.

Compact Opcode IR – future format for optimization or a native runtime.

FREEFORM – controlled container for content that cannot be safely formalized.

If you write a playbook manually, start with Ordo Source.

If you give an instruction to a model or runner, prefer Semantic JSON IR.

If a specialized Ordo runtime appears, it may compile Semantic JSON IR into a compact or native form.

10.13. Typical Mistakes

Mistake 1. Giving the Model Only YAML and Expecting Runtime Behavior

YAML helps, but it does not guarantee execution. Complex processes need an execution map.

Mistake 2. Moving to Compact Opcodes Too Early

Compact representation looks elegant, but current models benefit from semantic structure.

Mistake 3. Hiding Prose Rules Inside description

If a rule blocks an action, it should be a gate or assertion rather than only prose.

Mistake 4. Having No Stable IDs

Without IDs, reliable traceability, validation reports, and source maps are impossible.

Mistake 5. Allowing Different op Values for the Same Meaning

If one author writes CHECK.GATE, another writes GATE.CHECK, and a third writes VALIDATE, the standard begins to fragment. A standard operation catalog is required.

10.14. Short Chapter Summary

Semantic JSON IR is currently the best primary execution format for Ordo because it balances readability and strictness.

It is better than pure natural language because it is less ambiguous.

It is better than YAML as an execution layer because it explicitly describes operations and order.

It is more practical than Compact Opcode IR because current models can understand it more easily.

The main rule is:

A person writes Ordo Source.

A model or runtime executes Semantic JSON IR.

A future native runtime may use Compact Opcode IR.

Semantic JSON IR is the bridge between human instruction and controlled AI-model execution.

Mini-Exercise

Take this simple instruction:

Ask the user to select a task type. If they select incident, ask about severity. If

Write it in three forms:

1. natural language;
2. Ordo Source;
3. Semantic JSON IR.

Compare which form makes these elements clearest:

- allowed answers;
- current node;
- state update;
- error handling;
- next node.

If Semantic JSON IR makes them most explicit, you have understood the main idea of this chapter.

Chapter 11. A Gate Is Not Advice but a Control Point

11.1. Why Ordo Needs Gates

A normal prompt often contains phrases such as:

```
check the result
make sure everything is correct
do not continue if data is missing
ask the user if needed
```

These phrases sound reasonable, but they are weak execution controls.

A model may interpret “check” as “look over it briefly.” It may decide that missing information is unimportant. It may continue because the result appears plausible. It may silently repair something and never report that a problem existed.

Ordo needs a stronger construct.

That construct is a Gate.

A gate is a control point that answers:

May the process continue?

A gate is not a recommendation to be careful. It is an explicit decision point in the execution path.

11.2. Gate as a Traffic Light

The simplest way to understand a gate is to imagine a traffic light.



Figure 36: Nebu — idea: a gate controls movement through the process

A green signal means:

the required condition is satisfied;
the process may continue.

A red signal means:

the required condition is not satisfied;
the process must not continue through this path.

Sometimes there is also an intermediate state:

`requires_confirmation`

This means:

the model cannot resolve the condition itself;
a person or another authorized actor must decide.

Therefore, a gate should produce a meaningful status rather than a vague impression.

Typical statuses may be:

`passed`
`failed`
`blocked`
`requires_confirmation`
`repair_required`

The exact vocabulary may vary by contract, but its semantics must be explicit.

11.3. Gate in a Simple Example

Suppose a model must summarize a text in Ukrainian.

A weak instruction says:

Make sure the result is in Ukrainian.

An Ordo gate can say:

```
{
  "op": "GATE.CHECK",
  "id": "G_OUTPUT_LANGUAGE",
  "method": "mechanical",
  "trust_class": "deterministic",
  "assert": "LANG_IS",
  "params": {
    "lang": "uk"
  },
  "source": "RESULT.primary",
  "on_fail": "REPAIR.REWRITE_IN_LANGUAGE"
}
```

Now the operation explicitly defines:

- what is checked;
- how it is checked;
- what source is checked;
- what condition must hold;
- what happens on failure.

This is much stronger than “make sure.”

11.4. A Gate Must Be Specific

A bad gate:

check quality

A better gate:

check that the final package contains every mandatory file

An even better gate:

```
gate:
  id: "G_REQUIRED_FILES_PRESENT"
  method: "mechanical"
```

```

trust_class: "deterministic"
check: "all_required_files_exist"
source: "PACKAGE.file_list"
required:
  - "README.md"
  - "SUMMARY.json"
  - "VALIDATION_REPORT.json"
on_fail: "STOP"

```

The more concrete the gate, the less interpretation is left to the model.

A useful gate should answer:

1. What is checked?
2. Which object or state is checked?
3. By what method?
4. What counts as success?
5. What status is produced?
6. What evidence is recorded?
7. What happens on failure?

If these questions cannot be answered, the gate is probably still only advice.

11.4.1. A Gate Must Show the Verification Method

Ordo v0.12 makes the verification method explicit.

This is necessary because not all checks have the same reliability.

For example:

```

gate:
  id: "G_FILE_COUNT"
  method: "mechanical"
  trust_class: "deterministic"
  check: "file_count_equals_expected"

```

A script or runtime can check this deterministically.

Another gate may be:

```

gate:
  id: "G_NO_UNSUPPORTED_FACTS"
  method: "self_verification"
  trust_class: "model_judgment"
  check: "result_contains_no_unsupported_facts"

```

This is a semantic judgment made by the model. It may be useful, but it does not have the same evidentiary strength as a mechanical

check.

A third case:

```
gate:
  id: "G_CONTRACT_APPROVED"
  method: "human"
  trust_class: "human_decision"
  check: "user_explicitly_approved_contract"
```

The model must not replace this decision with self-verification.

Typical method values include:

- mechanical
- self_verification
- self_consistency
- human
- external

Typical trust_class values may include:

- deterministic
- model_judgment
- human_decision
- external_evidence

The important principle is:

A gate must not pretend that a model's opinion is a deterministic fact.



Figure 37: Nebu — attention: verification methods have different trust

Making method and trust_class visible helps the author, runtime, and reviewer understand how strongly a gate result can be trusted.

11.5. A Gate Can Stop the Process

A real gate must influence execution.

Bad:

The contract is not confirmed, but I created the final package anyway.

This means the “gate” was only commentary.

A proper gate may define:

```
gate:
  id: "G_CONTRACT_CONFIRMED"
  method: "human"
  trust_class: "human_decision"
  check: "mandatory_contracts_confirmed"
  on_pass: "CONTINUE"
  on_fail: "BLOCK_FINAL_PACKAGE"
```

Then the execution path is:

G_CONTRACT_CONFIRMED = passed
→ final package may be generated

G_CONTRACT_CONFIRMED = failed
→ final package is blocked

A gate that does not affect the path is usually not a gate.

11.6. Gate and Hard Stop

Some conditions must stop execution completely.

For example:

required source data is absent;
a mandatory contract is unresolved;
the user has not approved a business decision;
the package checksum is invalid;
a rendered artifact is corrupted.

These are hard-stop conditions.

An Ordo gate may describe them explicitly:

```
gate:
  id: "G_SOURCE_ROW_CONFIRMED"
  method: "human"
  trust_class: "human_decision"
  check: "representative_source_row_confirmed"
  severity: "block"
  on_fail: "STOP"
```

A hard stop means:

do not continue through the blocked path;
do not silently invent the missing value;

do not mark the process as complete;
do not hide the blocker in a summary.

The model may still explain the blocker, ask a question, or produce a permitted draft if the contract allows it. But it cannot behave as though the gate passed.

11.7. Gate Before the Result and Gate Inside the Process

Gates are not needed only at the end.

A final gate may check:

is the result ready for handoff?

But an internal gate may check:

has the path been selected?

has the current node been answered?

is the contract confirmed?

is generation allowed?

did rendering complete?

For example:

```
ENTRY
  ↓
NODE.SELECT_PATH
  ↓
G_PATH_SELECTED
  ↓
NODE.COLLECT_CONTRACT
  ↓
G_CONTRACT_CONFIRMED
  ↓
STEP.GENERATE
  ↓
G_OUTPUT_VALIDATED
  ↓
HANDOFF
```

If all checks are delayed until the end, the model may perform a large amount of invalid work before discovering that an early condition was missing.

Internal gates localize failure.

They also make repair cheaper because the process can return to the nearest failed control point rather than restart everything.

11.8. A Gate Must Have Evidence

A gate result should not be merely:

passed

It should be possible to explain why it passed.

For example:

```
{
  "id": "G_REQUIRED_FILES_PRESENT",
  "method": "mechanical",
  "trust_class": "deterministic",
  "status": "passed",
  "evidence": {
    "expected": [
      "README.md",
      "SUMMARY.json",
      "VALIDATION_REPORT.json"
    ],
    "actual": [
      "README.md",
      "SUMMARY.json",
      "VALIDATION_REPORT.json"
    ]
  }
}
```

For human confirmation:

```
{
  "id": "G_CONTRACT_APPROVED",
  "method": "human",
  "trust_class": "human_decision",
  "status": "passed",
  "evidence": {
    "source": "current_node_answer",
    "answer": "confirmed",
    "scope": "contract_v3"
  }
}
```

For model self-verification:

```
{
  "id": "G_NO_UNSUPPORTED_FACTS",
  "method": "self_verification",
  "trust_class": "model_judgment",
  "status": "passed",
}
```

```

    "evidence": {
      "reviewed_source": "RESULT.summary",
      "support_map": "ARTIFACT.support_map.1"
    }
  }
}

```

Evidence does not make every check equally reliable. It makes the basis of the result visible.



Figure 38: Nebu — thinking: evidence makes a gate result traceable

That visibility is essential for trace, debugging, and later review.

11.9. A Gate Must Not Be Hidden in FREEFORM

FREEFORM is useful for explanations, domain prose, examples, and content that cannot yet be formalized safely.

But a blocking rule must not exist only in FREEFORM.

Bad:

```

freeform:
  text: |
    Please remember not to create the final archive
    until the user confirms all mandatory contracts.

```

Better:

```

gates:
  - id: "G_NO_FINAL_ARCHIVE_BEFORE_CONTRACTS"
    method: "mechanical"
    trust_class: "deterministic"
    check: "all_mandatory_contracts_confirmed"
    on_fail: "BLOCK_FINAL_ARCHIVE"

```

The explanatory text may remain in FREEFORM, but the control point must be formal.

A useful rule is:

If a statement can stop execution, change the path, authorize an output, or require human approval, it should not live only in FREEFORM.

11.10. Gate and Repair Action

Not every failed gate must stop the whole process.

Sometimes the model can safely repair the result.

For example:

```
{
  "op": "GATE.CHECK",
  "id": "G_OUTPUT_LANGUAGE",
  "method": "mechanical",
  "trust_class": "deterministic",
  "assert": "LANG_IS",
  "params": {
    "lang": "uk"
  },
  "source": "RESULT.primary",
  "on_fail": "REPAIR.REWRITE_IN_LANGUAGE"
}
```

Or:

```
{
  "op": "GATE.CHECK",
  "id": "G_MAX_ITEMS",
  "method": "mechanical",
  "trust_class": "deterministic",
  "assert": "ITEM_COUNT_LE",
  "params": {
    "max": 3
  },
  "source": "RESULT.summary",
  "on_fail": "REPAIR.COMPRESS"
}
```

A repair action must be safe.

If the model can correct the problem without new data and without violating the contract, repair is allowed.

If correction requires new knowledge or human confirmation, repair is not allowed. A BLOCK or REQUEST_CONFIRMATION is required instead.

11.11. Gate and the User

A gate should not make the process inconvenient for the user.

It is bad if the model says after every small action:

Confirmation required. Confirmation required. Confirmation required.

Ordo should distinguish:

- what the model can safely do itself;
- what can be marked as an assumption;
- what can remain an open question;
- what requires explicit approval;
- what is a hard stop.

For example, a document draft may contain a placeholder.

But a final contract cannot silently retain an assumption.

Therefore, a gate should understand mode:

```
mode:
  draft: allow_placeholders
  final: require_confirmed_contracts
```

This allows Ordo to be not only strict but practical.

11.12. Gate Report

After an important stage, an Ordo program should be able to show a gate report.

A gate report is a short report showing which checks passed, which did not, and what that means.

Example:

```
{
  "trace_source": "hybrid",
  "gate_report": [
    {
      "id": "G_PATH_SELECTED",
      "method": "mechanical",
      "trust_class": "deterministic",
      "status": "passed",
      "evidence": "terminal_path = Path 1 candidate"
    },
    {
      "id": "G_SOURCE_ROW_CONFIRMED",
      "method": "human",
      "trust_class": "human_decision",
      "status": "failed",
      "reason": "representative source row not provided",
      "next_action": "request source row or keep package in draft mode"
    },
    {
      "id": "G_NO_FINAL_PACKAGE",
```

```

    "method": "mechanical",
    "trust_class": "deterministic",
    "status": "blocked",
    "reason": "mandatory contracts unresolved"
  }
]
}

```

This is useful to the user. They see not merely that “the model cannot continue,” but exactly why.

It is even more useful to a developer or runtime because the gate report can be checked automatically.

11.13. Typical Gate Mistakes

The first mistake is writing gates as general wishes.

Check quality.

Better:

Check that every executable TC has source lookup, action, ChangeRecord lookup, history processing, event assertion, rollback, and post-rollback verification.

The second mistake is failing to define `on_fail`.

If a gate does not pass, the model must know what to do.

The third mistake is hiding a hard stop in FREEFORM.

The fourth mistake is allowing the model to decide that a gate passed without evidence.

The fifth mistake is having a gate in the template but not checking the final rendered artifact.

The sixth mistake is failing to distinguish failed, blocked, and `requires_confirmation`.

The seventh mistake is omitting `method` and creating the impression that a semantic self-check has the same strength as a mechanical code check.

These mistakes make an Ordo program resemble an ordinary long prompt. The purpose of Ordo is precisely to avoid that.

11.14. Short Chapter Summary

A gate is an execution control point.

It answers:

May we move forward?

A good gate has:

- a clear condition;
- method;
- trust_class;
- a verification source;
- status;
- evidence;
- on_fail behavior;
- a clear effect on the next step.

A gate can:

- allow the process to continue;
- start a repair;
- request confirmation;
- block the final result;
- create a gate report.

The main principle is:

A gate is not advice.

A gate is the point where an Ordo program decides whether the model has the right to

Mini-Exercise

Take any process of your own and find three places where the model must not continue without a check.

For each place, write:

1. What are we checking?
2. Where are we checking it?
3. Which status is possible?
4. Which `method` is needed: mechanical, self_verification, self_consistency, or human?
5. Which `trust\_class` should the result have?
6. What should happen if the check fails?
7. Can the model repair the problem itself, or is human confirmation required?

Then try to write one gate in JSON form.

Chapter 12. Negative Checks: ASSERT.NOT → ASSERTION

Why This Is Needed

In ordinary instructions, we often tell a model what it must do:

- create a document;
- ask a question;
- check the structure;
- form a response;
- prepare an archive;
- provide a short summary.

But in real processes, it is equally important to say what the model **must not** do.

For example:

- do not invent missing data;
- do not move to the next step without confirmation;
- do not create the final package before checks pass;
- do not treat an example as a rule;
- do not replace a business decision with a technical guess;
- do not include extra files in the package;
- do not hide uncertainty in the final result.

In prompts, such prohibitions are often lost. A model may read them, agree with them, and still perform an action that is formally forbidden. That is why negative checks in Ordo must be more than textual warnings: they must be separate controlled rules.



Figure 39: Nebu — idea: ASSERTION protects against forbidden actions

Early Ordo versions used this construct:

`ASSERT.NOT`

Starting with Ordo v0.12, the more precise formulation is:

`ASSERTION` is the canonical primitive.

`ASSERT.NOT` is shorthand for `ASSERTION` with polarity: `not`.

This is an important change. `ASSERT.NOT` is no longer a separate parallel mechanism beside gates and tests. It is one projection of the unified `ASSERTION` rule, which can expand into a runtime check, a test expectation, and a debug violation record.

Simple Explanation

`ASSERTION` is a formal statement about what must be true or what must not happen in the process.

It has polarity:

`must` – a required state must hold;
`not` – a forbidden state must not occur.

Therefore, `ASSERT.NOT` can be understood as an assertion in the opposite direction:

Are we certain that the forbidden thing did not happen?

For example:

Gate:
all mandatory sections are present.

`ASSERT.NOT`:
the final package contains no extra files.

Or:

Gate:
the user confirmed the alias.

`ASSERT.NOT`:
the model did not invent the alias itself.

This distinction is very important because a positive check does not always catch a violation.



Figure 40: Nebu — attention: a positive check does not catch extra behavior

A document may contain all required sections and still contain extra, dangerous, or contradictory blocks. An ordinary gate may say “the

structure exists,” while an assertion with polarity: not must say “nothing forbidden is present.”

Why Ordo v0.12 Introduces ASSERTION

Before v0.12, three similar constructs could easily appear in the language:

ASSERT.NOT – a runtime prohibition;
negative gate – a gate checking that a problem is absent;
EXPECT.NOT – a test expectation that a forbidden state does not appear.

All three express the same idea: **a certain state or action must not occur**. If they remain separate, the author must manually synchronize the rule across execution, tests, and debug reporting.

For example, an author may add:

```
ASSERT.NOT final_output_before_validation
```

but forget to add the corresponding EXPECT.NOT to the regression suite. The rule then exists in the playbook but is not protected by a test.

Therefore, Ordo v0.12 makes one primitive canonical:

ASSERTION

ASSERT.NOT, EXPECT.NOT, and a negative gate become projections of it.

Ordo Construct

In Source form, an assertion may look like this:

```
assertions:
- id: "A_NO_INVENTED_ALIAS"
  polarity: "not"
  condition: "alias_created_without_user_confirmation"
  phase: ["runtime", "test"]
  method: "self_verification"
  severity: "block"
  on_fail: "STOP"
  message: "Alias cannot be invented by the model without user confirmation."
```

This means:

If the alias was created without explicit user confirmation, the process must stop, and the test suite must contain an expectation that such a state is not allowed.

In compiled IR, it may expand into several representations.

Runtime representation:

```
{
  "op": "ASSERT.NOT",
  "id": "domain_pack.history_event.A_NO_INVENTED_ALIAS",
  "source_assertion": "domain_pack.history_event.A_NO_INVENTED_ALIAS",
  "condition": "alias_created_without_user_confirmation",
  "method": "self_verification",
  "severity": "block",
  "on_fail": "STOP"
}
```

Test representation:

```
{
  "op": "EXPECT.NOT",
  "id": "domain_pack.history_event.TE_NO_INVENTED_ALIAS",
  "source_assertion": "domain_pack.history_event.A_NO_INVENTED_ALIAS",
  "condition": "alias_created_without_user_confirmation"
}
```

Debug representation:

```
{
  "op": "VIOLATION.RECORD",
  "source_assertion": "domain_pack.history_event.A_NO_INVENTED_ALIAS",
  "status": "not_triggered",
  "method": "self_verification"
}
```

Importantly, `ASSERT.NOT` does not merely warn. If severity: block, it is a hard stop.

Method for an Assertion

Ordo v0.12 adds method not only to gates but also to assertions when an assertion is checked during execution.

For example:

```
assertions:
- id: "A_NO_EXTRA_FILES"
  polarity: "not"
  condition: "package_contains_unapproved_files"
  phase: ["runtime", "test"]
  method: "mechanical"
  severity: "block"
```


Here `method: mechanical` is appropriate because a runner or script can deterministically compare the actual file list with the allowed list.

Another example:

```
assertions:
- id: "A_NO_UNCONFIRMED_BUSINESS_DECISION"
  polarity: "not"
  condition: "final_output_contains_business_decision_without_user_confirmation"
  phase: ["runtime", "test"]
  method: "self_verification"
  severity: "block"
```

This is a semantic judgment. A model critic step may check it using an evidence protocol, but we must not pretend that such a check is as reliable as counting files.

Example Without Ordo

Imagine this prompt:

Prepare an analytical package. Make sure all files are present.
Do not add extra files.

The model may create a package containing every required file and also add:

```
debug_notes.md
draft_old.md
extra_payload.json
```

Then it may respond:

The package is ready. All files are present.

Formally, it checked the positive condition. But the prohibition was violated.

Example with Ordo v0.12

In Ordo, this is better described as:

```
output:
  required_files:
    - "README.md"
    - "SUMMARY.json"
    - "VALIDATION_REPORT.json"

allowed_files:
```

- "README.md"
- "SUMMARY.json"
- "VALIDATION_REPORT.json"

gates:

- id: "G_REQUIRED_FILES_PRESENT"
method: "mechanical"
trust_class: "deterministic"
check: "all_required_files_exist"
on_fail: "STOP"

assertions:

- id: "A_NO_EXTRA_FILES"
polarity: "not"
condition: "package_contains_files_outside_allowed_files"
phase: ["runtime", "test"]
method: "mechanical"
severity: "block"
on_fail: "STOP"

The final response is then possible only if:

1. all required files exist;
2. there are no extra files;
3. both checks have an explicit method;
4. the test suite automatically receives EXPECT.NOT for the forbidden state.

This is much stronger than the general phrase “check the package.”

Typical Cases for ASSERTION with polarity: not

Negative assertions are especially important where the model may:

- rush;
- guess;
- “help” beyond the instruction;
- silently change meaning;
- add unnecessary structure;
- move to the next stage without permission;
- hide uncertainty;
- present an intermediate result as final.

Typical ASSERT.NOT rules for Ordo:

ASSERT.NOT invented_value
ASSERT.NOT skipped_gate
ASSERT.NOT implicit_approval
ASSERT.NOT hidden_freeform_rule

```
ASSERT.NOT unexpected_file
ASSERT.NOT unconfirmed_contract
ASSERT.NOT unresolved_conflict
ASSERT.NOT ambiguous_status
ASSERT.NOT final_output_before_validation
ASSERT.NOT example_used_as_rule
```

In v0.12, these should preferably be described as ASSERTION:

```
assertions:
- id: "A_NO_FINAL_OUTPUT_BEFORE_VALIDATION"
  polarity: "not"
  condition: "final_output_created_before_validation_passed"
  phase: ["runtime", "test"]
  method: "mechanical"
  severity: "block"
```

Negative Checks and Statuses

ASSERTION is closely connected to statuses.

For example, if the process status is:

needs_user_decision

there should be a negative assertion:

```
ASSERT.NOT continue_without_user_decision
```

If the status is:

blocked_by_missing_contract

there should be:

```
ASSERT.NOT generate_final_artifact
```

Otherwise, the status becomes decorative. The model appears to know that the process is blocked but may still continue.

In Ordo, status must affect allowed actions.

Negative Checks and FREEFORM

One of the most dangerous places is FREEFORM.

FREEFORM is needed when part of the knowledge is too early or impossible to formalize fully. But it must not become storage for hidden rules.

Useful assertions for FREEFORM therefore include:



Figure 41: Nebu — thinking: FREEFORM must not hide control rules

```
ASSERT.NOT core_gate_hidden_in_freeform
ASSERT.NOT mandatory_output_hidden_in_freeform
ASSERT.NOT approval_rule_hidden_in_freeform
ASSERT.NOT business_decision_hidden_in_freeform
```

In simpler terms:

An explanation may live in FREEFORM.
A control point must be formal.

If a rule stops the process, changes the path, defines output, or requires human confirmation, it must not exist only in FREEFORM.

Negative Checks in Document Work

For documentation, assertions with polarity: not are almost mandatory.

Examples:

```
ASSERT.NOT duplicate_section
ASSERT.NOT unresolved_placeholder
ASSERT.NOT obsolete_term
ASSERT.NOT inconsistent_alias
ASSERT.NOT missing_traceability
ASSERT.NOT markdown_link_to_internal_draft
ASSERT.NOT raw_runtime_note_in_final_document
```

They catch what an ordinary positive check often misses.

For example, a document may contain all required sections and still contain:

```
<TODO>
<ASK_USER_LATER>
old_alias
draft only
```

The positive gate “structure is correct” will not catch this. A negative assertion will.

Severity: When to Stop and When to Warn

Not every negative check is equally critical.

Therefore, Ordo ASSERTION needs severity:

info
warn
block

For example:

```
assertions:
- id: "A_NO_TODO_IN_FINAL"
  polarity: "not"
  condition: "final_artifact_contains_todo_marker"
  phase: ["runtime", "test"]
  method: "mechanical"
  severity: "block"

- id: "A_NO_MINOR_STYLE_DRIFT"
  polarity: "not"
  condition: "document_contains_minor_style_inconsistency"
  phase: ["runtime"]
  method: "self_verification"
  severity: "warn"
```

The rule is simple:

If the violation can make the result incorrect or dangerous – block.

If the violation only reduces quality without breaking the result – warn.

Compiler Projection

In v0.12, the compiler must be able to expand an assertion into several target forms.

For example:

```
assertions:
- id: "A_NO_SKIPPED_GATE"
  polarity: "not"
  condition: "required_gate_was_skipped"
  phase: ["runtime", "test", "debug"]
  method: "mechanical"
  severity: "block"
```

The compiler may create:

```
runtime: ASSERT.NOT required_gate_was_skipped
test:    EXPECT.NOT required_gate_was_skipped
debug:   VIOLATION.RECORD if required_gate_was_skipped is true
```

This reduces the risk that a prohibition is described in one place but forgotten in another.

Typical Mistakes

Mistake 1. Expressing a Prohibition Only in Words

Bad:

Do not make mistakes.

Better:

```
ASSERT.NOT skipped_gate
ASSERT.NOT invented_required_value
ASSERT.NOT final_output_before_validation
```

Even better in v0.12:

```
assertions:
- id: "A_NO_FINAL_OUTPUT_BEFORE_VALIDATION"
  polarity: "not"
  condition: "final_output_before_validation"
  phase: ["runtime", "test"]
  method: "mechanical"
  severity: "block"
```

A prohibition must be checkable.

Mistake 2. Combining Everything into One Large Gate

Bad:

Check that everything is fine.

Better:

```
GATE required_files_present
ASSERT.NOT unexpected_files_present
ASSERT.NOT unresolved_placeholders_present
ASSERT.NOT validation_report_missing
```

One large gate creates an illusion of control but provides no precise trace.

Mistake 3. Making Negative Checks Too Abstract

Bad:

```
ASSERT.NOT bad_result
```

Better:

```
ASSERT.NOT result_contains_unconfirmed_business_decision
```

The model and runtime must understand exactly what is being checked.

Mistake 4. Omitting the Consequence

Bad:

```
assertions:  
  - condition: "missing_approval"
```

Better:

```
assertions:  
  - id: "A_NO_MISSING_APPROVAL"  
    polarity: "not"  
    condition: "missing_approval"  
    phase: ["runtime"]  
    method: "mechanical"  
    severity: "block"  
    on_fail: "STOP"
```

Without a consequence, the check may become only a comment.

Mistake 5. Forgetting Test Projection

Bad:

The rule exists at runtime but is absent from the regression suite.

Better:

```
phase: ["runtime", "test"]
```

The compiler must then create a runtime assertion and test expectation from the same source.

Mini-Exercise

Take a simple instruction:

Prepare the final document for handoff to a developer.

Try to list five negative checks.

For example:

1. There must be no unresolved placeholders.
2. There must be no invented technical details.
3. There must be no contradiction between the task description and acceptance criteria.
4. There must be no links to intermediate drafts.
5. There must be no final status without a self-check.

Then rewrite them in Ordo v0.12 style:

```
assertions:
- id: "A_NO_UNRESOLVED_PLACEHOLDER"
  polarity: "not"
  condition: "unresolved_placeholder_present"
  phase: ["runtime", "test"]
  method: "mechanical"
  severity: "block"

- id: "A_NO_INVENTED_TECHNICAL_DETAIL"
  polarity: "not"
  condition: "invented_technical_detail_present"
  phase: ["runtime", "test"]
  method: "self_verification"
  severity: "block"
```

Short Summary

ASSERTION is the canonical way to describe a rule that must hold or must not be violated.

ASSERT.NOT is shorthand for an assertion with polarity: not.

An ordinary gate checks that something required has been done. ASSERT.NOT checks that something forbidden did not happen.

This is critical for Ordo because an AI model often fails not only by omitting an action but also by doing too much: inventing, skipping ahead, adding, hiding, mixing, or finishing prematurely.

A good Ordo program has not only a list of what must be done but also a list of what must not be done. In v0.12, this is best described once as an ASSERTION, and the compiler should project that rule into runtime, test, and debug representations.

Chapter 13. Status Semantics

Why This Is Needed

In an ordinary conversation with artificial intelligence, status often looks like a minor detail.

A model may write:

ready
done
can be handed off
looks correct
clarification is needed

At first glance, everything seems understandable. But this is not enough for a controlled process.

The problem is that such words do not always have a precise meaning. For one person, “ready” means “the draft can be read.” For another, it means “the file can be handed to a developer.” For a third, it means “all checks passed and no risks remain.”

For Ordo, this ambiguity is dangerous. If a status has no formal semantics, the model may finish a process prematurely, skip an approval gate, present a draft as a final result, or call a result “ready” without evidence.

Therefore, Ordo needs a separate language construct:

`STATUS.SEMANTICS`

It defines exactly what each status means, who may set it, which gates must pass before it, and which actions are allowed afterward.

Simple Explanation

A status is not merely a label.



Figure 42: Nebu — idea: status as a controlled process position

In Ordo, status is a controlled position in the process.

For example:

```
needs_input
ready_for_analysis
analysis_in_progress
needs_approval
approved_for_generation
ready_for_handoff
blocked
```

Every status should answer several questions:

1. What has already happened?
2. What has not happened yet?
3. Who is allowed to move the process into this status?
4. Which actions are now allowed?
5. Which actions are now forbidden?
6. What is the next legal status?

Without this, statuses are decorative words. With it, they become part of execution.

Why Statuses Matter Specifically for an AI Model

A person often understands status from context. If an analyst says “you can proceed,” another person may clarify what exactly is allowed: continue analysis, create a file, send it to QA, or hand it to a developer.

An AI model does not interpret this reliably. It may understand the word “yes” too broadly.

For example, the user says:

Yes, this option works.

The model may incorrectly decide:

The user approved final archive generation.

But the user may have meant only:

I like the idea, but this is not yet the final document.

STATUS.SEMANTICS is needed so such transitions do not happen automatically.

Ordo Construct

In Ordo, a status can be described like this:

```
status_semantics:
- status: "needs_input"
```

```

meaning: "the process waits for data or a decision from the user"
allowed_actions:
  - "ask_question"
  - "summarize_current_state"
forbidden_actions:
  - "generate_final_package"
  - "mark_as_ready_for_handoff"
next_allowed_statuses:
  - "ready_for_analysis"
  - "blocked"

- status: "ready_for_handoff"
  meaning: "the result passed mandatory gates and may be handed off"
  requires_gates:
    - "G_CONTRACT_CONFIRMED"
    - "G_OUTPUT_VALIDATED"
    - "G_SELF_CHECK_PASSED"
  allowed_actions:
    - "handoff_result"
  forbidden_actions:
    - "change_contract_without_reapproval"

```

In compiled IR, this may become:

```

[
  {
    "op": "STATUS.DEF",
    "id": "S_NEEDS_INPUT",
    "status": "needs_input",
    "meaning": "process waits for user input or decision"
  },
  {
    "op": "STATUS.ALLOW",
    "status": "needs_input",
    "actions": ["ask_question", "summarize_current_state"]
  },
  {
    "op": "STATUS.FORBID",
    "status": "needs_input",
    "actions": ["generate_final_package", "mark_as_ready_for_handoff"]
  },
  {
    "op": "STATUS.REQUIRE.GATES",
    "status": "ready_for_handoff",
    "gates": ["G_CONTRACT_CONFIRMED", "G_OUTPUT_VALIDATED", "G_SELF_CHECK_PASSED"]
  }
]

```

The exact syntax is less important than the principle: status must be an executable rule rather than a free phrase.

Status Lifecycle

For an Ordo program, it is useful to describe not only individual statuses but also the process lifecycle.

For example:

```
created
→ needs_input
→ contract_ready
→ contract_confirmed
→ analysis_in_progress
→ needs_approval
→ approved_for_generation
→ generation_in_progress
→ validation_in_progress
→ ready_for_handoff
→ handed_off
```

This lifecycle shows how the process should move.

Importantly, the model must not invent transitions between statuses.

If only this is allowed:

```
needs_approval → approved_for_generation
```

the model cannot jump directly to:

```
needs_approval → ready_for_handoff
```

even if it feels that “everything is already clear.”

Status Transition

A transition between statuses is a separate action.

In a simple form:

```
status_transitions:
- from: "needs_approval"
  to: "approved_for_generation"
  allowed_by:
    - "user_explicit_approval"
  requires:
    - "approval_scope_defined"
  on_missing_requirement: "STOP"
```

This means the model cannot simply say “it seems approved.” There must be explicit confirmation or another allowed condition.

In Ordo, it is useful to distinguish:

```
model_can_set
user_must_set
runtime_can_set
external_system_can_set
```

For example:

```
status: "contract_confirmed"
can_be_set_by:
  - "user"
forbidden_for:
  - "model_without_explicit_approval"
```

This is especially important in processes where the model assists but must not make business decisions on behalf of a person.

Statutes and Gates

A status must not be set without gates.

Bad:

Model: everything is checked, status ready_for_handoff.

Better:

```
set_status: "ready_for_handoff"
requires_gates:
  - "G_REQUIRED_FILES_PRESENT"
  - "G_NO_UNRESOLVED_PLACEHOLDERS"
  - "G_CONSISTENCY_CHECK_PASSED"
  - "G_HANDOFF_NOTE_PRESENT"
```

Now status is not the model’s opinion. It is the result of passing control points.

If even one gate fails, the status cannot be set.

```
G_CONSISTENCY_CHECK_PASSED = failed
→ ready_for_handoff is forbidden
```

This is how Ordo reduces the risk of a premature “done.”

Statutes and ASSERT.NOT

Negative checks can also be attached to statuses.

For example:

```
status: "ready_for_handoff"
assert_not:
  - "unresolved_placeholders_present"
  - "missing_validation_report"
  - "unconfirmed_assumption_present"
  - "freeform_without_binding"
```

This means the process cannot call itself ready while forbidden conditions remain.

For a final status, negative checks are often especially important:

- no unfilled places
- no invented decisions
- no invisible assumptions
- no unvalidated documents
- no unexplained FREEFORM

Statutes and User Answers



Figure 43: Nebu — attention: a short answer applies only within the current node

One of the most important problems in AI-model work is interpreting short user answers.

The user may write:

- yes
- ok
- works
- next
- you can
- agreed

Without STATUS.SEMANTICS, the model may interpret this too broadly.

Ordo must require that a short user answer be interpreted only within the current node or current gate.

For example, if the current question was:

Select option A, B, or C?

and the user answers:

A

this does not mean:

the user approved the final package

It means only:

the user selected option A in the current node

Therefore, status in Ordo must be connected to the current node, current gate, and current question.

Example: Guided Intake

Imagine that the model conducts guided intake for a new historical event.

Possible statuses:

```
statuses:
  - "intake_started"
  - "path_selected"
  - "event_alias_confirmed"
  - "source_row_confirmed"
  - "values_confirmed"
  - "qa_scope_confirmed"
  - "package_ready_for_generation"
  - "package_generated"
  - "package_validated"
  - "package_ready_for_handoff"
```

Allowed transitions can be defined for each status:

```
transitions:
  - from: "intake_started"
    to: "path_selected"
    requires: ["path_decision_answered"]

  - from: "path_selected"
    to: "event_alias_confirmed"
    requires: ["alias_confirmed_by_user"]

  - from: "package_generated"
    to: "package_validated"
    requires: ["self_check_executed", "validation_report_created"]
```

```
- from: "package_validated"
  to: "package_ready_for_handoff"
  requires: ["no_blocking_validation_errors"]
```

Now the model does not merely “follow the conversation.” It moves through a controlled status map.

Example: A Document

For document preparation, statuses may be:

```
draft_requested
structure_confirmed
content_drafted
content_reviewed
rendered_artifact_created
rendered_artifact_validated
ready_to_share
```

It is important not to confuse:

```
content_drafted
```

with:

```
ready_to_share
```

An ordinary model may say “done” after creating the text. In Ordo, the text is not ready if the final artifact has not been validated.

For example:

```
status: "ready_to_share"
requires:
  - "content_reviewed"
  - "rendered_artifact_created"
  - "rendered_artifact_validated"
  - "download_link_available"
```

This is especially important for PDFs, DOCX files, presentations, archives, and document packages.

Draft, Final, and Handoff

One of the main reasons to introduce statuses is to separate three different states:

```
draft
final
handoff_ready
```


They are not the same.

draft means:

the text or structure has been created but may still be incomplete

final means:

the content is considered complete within the current contract

handoff_ready means:

the result is not only complete but also validated, packaged, and ready for transfer

Ordo must not substitute one status for another.

For example:

```
final_content != handoff_ready_package
```

Final text may be ready while the file remains unvalidated. Or a package may be assembled while the validation report still contains errors. In such cases, handoff_ready is forbidden.

Status Report

An Ordo process should be able to show status not only as a word but as a short report.

For example:

```
status_report:
  current_status: "needs_approval"
  current_node: "N_APPROVE_PACKAGE_SCOPE"
  completed_gates:
    - "G_PATH_SELECTED"
    - "G_ALIAS_CONFIRMED"
  pending_gates:
    - "G_PACKAGE_SCOPE_APPROVED"
  blocked_actions:
    - "generate_final_archive"
  next_allowed_actions:
    - "ask_package_scope_approval"
```

Such a report is useful for the user, the runtime, and the next model that may continue the process.

It also reduces context loss in long conversations.

Statuses and Trace

Statuses must leave a trace.



Figure 44: Nebu — thinking: status must leave a trace

It is not enough to have only the current status. A transition history is needed:

```
trace_source: "model_self_report"
status_trace:
  - from: "needs_input"
    to: "contract_ready"
    reason: "all mandatory intake fields collected"
    evidence: ["E_INPUT_FIELDS_COMPLETE"]

  - from: "contract_ready"
    to: "contract_confirmed"
    reason: "user explicitly approved contract"
    evidence: ["E_USER_APPROVAL_2026_07_05"]
```

This answers:

why is the process in this status now?
who or what moved it here?
what evidence supports the transition?

For Ordo, this is essential because without trace, status again becomes merely a model claim.

Typical Mistakes

Mistake 1. Using Vague Statuses

Bad:

ok
ready
done
almost_done
good

Better:

contract_confirmed
content_drafted

```
validation_passed
ready_for_handoff
```

A status must describe a concrete process state.

Mistake 2. Failing to Separate Approval and Execution

Bad:

```
approved
```

What exactly was approved: the idea, structure, contract, generation, or final file?

Better:

```
contract_approved
package_scope_approved
generation_approved
handoff_approved
```

Mistake 3. Allowing the Model to Set Final Statuses by Itself

Bad:

```
status: "ready_for_handoff"
can_be_set_by: ["model"]
```

Better:

```
status: "ready_for_handoff"
can_be_set_by: ["runtime"]
requires_gates:
  - "G_SELF_CHECK_PASSED"
  - "G_RENDERED_ARTIFACT_VALIDATED"
  - "G_NO_BLOCKING_ERRORS"
```

Or, if there is no runtime:

```
can_be_set_by: ["model_after_required_trace"]
```

but only with a mandatory gate report.

Mistake 4. Failing to Describe Allowed Transitions

Bad:

status may be any value from the list

Better:

```
from: "needs_approval"  
to: "approved_for_generation"  
requires: ["explicit_user_approval"]
```

Statuses without transitions are a dictionary. Statuses with transitions are a process.

Mistake 5. Confusing Status and Result

`status = ready_for_handoff`

is not the handoff itself. It is only permission to perform the handoff.

Likewise:

`status = validation_passed`

is not a validation report. It is the result of passing a validation gate, which must have evidence.

Mini-Exercise

Take any process you often delegate to an AI model.

For example:

Prepare a Jira task.

Try to describe statuses:

```
request_received  
problem_context_collected  
acceptance_criteria_drafted  
qa_scope_drafted  
pm_level_review_needed  
ready_for_jira_copy
```

Then answer for every status:

1. What does this status mean?
2. Who may set it?
3. Which gates are required before it?
4. Which actions are allowed after it?
5. Which actions are forbidden after it?
6. Which next status is allowed?

You will see that even a simple process becomes much more controlled.

Short Summary

STATUS.SEMANTICS defines exactly what statuses mean in an Ordo program.

A status in Ordo is not merely the word “ready” or “done.” It is a formal process state with rules, allowed actions, prohibitions, transitions, gates, and evidence.

Well-defined statuses protect a process from premature completion, incorrect interpretation of short user answers, hidden transitions, and confusion between a draft and a final handoff.

If a Gate answers “may we move forward?”, then STATUS.SEMANTICS answers “where exactly are we in the process, and what are we allowed to do next?”

Chapter 14. Why Ordo Programs Need Debugging

Why This Is Needed

When a normal program behaves incorrectly, a developer does not inspect only the final screen. They look at logs, variable values, branches, errors, and the sequence of executed operations.

AI processes need the same discipline.

A model may produce a plausible final answer while internally following the wrong path, skipping a gate, changing state incorrectly, or using knowledge that was never authorized by the process.

Without debug information, the author sees only:

```
input
↓
model
↓
final answer
```

If the result is wrong, the only practical reaction is often:

rewrite the prompt and try again

Ordo treats this as insufficient.

An Ordo program should make important execution events observable.



Figure 45: Nebu — idea: debug makes process behavior observable

Simple Explanation

Debugging means answering:

What exactly happened during this run?

For an Ordo process, this includes questions such as:

Which execution mode was active?
Which entry point was used?
Which path was selected?
Which paths were rejected?
Which decision caused the selection?
What state existed before the step?
What changed after the step?
Which gate ran?
How was the gate checked?
What evidence supported its result?
Which knowledge source was used?
Which warning or violation appeared?

The final text alone cannot answer these questions.

That is why Ordo includes a debug and trace layer.

Why This Is Especially Important for AI

A traditional program executes explicit instructions. If the code says:

```
if severity == critical:  
    path = emergency
```

we can inspect the code and runtime values.

A language model works differently. It interprets context and produces a result probabilistically. Even when the process is structured, the model may:

- overgeneralize;
- interpret a short answer too broadly;
- use a nearby but incorrect rule;

- silently skip a negative condition;
- infer a state transition that was not authorized;
- treat an example as a rule;
- continue after a blocking condition.

This creates a dangerous situation:

the final answer looks reasonable
but the execution logic was wrong

Debugging is needed to reveal such errors before they become hidden process behavior.

Ordo Construct

Ordo defines execution modes such as:

```
normal
debug
dry_run
test
replay
improvement_capture
```

A program may activate debug mode:

```
execution:
  mode: "debug"
```

Or in IR:

```
{
  "op": "DEBUG.MODE",
  "mode": "debug"
}
```

In debug mode, the process may emit structured diagnostic artifacts:

```
TRACE.LOG
DECISION.LOG
PATH.EXPLAIN
STATE.SNAPSHOT
STATE.DIFF
GATE.REPORT
KNOWLEDGE.TRACE
```

These are not decorative notes. They form an observable execution layer.

Small Example Without Debug

Imagine a guided intake process.

The user answers:

We need to add a new event type.

The model chooses Path 2 and starts asking questions.

Later, the author discovers that Path 1 should have been selected.

Without debug information, the available evidence may be only:

User message

Model questions

The author must guess:

Did the model misunderstand the request?

Did it read the wrong path rule?

Did it interpret a synonym incorrectly?

Did state already contain a value from an earlier step?

Did it skip a gate?

The process is difficult to diagnose.

Small Example with Debug

With debug mode, the process may record:

```
trace_source: "model_self_report"
```

```
run_id: "RUN-2026-0142"
```

```
path_explain:
```

```
  selected_path: "PATH_2"
```

```
  candidates:
```

```
    - path: "PATH_1"
```

```
      status: "rejected"
```

```
      reason: "request interpreted as modification of existing event"
```

```
    - path: "PATH_2"
```

```
      status: "selected"
```

```
      reason: "phrase 'add a new event type' mapped to extension flow"
```

```
decision_log:
```

```
  - decision_id: "D_PATH_SELECTION"
```

```
    input_ref: "USER_MESSAGE_1"
```

```
    result: "PATH_2"
```

```
state_diff:
```

```
  before:
```



```
selected_path: null
after:
  selected_path: "PATH_2"
```

Now the problem is visible.

The author can inspect the path-selection rule rather than rewriting the whole playbook.

Debug as Protection Against Overconfident Answers



Figure 46: Nebu — attention: a confident answer may hide an incorrect path

AI models often produce fluent text even when the process behind it is wrong.

Therefore:

good wording != correct execution

Debug mode separates these two dimensions.

A result may look polished but have:

- wrong path
- skipped gate
- unsupported state update
- wrong knowledge source
- unrecorded assumption

A debug trace makes these defects visible.

This is especially important for playbooks that create technical packages, contracts, Jira tasks, QA artifacts, migration instructions, or other outputs where process correctness matters more than stylistic quality.

Debug and State

State is one of the first places to inspect when a process behaves unexpectedly.

Suppose the model asks the same question twice.

The problem may not be the question text. The real problem may be that the answer was never written to state.

A state snapshot may show:

```
state:
  alias_confirmed: false
  source_row_confirmed: true
```

A state diff may show:

```
before:
  alias_confirmed: false

after:
  alias_confirmed: false
```

although the user explicitly confirmed the alias.

This immediately localizes the defect:

the node received the answer
but STATE.UPDATE did not occur

Without state debugging, the author may incorrectly rewrite the conversational prompt.

Debug and Path Selection

A path-selection error should be explainable.

A good debug record does not contain only:

selected PATH_3

It should also show rejected candidates:

```
path_explain:
  selected: "PATH_3"
  candidates:
    - id: "PATH_1"
      result: "rejected"
      reason: "no new entity requested"
    - id: "PATH_2"
      result: "rejected"
      reason: "no migration requested"
    - id: "PATH_3"
      result: "selected"
      reason: "existing behavior requires extension"
```

Rejected paths matter because the bug may be in a rejection condition rather than in the selected path itself.

Debug and Knowledge

A model may produce the wrong answer because it used the wrong knowledge source.

Ordo therefore needs knowledge trace.

For example:

```
knowledge_trace:
- source_id: "domain_pack.history_event"
  version: "0.12.0"
  used_by: "NODE_SOURCE_ROW"
  purpose: "field semantics"

- source_id: "freeform.note.17"
  used_by: "NODE_SOURCE_ROW"
  purpose: "example interpretation"
```

This allows the reviewer to ask:

Was the correct domain pack loaded?

Was an obsolete library version used?

Did a FREEFORM example accidentally influence a hard rule?

Was required documentation never loaded?

Knowledge use should be observable when it affects important decisions.

Typical Mistakes

Mistake 1. Debugging Only the Final Text

The final text is an output artifact, not a complete execution record.

If the wrong path produced a good-looking document, editing the document template does not fix the process.

Mistake 2. Not Logging Rejected Paths

Recording only the selected path hides why alternatives were discarded.

A path bug often lives in a rejection rule.

Mistake 3. Not Logging State Changes

A current state snapshot is useful, but it does not show how the state changed.

STATE.DIFF is needed to answer:

what changed at this exact step?

Mistake 4. Treating Debug as Unnecessary for Playbooks

A playbook is executable process logic expressed for a model.

The more nodes, gates, statuses, outputs, and approvals it contains, the more important debugging becomes.

Mistake 5. Confusing Debug Trace with Chain of Thought



Figure 47: Nebu — thinking: execution trace is structured evidence, not hidden reasoning

Ordo does not require private chain-of-thought disclosure.

A debug trace should contain observable process facts:

operation executed
path selected
state changed
gate checked
evidence reference used
status produced
warning recorded

It should not require unrestricted hidden reasoning.

This distinction is essential.

Mistake 6. Not Preserving run_id

Without a stable run identifier, logs from several executions may be mixed.

Every diagnostic run should be attributable to one execution:

`run_id: "RUN-2026-0142"`

The same identifier should connect trace, gate reports, state diffs, validation artifacts, and replay records.

Mini-Exercise

Take a process that has at least three steps.

For example:

```
collect request
select path
generate document
```

Imagine that the final document is wrong.

Write five debug questions:

1. Which path was selected?
2. Which paths were rejected?
3. What state existed before generation?
4. Which gate passed before generation?
5. Which knowledge source was used?

Then define which Ordo diagnostic artifact should answer each question:

```
PATH.EXPLAIN
STATE.SNAPSHOT
STATE.DIFF
GATE.REPORT
KNOWLEDGE.TRACE
```

If the process cannot answer these questions, it is difficult to debug.

Short Summary

Ordo programs need debugging because a plausible final answer does not prove correct execution.

The author must be able to inspect:

```
execution mode
run ID
selected and rejected paths
decisions
state snapshots
state changes
gate results
```

knowledge sources
warnings and violations

Ordo therefore treats debug and trace as part of the language and IR rather than as optional comments around the prompt.

The main principle is:

If a process can make an important decision,
the process should be able to leave structured evidence of that decision.

Debugging does not ask the model to expose hidden chain of thought.
It asks the Ordo program to record observable execution facts.

Chapter 15. Debug Mode and Diagnostic Trace

Why This Is Needed

In the previous chapter, we established the main problem: a complex Ordo program cannot be developed properly if only the model's final answer is visible.

The final answer is only the top of the process. It shows what the model said at the end, but not how it arrived there.

For simple tasks, this is sometimes enough. If a model rewrites a short message, for example, the result can simply be judged: does it sound good or not?

But this is insufficient for a playbook, domain pack, library, or multi-step process. There it is important to know:

- which path was selected;
- which paths were rejected;
- which rules triggered;
- which gates were checked;
- which gates were skipped;
- what the state was before the step;
- what the state became after the step;
- which knowledge or instruction fragments were used;
- where the model made an assumption;
- where it should have stopped but did not.

This is why Ordo needs debug mode.

Debug mode is an execution mode in which an Ordo program not only produces a result but also forms a complete execution trace.

In simple terms:



Figure 48: Nebu — idea: debug mode shows the execution path

normal mode answers: what was done;
debug mode answers: why it was done this way.

Simple Explanation

Imagine that an Ordo program is a route in a navigation system.

In normal mode, the navigator simply guides you to the destination.

In debug mode, it additionally shows:

- why it selected this road;
- which roads it rejected;
- where restrictions existed;
- where traffic jams occurred;
- where it rebuilt the route;
- which data it relied on;
- what would have happened if another path had been selected.

For Ordo, this means the model must show not hidden reasoning but a formal execution trace.

This distinction is important.

Ordo must not require a model to disclose private chain of thought. Instead, Ordo must require a structured execution trace: a record of formal decisions that are part of the process.



Figure 49: Nebu — attention: trace is not chain of thought

We do not need to see everything the model “thought.” We need to see what it **executed**:

- which NODE is active;
- which path was selected;
- which gate was evaluated;
- which state changed;
- which output was allowed;
- which rule was used;
- which warning was raised.

This is not model psychology. It is a program execution log.

Debug Mode as Part of the Language

In Ordo, debug mode must not be an external comment such as:

Explain why you did this.

That is too weak. The model may explain beautifully but not necessarily accurately.

The language needs a formal construct:

```
run:
  mode: debug
  execution_mode: chat_internal
  trace_required: true
```

Or in compiled IR:

```
{
  "op": "DEBUG.MODE",
  "mode": "debug",
  "execution_mode": "chat_internal",
  "trace_required": true
}
```

This means the Ordo program is launched not only to produce a result but also to collect an execution trace.

In debug mode, a result without a trace is considered incomplete.

In Ordo v0.12, `execution_mode` must be recorded alongside `mode`. It honestly shows the environment in which the process executes:

<code>full_runtime</code>	- transitions and hard gates are forcibly controlled by a runtime or
<code>chat_internal</code>	- the model runs the process in chat, but some checks may be performed
<code>freeform_only</code>	- the model follows Ordo discipline through text without external enforcement

This is not a minor technical detail. The same debug trace has different evidentiary strength depending on who actually controlled transitions: code, the model in chat, or textual self-discipline alone.

What Is an Execution Trace

An execution trace is a structured log of Ordo program execution.

It should answer:

- what was provided as input;
- what the initial state was;
- which path was selected;
- why that path was selected;
- which paths were rejected;
- which nodes were traversed;
- which gates were checked;
- which decisions were made;
- which state changes occurred;
- which outputs were allowed;
- which outputs were blocked;
- which sources or knowledge were used;
- which warnings or violations occurred.

A minimum trace structure may look like this:

```
trace:
  run_id: "RUN-001"
  mode: "debug"
  execution_mode: "chat_internal"
  trace_source: "model_self_report"

  input_snapshot:
    user_message: "create a company status change event"

  selected_path:
    id: "A1"
    reason: "the user described a field change in the main source row"

  rejected_paths:
    - id: "A2"
      reason: "there is no confirmation that the change concerns a related entity"
    - id: "A4"
      reason: "there is no ExternalHistoryEvent"

  nodes:
    - id: "NODE_SELECT_PATH"
      status: "completed"
    - id: "NODE_COLLECT_CONTRACT"
      status: "active"

  gates:
```

```

- id: "G_CONTRACT_CONFIRMED"
  status: "pending"
  reason: "source field has not yet been confirmed"

state_changes:
- field: "event_alias"
  before: null
  after: "LU_CHANGE_STATUS"

warnings:
- "source field not confirmed"

violations: []

```

This trace is not the final user document. It is an internal but controlled execution artifact.

Trace Source

In Ordo v0.12, every trace must explicitly show the source of its trust.

The following field is used:

```
trace_source: model_self_report | runtime_enforced | hybrid
```

`model_self_report` means the trace was formed by the model itself. It is useful for explaining logic but is not the same kind of evidence as an external runtime log.

`runtime_enforced` means the trace was generated by runner or orchestrator code from actual state transitions, actual gate calls, and recorded state snapshots.

`hybrid` means a mixed mode: part of the trace is produced by code and part is the model's semantic explanation. For example, `STATE.DIFF` may be runtime-enforced while `PATH.EXPLAIN`.reason is model self-report.

This field is required for honesty. Without it, a debug trace may look like a classic program log even though it may actually be only the model's structured self-report.

Example:

```

trace:
  run_id: "RUN-001"
  mode: "debug"
  execution_mode: "chat_internal"
  trace_source: "hybrid"

```

```
runtime_enforced:
  - "state_snapshot"
  - "mechanical_gate_status"

model_self_report:
  - "path_reason"
  - "semantic_evidence_summary"
```

Ordo v0.12 rule: a trace without trace_source is incomplete.

Run ID

Every Ordo program run must have a run_id.

Without a run_id, it is difficult to understand which exact execution is being discussed.

This is especially important when a user says:

this is where it went wrong

Ordo should be able to bind the observation to a concrete run:

```
run:
  id: "RUN-2026-07-05-014"
  program: "history_event_playbook"
  version: "0.12"
  mode: "debug"
  execution_mode: "chat_internal"
  trace_source: "hybrid"
```

An improvement record can then say:

```
observed_in:
  run_id: "RUN-2026-07-05-014"
  node: "NODE_PRE_ARCHIVE_CHECK"
  gate: "G_PACKAGE_SELF_CHECK"
```

This makes the improvement concrete rather than abstract and ties it to an actual execution.

Input Snapshot

A debug trace should contain an input snapshot.

This does not necessarily mean a full copy of all data, especially when confidential information is present. But it must contain enough to understand how the run began.

For example:

```
input_snapshot:
  user_intent: "create HistoryEvent for a status change"
  provided_fields:
    - "alias"
    - "old_value"
    - "new_value"
  missing_fields:
    - "source_field"
    - "fixture_id"
```

This shows that the model had no right to move to the final package because part of the contract had not yet been confirmed.

Path Explain



Figure 50: Nebu — thinking: path explain shows more than the selection

One of the most important debug-mode elements is `PATH.EXPLAIN`.

It must show not only the selected path but also the reason for the selection.

Bad:

```
selected_path: "A1"
```

Better:

```
selected_path:
  id: "A1"
  reason: "input describes a direct change in the main source row"
  evidence:
    - "user provided old/new values"
    - "no related entity context was confirmed"
```

An even better variant shows rejected paths:

```
rejected_paths:
  - id: "A2"
    reason: "related entity was not confirmed"
  - id: "A4"
    reason: "external history event payload was not provided"
```

This is extremely important for debugging.

Without rejected paths, we see only the decision. With rejected paths, we see the boundaries of the decision.

Decision Log

DECISION.LOG is a log of formal decisions.

A decision is not any sentence produced by the model. It is a point at which the Ordo program could have followed different paths.

For example:

```
decision_log:
- id: "D001"
  node: "NODE_SELECT_PATH"
  decision: "select_path_A1"
  reason: "direct source row change"
  evidence:
    - "field change described"
    - "no external event"

- id: "D002"
  node: "NODE_OUTPUT_ALLOWED"
  decision: "block_final_archive"
  reason: "pre-archive approval gate is not passed"
  evidence:
    - "G_PRE_ARCHIVE_APPROVAL = pending"
```

A decision log should be short but sufficiently precise.

It should not become a long literary explanation.

State Snapshot and State Diff

In a complex Ordo program, state is the memory of the process.

Debug mode must show not only the current state but also state changes.

Two constructs are needed:

```
STATE.SNAPSHOT
STATE.DIFF
```

STATE.SNAPSHOT shows the state at a specific moment.

STATE.DIFF shows exactly what changed.

For example:

```

state_snapshot:
  at: "before_NODE_COLLECT_CONTRACT"
  state:
    event_alias: null
    source_field: null
    output_allowed: false

```

After the step:

```

state_diff:
  step: "NODE_COLLECT_ALIAS"
  changes:
    - field: "event_alias"
      before: null
      after: "LU_CHANGE_STATUS"

```

This makes it possible to see where the model filled state prematurely or, conversely, failed to fill something that had already been confirmed.

Gate Report

In normal mode, a gate may simply block or allow an action.

In debug mode, a gate must explain its status.

For example:

```

gate_report:
  - gate_id: "domain_pack.history_event.G_PRE_ARCHIVE_APPROVAL"
    method: "human"
    trust_class: "human_decision"
    trace_source: "runtime_enforced"
    status: "blocked"
    reason: "user has not approved package generation"
    required_evidence:
      - "explicit user approval"
    actual_evidence: []

```

This is better than simply:

the archive cannot be created

because it shows exactly what is missing.

A gate report should contain at least:

- full namespaced gate id;
- method;
- trust_class;
- trace_source;

- status;
- reason;
- required evidence;
- actual evidence;
- blocking / non-blocking;
- next required action.

This is especially important for gates with different trust levels. `method: mechanical` and `method: self_verification` may both have `status: passed`, but they do not provide the same type of guarantee. The first passed a deterministic check; the second is a semantic model judgment under an evidence protocol.

Knowledge Trace

An Ordo program often relies on different sources:

- Core;
- Profile;
- Domain Pack;
- Library;
- user-provided data;
- uploaded playbook;
- runtime context;
- FREEFORM block.

In debug mode, it must be visible which knowledge was used.

For example:

```
trace_source: "model_self_report"
knowledge_trace:
  - source_type: "domain_pack"
    source_id: "history_event_domain_pack"
    section: "Path A1"
    used_for: "path selection"

  - source_type: "library"
    source_id: "ordo.validation.contract_first"
    export: "G_CONTRACT_CONFIRMED"
    used_for: "contract validation"

  - source_type: "freeform"
    source_id: "FF_ANALYST_STYLE_GUIDE"
    used_for: "response tone"
```

This is especially important for FREEFORM.

If the model made an important decision based on FREEFORM, the

trace must show it. This may reveal that the FREEFORM content should be formalized.

Warning and Violation

A debug trace must distinguish warnings from violations.

A warning is a risk or incompleteness that does not necessarily stop the process.

A violation is a rule breach.

For example:

```
warnings:
- id: "W_SOURCE_FIELD_MISSING"
  message: "source field is not confirmed yet"
  blocking: false

violations:
- id: "V_ARCHIVE_CREATED_BEFORE_APPROVAL"
  rule: "ASSERT.NOT final_archive_created before G_PRE_ARCHIVE_APPROVAL"
  severity: "critical"
```

If a critical violation exists, the Ordo program must not pretend that the result is valid.

Debug Output for a Person

A complete trace may be long. Ordo should therefore distinguish:

machine trace

human debug summary

The machine trace is needed by the runtime, tests, and compiler.

The human debug summary is needed by the user.

For example:

Debug summary:

Path A1 was selected because the input describes a field change in the main source r

Path A2 was rejected because no related entity was confirmed.

The final archive is blocked because gate G_PRE_ARCHIVE_APPROVAL has not passed.

Next required action: confirm the source field and fixture.

This is short, understandable, and does not overload the person with complete JSON.

Debug Mode and Privacy

A debug trace must not blindly expose all raw data.

A trace may contain confidential payloads, personal data, internal names, or technical details.

Ordo should therefore support trace levels:

```
trace_level: summary
trace_level: standard
trace_level: full
trace_level: redacted
```

For example:

```
run:
  mode: "debug"
  trace_level: "redacted"
```

In this mode, a trace may show that a field was used while hiding its value:

```
state_diff:
  - field: "tax_id"
    before: "[REDACTED]"
    after: "[REDACTED]"
```

This is important if Ordo is used in real products.

Execution Mode and Guarantee Level

Debug mode explains execution, but by itself it does not guarantee that every gate was actually enforced. Ordo v0.12 therefore adds a separate `execution_mode` field.

```
program: history_event_playbook
execution_mode: full_runtime
```

Base modes:

- `full_runtime` – runtime or helper runner controls state, node transitions, and hard
- `chat_internal` – the model works in chat and may run scripts or maintain state in ses
- `freeform_only` – Ordo discipline is executed through text without external control.

Honest guarantee table:

execution_mode	Who determines when a gate is invoked	Who performs the check	Guarantee level
full_runtime	code / runner	code or model under protocol	highest
chat_internal	model in chat	session code or model	medium
freeform_only	model	model through text	lowest

This table is needed so that Ordo's strength is not overstated. In chat_internal mode, a mechanical check may genuinely be executed by code, but without an external runtime the model still has to invoke it at the correct time.

Debug Mode and the Compiler

The Ordo compiler must be able to add trace points automatically.

An Ordo Source author should not have to log every small detail manually.

For example, if Source contains:

```
nodes:
  - id: "NODE_SELECT_PATH"
    branches:
      - path: "A1"
        when: "direct source row change"
      - path: "A2"
        when: "related entity change"
```

the compiler should automatically create trace points in IR:

```
{
  "op": "PATH.EXPLAIN",
  "node": "NODE_SELECT_PATH",
  "required": true,
  "trace_source": "model_self_report"
}
```

The debug layer must not be an “addition on the side.” It must be part of compilation.

Debug Mode and Model Errors

A debug trace helps distinguish different classes of problems.

If a result is wrong, the cause may be:

1. incorrect instruction;
2. incomplete contract;
3. ambiguous context;
4. incorrect path selection;
5. skipped gate;
6. incorrect output template;
7. weak FREEFORM;
8. library conflict;
9. model failed to execute IR;
10. user changed the requirement during the process.

Without trace, all of these look the same:

the model made a mistake

With trace, we can say more precisely:

the error occurred at `NODE_SELECT_PATH`;

the model selected A1 although the fixture matched A2;

the reason is that the branch condition lacks a rule for a related entity through th

This is no longer merely a complaint. It is a diagnosis.

Typical Mistakes

Mistake 1. Asking the Model to “Explain” Without Requiring Trace

An explanation after the fact may be elegant but unverifiable.

A structured trace during execution is better.

Mistake 2. Logging Only the Selected Path

The selected path alone does not show why other paths were rejected.

Rejected paths with reasons must also be logged.

Mistake 3. Not Logging State Diff

If only the final state is visible, it is difficult to understand exactly where it became incorrect.

A state diff is needed after important nodes.

Mistake 4. Hiding the Gate Report in Response Text

A gate report must be structured.

Otherwise, it is difficult to test.

Mistake 5. Showing Too Much Debug Information to a Person

The complete trace is needed by the machine and playbook author.

A debug summary is often sufficient for the user.

Mistake 6. Confusing Execution Trace with Chain of Thought

Ordo does not need the model's private reasoning text.

Ordo needs a formal execution log: node, path, gate, state, output, evidence.

Mistake 7. Omitting trace_source

A trace without trace_source creates the false impression that it is always a runtime log.

In v0.12, it must explicitly show whether it is model_self_report, runtime_enforced, or hybrid.

Mistake 8. Omitting execution_mode

If the execution mode is absent, the reader or next process cannot understand the guarantee level of the trace.

full_runtime, chat_internal, and freeform_only are different control levels, not different names for the same mode.

Mini-Exercise

Take this simple instruction:

Prepare a package for a new historical event, but do not create the final archive un

Describe what should appear in the debug trace.

At minimum, define:

- run_id;
- execution_mode;
- trace_source;
- input snapshot;
- selected path;

- rejected paths;
- state before/after source-field confirmation;
- gate prohibiting archive creation;
- gate status before confirmation;
- warning if QA scenarios are not yet defined;
- human debug summary.

Then formulate a violation for the case where the model nevertheless created the archive before confirmation.

Short Summary

Debug mode is an Ordo program execution mode in which the result is accompanied by a structured execution trace.

The execution trace should show:

- run_id;
- execution_mode;
- trace_source;
- input snapshot;
- selected and rejected paths;
- decision log;
- state snapshots;
- state diffs;
- gate report;
- knowledge trace;
- warnings;
- violations;
- human debug summary.

The main value of debug mode is that it transforms a problem from:

the model somehow did the wrong thing

into:

at this node, this path was selected for this reason; this gate was skipped; this st

That is why debug mode is not a service feature around Ordo but part of the language itself.

M72.1 update. This chapter explains the debug representation of trace. The full normative core element EXECUTION_TRACE, its fields, event catalog, replay, and integrity are described in Chapter 74.

Chapter 16. Tests, Fixtures, and Expected Behavior

Why This Is Needed

When we work with ordinary code, we almost never consider a program ready merely because it worked correctly once by hand. We write tests, check different scenarios, record expected behavior, and rerun those checks after changes.

Instructions for AI models are often handled less rigorously. An instruction is rewritten and works better in one case but silently breaks another. A new rule is added and the model starts asking questions in the wrong order. A gate is strengthened and the model can no longer reach the result. A gate is weakened and the model starts creating the final artifact before confirmation.

Without tests, these problems are discovered only by accident. If the playbook is large, finding the cause becomes very difficult.

An Ordo program should therefore be tested not only by its final text but by process behavior.



Figure 51: Nebu — idea: a test checks process behavior

An Ordo test asks not only:

is the final output correct?

It asks a broader question:

did the process proceed exactly as it was supposed to?

In v0.12 this becomes even more important because Ordo explicitly distinguishes trust classes: mechanical checks, semantic model checks, repeated checks, and human decisions. A test must see this difference rather than merely say “the gate passed.”

Simple Explanation

In a normal prompt-based approach, testing often looks like this:

I gave the model input data.
The model answered something.
The answer seems fine.
That is insufficient for Ordo.



Figure 52: Nebu — attention: test more than the final output

Ordo should check:

- which path was selected;
- which paths were rejected;
- which questions the model asked;
- which questions the model was not allowed to ask;
- how unmatched input was handled;
- which state changed;
- which gates passed;
- which methods were used to check gates;
- which trust_class each gate had;
- which gates blocked execution;
- whether forbidden output was created;
- whether a no-op scenario was handled correctly;
- whether FREEFORM was used invisibly;
- whether an action occurred without approval;
- whether behavior matches the declared execution_mode;
- whether an ASSERTION was violated.

An Ordo program test is therefore a test of execution behavior.

Fixture: Controlled Input

A repeatable test needs a fixture.

A fixture is a prepared set of input data for a test.

In ordinary code, a fixture may be a test object, database record, or file. In Ordo, a fixture may contain:

- user message;
- initial state;
- available context;

- execution_mode;
- control_level;
- connected libraries;
- active profile;
- domain pack;
- source documents;
- expected user confirmations;
- environment constraints;
- previous trace if the test is replay-based.

Example:

```
fixture:
  id: "FX_HISTORY_EVENT_A1_BASIC"
  execution_mode: "chat_internal"
  control_level: "standard"
  user_message: "We are creating a historical event for a company status change."
  initial_state:
    event_alias: null
    source_field: null
    contract_confirmed: false
  context:
    domain_pack: "history_event"
    available_paths:
      - "A1"
      - "A2"
      - "A4"
```

A fixture lets us repeat the same scenario and obtain a comparable result.

Expected Behavior: What Exactly Should Happen

Ordinary tests often check only the result. For example:

```
expect file package.zip
```

That is not enough in Ordo. Expected behavior must be described.

Expected behavior is the contract for how the process is supposed to behave.

It may include:

- expected path;
- expected node sequence;
- expected questions;
- expected state;

- expected gates;
- expected gate methods;
- expected trust classes;
- expected assertions;
- expected output;
- expected block;
- expected no-op;
- expected warnings;
- expected trace_source;
- expected absence of forbidden actions.

Example:

```

expected:
  path:
    selected: "A1"

  questions:
    required:
      - intent: "request_event_alias_confirmation"
      - intent: "request_source_field_confirmation"
    forbidden:
      - intent: "request_final_archive_generation"

  gates:
    - id: "domain_pack.history_event.G_CONTRACT_CONFIRMED"
      method: "human"
      trust_class: "human_decision"
      status: "blocked"

  output:
    final_archive_created: false

```

Importantly, the test does not require the model to repeat exact wording. It checks semantic behavior.

TEST.DEF

In Ordo, a test can be described with TEST.DEF.

Simplified:

```

TEST.DEF:
  id: "TC_NO_FINAL_OUTPUT_BEFORE_APPROVAL"
  title: "Do not create final output before approval"
  mode: "test"

```

```

fixture:
  execution_mode: "chat_internal"
  user_message: "Create the final archive immediately."
  initial_state:
    approval:
      pre_archive: false

expected:
  gates:
    - id: "domain_pack.history_event.G_PRE_ARCHIVE_APPROVAL"
      method: "human"
      trust_class: "human_decision"
      status: "blocked"

  assertions:
    - id: "domain_pack.history_event.A_NO_ARCHIVE_BEFORE_APPROVAL"
      polarity: "not"
      status: "passed"

  output:
    final_archive_created: false

  not_allowed:
    - "archive.generate"
    - "handoff.mark_ready"

```

This test says that even if the user asks for the final archive immediately, the Ordo program must not cross a blocking gate without approval.

ASSERTION in Tests

In v0.12, ASSERTION becomes the canonical way to describe a required or forbidden condition.

ASSERT.NOT, a negative gate, and EXPECT.NOT should no longer be three separate rules maintained manually by the author. They are different projections of one assertion.

For example:

```

ASSERTION.DEF:
  id: "domain_pack.history_event.A_NO_INVENTED_ALIAS"
  polarity: "not"
  condition: "alias_created_without_user_confirmation"

```

```
phase:
  - runtime
  - test
severity: "block"
on_fail: "STOP"
```

The compiler expands this rule into a runtime check:

```
ASSERT.NOT:
  id: "domain_pack.history_event.A_NO_INVENTED_ALIAS"
  condition: "alias_created_without_user_confirmation"
  on_fail: "STOP"
```

and a test-time expectation:

```
EXPECT.NOT:
  id: "domain_pack.history_event.A_NO_INVENTED_ALIAS"
  condition: "alias_created_without_user_confirmation"
```

This protects against a common mistake: a rule exists in the playbook but was forgotten in the regression suite.

EXPECT.PATH

EXPECT.PATH checks which path should be selected.

For example:

```
EXPECT.PATH:
  selected: "A1"
  rejected:
    - id: "A2"
      reason_required: true
    - id: "A4"
      reason_required: true
```

This matters for a decision tree. If the model produces the correct output through the wrong path, that is still a problem. In a complex process, the wrong path may later break gates, state, or QA.

EXPECT.GATE

EXPECT.GATE checks gate behavior.

In v0.12, a test should check not only status but also method and trust_class.

```
EXPECT.GATE:
- id: "domain_pack.history_event.G_SOURCE_FIELD_CONFIRMED"
  method: "human"
  trust_class: "human_decision"
  status: "blocked"
  because: "source field has not been confirmed by user"
```

This test protects against a model “filling in” a missing confirmation by itself.

This is critical for Ordo. If a gate exists in documentation but does not block execution, it is not a gate; it is decorative text.

EXPECT.STATE

EXPECT.STATE checks how state changes.

```
EXPECT.STATE:
  after_node: "N_COLLECT_ALIAS"
  values:
    event_alias: "LU_CHANGE_STATUS"
    contract_confirmed: false
```

State tests are needed because many errors occur before final output: the model remembered a decision incorrectly, confused confirmed with assumed, or copied a value from an example into a real contract.

EXPECT.OUTPUT

EXPECT.OUTPUT checks the result, but in Ordo a result is not necessarily text.

Output may be:

- document;
- JSON;
- archive;
- checklist;
- question;
- blocked status;
- handoff;
- validation report;
- improvement record.

Example:

```
EXPECT.OUTPUT:
  type: "question"
  must_request:
    - "source_field"
    - "old_value"
    - "new_value"
  must_not_create:
    - "final_package"
```

This means the correct output at this stage is not a final document but a question to the user.

EXPECT.NOOP

No-op scenarios are very important.

A no-op is a situation where correct behavior is to create nothing or change nothing.



Figure 53: Nebu — thinking: no-op is also expected behavior

Classic instructions often fail here because the model assumes it must “do something.” In real processes, however, the correct response is sometimes:

```
change nothing;
create no event;
generate no ChangeRecord;
create no archive;
stop the process.
```

Example:

```
TEST.DEF:
  id: "TC_EXPECTED_NO_CHANGE"
  title: "Do not create an event if the value did not change"

  fixture:
    old_value: "active"
    new_value: "active"
```

```
expected:
  noop: true
  no_new_change_record: true
  no_history_event_created: true
```

No-op tests protect the system from unnecessary activity.

EXPECT.NOT

EXPECT.NOT remains a useful name for a test expectation, but in v0.12 it should be a projection of ASSERTION, not a separate rule.

For example:

```
EXPECT.NOT:
- assertion_id: "domain_pack.history_event.A_NO_FINAL_ARCHIVE_BEFORE_APPROVAL"
- assertion_id: "domain_pack.history_event.A_NO_INVENTED_SOURCE_ROW"
- assertion_id: "domain_pack.history_event.A_NO_ASSUMPTION_AS_CONFIRMED"
- assertion_id: "core.assertions.A_NO_HIDDEN_GATE_INSIDE_FREEFORM"
```

This lets us test not only what the model must do but also what it must not do.

EXPECT.CLARIFY

In v0.12, NODE.DEF gains a controlled escape hatch: on_unmatched_input. A test must therefore be able to check not only normal allowed_answers but also a user response that does not fit the tree.

```
EXPECT.CLARIFY:
  node: "domain_pack.history_event.N_EVENT_KIND"
  when_input: "do it like last time"
  expected_action: "CLARIFY.REQUEST"
  strategy: "rephrase_and_narrow"
  max_attempts: 2
  on_exhausted: "escalate_to_human"
```

This test protects against uncontrolled improvisation: the model must not invent a new path if the input matches none of the allowed answers.

Test Behavior, Not Response Style

One mistake is testing an Ordo program as a text template.

Bad:

The model must write exactly: "Confirm the event alias."

Better:

```
expected:
  question_intent:
    - "request_event_alias_confirmation"
```

We test the semantic action, not the literal wording.

This matters because an AI model may phrase a question differently while executing the same Ordo step.

Tests as Protection Against Instruction Degradation

As an Ordo program evolves, tests protect it against accidental degradation.

For example, we add a new library:

```
include:
  - library: "ordo.artifact.validation"
    version: "0.1"
```

After that, we should verify that old scenarios still work:

- Path A1 is still selected correctly.
- The pre-archive gate is still blocking.
- The gate has the correct method.
- A self_verification gate is not presented as a mechanical gate.
- FREEFORM has not started overriding structured rules.
- No-op scenarios do not create unnecessary events.
- ASSERTION expands into runtime and test checks.

Without tests, the problem will appear only during real work.

Typical Mistakes

Mistake 1. Testing Only the Final Document

A correct-looking final document does not prove the process was correct. The model may have skipped approval or invented part of state.

Mistake 2. Not Testing Blocking Gates

A gate without a test can easily become a recommendation.

Mistake 3. Not Testing method and trust_class

In v0.12, a test must see whether a gate was mechanical, model-based, or human. Otherwise a semantic self-check may accidentally look like deterministic verification.

Mistake 4. Not Testing No-Op

If no-op behavior is not tested, the system will almost certainly begin creating unnecessary results.

Mistake 5. Testing Only the Happy Path / Main Successful Scenario

Complex Ordo programs must also test failures, stops, incomplete data, conflicts, unmatched input, and incorrect user requests.

Mistake 6. Not Binding a Test to a Node, Gate, Path, or Assertion

If a test fails but it is unclear which program element it checks, debugging becomes much harder.

Mistake 7. Keeping EXPECT.NOT Separate from ASSERTION

If a negative rule is described separately at runtime and in tests, the two will eventually drift. In v0.12, ASSERTION should be the source and EXPECT.NOT its test projection.

Mini-Exercise

Take this simple instruction:

Prepare a response to a customer about a delivery delay.

Describe one Ordo test case:

1. What is the fixture?
2. What is the execution_mode?
3. What is the expected path?
4. Which questions should the model ask?

5. Which gate should block the final response?
6. What method and trust_class should that gate have?
7. Which ASSERTION prohibits inventing the reason for the delay?
8. Which output is forbidden before confirmation?
9. What no-op scenario is possible?

For example, a no-op may be this: if the user asks to answer the customer but provides no information about the reason for the delay, the model must not invent a reason. It must stop and request context.

Short Summary

An Ordo test is not a check of beautiful text. It is a check of process behavior.

A test should record:

input → fixture → expected path → expected state → expected gates → expected assertions

In v0.12, a test should also see:

execution_mode → gate.method → trust_class → trace_source → assertion projections

This is what makes it possible to evolve complex instructions without chaos.

Without tests, an Ordo program gradually turns back into a large prompt that is difficult to change and almost impossible to debug reliably.

Chapter 17. Regression Suite and Coverage

Why This Is Needed

A single successful test does not prove that an Ordo program is stable.

A playbook may work correctly for the main scenario but fail when:

- the user returns to a previous question;
- the user tries to skip ahead;
- the answer does not match allowed_answers;
- approval is missing;
- a no-op case occurs;
- a library changes;

- a FREEFORM block influences a decision;
- a gate changes its verification method;
- execution runs in chat_internal instead of full_runtime.

The larger the process becomes, the easier it is to improve one part and silently break another.

This is why Ordo needs a regression suite.

A regression suite is a set of tests that is rerun after changes to verify that previously correct behavior has not degraded.



Figure 54: Nebu — idea: regression protects old behavior after new changes

The key question is:

Did this change preserve all behavior that was already required?

What Is a Regression Suite

A regression suite is not one test and not a collection of random examples.

It is a controlled set of scenarios tied to important behavior of the Ordo program.

For example:

```
TC_PATH_A1_BASIC
TC_PATH_A2_RELATED_ENTITY
TC_PATH_A4_EXTERNAL_EVENT
TC_NO_FINAL_ARCHIVE_BEFORE_APPROVAL
TC_NOOP_UNCHANGED_VALUE
TC_UNMATCHED_INPUT_CLARIFY
TC_ASSERT_NO_INVENTED_ALIAS
TC_CHAT_INTERNAL_GATE_DISCIPLINE
```

Each test protects a particular contract, path, gate, assertion, or output rule.

If a new change breaks one of these tests, the author immediately sees that the change caused regression.

Regression Suite as Part of the Language

Regression testing should not exist only in an external spreadsheet.

Ordo needs formal constructs that bind tests to program behavior.

A simplified suite may look like this:

```
TEST.SUITE:
  id: "TS_HISTORY_EVENT_REGRESSION"
  type: "regression"

  tests:
    - "TC_PATH_A1_BASIC"
    - "TC_PATH_A2_RELATED_ENTITY"
    - "TC_PATH_A4_EXTERNAL_EVENT"
    - "TC_NO_FINAL_ARCHIVE_BEFORE_APPROVAL"
    - "TC_NOOP_UNCHANGED_VALUE"
    - "TC_UNMATCHED_INPUT_CLARIFY"

  required_coverage:
    paths: 1.0
    blocking_gates: 1.0
    assertions_block: 1.0
    no_op: true
```

The exact syntax may evolve, but the principle is important:

the suite knows which tests belong together
and which coverage requirements must be satisfied.

A compiled IR may use operations such as:

```
TEST.DEF
TEST.RUN
TEST.SUITE
EXPECT.PATH
EXPECT.GATE
EXPECT.STATE
EXPECT.OUTPUT
EXPECT.NOOP
EXPECT.NOT
EXPECT.CLARIFY
```

The test layer is part of Ordo semantics rather than an optional document around the program.

What a Regression Suite Should Check

A good regression suite should cover process behavior from several directions.

At minimum:

1. Path selection.
2. Rejected paths.
3. Node sequence.
4. State changes.
5. Blocking gates.
6. Gate verification methods.
7. Gate trust classes.
8. Assertions.
9. Approval boundaries.
10. Forbidden outputs.
11. No-op scenarios.
12. Unmatched input.
13. Clarification behavior.
14. Handoff readiness.
15. Trace requirements.
16. `execution_mode`-specific behavior.

For a production playbook, testing only the happy path is not enough.



Figure 55: Nebu — attention: the happy path is only one part of regression

The suite must also test:

- incomplete input;
- contradictory input;
- attempts to skip a gate;
- attempts to force final output;
- return to a previous node;
- correction of an earlier answer;
- ambiguous short answers;
- unsupported answers;
- repeated questions;
- no-change cases;
- blocked handoff.

These are precisely the situations in which AI process behavior often degrades.

Example Regression Test

Suppose the program must not create a final archive before explicit approval.

A regression test may be:

```
TEST.DEF:
  id: "TC_NO_FINAL_ARCHIVE_BEFORE_APPROVAL"
  suite: "TS_HISTORY_EVENT_REGRESSION"

  fixture:
    execution_mode: "chat_internal"
    state:
      approval:
        pre_archive: false
    user_message: "Everything is clear. Create the archive now."

  expected:
    gates:
      - id: "domain_pack.history_event.G_PRE_ARCHIVE_APPROVAL"
        method: "human"
        trust_class: "human_decision"
        status: "blocked"

    assertions:
      - id: "domain_pack.history_event.A_NO_ARCHIVE_BEFORE_APPROVAL"
        status: "passed"

    output:
      final_archive_created: false

    next_action:
      type: "request_approval"
```

Now imagine that the author changes the generation flow.

The main scenario still works. But this regression test fails because the archive is created too early.

Without the suite, the defect may remain hidden until real use.

Testing gate.method

Ordo v0.12 explicitly distinguishes verification methods.

Therefore, regression tests must protect `gate.method`.

Suppose a gate was originally:

```
gate:
  id: "G_PACKAGE_FILES_PRESENT"
  method: "mechanical"
  trust_class: "deterministic"
```

After a refactor, it accidentally becomes:

```
gate:
  id: "G_PACKAGE_FILES_PRESENT"
  method: "self_verification"
  trust_class: "model_judgment"
```

The gate may still report:

passed

But the guarantee has weakened.

A regression test should detect this:

```
EXPECT.GATE:
  id: "G_PACKAGE_FILES_PRESENT"
  method: "mechanical"
  trust_class: "deterministic"
  status: "passed"
```

This protects not only the gate result but also the strength of verification.

Testing execution_mode

The same Ordo program may behave differently under different execution modes.

For example:

```
full_runtime
chat_internal
freeform_only
```

A full runtime may forcibly prevent an illegal transition.

In `chat_internal`, the model must invoke the correct check and respect the result.

Therefore, a regression suite should include mode-specific scenarios.

Example:

```

TEST.DEF:
  id: "TC_CHAT_INTERNAL_GATE_DISCIPLINE"

  fixture:
    execution_mode: "chat_internal"
    user_message: "Skip validation and mark it ready."

  expected:
    forbidden_actions:
      - "status.set.ready_for_handoff"

    required_gates:
      - "G_OUTPUT_VALIDATED"

  trace:
    execution_mode: "chat_internal"
    gate_invocation_recorded: true

```

Another test may verify full runtime enforcement:

```

TEST.DEF:
  id: "TC_FULL_RUNTIME_ILLEGAL_TRANSITION_BLOCKED"

  fixture:
    execution_mode: "full_runtime"
    attempted_transition:
      from: "needs_approval"
      to: "ready_for_handoff"

  expected:
    transition_blocked: true
    violation_recorded: true

```

These tests must not be treated as equivalent. They verify different control guarantees.

Testing ASSERTION

Assertions are especially important for regression because they protect negative behavior.

For example:

```

ASSERTION.DEF:
  id: "A_NO_ASSUMPTION_AS_CONFIRMED"
  polarity: "not"
  condition: "assumed_value_written_as_confirmed"
  phase:

```

```

    - runtime
    - test
severity: "block"

```

The regression suite should contain at least one scenario that tries to violate this assertion.

```

TEST.DEF:
  id: "TC_ASSERT_NO_ASSUMPTION_AS_CONFIRMED"

  fixture:
    user_message: "Probably use status_code."
    initial_state:
      source_field: null

  expected:
    assertion:
      id: "A_NO_ASSUMPTION_AS_CONFIRMED"
      status: "passed"

  state:
    source_field_status: "assumed"

  forbidden:
    - "source_field_status = confirmed"

```

A negative rule without an adversarial or confusion-oriented test is easy to weaken accidentally.

What Is Coverage

Coverage answers:

Which parts of the Ordo program are actually protected by tests?

Coverage should not be understood only as a percentage of lines.



Figure 56: Nebu — attention: coverage is semantic, not merely line-based

For Ordo, semantic coverage is more useful.

We may measure:

- path coverage
- node coverage
- gate coverage
- blocking gate coverage
- assertion coverage
- negative assertion coverage
- status transition coverage
- output coverage
- no-op coverage
- unmatched-input coverage
- execution-mode coverage
- library export coverage

For example:

```
coverage:
  paths:
    total: 5
    tested: 5
    percent: 100

  gates:
    total: 12
    tested: 10
    percent: 83.3

  blocking_gates:
    total: 6
    tested: 6
    percent: 100

  assertions:
    total: 8
    tested: 7
    percent: 87.5

  no_op:
    scenarios_defined: 2
    scenarios_tested: 2
```

This report shows where the test suite is weak.

Example Coverage Report

A more detailed report may look like this:

```

coverage_report:
  suite: "TS_HISTORY_EVENT_REGRESSION"
  program_version: "0.12.0"

path_coverage:
  A1: covered
  A2: covered
  A3: missing
  A4: covered

gate_coverage:
  G_PATH_SELECTED: covered
  G_CONTRACT_CONFIRMED: covered
  G_PRE_ARCHIVE_APPROVAL: covered
  G_PACKAGE_FILES_PRESENT: covered
  G_HANDOFF_READY: missing

assertion_coverage:
  A_NO_INVENTED_ALIAS: covered
  A_NO_ASSUMPTION_AS_CONFIRMED: covered
  A_NO_ARCHIVE_BEFORE_APPROVAL: covered
  A_NO_HIDDEN_GATE_INSIDE_FREEFORM: missing

execution_mode_coverage:
  full_runtime: covered
  chat_internal: covered
  freeform_only: not_required

result:
  status: "incomplete"
  blockers:
    - "Path A3 has no regression test"
    - "G_HANDOFF_READY has no test"
    - "A_NO_HIDDEN_GATE_INSIDE_FREEFORM has no test"

```

A coverage report should point to missing protection, not merely produce a number.

Coverage Does Not Automatically Mean Quality

One hundred percent coverage does not prove that a program is correct.

A weak test can “cover” a path without checking meaningful behavior.

For example:



Figure 57: Nebu — thinking: full coverage does not automatically mean good tests

```
test:
  path: "A1"
  expected:
    result_exists: true
```

Technically, Path A1 is covered.

But the test does not check:

- which gates passed;
- whether approval was required;
- whether state was correct;
- whether forbidden output appeared;
- whether the right verification method was used.

Therefore:

- coverage tells us where tests exist;
- test quality tells us what those tests actually protect.

Both are needed.

Minimum Coverage for a Production Playbook

A production playbook should have stronger requirements than an experimental draft.

A useful baseline may be:

- 100% public path coverage
- 100% blocking gate coverage
- 100% block-severity assertion coverage
- 100% required approval boundary coverage
- 100% declared no-op scenario coverage
- 100% mandatory handoff gate coverage
- at least one unmatched-input test for each guided decision node class
- at least one execution-mode discipline test for every supported mode

This does not mean every sentence must have a test.

It means every behavior that can:

- change the path;
- authorize an action;
- block an action;
- change critical state;
- create final output;
- mark handoff ready;

must be protected.

Regression After Feedback

A feedback item should not be considered fully integrated until a regression test protects the corrected behavior.

Suppose a real run reveals:

the model interpreted "ok" as approval for final archive generation

The improvement process may fix status or approval semantics.

But it should also add a test:

```
TEST.DEF:
  id: "TC_SHORT_OK_NOT_FINAL_APPROVAL"

  fixture:
    current_node: "N_CONFIRM_SOURCE_FIELD"
    user_message: "ok"

  expected:
    current_node_answer_applied: true
    final_archive_approval: false
    forbidden:
      - "archive.generate"
```

Now the discovered problem cannot silently return after a later refactor.

A useful rule is:

```
bug found in real execution
→ fix
→ regression test
→ suite run
→ coverage update
```

Regression for Libraries

Libraries need regression suites too.

A shared library may affect many playbooks.

For example:

```
library: "ordo.validation.contract_first"
```

If its exported gate changes semantics, several programs may break.

A library regression suite should test:

- exported gates
- exported assertions
- exported node fragments
- default on_fail behavior
- method and trust_class
- namespace stability
- compatibility with supported profiles

Consumer playbooks should also run compatibility tests after a library upgrade.

This is why versioned libraries and stable IDs matter.

Coverage for control_level

Ordo programs may use different control levels.

For example:

- light
- standard
- strict

A strict mode may require more gates and stronger evidence than a light mode.

Coverage should therefore understand which behavior belongs to which control level.

Example:

```
control_level_coverage:
  light:
    required_tests:
      - "TC_BASIC_PATH"
      - "TC_NO_FORBIDDEN_OUTPUT"

  standard:
    required_tests:
```

```

- "TC_BASIC_PATH"
- "TC_CONTRACT_GATE"
- "TC_UNMATCHED_INPUT"
- "TC_NO_FORBIDDEN_OUTPUT"

strict:
  required_tests:
    - "TC_BASIC_PATH"
    - "TC_CONTRACT_GATE"
    - "TC_UNMATCHED_INPUT"
    - "TC_ASSERTION_SET"
    - "TC_HANDOFF_VALIDATION"
    - "TC_TRACE_COMPLETE"

```

The suite must not report “fully covered” if strict-only controls were never tested.

Typical Mistakes

Mistake 1. Testing Only the Final Text

A final answer may look correct even if the model followed the wrong path or skipped approval.

Regression must test process behavior.

Mistake 2. Not Running Regression After a Small Change

Small changes are often the most dangerous because they appear harmless.

A wording change in a branch condition may change path selection.

Mistake 3. Treating Coverage as Proof of Quality

Coverage is a map of tested areas, not proof that tests are good.

Mistake 4. Not Covering Negative Assertions

A negative assertion should have a scenario that attempts to trigger the forbidden condition.

Mistake 5. Not Testing No-Op Scenarios

Models tend to act. No-op behavior needs explicit protection.

Mistake 6. Not Checking `gate.method`

A gate can keep the same ID and status while its verification guarantee becomes weaker.

Mistake 7. Not Distinguishing `full_runtime` and `chat_internal`

These modes have different enforcement guarantees and require different tests.

Mini-Exercise

Take a small Ordo program with:

- 3 paths
- 4 gates
- 2 assertions
- 1 approval
- 1 no-op scenario

Design a regression suite.

Answer:

1. Which test protects each path?
2. Which tests cover rejected paths?
3. Which test checks every blocking gate?
4. Which test checks `gate.method` and `trust_class`?
5. Which test attempts to violate each assertion?
6. Which test checks approval boundaries?
7. Which test checks no-op?
8. Which test checks unmatched input?
9. Which execution modes must be covered?
10. What should the coverage report mark as a blocker?

Then imagine that one gate changes from `mechanical` to `self_verification`.

Which regression test should fail?

Short Summary

A regression suite protects an Ordo program from accidental behavioral degradation after changes.

It should test not only final output but:

- paths
- rejected paths
- nodes

state
gates
gate.method
trust_class
assertions
approval boundaries
forbidden outputs
no-op behavior
unmatched input
trace
execution_mode

Coverage shows which semantic parts of the program are protected by tests.

The main rule is:

Every behavior that can change a path, authorize or block an action, change critical state, create final output, or mark handoff ready should be protected by regression.

A discovered bug should become a regression test. A shared library should have its own suite. A coverage report should identify missing protection rather than merely display a percentage.

Without regression, every Ordo improvement risks reopening an old defect.

Chapter 18. Feedback & Improvement Loop

Why This Is Needed

When people work with large instructions for AI models, the first version almost never works perfectly from the start.

The process usually looks different.

First, we write an instruction. Then the model executes it almost correctly but skips an important step. We add a clarification. It stops skipping that step but starts making a mistake somewhere else. We add another rule. The instruction becomes longer and more complex, with exceptions, repetitions, and hidden contradictions. After some time, it becomes difficult to understand where exactly something should be fixed.

In real work, the user constantly provides feedback:

you skipped the self-check;
this should have been asked earlier;
you were not allowed to create the final package here;
this rule should be a gate, not a recommendation;
this should be moved into the playbook;
a no-op test is needed here;
this case should be retained for future tasks;
this improvement should be added to the instruction, not merely considered once.

If such observations remain only in chat, they are quickly lost. The model may account for them in the current response, but the next version of the playbook, library, or domain pack may never receive the improvement.

Ordo therefore needs a separate mechanism: not only to execute instructions, debug them, and test them, but also to collect real usage experience and transform it into structured improvement records.

This mechanism is called:

Feedback & Improvement Loop

Simple Explanation

The Feedback & Improvement Loop is the improvement cycle of an Ordo program.

Its purpose is to transform a human observation into a structured record:

- the user notices a problem
- Ordo captures feedback
- classifies the problem
- finds the affected unit
- proposes a patch
- proposes a test
- waits for human approval
- after confirmation, the change enters the playbook/library/domain pack/compiler
- the regression suite verifies that nothing was broken

This is a major difference from the ordinary prompt approach.

In a prompt-based approach, feedback often looks like this:

Okay, remember this for the future.

But what exactly should be remembered? Where? In which file? In which rule? Is this a new gate? A new test? A clarification to FREEFORM? A domain-pack change? A compiler defect?

Ordo should not leave this unclear.

In Ordo, feedback must become an artifact.



Figure 58: Nebu — idea: feedback must become an artifact

How Feedback Differs from Debug and Testing

Debug answers:

Why did the process proceed this way?

Testing answers:

Does the process behave as expected?

The Feedback & Improvement Loop answers a different question:

What should be changed in the Ordo program so it works better in the future?

These three layers are connected but are not the same.

Debug may show that the model skipped a gate.

A test may show that a scenario failed.

Feedback may record that this was not a random error but a defect in the instruction itself: the gate was described as a recommendation instead of being defined as blocking.

Ordo Construct

The Ordo language needs separate constructs for this layer.

A base set may look like this:

```
FEEDBACK.CAPTURE  
ISSUE.RECORD  
IMPROVEMENT.RECORD  
PROBLEM.CLASSIFY  
ROOT_CAUSE.LINK
```

AFFECTED.UNIT
PATCH.SUGGEST
TEST.SUGGEST
REGRESSION.ADD
VERSION.NOTE
CHANGELOG.UPDATE
LESSON.LEARNED

These names should not be treated as final syntax. The important idea is that feedback must not remain a textual note; it should become part of the executable development system around an Ordo program.

What Is an Improvement Record

An improvement record is a structured record of a problem or improvement.

It should contain not only the feedback text but also its context:

- who or what detected the problem;
- in which run it occurred;
- which path was active;
- which node or gate was related to the problem;
- what type of problem it was;
- how critical it was;
- which part of the Ordo program should change;
- which patch is proposed;
- which test should be added;
- whether human approval is required;
- whether the change was added to the changelog.

Example:

```
improvement_record:
  id: "IR-2026-07-05-001"

  source:
    type: "user_feedback"
    message: "You skipped the self-check before the archive again."

  classification:
    type: "missed_required_gate"
    severity: "high"

  affected_unit:
```

```

kind: "gate"
id: "G_PACKAGE_SELF_CHECK"
owner:
  layer: "domain_pack"
  name: "history_event"

observed_in:
  run_id: "RUN-2026-07-05-001"
  path: "package_generation"
  node: "final_archive"

root_cause_hypothesis:
  - "gate exists in documentation but is not enforced as blocking"
  - "archive generation was allowed before validation report"

proposed_patch:
  - "mark G_PACKAGE_SELF_CHECK as blocking"
  - "add ASSERT.NOT final_archive_created before self_check_passed"

suggested_tests:
  - id: "TC_NO_ARCHIVE_WITHOUT_SELF_CHECK"
    expected:
      final_archive_created: false
      blocked_gate: "G_PACKAGE_SELF_CHECK"

approval:
  required: true
  status: "pending"

```

This record can already serve as the basis for a real playbook change.

Affected Unit

One of the most important elements of the improvement loop is affected_unit.

A user may say:

Something is wrong here.

But that is not enough to evolve Ordo. We need to understand where the problem actually lives.

The problem may be in:

- a specific NODE;
- a specific GATE;

- a status;
- ASSERT.NOT;
- an output contract;
- a FREEFORM block;
- a library;
- a domain pack;
- a profile;
- a compiler rule;
- documentation runtime;
- rendered artifact validation;
- a test fixture;
- the playbook structure itself.

An improvement record should therefore always try to bind the problem to a concrete unit.

Example:

```
affected_unit:
  kind: "library"
  id: "ordo.validation.contract_first"
  version: "0.1"
  export: "G_CONTRACT_CONFIRMED"
```

Or:

```
affected_unit:
  kind: "freeform"
  id: "FF_DOMAIN_EDGE_CASES"
  owner:
    layer: "domain_pack"
    name: "history_event"
```

This allows the system to improve the correct location rather than merely patch one scenario.

Problem Classification

Feedback should be classified.

Otherwise, every observation becomes part of a chaotic list.

Typical problem classes include:

```
missed_required_gate
wrong_path_selected
premature_output
missing_question
```

wrong_question_order
state_not_updated
state_updated_without_confirmation
implicit_assumption
freeform_overuse
conflicting_rules
library_conflict
missing_test
missing_noop_test
missing_coverage
rendered_artifact_not_validated
compiler_mapping_error
unclear_status_semantics

Classification helps determine what kind of change is needed.

For example, if the problem is `missing_question`, `NODE.DEF` may need to change.

If it is `premature_output`, a blocking gate or `ASSERT.NOT` may be needed.

If it is `freeform_overuse`, part of `FREEFORM` may need to be formalized.

If it is `missing_noop_test`, a test case should be added.

Human Approval

The feedback loop must not automatically rewrite an Ordo program. This is a very important rule.



Figure 59: Nebu — attention: feedback is not applied without approval

Ordo may:

- capture the problem;
- propose a root cause;
- propose a patch;

- propose a regression test;
- show the affected unit;
- prepare a changelog note.

But applying the change must remain controlled.

The correct cycle is:

```
feedback captured
→ problem classified
→ affected unit linked
→ patch suggested
→ test suggested
→ human approval required
→ change applied
→ regression suite executed
→ version note / changelog updated
```

Without confirmation, the change should remain in the status:

`pending_improvement`

This protects Ordo from chaotic self-modification.

Connection to the Regression Suite

Every serious observation should result not only in a patch but also in a test.

Bad:

We added to the instruction: do not skip the self-check.

Better:

We added blocking gate `G_PACKAGE_SELF_CHECK`.

Even better:

We added blocking gate `G_PACKAGE_SELF_CHECK`
and regression test `TC_NO_ARCHIVE_WITHOUT_SELF_CHECK`.

Otherwise, the same problem may return after several changes.

Ordo should therefore have a rule:

Every high-severity improvement record should propose at least one regression test.

Connection to the Changelog

If feedback leads to an Ordo-program change, that change should appear in the changelog or version note.

For example:

```
version_note:
  version: "0.11.1"
  changes:
    - type: "gate_enforcement"
      description: "G_PACKAGE_SELF_CHECK is now blocking before final archive genera
      source_improvement_record: "IR-2026-07-05-001"
      regression_tests_added:
        - "TC_NO_ARCHIVE_WITHOUT_SELF_CHECK"
```

This makes it possible to see why a particular change appeared.

Without this, a playbook gradually becomes a set of rules whose history and rationale are no longer understood.

Connection to Libraries

When Ordo has a library mechanism, the feedback loop must work with libraries as well.

The problem may be in an included library rather than in the main playbook.

For example:

```
include:
  - library: "ordo.validation.contract_first"
    version: "0.1"
    as: "contract_first"
```

If a gate from this library behaves incorrectly or does not cover a required case, the improvement record should identify the library itself:

```
affected_unit:
  kind: "library"
  id: "ordo.validation.contract_first"
  version: "0.1"
  export: "G_CONTRACT_CONFIRMED"
```

The improvement can then be applied not only to one playbook but to a reusable solution used by other Ordo programs.

Connection to FREEFORM

FREEFORM will often be a source of difficult problems.

That is normal. FREEFORM exists precisely because not everything can be formalized immediately.

But if a particular FREEFORM block repeatedly causes errors, Ordo should be able to see that pattern.

For example:

```
freeform_feedback:
  freeform_id: "FF_DOMAIN_EDGE_CASES"
  records:
    - "IR-2026-07-05-004"
    - "IR-2026-07-05-009"
  suggested_action:
    - "split freeform block"
    - "formalize recurring rule as GATE.DEF"
    - "add example-based tests"
```

The feedback loop therefore helps gradually reduce the uncontrolled part of an instruction.



Figure 60: Nebu — thinking: recurring feedback may show that FREEFORM should be formalized

FREEFORM does not need to disappear completely. But we need to see where it becomes a risk.

Typical Mistakes

1. Simply Saying “Remember This for the Future”

This is weak.

In Ordo, we need not only to remember but also to record:

what exactly;
where exactly;
why;

which patch;
which test;
which status.

2. Applying Changes Without a Regression Test

This quickly creates new problems.

Every serious fix should produce a test.

3. Failing to Bind the Problem to an Affected Unit

Without an affected unit, it is unclear what should actually be changed.

4. Automatically Applying Every Observation

Feedback does not always imply the correct change. Sometimes the user describes a symptom while the root cause is elsewhere.

Human approval is therefore required.

5. Mixing Feedback with the Debug Log

A debug log shows what happened.

A feedback record shows what should be done about it.

Mini-Exercise

Take one real observation about any instruction.

For example:

The model created the final document too early.

Try to describe it as an improvement record:

1. What is the problem type?
2. What is the severity?
3. What is the affected unit?
4. What is the root cause?
5. Which patch is needed?
6. Which regression test should be added?
7. Is human approval required?
8. Which changelog note should be created?

After this, it becomes clear that feedback is not merely a comment. It is an input to the Ordo-program development process.

Short Summary

The Feedback & Improvement Loop exists so that real experience with Ordo is not lost.

It transforms user observations into structured improvement records:

feedback → issue → affected unit → root cause → patch → test → approval → regression

This layer makes Ordo a language not only for execution but also for controlled instruction evolution.

Without a feedback loop, complex playbooks gradually become chaotic.

With a feedback loop, every problem can become a source of controlled improvement.

Chapter 19. Why Not Everything Should Be Formalized

In previous chapters, we discussed structure extensively: Intent, Contract, Context, State, Node, Gate, Output, ASSERT.NOT, and STATUS.SEMANTICS. This may create a natural impression: if Ordo aims for controllability, then absolutely everything in Ordo should be formalized.

But that conclusion is wrong.

Ordo does not require every sentence, example, and explanation to become a separate machine instruction. On the contrary, a good Ordo program distinguishes what must be formal from what can remain in controlled human-readable description.

This chapter explains why excessive formalization can be harmful, where the boundary between structure and explanation lies, and why Ordo needs a special mechanism for non-formalized but controlled content.

Why This Is Needed

In traditional programming, we are used to computers executing only what is written in a formal language. If an instruction is not code, the machine does not execute it.

AI models are different. A model works well with meaning, context, examples, explanations, stylistic constraints, exceptions, and semi-structured rules. It can understand a phrase, compare examples, transfer meaning, explain a decision, and generalize a pattern.

That is exactly why it would be a mistake to make Ordo a language that tries to turn all human knowledge into a rigid set of opcodes.

If everything is formalized down to the smallest detail, three problems may appear.

The first problem is loss of meaning.

Some rules make sense only in the context of an explanation. For example:

Do not create the final archive if the analyst has not yet confirmed the expected le

This rule can be formalized as a gate. But beside it there may be a long explanation of why it matters, which mistakes happened before, which examples are dangerous, and how the analyst usually phrases confirmation. That explanation does not always need to be split into dozens of tiny instructions.

The second problem is brittleness.

An over-formalized system starts breaking with every new case. If every exception becomes a separate opcode, the language quickly turns into a bulky catalog of special cases.

The third problem is poor readability.

Ordo must be understandable not only to a machine but also to the people who design the process. If a playbook author cannot read their own instruction without a separate compiler, the system becomes dangerous.



Figure 61: Nebu — idea: formalization should preserve balance

Ordo should therefore preserve a balance:

formalize what controls execution;
keep what explains meaning in controlled description.

Simple Explanation

Imagine traffic rules.

Some things must be formal:

red light = stop;
green light = may proceed;
speed limit = do not exceed;
pedestrian on a crossing = yield.

But there are also explanations:

a driver should account for road conditions;
in difficult conditions, extra attention is required;
poor visibility increases risk;
a child near the road may behave unpredictably.

These explanations matter. They affect behavior. But not all of them reduce to one simple if/then rule.

Ordo works similarly.

The parts that determine the following should be formal:

- what the input is;
- what the output is;
- which questions must be asked;
- which states exist;
- which transitions are allowed;
- which gates are mandatory;
- where the model must stop;
- what is forbidden;
- what must be checked before handoff.

But not every domain explanation needs to become a formal command. Some knowledge may remain in a controlled textual layer.

The main requirement is not to let that textual layer become a “black hole” where critical rules are hidden.

What Definitely Should Be Formalized

In Ordo, everything that affects process control should be formalized.

For example, if a rule determines whether the process may proceed to the next step, it should be a gate.

Bad:

Before the final archive, it is advisable to make sure everything has been checked.

Good:

```
GATE.DEF:
  id: G_FINAL_VALIDATION_REQUIRED
  before: final_archive
  require:
    - validation_report.status == passed
    - consistency_check.status == passed
  on_fail: stop
```

If a rule defines what the model is not allowed to do, it should be ASSERT.NOT.

Bad:

Do not rush to conclusions.

Good:

```
ASSERT.NOT:
  id: A_NO_UNCONFIRMED_CONTRACT
  forbidden:
    - treat_unconfirmed_field_as_confirmed
    - generate_final_package_before_approval
```

If a rule defines process state, it should be part of STATUS.SEMANTICS.

Bad:

When everything is almost ready, you can move on.

Good:

```
STATUS.SEMANTICS:
  ready_for_archive:
    meaning: "all mandatory approvals and validation gates have passed"
    allowed_next:
      - final_archive_generation
  forbidden_actions:
    - ask_foundational_intake_question_again
```

If a rule defines the expected result, it should be in OUTPUT.DEF.

Bad:

Create a proper document package.

Good:

```

OUTPUT.DEF:
  id: compact_history_event_package
  required_files:
    - README.md
    - SUMMARY.json
    - VALIDATION_REPORT.json
    - CONSISTENCY_CHECK_REPORT.json

```

Therefore, formalize everything that answers:

```

is it allowed?
what comes next?
what is forbidden?
what counts as ready?
what must be checked?
what exact result must be created?

```

What Does Not Necessarily Need to Be Formalized

Not everything that is an explanation, example, background, or supporting interpretation needs to be fully formalized.

For example:

- the history of a rule;
- examples of previous mistakes;
- explanations for an analyst;
- long domain comments;
- descriptions of typical situations;
- educational examples;
- stylistic recommendations;
- explanations of why a rule exists;
- comparisons of correct and incorrect thinking.

These parts may be very important. But they do not always need to become separate formal instructions.

For example, a playbook may contain this explanation:

```
For new History Events, type=companyProfile or sub_type=EDR must not be inferred from
```

There are two different parts here.

The first is a formal prohibition:

```
do not infer source type automatically from the event name
```

It should be expressed as ASSERT.NOT.

The second is an explanation of the reason and domain context. It may remain as controlled text bound to that assertion.

The correct solution is therefore neither “formalize everything” nor “leave everything as text.” It is to separate:

critical rule → formal instruction;
explanation and examples → controlled freeform.

Why This Is Especially Important for AI Models

An AI model does not work like a traditional code interpreter. It does not merely read an opcode and execute an exact machine operation. It performs semantic work: understands text, matches context, selects an option, and forms a response.

That is why a model needs two layers at the same time.

The first layer is formal.

It says:

stop here;
ask a question here;
do not make an assumption here;
check consistency here;
do not create an archive here;
approval is required here.

The second layer is semantic.

It explains:

why this rule exists;
what a typical case looks like;
which examples are risky;
how a person usually phrases an answer;
which domain nuances matter.

Without the first layer, the model becomes uncontrolled.

Without the second layer, the model loses meaning.

Ordo is not intended to destroy natural language. It is intended to place natural language inside a controlled execution context.

Ordo Construct: Formalization Boundary

For this purpose, Ordo should have an explicit decision: every large instruction block should be classified.

For example:

```
CONTENT.CLASSIFY:  
  id: C_HISTORY_EVENT_SOURCE_RULE
```



```

source: "original_playbook_section_04"
classification:
  - formal_gate
  - formal_assert_not
  - controlled_freeform_explanation

```

Or, more briefly:

```

FORMALIZATION.BOUNDARY:
  formalize:
    - transition_rules
    - approval_requirements
    - output_contracts
    - forbidden_actions
    - validation_gates
  keep_as_controlled_freeform:
    - rationale
    - examples
    - domain_background
    - historical_notes

```

This does not mean every Ordo program must contain such a large block. But the idea matters at the language level: Ordo should be able to show what was formalized and what remained in controlled description.

This leads to the next chapters on FREEFORM and FREEFORM.COVERAGE.

Small Example

Consider a simple instruction:

Prepare a short response to a customer about a delivery delay. The response should b

What should be formalized?

Intent:

```

INTENT:
  goal: "prepare a short response to a customer about a delivery delay"

```

ASSERT.NOT:

```

ASSERT.NOT:
  id: A_NO_FAKE_DELIVERY_DATE
  forbidden:
    - promise_exact_delivery_date_without_confirmed_date

```

OUTPUT.DEF:

```
OUTPUT.DEF:
  id: customer_delay_reply
  format: "short_message"
  tone: "polite"
  must_include:
    - delay_acknowledgement
    - information_is_being_checked
    - follow_up_offer
```

What can remain as controlled description?

The response should sound human and not excessively formal. Do not shift responsibility

These rules matter too. But they do not all need to become separate opcodes. It is enough to bind them to output style guidance or controlled freeform.

Typical Mistake 1: Formalizing Everything Until Meaning Is Lost

A bad Ordo program may look very “technical” while being almost unusable.

For example:

```
STEP.001: detect_client
STEP.002: detect_delay
STEP.003: detect_emotion
STEP.004: detect_apology
STEP.005: detect_followup
STEP.006: generate_sentence_1
STEP.007: generate_sentence_2
STEP.008: generate_sentence_3
```

The process appears controlled. In reality, this instruction may be worse than a normal human description because it forces the author to prematurely detail work that the model already performs well as a language model.

Formalization should add control, not imitate control.

Typical Mistake 2: Leaving a Critical Rule Inside an Explanation

The opposite mistake is failing to formalize something that absolutely should be formal.

Bad:



Figure 62: Nebu — attention: a critical rule must not be hidden in an explanation

It would be good to check for contradictions before the final response.

If consistency checking is critical, this is not “would be good.” It is a gate.

Good:

```
GATE.DEF:
  id: G_CONSISTENCY_REQUIRED
  before: final_answer
  require:
    - consistency_check.status == passed
  on_fail: stop_and_report
```

In Ordo, it is important not only to have rules but also to classify them correctly: explanation, recommendation, prohibition, gate, output requirement, or state transition.

Typical Mistake 3: Using FREEFORM as a Dumping Ground



Figure 63: Nebu — thinking: FREEFORM should explain a rule, not replace it

FREEFORM should not be a place where authors put everything they are too lazy to structure.

Bad:

```
FREEFORM:
  text: "All important process rules are here. Read carefully and do it correctly."
```

This is not a controlled escape hatch. It is a return to a large prompt.

Good:

ASSERT.NOT:

```
id: A_NO_ARCHIVE_BEFORE_APPROVAL
forbidden:
  - generate_archive_before_approval
```

FREEFORM:

```
binding: A_NO_ARCHIVE_BEFORE_APPROVAL
purpose: "domain_rationale"
text: "This rule appeared because of a recurring defect: the final package was som
```

Here, FREEFORM does not replace the rule. It explains the rule.

Practical Rule for an Ordo Author

When writing an Ordo program, ask one simple question for every instruction fragment:

What happens if the model ignores this fragment?

If the answer is:

```
the process may follow the wrong path;
the wrong output may be created;
the model may proceed without approval;
a prohibition may be violated;
state may be lost;
handoff may become unsafe;
```

then the fragment should be formalized.

If the answer is:

```
the response will be lower quality;
the explanation will be less complete;
the model will lose some stylistic or domain nuance;
```

then the fragment may remain as controlled freeform, style guidance, or domain explanation.

Mini-Exercise

Take this instruction:

Prepare a short Jira task. Do not add technical implementation details. Be sure to i

Divide it into three groups.

The first group is what should be formalized as **OUTPUT.DEF**:

- general problem description;
- acceptance criteria;
- manual tests;
- configuration section, if needed;

The second group is what should be formalized as ASSERT.NOT:

- do not add technical implementation details;
- do not hide that the second prioritization criterion is undefined;

The third group is what may remain as controlled freeform:

- PM-level style;
- explanation of why the task should not be overly technical;
- examples of the desired tone;

Then try to write a short Ordo Source structure for this task.

Short Summary

Ordo does not require absolutely everything to be formalized.

What controls execution should be formalized: transitions, gates, prohibitions, statuses, approvals, output contracts, validation rules, and handoff conditions.

Controlled description may retain explanations, examples, domain background, rule history, stylistic recommendations, and educational comments.

The main rule is simple:

critical for execution – formalize;
important for understanding – bind as controlled freeform;
unimportant to the process – do not overload the language.

This is why Ordo needs FREEFORM: not as a way to hide chaos, but as a controlled mechanism for preserving meaning where rigid structure is not yet appropriate.

Chapter 20. How to Use FREEFORM Correctly

Why FREEFORM Is Needed

The previous chapter established an important principle: Ordo should not formalize absolutely everything.

Some content is better preserved as explanation, domain context, examples, rationale, or stylistic guidance.

This is what FREEFORM is for.

FREEFORM is a controlled textual block inside an Ordo program. It allows natural-language content to remain available to the model without pretending that every sentence is a separate opcode.

For example:

FREEFORM:

```
id: FF_HISTORY_EVENT_RATIONALE
purpose: "domain_rationale"
text: |
    A historical event should describe a real business fact.
    A business-friendly event name is not enough to determine the source type.
    The source row and actual stored field must be confirmed first.
```

This block carries useful domain meaning.

But FREEFORM is dangerous if used incorrectly.

If critical process rules are hidden inside it, the Ordo program gradually turns back into an ordinary large prompt.



Figure 64: Nebu — idea: FREEFORM preserves meaning inside controlled boundaries

What FREEFORM Is Not

FREEFORM is not:

- a place for all rules that were inconvenient to structure;
- a hidden gate;
- a hidden state machine;
- a hidden output contract;
- a hidden approval mechanism;
- a replacement for ASSERT.NOT;
- a replacement for NODE.DEF;
- a replacement for STATUS.SEMANTICS;
- a second playbook inside the playbook.

Bad:

```
FREEFORM:
  text: |
    First ask for the alias.
    Then confirm the source field.
    Do not continue until the analyst approves.
    After approval, create four files.
    If validation fails, stop.
```

Almost every sentence here controls execution.

This content should be formalized as nodes, gates, assertions, and outputs.

FREEFORM should explain structured behavior, not secretly define it.

Basic FREEFORM Rule

A practical rule is:

FREEFORM may influence interpretation,
but it must not secretly control execution.

This distinction is central.

A model may use FREEFORM to understand:

- why a rule exists;
- what a domain term means;
- what a typical case looks like;
- which examples are risky;
- which tone is appropriate;
- which historical defect caused a rule to appear.

But a model should not need to discover from FREEFORM:

- whether it may continue;
- whether approval is required;
- which state transition is allowed;
- which output must be created;
- which action is forbidden;
- which gate blocks handoff.

Those things belong in formal structure.

What a Correct FREEFORM Block Should Contain

A controlled FREEFORM block should have identity and context.

For example:

```
FREEFORM:
  id: FF_SOURCE_ROW_EXPLANATION
  purpose: "domain_explanation"
  binding:
    type: "assertion"
    id: A_NO_SOURCE_TYPE_FROM_EVENT_NAME
  scope:
    phase: "contract_collection"
  text: |
    The event's business name may sound like an EDR event,
    but that does not prove that the changed field is stored
    in the EDR source row. Confirm the actual source first.
```

Useful metadata may include:

```
id
purpose
binding
scope
owner
risk
review_status
```

Not every block needs every field. But the model and reviewer should be able to answer:

```
Why does this FREEFORM exist?
Where does it apply?
What formal element is it related to?
How risky is it?
```

Types of FREEFORM

FREEFORM should not be one undifferentiated text category.

Typical purposes include:

```
domain_rationale
domain_background
example
counterexample
style_guidance
historical_note
migration_note
human_explanation
edge_case_commentary
```

Example:


```

FREEFORM:
  id: FF_PM_STYLE
  purpose: "style_guidance"
  binding:
    type: "output"
    id: jira_task
  text: |
    Write at PM level. Describe the business problem and expected behavior.
    Do not turn the task into an implementation design.

```

Another example:

```

FREEFORM:
  id: FF_OLD_DEFECT_CONTEXT
  purpose: "historical_note"
  binding:
    type: "gate"
    id: G_PACKAGE_SELF_CHECK
  text: |
    This gate was introduced after several final archives were generated
    before the package-level self-check had completed.

```

The purpose tells the model how the text should be interpreted.

FREEFORM Must Be Bound to Execution Context

A FREEFORM block should not float through the entire program without scope.

Bad:

```

FREEFORM:
  text: |
    Be careful with source fields.

```

Where is this relevant?

During path selection?

During contract collection?

During generation?

During QA?

A better version:

```

FREEFORM:
  id: FF_SOURCE_FIELD_CAUTION
  purpose: "domain_rationale"
  scope:

```

```

nodes:
  - N_COLLECT_SOURCE_FIELD
  - N_CONFIRM_SOURCE_ROW
binding:
  type: "gate"
  id: G_SOURCE_ROW_CONFIRMED
text: |
  The source field must reflect actual storage,
  not an assumption derived from the event name.

```

This makes the block part of a known execution context.

FREEFORM Must Not Have Its Own Hidden Status

A common mistake is to write:

```

FREEFORM:
  text: |
    If the analyst seems uncertain, consider the contract preliminary.
    When the explanation looks complete, treat the contract as ready.

```

This text creates hidden status semantics.

What does preliminary mean?

What does ready mean?

Which actions are allowed?

Which gates are required?

The correct solution is to formalize status:

```

STATUS.SEMANTICS:
  contract_draft:
    meaning: "contract contains unresolved assumptions"
    allowed_actions:
      - ask_clarifying_question
      - update_contract
    forbidden_actions:
      - generate_final_package

  contract_confirmed:
    meaning: "mandatory contract fields are explicitly confirmed"
    allowed_next:
      - generation

```

FREEFORM may explain the distinction, but it should not define it secretly.

FREEFORM Must Not Create New Outputs

Another dangerous pattern is:

```
FREEFORM:
  text: |
    Also prepare a short migration note and a separate checklist
    if the change affects several modules.
```

This introduces outputs that do not exist in `OUTPUT.DEF`.

A model may create them inconsistently.

Instead:

```
OUTPUT.DEF:
  id: migration_note
  when: "multi_module_change == true"
```

```
OUTPUT.DEF:
  id: module_checklist
  when: "multi_module_change == true"
```

FREEFORM may explain why these outputs are useful, but the output contract should be formal.

FREEFORM and Examples

Examples are one of the best uses of FREEFORM.

For example:

```
FREEFORM:
  id: FF_ALIAS_EXAMPLES
  purpose: "example"
  binding:
    type: "node"
    id: N_COLLECT_ALIAS
  text: |
    Good aliases are stable and describe the event meaning:
    LU_CHANGE_STATUS
    LU_CHANGE_NAME
```

Avoid aliases based on temporary ticket numbers or implementation details.

Examples help the model interpret a formal rule.

But an example must not silently become a mandatory value.

If the only examples are:

LU_CHANGE_STATUS
LU_CHANGE_NAME

the model must not assume that all aliases begin with LU_CHANGE_ unless a formal rule says so.

This is why examples should be explicitly marked as examples.

FREEFORM and Human Style

Style guidance often belongs in FREEFORM.

For example:

```
FREEFORM:
  id: FF_ANALYST_CONVERSATION_STYLE
  purpose: "style_guidance"
  binding:
    type: "profile"
    id: analyst_guided_intake
  text: |
    Ask one focused question at a time.
    Use domain language the analyst already uses.
    Avoid exposing internal IR names unless they help resolve ambiguity.
    Keep confirmations concise.
```

Some style rules may eventually become formal if they affect process safety.

For example:

ask one question at a time

may be only style guidance in one program.

But in another program, asking several questions at once may make state mapping unreliable. Then it should become a formal interaction rule.

The same sentence can therefore belong to different layers depending on its execution impact.

FREEFORM and Migration of Old Playbooks

Old playbooks often contain large prose sections.

The wrong migration strategy is:

copy the whole section into FREEFORM

That preserves the old prompt but does not create an Ordo program.

A better migration process is:

1. Read the prose section.
2. Identify execution-control statements.
3. Extract gates.
4. Extract assertions.
5. Extract statuses.
6. Extract output requirements.
7. Extract node transitions.
8. Leave rationale, examples, and context in FREEFORM.
9. Bind each remaining FREEFORM block to the relevant formal element.

For example, an old paragraph may say:

Before creating the archive, review the package carefully.
The archive has caused repeated issues in the past because missing files were discovered only after handoff. Make sure README, SUMMARY, and validation reports exist. If anything is missing, stop and fix it.

Migration should produce:

```
GATE.DEF:
  id: G_REQUIRED_FILES_PRESENT
  method: "mechanical"
  trust_class: "deterministic"
  before: final_archive
  require:
    - README.md
    - SUMMARY.json
    - VALIDATION_REPORT.json
  on_fail: repair
```

and:

```
FREEFORM:
  id: FF_REQUIRED_FILES_HISTORY
  purpose: "historical_note"
  binding:
    type: "gate"
    id: G_REQUIRED_FILES_PRESENT
  text: |
    This gate exists because missing files were previously discovered
    only after handoff.
```

This is controlled migration.

Signs of Incorrect FREEFORM

There are several warning signs.

Sign 1. FREEFORM Contains Words Such as “Mandatory,” “Forbidden,” or “Must Not”

These words do not automatically mean the block is wrong.

But they are a signal that a formal rule may be hidden inside.

Example:

You must not create the archive before approval.

This is probably an assertion or gate.

The reviewer should ask:

Where is the formal control?

Sign 2. FREEFORM Defines the Order of Steps

Example:

First ask A, then B, then validate C, and only after that create D.

This is process topology.

It probably belongs in nodes and transitions.

Sign 3. FREEFORM Creates a New Document

Example:

Also create a separate risk report.

This belongs in OUTPUT.DEF.

Sign 4. FREEFORM Changes the Meaning of a Status

Example:

For this process, "ready" also means QA has approved the package.

This belongs in STATUS.SEMANTICS.

Sign 5. FREEFORM Allows a Gate to Be Bypassed

Example:

If the model is sufficiently confident, it may continue without explicit approval.

This is a critical conflict.

If approval is required by a formal gate, FREEFORM must not override it.



Figure 65: Nebu — attention: FREEFORM must not bypass formal control

How to Decide: Formalize or Keep as FREEFORM

Use a simple decision sequence.

Ask:

1. Does this fragment change the execution path?
2. Does it allow or block an action?
3. Does it define a status?
4. Does it change state?
5. Does it require approval?
6. Does it create an output?
7. Does it define a validation condition?
8. Does it prohibit behavior?

If the answer to any of these is yes, the fragment probably needs formalization.

If all answers are no, ask:

Does the fragment help the model understand meaning, examples, context, or style?

If yes, it is a good FREEFORM candidate.

A compact heuristic is:

controls behavior → formalize
explains behavior → FREEFORM

Example: Incorrect and Correct Versions

Incorrect:

FREEFORM:

text: |

Ask the analyst for the source field.
Do not continue until it is confirmed.
Then create the Jira task.
The task must contain acceptance criteria and manual tests.
Never include implementation details.

This is a hidden process.

Correct:

```

NODE.DEF:
  id: N_COLLECT_SOURCE_FIELD
  required_input:
    - source_field

GATE.DEF:
  id: G_SOURCE_FIELD_CONFIRMED
  method: "human"
  trust_class: "human_decision"
  require:
    - source_field.status == confirmed
  on_fail: block

OUTPUT.DEF:
  id: jira_task
  must_include:
    - acceptance_criteria
    - manual_tests

ASSERT.NOT:
  id: A_NO_IMPLEMENTATION_DETAILS
  forbidden:
    - implementation_design_in_jira_task

FREEFORM:
  id: FF_JIRA_PM_RATIONALE
  purpose: "style_guidance"
  binding:
    type: "output"
    id: jira_task
  text: |
    The Jira task is intended for PM-level communication.
    It should describe the problem and expected behavior without
    prescribing the developer's implementation approach.
```

Now the process control is formal, while the explanation remains natural.

FREEFORM Must Be Visible in Reports

A reviewer should be able to see how much of an Ordo program remains in FREEFORM.

A report may show:

```
freeform_report:
  total_blocks: 12

  by_purpose:
    domain_rationale: 4
    example: 3
    style_guidance: 2
    historical_note: 2
    edge_case_commentary: 1

  unbound_blocks: 1
  high_risk_blocks: 2
```

This does not mean FREEFORM is bad.

It means FREEFORM is visible.

Invisible FREEFORM debt is dangerous. Visible FREEFORM can be reviewed and improved.

High-Risk FREEFORM

Some FREEFORM blocks deserve additional attention.

A block may be high-risk if it:

- contains prohibition language;
- mentions approval;
- describes ordering;
- references final output;
- describes stop conditions;
- mentions state transitions;
- has no binding;
- is very large;
- repeatedly appears in feedback records;
- conflicts with formal structure.

Example:

```
FREEFORM:
  id: FF_ARCHIVE_EDGE_CASES
  risk: "high"
  review_status: "needs_formalization_review"
```

A high-risk block does not automatically mean an error.

It means:

review whether hidden execution logic exists here.

FREEFORM as Formalization Debt

Sometimes an author knows that a block should eventually be formalized but does not yet have enough understanding.

Ordo should allow this to be recorded explicitly.

For example:

```
FREEFORM:
  id: FF_COMPLEX_PRIORITY_RULES
  purpose: "edge_case_commentary"
  risk: "high"
  formalization_debt:
    status: "open"
    reason: "priority semantics are not stable yet"
    target:
      - "STATUS.SEMANTICS"
      - "GATE.DEF"
```

This is better than pretending the block is fully controlled.

The language can then report formalization debt.



Figure 66: Nebu — thinking: visible debt is safer than hidden pseudo-formalization

How the Model Should Work with FREEFORM

A model executing an Ordo program should follow this discipline:

1. Read formal structure first.
2. Determine the active node, state, and gates.
3. Load only relevant FREEFORM blocks by scope and binding.
4. Use FREEFORM to interpret or explain the formal context.
5. Never use FREEFORM to override a formal prohibition or blocking gate.
6. If FREEFORM conflicts with formal structure, record a conflict.
7. If FREEFORM repeatedly determines execution, flag it for formalization review.

This ordering matters.

The model should not read a large FREEFORM block and then reinterpret the formal program around it.

Formal structure has priority.

Priority of Formal Structure

The base priority rule is:

formal control > controlled FREEFORM

For example:

```
GATE.DEF:
  id: G_APPROVAL_REQUIRED
  require:
    - approval.status == confirmed
  on_fail: block
```

and:

```
FREEFORM:
  text: |
    In obvious cases, approval may sometimes be unnecessary.
```

This is a conflict.

The gate wins.

The model should not silently choose the FREEFORM interpretation.

It should record something like:

```
conflict:
  type: "freeform_vs_formal"
  formal_unit: "G_APPROVAL_REQUIRED"
  freeform_unit: "FF_APPROVAL_NOTE"
  resolution: "formal_control_applied"
  improvement_suggested: true
```

This makes the inconsistency visible.

Mini-Exercise

Take this prose block:

Before generating the final package, ask the analyst to confirm the event alias and

Split it into:

```
NODE.DEF
GATE.DEF
OUTPUT.DEF
ASSERT.NOT
FREEFORM
```

A possible classification is:

ask for alias and source field → NODE.DEF
require confirmation → GATE.DEF
required archive files → OUTPUT.DEF
no implementation details → ASSERT.NOT
PM-level rationale → FREEFORM

Then add binding and purpose to the FREEFORM block.

Short Summary

FREEFORM is a controlled natural-language layer inside an Ordo program.

It exists to preserve:

rationale
examples
domain background
style guidance
historical context
edge-case commentary
human explanation

It must not secretly define:

paths
gates
statuses
state transitions
approvals
forbidden actions
outputs
validation conditions

The main rule is:

controls behavior → formalize
explains behavior → FREEFORM

A good FREEFORM block has identity, purpose, scope, and binding. It is visible in reports, may be marked as high-risk, and may carry explicit formalization debt.

Formal structure always has priority over FREEFORM.

FREEFORM is not a dumping ground for unstructured instructions. It is a controlled mechanism for preserving meaning without sacrificing execution control.

Chapter 21. FREEFORM.COVERAGE

In the previous chapters, we established that Ordo needs FREEFORM. Not everything can or should be converted into strict opcodes, tables, and gates. Some knowledge will always remain as explanations, examples, domain nuances, historical notes, or instructions for a person.

But this creates a new problem.

If FREEFORM is allowed, we need to understand:

- how much important logic remains in FREEFORM;
- where exactly it is located;
- what it affects;
- whether it can be tested;
- whether critical rules are hidden there;
- what should be formalized later.

This is why Ordo needs the FREEFORM.COVERAGE construct.

Why FREEFORM.COVERAGE Is Needed

Without coverage, FREEFORM quickly becomes a black box.

At first glance, a playbook may look structured: it has paths, nodes, gates, outputs, and statuses. But some of the real rules may still remain in free text.

For example:

In complex cases, do not create the final package without an additional check.

If this is merely a phrase in FREEFORM, the model may follow it once, skip it another time, interpret it differently, or fail to understand that it is a blocking rule.

FREEFORM.COVERAGE exists to make such places visible.

It answers:

Which part of the process has already been formalized, and which part still lives in

FREEFORM Is Not an Error

It is important not to treat FREEFORM as something bad.

In a good Ordo document, FREEFORM may be completely normal and necessary. The problem is not that it exists. The problem is not knowing what role it plays.

There is safe FREEFORM:

explanations for an analyst;
wording examples;
domain context descriptions;
historical reasons for a rule;
term explanations;
style recommendations.

And there is dangerous FREEFORM:

hidden gates;
hidden prohibitions;
conditions for transitions between paths;
output creation rules;
process stop conditions;
exceptions that change model behavior.

FREEFORM.COVERAGE helps distinguish these two cases.

What Exactly Should Be Covered

Coverage should show more than the amount of text in FREEFORM. Text volume itself is not the main issue.

It is much more important to understand whether FREEFORM affects execution.

For every FREEFORM block, we need to know:

block id;
where it is located;
which path/node/gate/output it is bound to;
what role it performs;
whether it affects model decisions;
whether it contains rules;
whether it contains examples;
whether tests are required for it;
whether improvement records are linked to the block;
whether it should be formalized later.

In a simple form:

```
freeform_coverage:
  entries:
    - id: "FF_DOMAIN_CONTEXT_01"
      location: "History Event Domain Pack / Path A1"
      role: "domain_explanation"
      affects_execution: false
      test_required: false
      formalization_needed: false
```

```

- id: "FF_EDGE_CASE_02"
  location: "Package Generation / Final Archive"
  role: "conditional_rule"
  affects_execution: true
  test_required: true
  formalization_needed: true
  suggested_formalization:
    - "convert to GATE.DEF"
    - "add ASSERT.NOT before final archive"

```

Roles of FREEFORM Blocks

For coverage to be useful, every FREEFORM block should have a role.

Base roles may include:

```

explanation
example
note
warning
domain_context
style_guidance
edge_case
conditional_rule
human_instruction
migration_note
implementation_hint

```

Not all roles have the same risk.

For example:

```

example – usually low risk;
explanation – low or medium risk;
warning – medium risk;
edge_case – medium or high risk;
conditional_rule – high risk;
human_instruction – depends on context;
implementation_hint – high risk if it affects output or tests.

```

Coverage should therefore do more than count blocks. It should provide a risk view.

Risk View

Ordo can use a simple risk classification:

low
medium
high
critical

For example:

```
freeform_risk_summary:
  total_entries: 12
  low: 6
  medium: 3
  high: 2
  critical: 1

critical_entries:
- id: "FF_NO_ARCHIVE_WITHOUT_SELF_CHECK"
  reason: "contains blocking behavior but is not represented as gate"
  action: "formalize_before_release"
```

If a FREEFORM block has critical risk, the Ordo program should not be considered ready for production use until a decision is made:

formalize it;
cover it with a test;
or explicitly accept the risk.

FREEFORM.COVERAGE and Tests

FREEFORM that affects behavior should be covered by tests.

For example, if FREEFORM says:

If the user asks to create an archive before approval, the process must stop.

This is no longer merely an explanation. It is a behavioral rule.

It needs a test case:

```
test:
  id: "TC_FREEFORM_NO_ARCHIVE_BEFORE_APPROVAL"
  method: human
  trust_class: human_decision

fixture:
  user_message: "create the archive immediately"

expected:
  gate:
    id: "G_PRE_ARCHIVE_APPROVAL"
    status: "blocked"
```



```
output:
  archive_created: false
```

Coverage should show that this FREEFORM block has either already been formalized as a gate or is at least checked by a test.

FREEFORM.COVERAGE and the Improvement Loop

If a user repeatedly points out a problem originating in a FREEFORM block, that is a strong signal.

For example:

The edge case was interpreted incorrectly again.

The improvement record should then refer not only to the run or node but also to the specific FREEFORM block:

```
improvement_record:
  id: "IR-002"
  classification:
    type: "ambiguous_freeform_rule"
    severity: "high"

  affected_unit:
    kind: "freeform"
    id: "FF_EDGE_CASE_02"

  proposed_patch:
    - "split FREEFORM block into explanation and rule"
    - "convert rule part into GATE.DEF"
    - "add regression test"
```

This turns FREEFORM.COVERAGE from a report into part of the Ordo-program improvement cycle.

Formalization Over Time

In the first version of a playbook, some logic may remain in FREEFORM. That is normal.

But Ordo should support gradual formalization.

The cycle may look like this:

```
FREEFORM explanation
→ feedback reveals a problem
→ debug trace shows the location
→ coverage marks high risk
```

- author formalizes the rule
- gate/assertion/test is added
- regression suite verifies the change

FREEFORM.COVERAGE therefore helps evolve an Ordo program without abruptly rewriting the entire document.

Minimum FREEFORM.COVERAGE Report

For simple Ordo programs, a short report is enough:

```
freeform_coverage_report:
  total_entries: 5
  execution_affecting_entries: 1
  tested_entries: 1
  high_risk_entries: 0
  formalization_required: false
```

Large playbooks need a more detailed report:

```
freeform_coverage_report:
  total_entries: 28
  by_role:
    explanation: 8
    example: 6
    domain_context: 5
    edge_case: 4
    conditional_rule: 3
    warning: 2

  execution_affecting:
    total: 7
    covered_by_tests: 5
    not_covered:
      - "FF_EDGE_CASE_07"
      - "FF_STATUS_WARNING_02"

  formalization_candidates:
    - id: "FF_EDGE_CASE_07"
      suggested_target: "GATE.DEF"
    - id: "FF_STATUS_WARNING_02"
      suggested_target: "STATUS.SEMANTICS"

  release_status: "blocked_until_review"
```

Typical Mistakes

The first mistake is counting only the amount of FREEFORM.

There may be a large amount of safe FREEFORM and very little dangerous FREEFORM. Or there may be one short FREEFORM block that effectively changes the entire process behavior.

The second mistake is failing to bind FREEFORM to a specific execution location.

If a block is not bound to a path, node, gate, output, or domain rule, it is difficult to test and improve.

The third mistake is leaving a blocking rule in FREEFORM.

Anything that should stop the process must be a gate or assertion.

The fourth mistake is failing to test FREEFORM edge cases.

If FREEFORM describes a complex exception, it needs a test case.

The fifth mistake is failing to move recurring FREEFORM problems into the improvement backlog.

If the same error occurs several times, it is no longer random. It is a signal that the Ordo program should be improved.

Mini-Exercise

Take any playbook fragment containing free text and answer five questions:

1. Is this an explanation, example, warning, or rule?
2. Does this text affect a model decision?
3. Could an incorrect interpretation of this text break the process?
4. Is there a test for it?
5. Should it be formalized as a gate, assertion, status, or output rule?

If the answer to the second or third question is “yes,” this FREEFORM block should be visible in the coverage report.

Short Summary

FREEFORM.COVERAGE exists so that Ordo does not turn back into one large uncontrolled prompt.

FREEFORM preserves human meaning, domain explanations, and complex nuances. But everything that affects execution must be visible, bound, risk-assessed, and, when necessary, covered by tests.

The main idea of this chapter is simple:

FREEFORM is allowed, but it must not be invisible.

In a mature Ordo program, every important FREEFORM block should answer three questions:

where is it used;

what does it affect;

how do we verify that it does not break the process.

Chapter 22. Ordo Core

Why Ordo Core Is Needed

By this point, we have already examined many individual parts of Ordo: intent, contract, state, nodes, gates, output, status semantics, debug, tests, the feedback loop, and FREEFORM. But if these concepts remain merely a collection of ideas, every Ordo-program author will assemble them differently.

One author will call the starting node `start`. Another will call it `entry`. A third will use `initial_question`. A fourth will hide the beginning of the process inside a long text block.

The same may happen with gates, statuses, output, state, checks, trace, and FREEFORM. Formally, everyone will be “writing Ordo,” but in practice every program will live by its own rules.

This is why Ordo Core is needed.

Ordo Core is the minimum mandatory set of concepts, rules, and constructs without which an Ordo program is not considered a complete Ordo program.

Core does not describe the entire domain. It does not know what a historical event, monitoring event, legal opinion, company check, or QA package is. Core is responsible for something else: the basic shape of controlled execution.

Simply put:

Ordo Core is the skeleton of the language.

Profiles, domain packs, libraries, and concrete playbooks are then layered onto this skeleton.

What Is Included in Ordo Core

Ordo Core should answer several basic questions:



Figure 67: Nebu — idea: Core as the skeleton of the language

Where does execution begin?
Which contract must be confirmed?
What state is maintained during the process?
Which process nodes exist?
Which answers may be accepted?
Which gates block or allow a transition?
Which output must be created?
Which negative assertions prohibit incorrect actions?
How is the execution trace recorded?
How are FREEFORM and its coverage represented?

Core is therefore not a library of ready-made solutions and not a domain pack. It is the basic grammar of execution.

In the first version, Ordo Core can be understood through these key blocks:

ENTRY.DEF
NODE.DEF
STATE.SCHEMA
ANSWER.REGISTRY
OUTPUT.DEF
ASSERT.NOT
STATUS.SEMANTICS
ASSUMPTION.LEDGER
FREEFORM.COVERAGE
TRACE.REQUIRE
GATE.REQUIRE

We will now examine them in simple terms.

ENTRY.DEF

ENTRY.DEF describes how an Ordo program begins.

In an ordinary prompt, the beginning is often informal. The user writes something, the model interprets it somehow, and starts answering.

In Ordo, the start must be defined.

For example:

```
entry:
  id: "ENTRY_MAIN"
  accepts:
    - "new_user_request"
    - "uploaded_playbook"
    - "existing_state"
  first_node: "NODE_CLASSIFY_REQUEST"
```

This means that when a new request arrives, the program must not immediately create the final result. It must move to the first defined node and classify the request.

ENTRY exists so the model does not invent “where to begin.”

NODE.DEF

NODE.DEF describes one step or node in the process.

A node is not merely a paragraph of instructions. It is a place where the model must perform a particular action: ask a question, accept an answer, update state, select a path, check a gate, or move forward.

Example:

```
node:
  id: "NODE_COLLECT_ALIAS"
  purpose: "collect event alias"
  asks:
    question: "What is the event alias?"
  writes_to_state:
    - "event.alias"
  next:
    when_answered: "NODE_COLLECT_SOURCE_FIELD"
```

This is no longer merely advice to “ask for the alias.” It is a formal part of the execution flow.

Core does not define which exact nodes every domain needs. But Core defines that a node should have an id, purpose, expected action, state impact, and transition rule.

STATE.SCHEMA

STATE.SCHEMA describes which data the process remembers during execution.

Without state, the model can easily lose context. It may ask the same question twice, forget a confirmation, confuse a draft with a final result, or treat an assumption as fact.

Example:

```
state_schema:
  event:
    alias:
      type: "string"
      required: true
      status: "unconfirmed"
    source_field:
      type: "string"
      required: true
      status: "unconfirmed"
  approvals:
    pre_archive:
      type: "boolean"
      default: false
```

A state schema does not guarantee that the model will never make a mistake. But it provides a clear map of what must be remembered and what status each item has.

ANSWER.REGISTRY

ANSWER.REGISTRY describes which types of user answers an Ordo program can accept.

This is important for guided intake. A user does not always answer in a perfectly structured form. They may write:

```
yes
confirmed
no
I changed my mind
go back
continue
that's not it
```

Without a registry, the model decides what each answer means every time. That is dangerous.

Example:

```
answer_registry:
  confirm:
    examples:
```

```

      - "yes"
      - "confirmed"
      - "ok"
    effect: "mark_current_contract_part_confirmed"

  reject:
    examples:
      - "no"
      - "doesn't work"
      - "that's not it"
    effect: "keep_state_unconfirmed"

  go_next:
    examples:
      - "next"
      - "continue"
    effect: "advance_if_current_gate_passed"

```

Core should not know every phrase in every language. But Core should require important answers to be classified and to have a defined effect.

OUTPUT.DEF

OUTPUT.DEF describes exactly what an Ordo program must create.

Output is not merely “give an answer.” It is a defined result structure.

For example:

```

output:
  id: "FINAL_PACKAGE"
  type: "archive"
  required_files:
    - "README.md"
    - "SUMMARY.json"
    - "VALIDATION_REPORT.json"
    - "CONSISTENCY_CHECK_REPORT.json"
  gates_before_creation:
    - "G_CONTRACT_CONFIRMED"
    - "G_PRE_ARCHIVE_APPROVED"
    - "G_SELF_CHECK_PASSED"

```

Output definition prevents the model from mixing a draft, explanation, final document, and handoff into one chaotic text.

ASSERT.NOT

ASSERT.NOT represents negative checks. They describe what an Ordo program is not allowed to do.

For example:

```
assert_not:
- id: "NO_FINAL_ARCHIVE_BEFORE_APPROVAL"
  condition: "pre_archive_approval != true"
  forbidden_action: "create_final_archive"
```

This is especially important in Core. Positive rules say what should be done. But for an AI model, it is often even more important to state explicitly what must not be done.

This especially includes:

- do not create final output before approval;
- do not invent missing data;
- do not mark unconfirmed as confirmed;
- do not hide a gate in FREEFORM;
- do not treat an example as a rule;
- do not change domain logic without explicit instruction.

STATUS.SEMANTICS

STATUS.SEMANTICS describes the meaning of statuses.

In complex processes, words such as “ready,” “confirmed,” “draft,” “blocked,” and “passed” can easily become mixed together.

Core should require statuses to have clear semantics.

For example:

```
status_semantics:
  draft:
    meaning: "created but not approved"
    allows_final_handoff: false

  confirmed:
    meaning: "explicitly approved by user"
    allows_gate_pass: true

  blocked:
    meaning: "execution cannot continue until condition is resolved"
    allows_next_step: false
```

This is especially important in Ordo because the language works not only with text but with process.

ASSUMPTION.LEDGER

ASSUMPTION.LEDGER is a record of assumptions.

A model often has to make assumptions. The problem is not the assumptions themselves but the fact that they can become invisible.

Core should require important assumptions to be recorded.

For example:

```
assumption_ledger:
  - id: "A-001"
    assumption: "source field belongs to EDR factual data"
    reason: "user provided EDR-like payload"
    status: "needs_confirmation"
    used_for:
      - "path_selection"
```

If an assumption affects a path, gate, or output, it cannot remain hidden.

FREEFORM.COVERAGE

Core does not prohibit FREEFORM. On the contrary, it recognizes that part of an instruction may remain in natural language.

But Core should require FREEFORM to be controlled.

This is why FREEFORM.COVERAGE is needed:

```
freeform_coverage:
  entries_total: 5
  structured_bindings: 4
  unbound_entries:
    - "FF_DOMAIN_EXAMPLES"
  risk: "medium"
```

This makes it possible to see which parts of the playbook are not yet formalized and where errors may arise.

TRACE.REQUIRE

After the introduction of the Debug, Test & Improvement Layer, Core should include a minimum trace requirement.

Even when a run is not launched in full debug mode, a complex Ordo program should leave basic execution traces:

- which entry was used;
- which path was selected;

- which gates passed;
- which gates were blocked;
- which state fields changed;
- which output was created;
- which warnings were recorded.

This does not always need to be a large detailed log. But execution should not be completely opaque.

GATE.REQUIRE

Core should also require important transitions to be protected by gates.

For example, final output should not be created merely because the user wrote “go ahead.” The conditions must be checked.

```
gate_require:
  before:
    - action: "create_final_output"
      required_gates:
        - "G_CONTRACT_CONFIRMED"
        - "G_VALIDATION_PASSED"
```

This is one of Ordo’s main principles: a gate is not a recommendation but a control point.

How Core Differs from a Profile

Core is responsible for the base language.

A Profile is responsible for the process type.

For example:

Core says: output must be defined.

Profile says: for documentation, output must have a template, rendered validation, a

Core says: gates must be explicit.

Profile says: a QA package needs manual QA readiness and automation readiness gates.

Core says: state must be described.

Profile says: for guided intake, state must contain current node, confirmed answers,

A profile therefore extends Core but does not replace it.

How Core Differs from a Domain Pack

A Domain Pack knows the subject domain.

Core does not know what HistoryEvent, ChangeRecord, External-HistoryEvent, Monitoring Center, EDR, source row, or QA package means.

A Domain Pack describes:

- domain vocabulary;
- domain-specific paths;
- domain gates;
- domain statuses;
- domain outputs;
- domain examples;
- domain no-op rules.

Core ensures that all of this is executable, controlled, and verifiable.

How Core Differs from a Library

A Library is a reusable ready-made solution.

For example, a library may contain a ready-made set of contract-first gates or rendered artifact validation.

Core defines how such things are connected, checked, and prevented from conflicting with the program.

In short:

Core – base language.

Profile – process style/type.

Domain Pack – subject domain.

Library – reusable fragment.



Figure 68: Nebu — thinking: do not mix Core, Profile, Domain Pack, and Library

Typical Mistakes

Mistake 1. Making Core Too Large

Core should not contain everything. If historical events, QA packages, legal opinions, API configurations, and document templates are all placed in Core, it becomes unwieldy.

Core should remain minimal.



Figure 69: Nebu — attention: Core should not become too large

Mistake 2. Hiding Core Rules in FREEFORM

If a rule blocks execution, it should not live only in explanatory text.

Bad:

It is advisable to perform a self-check before the archive.

Good:

```
gate:
  id: "G_SELF_CHECK_PASSED"
  method: mechanical
  trust_class: deterministic
  blocking: true
```

Mistake 3. Treating Core as Documentation

Core is not a description for reading. It is part of the executable process model.

Mistake 4. Allowing a Domain Pack to Break Core

A Domain Pack may extend Core but should not cancel base rules without an explicit override and trace.

Mini-Exercise

Take any complex prompt you have used before and try to identify its Core part:

1. Where does the process begin?
2. Which nodes or steps exist?
3. Which state must be maintained?
4. Which gates should be blocking?
5. Which output is expected?
6. Which actions must be explicitly prohibited?

7. Which assumptions may arise?
8. What should be logged for debug?

If these questions have no answers, you do not yet have an Ordo program, only an informal instruction.

Short Summary

Ordo Core is the minimum mandatory set of constructs that makes an Ordo program controllable.

Core does not describe the domain and does not replace a domain pack. It defines the basic execution shape:

entry → node → state → gate → output → trace

Without Core, Ordo will become a collection of attractive prompt templates.

With Core, it becomes a language for controlling AI-model behavior.

Chapter 23. Ordo Profiles

Why Profiles Are Needed

In the previous chapter, we examined Ordo Core — the minimum set of constructs without which an Ordo program cannot be controlled. Core defines the basic execution shape: entry, node, state, gate, output, and trace.

But in real work, this is not enough.

One Ordo program may create an analytical package. Another may guide a user through guided intake. A third may produce QA documentation. A fourth may validate a rendered artifact. A fifth may work with approvals, evidence, templates, and a document catalog.

All of these scenarios use Core, but they need different additional rules.

This is why Ordo needs Profiles.

Simply put:

An Ordo Profile is a standard set of additional rules and constructs for a particular

Core says: “every Ordo program must have controlled execution.”

A Profile says: “this is how execution should be controlled for a particular class of tasks.”



Figure 70: Nebu — idea: Profile as an operating mode

Why Not Put Everything in Core

It may seem that every useful rule should be added directly to Core. For example:

- document rules;
- QA rules;
- approval rules;
- rendered artifact validation rules;
- evidence matrix rules;
- documentation splitting rules;
- package generation rules.

But Core would quickly become too large.

Core should remain minimal and stable. It should not know every possible task type. Otherwise, Ordo will lose flexibility.

Ordo therefore separates:

Core	→ base execution language
Profile	→ standard operating mode
Domain	→ specific subject domain
Library	→ reusable ready-made parts



Figure 71: Nebu — thinking: separation of Core, Profile, Domain, and Library

This resembles ordinary programming. A language has base syntax, while separate frameworks and libraries are used for web, testing, databases, or UI.

In Ordo, a Profile serves as such a standard operating mode for a particular class of tasks.

What a Profile May Contain

A Profile should not contain everything. It should contain only rules that recur across many Ordo programs of the same type.

For example, a Documentation Profile may contain:

- template handling rules;
- rules for splitting large documents;
- rules for selecting required documents;
- rendered artifact validation rules;
- catalog / selected docs rules;
- self-check rules before handoff.

A QA Profile may contain:

- test case structure;
- fixture rules;
- expected behavior;
- negative scenarios;
- regression suite;
- coverage report;
- manual QA runbook rules.

An Approval Profile may contain:

- what requires human confirmation;
- which gates are blocking;
- which decisions the model may not make itself;
- how approval is recorded in state;
- how final output is prohibited without approval.

A Profile is therefore not a subject domain. It is an execution style and mode.

Example: Documentation Profile

Imagine an Ordo program that creates a document package.

Without a Profile, the instruction may look like this:

Create a README, passport, task, QA document, and validation report.
Before the archive, check that everything is consistent.

A human can understand this, but the description lacks structure for controlled execution.

A Documentation Profile may formalize it like this:

```
profile:
  id: "ordo.profile.documentation"
  version: "0.1"
```



```

rules:
  - id: "DOC.CATALOG"
    description: "all package documents must be registered in the catalog"

  - id: "DOC.SELECT"
    description: "before responding, the model must determine which documents are re

  - id: "TEMPLATE.BIND"
    description: "each output document must be bound to a template or explicitly des

  - id: "RENDER.VALIDATE"
    description: "validate not only the template but the final rendered artifact"

```

Now every Ordo program that uses this Profile receives standard documentation-execution rules.

Profile as a Behavior Contract



Figure 72: Nebu — attention: a Profile changes the execution contract

A Profile does not merely add useful advice. It changes the execution contract.

If an Ordo program uses the Documentation Profile, the model can no longer behave as though it were simply writing text.

It must:

- know which documents exist;
- select relevant documents for the current step;
- not confuse a template with a rendered artifact;
- not create a final package without a self-check;
- record which documents were used;
- report a missing required artifact;
- execute validation gates.

This is an important distinction.

In an ordinary prompt, you can write: “check consistency.”

In a Profile, you can say: “consistency is a mandatory gate before handoff.”

Profile in Ordo Source

In human-readable Ordo Source, Profile inclusion may look like this:

```
ordo:
  version: "0.11"

profiles:
  use:
    - id: "ordo.profile.documentation"
      version: "0.1"
    - id: "ordo.profile.qa"
      version: "0.1"
    - id: "ordo.profile.approval"
      version: "0.1"
```

Such a program immediately communicates:

```
this is not merely text generation;
this is a documentation process;
it has a QA structure;
some decisions require human approval.
```

Profile in Compiled IR

In compiled IR, this may become a set of operations:

```
[
  {
    "op": "PROFILE.USE",
    "id": "P1",
    "profile": "ordo.profile.documentation",
    "version": "0.1"
  },
  {
    "op": "PROFILE.USE",
    "id": "P2",
    "profile": "ordo.profile.qa",
    "version": "0.1"
  },
  {
    "op": "PROFILE.BIND_RULES",
    "id": "P3",
    "profiles": ["P1", "P2"]
  }
]
```

```
}  
]
```

After this, the compiler or runtime must know that additional rules are active in the program.

For example, if the Documentation Profile is active, final package generation cannot occur without RENDER.VALIDATE or an equivalent gate.

Typical Profile Constructs

In the current Ordo concept, the following Profile-level constructs can be identified:

```
TEMPLATE.BIND  
EVIDENCE.MATRIX  
APPROVAL.REQUIRE  
DOC.SPLIT  
DOC.CATALOG  
DOC.SELECT  
RENDER.VALIDATE  
QA.CASE.DEF  
QA.RUNBOOK.DEF  
PACKAGE.SELF_CHECK  
HANDOFF.NOTE
```

This is not necessarily a complete list. But it shows the difference between Core and Profile.

Core says:

there must be a gate

A Profile says:

before the archive, there must be a rendered artifact validation gate

Core says:

there must be an output

A Profile says:

the output must comply with template binding and be included in the document catalog

Core says:

there must be state

A Profile says:

state must contain approval status, selected docs, and validation result

Profile and Domain Pack

Profiles are often confused with Domain Packs. The distinction must be very clear.

A Profile answers:

what type of process is this?

A Domain Pack answers:

what subject domain is this process about?

For example, a History Event Playbook may use:

Documentation Profile

QA Profile

Approval Profile

Debug/Test Profile

History Event Domain Pack

Profiles provide general rules for documents, QA, and approval.

The Domain Pack provides historical-event specifics:

- Path A1/A2/A3/A4/A5;
- HistoryEvent;
- ChangeRecord;
- source row;
- no-op rules;
- event alias;
- history event package structure.

These are different layers.

If they are mixed, the Ordo program becomes difficult to maintain.

Profile and Libraries

Later in the book, we will discuss Ordo Libraries separately. But it is already useful to establish the distinction.

A Profile is a standard behavior mode.

A Library is a reusable package of ready-made parts.

For example:

`ordo.profile.qa`

may define that a QA process must have fixtures, expected behavior, and a regression suite.

A library:

`ordo.library.qa.history_event_basic_tests`

may contain ready-made test cases for a specific type of History Event.

A Profile therefore defines the rules of the game, while a Library may provide reusable elements for that game.

Why Profiles Matter to the Compiler

The Ordo compiler must understand active Profiles.

Without this, it cannot validate a program correctly.

For example, if the QA Profile is active, the compiler may require:

- presence of TEST.DEF;
- presence of at least a minimum coverage report;
- a negative test for blocking gates;
- expected behavior for key paths.

If the Documentation Profile is active, the compiler may check:

- whether all required documents are described;
- whether a document catalog exists;
- whether a validation report exists;
- whether an archive is created before self-check;
- whether rendered artifact validation has been replaced by template validation.

A Profile is therefore also a mechanism for compiler validation.

Typical Mistakes

Mistake 1. Making a Profile Too Domain-Specific

Bad:

Profile for changing a company's authorized capital

This is already a domain pack or library, not a Profile.

A Profile should be broader:

Documentation Profile

QA Profile

Approval Profile

Guided Intake Profile

Mistake 2. Putting Every Rule in Core

If everything becomes Core, Core becomes unmanageable.

Core should remain small. Profiles should extend it for standard operating modes.

Mistake 3. Not Declaring the Active Profile

If an Ordo program effectively works as a QA process but does not use the QA Profile, the compiler cannot require the necessary tests.

Bad:

somewhere in the text it says that QA should be checked

Good:

```
profiles:
  use:
    - id: "ordo.profile.qa"
      version: "0.1"
```

Mistake 4. Mixing a Profile and a Library

A Profile should not become a warehouse of ready-made examples for every case.

Reusable ready-made parts belong in libraries.

A Profile should define rules, obligations, and gates.

Mini-Exercise

Take any complex process you want to execute through an AI model.

For example:

- creating an analytical package;
- reviewing a document;
- preparing QA instructions;
- guided intake for a new task;
- reviewing code changes;
- preparing a developer handoff.

Try to answer:

1. Which Core is needed here?
2. What type of process is this?
3. Which Profile fits best?
4. Is a Documentation Profile needed?
5. Is a QA Profile needed?
6. Is an Approval Profile needed?
7. Which gates should the Profile add?

8. Which checks should the compiler perform?
9. What is domain-specific and should not enter the Profile?

If you can answer these questions, you are already beginning to think in Ordo layers rather than prompts.

Short Summary

Ordo Profile is a standard set of additional rules for a particular type of Ordo process.

Core defines the base execution language.

A Profile defines the operating mode.

A Domain Pack defines the subject domain.

A Library provides reusable ready-made parts.

Correct separation of these layers allows Ordo to remain simple, extensible, and suitable for large playbooks.

Without Profiles, every complex playbook will reinvent documentation, QA, approval, and validation rules.

With Profiles, these rules become reusable and understandable to the compiler.

Chapter 24. Domain Packs

Why Domain Packs Are Needed

In the previous chapters, we divided Ordo into several layers.

Ordo Core defines the minimum execution language.

Ordo Profiles add rules for standard operating modes: documentation, QA, approval, guided intake, debug, and testing.

But this is still not enough for real processes.

Almost every serious process has its own subject-matter logic.

For example, creating a historical event has its own concepts:

```
HistoryEvent
ChangeRecord
source row
alias
display name
old value
new value
```

Path A1
Path A2
Path A4
no-op
rollback
manual QA package
automation spec

A monitoring event has different concepts:

monitoring event
configuration
translation tree
sandbox evaluate
REST runbook
business passport
technical package
trigger condition
expected notification

Legal analysis has yet another vocabulary:

jurisdiction
legal basis
risk factor
evidence
finding
recommendation
exception

These concepts should not be placed in Core.

They are not exactly Profiles either. A Profile says: “this process has QA,” “this process requires approval,” or “this process uses documentation runtime.”

A Domain Pack says: “this subject area has these objects, rules, paths, gates, and edge cases.”

Ordo therefore needs a separate layer:

Domain Pack



Figure 73: Nebu — idea: Domain Pack as Ordo’s subject-matter layer

Simple Explanation

A Domain Pack is a package of subject-matter knowledge and rules for a specific domain.

Very simply:

Core is Ordo's grammar.

Profile is the operating style.

Domain Pack is knowledge of a specific domain.

Or:

Core knows what NODE and GATE are.

Profile knows that a QA process needs tests and coverage.

Domain Pack knows that History Event work must distinguish A1 from A2, no-op from real change, and a source row from a generated event.

A Domain Pack turns the general Ordo language into a tool for concrete work.



Figure 74: Nebu — thinking: a Domain Pack does not replace Core or Profile

Why a Domain Pack Cannot Be Replaced by a Long Prompt

Of course, a large subject-matter description can be written directly in a prompt.

But familiar problems then appear:

- the model does not know which rules are mandatory;
- examples become mixed with rules;
- edge cases are lost;
- gates do not behave as blocking controls;
- new scenarios are added chaotically;
- there is no test coverage;
- it is unclear which part of the instruction should be changed;
- user feedback is not linked to a specific rule or path.

A Domain Pack is intended to solve this.

It structures the domain:

- which objects exist;
- which statuses exist;
- which paths exist;
- which questions must be asked;
- which gates are blocking;
- which outputs are created;
- which edge cases exist;
- which rules remain in controlled FREEFORM;
- which tests verify behavior.

What a Domain Pack Contains

A minimum Domain Pack may contain:

DOMAIN.DEF
 DOMAIN.VOCABULARY
 DOMAIN.OBJECTS
 DOMAIN.PATHS
 DOMAIN.RULES
 DOMAIN.GATES
 DOMAIN.STATUS
 DOMAIN.OUTPUTS
 DOMAIN.FREEFORM
 DOMAIN.TESTS
 DOMAIN.COVERAGE

DOMAIN.DEF

This defines the subject domain.

For example:

```
domain:
  id: "history_event"
  name: "History Event Domain Pack"
  version: "0.1"
  purpose: "Guided intake and package generation for history event creation or update"
```

This block answers:

What is this domain?
 Why does it exist?
 Which task does it help perform?

DOMAIN.VOCABULARY

This is the term dictionary.

For example:

```
vocabulary:
  HistoryEvent: "final internal historical event shown in company/person history"
  ChangeRecord: "technical record describing detected source-level change"
  source_row: "input source object from which change is detected"
  no_op: "case where no new event should be created"
```

The vocabulary is not decorative. It reduces the risk that the model will confuse similar concepts.

For example, without a defined distinction between ChangeRecord and HistoryEvent, the model may treat a technical record as a completed historical event.

DOMAIN.OBJECTS

This describes the main domain objects.

```
objects:
  - id: "HistoryEvent"
    required_fields:
      - "alias"
      - "display_name"
      - "event_date"
      - "old_value"
      - "new_value"
  - id: "ChangeRecord"
    required_fields:
      - "field"
      - "old"
      - "new"
      - "status"
```

In a real Domain Pack, these fields may be more complex. The basic idea is simple: the model must know which entities it works with.

DOMAIN.PATHS

This describes the main scenarios.

For example:

```
paths:
  - id: "A1"
    name: "direct source row field change"
    when:
      - "change is detected directly in source row"
```

```

requires:
  - "source field confirmed"
  - "old/new values confirmed"

- id: "A2"
  name: "related entity change"
  when:
    - "change belongs to entity related through identification center"
  requires:
    - "main entity confirmed"
    - "related entity confirmed"
    - "relation context confirmed"

```

Paths prevent the model from guessing a scenario based on a general impression.

The model must select a path and explain why.

DOMAIN.RULES

These are subject-matter rules.

For example:

```

rules:
  - id: "R_NO_EVENT_FOR_NOOP"
    text: "If old and new normalized values are equal, no HistoryEvent should be created"
    enforcement: "blocking"

  - id: "R_CURRENCY_NORMALIZATION"
    text: "Capital amount comparison must use normalized amount and normalized currency"
    enforcement: "required"

```

A rule should not be merely a paragraph in documentation. It should have an identifier and enforcement semantics.

DOMAIN.GATES

These are domain-specific control points.

```

gates:
  - id: "G_SOURCE_FIELD_CONFIRMED"
    type: "approval"
    blocking: true
    description: "Source field must be confirmed before generating event passport."

  - id: "G_NOOP_CHECK_DONE"
    type: "validation"

```

```
blocking: true
description: "No-op check must be completed before event creation decision."
```

A Domain Pack may use general Profile gates while adding its own subject-specific gates.

DOMAIN.OUTPUTS

This describes the results the process must create.

For example:

```
outputs:
- id: "history_event_passport"
  format: "markdown"
  required: true

- id: "jira_task"
  format: "markdown"
  required: true

- id: "manual_qa_package"
  format: "markdown"
  required: true
```

This matters because different domains have different outputs.

DOMAIN.FREEFORM

Even in a Domain Pack, some knowledge may remain in controlled FREEFORM.

For example:

```
freeform:
- id: "FF_DOMAIN_EXAMPLES"
  purpose: "Examples of valid and invalid history event descriptions."
  binding:
    applies_to:
      - "DOMAIN.PATHS"
      - "DOMAIN.RULES"
  must_not_contain:
    - "blocking gates"
    - "status semantics"
```

A Domain Pack therefore does not prohibit free text. It controls it.

DOMAIN.TESTS

A Domain Pack should contain or include tests.

```
tests:
  - id: "TC_A1_DIRECT_FIELD_CHANGE"
    expects:
      path: "A1"
      required_gates:
        - "G_SOURCE_FIELD_CONFIRMED"

  - id: "TC_NOOP_NORMALIZED_VALUES_EQUAL"
    expects:
      noop: true
      history_event_created: false
```

Without tests, a Domain Pack quickly begins to break as it changes.

Domain Pack as a Contract Between People and the Model

A Domain Pack is not merely a technical specification.

It is a contract between:

- the author of the domain logic;
- the analyst;
- the model;
- compiler/runtime;
- tests;
- future users.

The domain author says:

This is how we work in this subject area.
These are our concepts.
These are our paths.
These are our gates.
This is where the model must stop.
This is what it must not do.
This is how we verify that it has not broken.

The model should not invent domain logic from nothing.

It should execute the Domain Pack.

Example: History Event Domain Pack

The main parts of a History Event Domain Pack may look like this:

DOMAIN.DEF:
 History Event guided intake and package generation

DOMAIN.VOCABULARY:
 HistoryEvent
 ChangeRecord
 source row
 ExternalHistoryEvent
 no-op
 rollback
 manual QA
 automation spec

DOMAIN.PATHS:
 A1 – direct source row field change
 A2 – related entity change through identification center
 A3 – generated/calculated data change
 A4 – external history event input
 A5 – correction / rollback / special case

DOMAIN.GATES:
 G_ALIAS_CONFIRMED
 G_DISPLAY_NAME_CONFIRMED
 G_SOURCE_ROW_CONFIRMED
 G_VALUES_CONFIRMED
 G_NOOP_CHECK_DONE
 G_PRE_ARCHIVE_APPROVAL
 G_SELF_CHECK_DONE

DOMAIN.OUTPUTS:
 README
 SUMMARY
 VALIDATION_REPORT
 CONSISTENCY_CHECK_REPORT
 HISTORY_EVENT_PASSPORT
 JIRA_TASK
 IMPLEMENTATION_PROMPT
 QA_PACKAGE
 PROCESS_IMPROVEMENT_FEEDBACK
 QA_AUTOMATION_SPEC
 QA_AUTOMATION_README

This is no longer a “long prompt.” It is a domain rule system.

Example: Monitoring Event Domain Pack

Another domain has a different package.

DOMAIN.DEF:

Monitoring Event business and technical package generation

DOMAIN.VOCABULARY:

monitoring event
business passport
registry row
Monitoring Center config
translation tree
sandbox evaluation
REST runbook
notification payload

DOMAIN.PATHS:

business passport creation
technical config package
sandbox QA
REST execution runbook
registry row after Confluence URL

DOMAIN.GATES:

G_BUSINESS_CONFIRMATION
G_CONFIG_VALUES_CONFIRMED
G_SANDBOX_EVALUATION_READY
G_REST_RUNBOOK_SELF_CONTAINED
G_CONFLUENCE_URL_REQUIRED_BEFORE_REGISTRY_ROW

This Domain Pack may use the same Core and Profiles, but its domain rules are different.

Relationship Between a Domain Pack and Core

A Domain Pack must not redefine Core.



Figure 75: Nebu — attention: a Domain Pack must not rewrite Core

Bad:

The Domain Pack changes the meaning of NODE or GATE.

Good:

The Domain Pack uses NODE and GATE for its subject-matter rules.

Core should remain stable.

A Domain Pack should extend the language rather than rewrite its foundation.

Relationship Between a Domain Pack and Profiles

A Domain Pack may require specific Profiles.

For example:

```
domain:
  id: "history_event"

requires_profiles:
  - "ordo.profile.guided_intake"
  - "ordo.profile.documentation"
  - "ordo.profile.qa"
  - "ordo.profile.approval"
  - "ordo.profile.debug_test_improvement"
```

This means:

Core alone is not enough for this domain.

Guided intake, documentation, QA, approval, and debug/test/improvement are mandatory

Profiles provide general mechanisms.

The Domain Pack fills them with subject-matter meaning.

Relationship Between a Domain Pack and Libraries

A Domain Pack may use libraries.

For example:

```
include:
  - library: "ordo.validation.noop_checks"
    version: "0.1"
    as: "noop"

  - library: "ordo.qa.manual_runbook"
    version: "0.1"
    as: "manual_qa"
```

But a Domain Pack is not simply a library.

The distinction is:

Library – a reusable ready-made solution.

Domain Pack – a subject-matter rule system for a specific domain.

A Domain Pack may include many libraries while remaining the owner of domain semantics.

Domain Pack Versioning

A Domain Pack must have a version.

```
domain:
  id: "history_event"
  version: "0.10"
```

This is necessary because changes to domain rules may change model behavior.

For example, if version 0.10 interpreted Path A4 ExternalHistoryEvent one way and version 0.11 clarified the rules, old tests may behave differently.

It is therefore important to know:

- which Domain Pack version was used;
- which tests passed;
- which compatibility checks were performed;
- which breaking changes were introduced;
- whether the changelog was updated.

Domain Pack and the Compiler

The Ordo compiler must be able to validate a Domain Pack.

Minimum checks include:

- DOMAIN.DEF exists;
- vocabulary has no critical gaps;
- paths have selection conditions;
- blocking gates have enforcement;
- outputs have required/optional status;
- tests cover the main paths;
- FREEFORM blocks have binding;
- the Domain Pack does not redefine Core without explicit permission;
- required Profiles are active;
- libraries have pinned versions.

For complex Domain Packs, the compiler should also generate:

- coverage report;
- conflict report;
- unresolved ambiguity report;
- compatibility report;
- improvement backlog.

Domain Pack and Debug

Debug mode should show exactly which parts of a Domain Pack were used.

For example:

```
trace_source: "model_self_report"
knowledge_trace:
  - source: "history_event_domain_pack"
    version: "0.10"
    section: "DOMAIN.PATHS.A1"
    used_for: "path selection"

  - source: "history_event_domain_pack"
    version: "0.10"
    section: "DOMAIN.GATES.G_NOOP_CHECK_DONE"
    used_for: "gate evaluation"
```

Without this, the user sees only the answer.

With it, the user can see which domain rule caused the decision.

Domain Pack and the Improvement Loop

When a user identifies a problem, the improvement record should bind it to the Domain Pack.

For example:

```
improvement_record:
  classification:
    type: "missing_domain_gate"
    severity: "high"

  affected_unit:
    kind: "domain_pack"
    id: "history_event"
    version: "0.10"
    section: "DOMAIN.GATES"

  proposed_patch:
```

- "add blocking gate G_MANUAL_QA_INSTRUCTIONS_ARE_ACTIONABLE"
- "add regression test TC_MANUAL_QA_NOT_TOO_GENERIC"

This is important: the problem is not lost in chat but becomes a change to a specific part of domain logic.

Typical Mistakes

Mistake 1. Making a Domain Pack One Continuous Text

If a Domain Pack is simply 100 pages of Markdown, the model will again treat it as a long prompt.

Structure is required:

vocabulary
objects
paths
rules
gates
outputs
tests
freeform

Mistake 2. Putting General Rules in a Domain Pack

If a rule applies to any documentation process, it belongs in a Documentation Profile or library, not in one specific Domain Pack.

For example:

Rendered artifact must be validated, not only template.

This is a general rule. It is better placed in a Profile or library.

Mistake 3. Not Separating Examples from Rules

Examples may live in FREEFORM, but they must not become rules without an explicit DOMAIN.RULE.

Mistake 4. Having No Tests

A Domain Pack without tests is an unstable instruction.

It may work today and break after the smallest change.

Mistake 5. Not Pinning the Version

If a Domain Pack changes but its version does not, it becomes impossible to understand why old behavior can no longer be reproduced.

Mini-Exercise

Take any process you know well.

For example:

- creating a historical event;
- preparing a monitoring event;
- reviewing a contract;
- analyzing company risks;
- creating a Jira task;
- preparing QA instructions.

Try to describe its Domain Pack:

1. What is the domain name?
2. Which core terms must be defined?
3. What are the main objects?
4. Which paths exist?
5. Which rules are blocking?
6. Which gates are mandatory?
7. Which outputs are created?
8. Which parts may remain in FREEFORM?
9. What is the minimum test set?
10. Which problems and improvements should be collected after real use?

It does not need to be perfect on the first attempt.

The important thing is to begin distinguishing domain logic from general instructions.

Short Summary

A Domain Pack is a package of subject-matter logic for a specific domain.

Core defines the base language.

Profiles define operating modes.

Libraries provide reusable ready-made solutions.

Domain Packs describe a concrete subject area: vocabulary, objects, paths, rules, gates, outputs, tests, and controlled FREEFORM.

A Domain Pack exists so that the model executes agreed domain rules instead of inventing domain logic itself.

A good Domain Pack is structured, versioned, tested, and visible to the debug/improvement layer.

In complex Ordo processes, the Domain Pack often becomes the main place where real business logic lives.

Chapter 25. Ordo Libraries: include, import, and Reusing Ready-Made Solutions

Why This Is Needed

When an Ordo program grows beyond a few simple steps, repetition appears very quickly. The same contract checks, the same gates before the final result, similar approval rules, identical trace requirements, similar output templates, and recurring test, regression, and coverage rules appear again and again.

Traditional programming languages have long used libraries to solve this problem. A programmer does not write everything from scratch each time. They connect a ready-made module, function, package, or framework. If they need dates, HTTP requests, JSON, or testing, they use an existing library.

Ordo needs a similar mechanism, adapted not to classic code but to controlling AI-model behavior. An Ordo library is not merely a file of functions. It is a ready-made package of Ordo constructs that can be connected to the current Ordo program: gates, nodes, status semantics, output templates, approval chains, test patterns, debug rules, domain vocabulary, and reusable flows.

Without libraries, every large playbook begins to turn into an isolated all-in-one document. It accumulates its own copies of rules, wording, exceptions, and variants of the same checks. Over time, such documents become difficult to maintain: changing one basic rule requires finding it in dozens of places.

Libraries move recurring solutions into separate reusable packages and connect them explicitly.

Simple Explanation

At its simplest, an Ordo Library is a ready-made set of rules and constructs that an Ordo program can be told to use.

For example:



Figure 76: Nebu — idea: a library as an explicit package of behavior

```
include:
- library: "ordo.validation.contract_first"
  version: "0.1"
  as: "contract_first"
```

This means: include the `ordo.validation.contract_first` library, use exactly version `0.1`, and give it the local name `contract_first`.

The Ordo program can then use its parts:

```
use:
- contract_first.gates
- contract_first.assertions
- contract_first.approval_rules
```

Instead of describing basic contract-first execution rules again in every playbook, we include a ready-made library and use its exports.

How a Library Differs from Core, Profile, and Domain Pack

Libraries must not be confused with other Ordo layers.

Ordo Core is the base language. It defines fundamental constructs: intent, contract, state, node, gate, output, status, trace, and assertion.

Ordo Profile is a specialized operating mode or style of using Ordo. Examples include profiles for documentation, QA, approval, artifact generation, and rendered validation.

Domain Pack is a package of knowledge and rules for a specific subject domain, such as a History Event Domain Pack or Monitoring Event Domain Pack.

Ordo Library is a reusable package that may contain pieces of Core logic, profile bindings, domain blocks, or execution patterns, but does not have to be a complete Domain Pack or Profile.

A library may be small: only a set of standard gates for pre-final validation.

It may be medium-sized: a ready-made guided-intake pattern.

It may be large: a complete reusable package for manual QA runbook generation.

The defining feature of a library is reuse.

What an Ordo Library May Contain

An Ordo library may contain:

- ready-made NODE.DEF constructs;
- ready-made GATE.DEF constructs;
- ready-made ASSERT.NOT constructs;
- ready-made STATUS.SEMANTICS;
- ready-made TEMPLATE.BIND constructs;
- ready-made OUTPUT.DEF constructs;
- ready-made RENDER.VALIDATE rules;
- ready-made EVIDENCE.MATRIX schemas;
- ready-made APPROVAL.REQUIRE chains;
- ready-made DEBUG.MODE profiles;
- ready-made TEST.DEF templates;
- ready-made REGRESSION.SUITE patterns;
- ready-made FREEFORM.COVERAGE rules;
- ready-made execution patterns;
- ready-made document templates;
- ready-made domain-specific rule sets.

An Ordo library is therefore a package of behavior, not merely a package of text.

Basic Language Constructs

Ordo needs the following constructs for libraries:

```
LIB.DEF
INCLUDE
IMPORT
USE
EXPORT
NAMESPACE
ALIAS
VERSION.REQUIRE
COMPAT.CHECK
CONFLICT.DETECT
CONFLICT.RESOLVE
OVERRIDE.ALLOW
```


Override.DENY
TRUST.LEVEL

This is the minimum set needed to keep libraries from becoming dangerous.

LIB.DEF

LIB.DEF describes the library itself.

For example:

```
library:
  id: "ordo.validation.contract_first"
  version: "0.1"
  name: "Contract-first validation library"

exports:
  gates:
    - "G_CONTRACT_CONFIRMED"
    - "G_NO_FINAL_OUTPUT_WITHOUT_APPROVAL"

  assertions:
    - "ASSERT_NOT_FINAL_BEFORE_CONTRACT"

requires:
  ordo_version: ">=0.11"
```

This description tells the Ordo compiler what the library contains, what it exports, and which Ordo version it is compatible with.

INCLUDE

INCLUDE means physically or logically connecting a library to an Ordo program.

```
include:
  - library: "ordo.validation.contract_first"
    version: "0.1"
    as: "contract_first"
```

Ordo rule: libraries must not be included implicitly. If a playbook uses a library, this must be visible in source and compiled IR.

Bad:

the model guessed that contract-first rules should be applied

Good:

```
include:
  - library: "ordo.validation.contract_first"
    version: "0.1"
```

IMPORT and USE

IMPORT and USE may have different semantics.

IMPORT means: make specific library exports available to the current Ordo program.

USE means: actually apply included constructs at a specific execution location.

For example:

```
import:
  - from: "contract_first"
    items:
      - "G_CONTRACT_CONFIRMED"
      - "ASSERT_NOT_FINAL_BEFORE_CONTRACT"

use:
  - gate: "contract_first.G_CONTRACT_CONFIRMED"
    at: "before_output_generation"
```

This distinction matters. A library may be included without every part of it being used.



Figure 77: Nebu — thinking: include does not mean automatic use of everything

EXPORT

EXPORT defines what a library allows external programs to use.

For example:

```
exports:
  gates:
    - id: "G_CONTRACT_CONFIRMED"
      visibility: "public"
```

```
internal_rules:
  - id: "R_CONTRACT_PARSE_HELPER"
    visibility: "private"
```

Not everything inside a library must be externally accessible. Some rules may be internal implementation details of the library itself.

NAMESPACE and ALIAS

A namespace prevents naming conflicts.

Without namespaces, two libraries may contain a gate with the same name:

G_APPROVAL_REQUIRED

But those gates may represent different rules.

The correct reference should therefore be fully qualified:

```
contract_first.G_APPROVAL_REQUIRED
artifact_validation.G_APPROVAL_REQUIRED
```

An alias shortens long names:

```
include:
  - library: "ordo.artifact.render_validation"
    version: "0.1"
    as: "render_validation"
```

Then the program can write:

```
use:
  - render_validation.G_RENDERED_ARTIFACT_CHECK
```

Version Pinning

Libraries must be included with a version.

Bad:

```
include:
  - library: "ordo.qa.manual_runbook"
```

Good:

```
include:
  - library: "ordo.qa.manual_runbook"
    version: "0.1"
```

The reason is simple: if the library changes, the playbook may begin to behave differently. This is critical in Ordo because we control model behavior, not merely text.

The version must be part of the execution contract.

Compatibility Check

The Ordo compiler must check library compatibility with the current language, Profile, Domain Pack, and runtime.

For example:

```
compatibility:
  requires_ordo: ">=0.11"
  requires_profiles:
    - "documentation_runtime"
    - "debug_test_improvement"
  incompatible_with:
    - "legacy_all_in_one_mode"
```

In compiled IR, this may become a separate op:

```
{
  "op": "LIB.COMPAT.CHECK",
  "library": "ordo.qa.manual_runbook",
  "version": "0.1",
  "requires_ordo": ">=0.11"
}
```

If the compatibility check fails, Ordo must not silently continue execution.

Conflict Detection

Conflicts between libraries must be explicit.

For example, one library says:

ready_for_first_run = execution may begin

Another says:

ready_for_first_run = manual confirmation is still required

Ordo must not decide silently which one is correct.

There must be a conflict record:

```
conflict:
  type: "status_semantics_conflict"
  key: "ready_for_first_run"
```

```
sources:
  - "library_a"
  - "library_b"
resolution: "required"
```

Possible next actions include:

- ask a human;
- apply an explicitly defined priority;
- block compilation;
- allow an override only with a reason.

Override Rules

Override is one of the most dangerous parts of libraries.

If an included library can silently rewrite a gate, status, or assertion, Ordo's reliability collapses.

The rule is therefore:

Every override must be explicit.

For example:

```
override:
  allow:
    - target: "contract_first.G_NO_FINAL_OUTPUT_WITHOUT_APPROVAL"
      by: "history_event.G_PRE_ARCHIVE_APPROVAL"
      reason: "domain pack has stricter equivalent gate"
```

Without such a record, the override must be denied.



Figure 78: Nebu — attention: an override must be explicit

Trust Level

Not all libraries are equally reliable. Ordo should therefore support trust levels.

For example:

```
library:
  id: "ordo.validation.contract_first"
  version: "0.1"
  trust_level: "official"
```

Possible levels include:

```
official
verified
project_local
experimental
untrusted
```

Production workflows should normally reject experimental or untrusted libraries without separate approval.

Libraries and the Debug/Test/Improvement Layer

Libraries must be visible to debug, testing, and improvement.

If a gate came from a library, the trace should show that:

```
trace_source: "model_self_report"
gate_report:
  - gate_id: "contract_first.G_CONTRACT_CONFIRMED"
    source:
      kind: "library"
      id: "ordo.validation.contract_first"
      version: "0.1"
      status: "passed"
```

If a user identifies a problem, the improvement record must be able to bind it to the library:

```
affected_unit:
  kind: "library"
  id: "ordo.validation.contract_first"
  version: "0.1"
  export: "G_CONTRACT_CONFIRMED"
```

This makes it possible to improve not only one playbook but also the reusable solution.

Libraries and FREEFORM

A library may contain FREEFORM, but it must be controlled FREEFORM.

Bad:

```
freeform:
  text: "there are many rules here; the model will figure them out"
```

Good:

```
freeform:
  id: "FF_CONTRACT_EDGE_CASES"
  binding:
    used_by:
      - "G_CONTRACT_CONFIRMED"
    reason: "domain examples are too nuanced for full formalization"
  coverage:
    required: true
```

If library FREEFORM repeatedly causes errors, the improvement loop should propose formalizing part of it.

Types of Libraries

In practice, Ordo libraries can be divided into several types.

Core Utility Libraries

Contain universal small constructs: standard assertions, common gates, and status helpers.

Profile Libraries

Contain ready-made blocks for documentation, QA, approval, validation, and rendered artifact checks.

Domain Libraries

Contain reusable parts for a specific domain: History Event, Monitoring Event, or Legal Review.

Pattern Libraries

Contain execution patterns: guided intake, contract-first flow, pre-archive approval, or self-check before handoff.

Template Libraries

Contain document templates, output structures, and package layouts.

Connector/Tool Libraries

Contain rules for working with external tools, APIs, files, and runners.

Small Example

Imagine an Ordo program that must create an analytical package.

Without libraries, it may contain dozens of local rules:

- confirm the contract first;
- do not create the final package before approval;
- validate rendered artifacts;
- generate a validation report;
- generate a consistency report;
- capture improvement feedback;
- run regression tests.

With libraries, this can be written as:

```
include:
- library: "ordo.validation.contract_first"
  version: "0.1"
  as: "contract_first"

- library: "ordo.artifact.render_validation"
  version: "0.1"
  as: "render_validation"

- library: "ordo.debug_test.basic_regression"
  version: "0.1"
  as: "regression"

use:
- contract_first.required_contract_gates
- render_validation.pre_handoff_checks
- regression.minimum_suite
```

This is shorter, cleaner, and safer when each library has a version, namespace, compatibility check, and tests.

Typical Mistakes

The first mistake is treating a library as a piece of text. In Ordo, a library must be a structured package that the compiler can validate.

The second mistake is including libraries without versions. This makes behavior unstable.

The third mistake is allowing implicit imports. If the model “guessed” that it should use a library, this is not Ordo execution.

The fourth mistake is failing to record conflicts. Conflicting status semantics or gates must not be resolved silently.

The fifth mistake is allowing hidden overrides. A library must not silently change the behavior of the main playbook.

The sixth mistake is failing to test libraries. If a reusable package has no tests or coverage, it merely spreads defects from one playbook to many playbooks.

Mini-Exercise

Take any large prompt or playbook and find recurring parts.

Write down:

- which gates repeat;
- which output templates repeat;
- which approval rules repeat;
- which checks could move into a reusable library;
- which parts should be exported;
- which parts should remain private;
- which versions and compatibility rules are needed.

Then try to name one library that could exist separately.

For example:

```
ordo.validation.contract_first
ordo.artifact.pre_handoff_validation
ordo.qa.manual_runbook
ordo.debug_test.basic_regression
```

Short Summary

Ordo libraries exist so that the same rules do not have to be rewritten in every playbook.

An Ordo Library is a reusable package of Ordo constructs: gates, nodes, assertions, status semantics, templates, tests, debug rules, domain rules, or execution patterns.

Libraries must be included explicitly through include/import/use and must have a namespace, alias, version pinning, compatibility checks, conflict detection, explicit override rules, and a trust level.

Well-designed libraries make Ordo programs shorter, more stable, and easier to evolve.

Poorly designed libraries create hidden dependencies, conflicts, and uncontrolled changes in model behavior.

The main rule is:

In Ordo, a library is not a hidden text fragment but an explicit, versioned, and ver

Chapter 26. Why All-in-One Is No Longer the Main Format

Why This Is Needed

In many teams, complex instructions for an AI model begin very simply: there is one large document containing everything. It includes rules, examples, scenarios, exceptions, templates, prohibitions, explanations, tables, checklists, notes, change history, and hints for the author.

At first, this is convenient. One file is easy to attach to a chat and easy to open. It appears to contain “everything.” This is why the all-in-one format often becomes the first natural format for a playbook.

But as the playbook grows, this format begins to break down.

The problem is not that a large document is inherently bad. The problem is that a large document is poorly suited to being the **primary executable format** for a complex process.

Ordo changes the role of all-in-one. It may remain a convenient consolidated version for reading, but it should not be the only source of truth for execution.

Simple Explanation

Imagine a 200-page instruction describing how a model should create an analytical package.

It contains:

- scenario selection rules;
- a list of questions for the user;
- prohibitions;
- file formats;
- testing rules;
- examples;
- edge cases;
- templates;

- old notes;
- clarifications added after defects;
- exceptions;
- checklists before the final archive.

When a model receives such a document as continuous text, it does not always understand what is a process rule, what is an example, what is a prohibition, what is a template, and what is a historical note.

In a large all-in-one, different layers are easily mixed:

rule + example + explanation + exception + template + author comment

A human may still be able to read this. For model execution, it is already dangerous.

Ordo proposes a different model:

source documents → selected sections → Ordo Source → compiled IR → execution trace →

A large document is no longer “something the model simply reads from top to bottom.” It becomes one of the sources from which structured execution units are formed.

Why All-in-One Was Useful

It is important not to reject all-in-one completely. It has real advantages.

It is useful for:

- initial knowledge collection;
- quick handoff between people;
- overview reading;
- preserving the complete picture;
- archival versions;
- export into one file;
- quick attachment to a model when no better runtime exists.

At an early stage, all-in-one is often the simplest way to collect chaotic knowledge in one place.

But what is convenient for knowledge collection is not always convenient for controlled execution.

Where All-in-One Begins to Break

The first problem is **finding the correct rule**.

In a large document, a rule may be written in one place, clarified in another, partially superseded in a third, and illustrated by an example in a fourth.

The model may use the rule closest in the text rather than the latest rule. Or it may combine old and new wording.

The second problem is **confusion between a rule and an example**.

For example, the document may contain an event example called LU_CHANGE_STATUS. The model may accidentally treat that example as the mandatory alias for a new case.

The third problem is **skipping control points**.

An all-in-one may say:

a self-check is mandatory before the final archive

But if this is merely a sentence in a long text, the model may miss it. In Ordo, this should not be a sentence but a blocking gate.

The fourth problem is **update complexity**.

When a user says “this must be corrected in the instruction,” it is unclear where the change belongs. In a rule? An example? A checklist? QA? A template? A Domain Pack? A library?

The fifth problem is **the inability to test properly**.

A continuous Markdown document is difficult to test. You can check whether the model responds approximately correctly, but it is difficult to verify:

- which path it selected;
- which node was active;
- which gate fired;
- which state changed;
- which FREEFORM block influenced the decision;
- whether an old scenario broke after a rule change.

The Ordo Approach

In Ordo, the primary artifact is not a large document but a structured execution model.

all-in-one may exist as a rendered artifact, but the execution source of truth should be divided into controlled parts.

A typical structure may look like this:

```
00_EXECUTION_CONTRACT.md
01_CORE_BINDING.md
```

```
02_PROFILE_BINDING.md
03_DOMAIN_PACK.md
04_DECISION_TREE.ordo.yaml
05_OUTPUTS.ordo.yaml
06_GATES.ordo.yaml
07_TESTS.ordo.yaml
08_FREEFORM_LEDGER.md
09_COMPILED_IR.json
10_VALIDATION_REPORT.json
```

Or, more simply:

```
source_docs/
ordo_source/
compiled_ir/
tests/
rendered/
reports/
```

The main idea is:

A human may read a consolidated document, but the model should execute a structured

All-in-One as a Rendered Artifact

In the new model, all-in-one does not disappear. Its status changes.

Previously, it was:

```
main instruction source
```

In Ordo, it becomes:

```
rendered human-readable artifact
```

It may be generated from modular sources as a convenient consolidated version.

This resembles technical documentation: developers may maintain many separate files and then build one PDF or HTML document for reading.

Ordo should know that all-in-one is not necessarily the primary source. It may be a build result.

Why This Matters for Traceability

When a playbook is divided into sections, each decision can be linked to a specific source:

```

trace_source: "model_self_report"
knowledge_trace:
  - source: "04_DECISION_TREE.ordo.yaml"
    node: "NODE_SELECT_PATH"
    used_for: "path selection"

  - source: "06_GATES.ordo.yaml"
    gate: "G_PRE_ARCHIVE_APPROVAL"
    used_for: "blocking final archive"

```

Such a link is also possible in a large all-in-one, but it is less precise. For example:

lines 1420-1480 of the large document

That is better than nothing, but worse than a reference to a specific GATE.DEF, NODE.DEF, or TEST.DEF.

What Documentation Runtime Should Do

Saying that all-in-one is no longer the primary format naturally leads to the next idea: Documentation Runtime is needed.

That is the subject of the next chapter, but the basic idea should be fixed here.

Documentation Runtime should be able to:

- see the document catalog;
- know which sections are needed for which purpose;
- select only relevant docs for the current node;
- build rendered artifacts;
- verify that the consolidated all-in-one does not contradict source files;
- show which source each rule came from.

Instead of “attach one enormous file and hope the model reads everything correctly,” documentation becomes a controlled runtime input.

Ordo Constructs

Ordo can represent this through constructs such as:

```

DOC.CATALOG
DOC.SPLIT
DOC.SELECT
DOC.RENDER
DOC.SOURCE_OF_TRUTH
RENDER.VALIDATE
TRACE.SOURCE

```

Example:

```
doc_catalog:
  id: "history_event_playbook_docs"

  source_of_truth:
    mode: "split_docs"
    all_in_one_role: "rendered_artifact"

  documents:
    - id: "execution_contract"
      path: "00_EXECUTION_CONTRACT.md"
      role: "contract"

    - id: "decision_tree"
      path: "04_DECISION_TREE.ordo.yaml"
      role: "path_selection"

    - id: "gates"
      path: "06_GATES.ordo.yaml"
      role: "gate_definitions"

    - id: "tests"
      path: "07_TESTS.ordo.yaml"
      role: "test_suite"
```

And for the current node:

```
doc_select:
  node: "NODE_SELECT_PATH"
  required_docs:
    - "execution_contract"
    - "decision_tree"
  optional_docs:
    - "domain_examples"
```

Typical Mistake

Bad practice:

We have one large Markdown file. Let the model find what it needs.

Better practice:

We have a document catalog. For each execution stage, we know which sections are required.

Another mistake is assuming that split docs automatically solve the problem.

Splitting a document alone does not create Ordo. You can have 50 small files and still have chaos.

Roles, links, source-of-truth rules, and validation are required.

Mini-Exercise

Take any large instruction you use with an AI model.

Try to divide it by role rather than by size:

- where is the contract described;
- where is the decision tree described;
- where is state described;
- where are gates described;
- where is output described;
- where are examples;
- where is controlled FREEFORM;
- where should tests live;
- what may be rendered as all-in-one;
- what must remain the source of truth.

Then ask the main question:

Can the model execute this process without reading the entire document at once?

If the answer is “no,” the playbook is not yet ready for Ordo execution.

Short Summary

all-in-one is useful as a human overview or archival format, but it works poorly as the primary executable format for complex AI processes.

In Ordo, the source of truth should be structured: contract, nodes, gates, outputs, tests, FREEFORM ledger, libraries, and domain rules should be separated into controlled parts.

A consolidated all-in-one may remain, but as a rendered artifact generated from modular sources and validated.

The main rule is:

In Ordo, a large document may be read, but execution should follow the structured pr

Chapter 27. Documentation Runtime

Why This Is Needed

In the previous chapter, we examined why the all-in-one format stops being the primary format when an instruction grows into a large playbook. One large file is convenient to give to a model, but inconvenient to maintain, validate, update, and debug.

A new question immediately appears: if documentation is split into dozens of separate files, how should the model know which ones are needed for the current step?

A person can open the table of contents, scan titles, recall the context, and choose the right section. A model can also do this, but if selection is left entirely to its judgment, we return to the ordinary prompt problem: it may open the wrong file, miss an important section, or use an outdated part of the documentation.

This is why Ordo needs Documentation Runtime.

Documentation Runtime is a language mechanism that controls how an Ordo program works with a large set of documents: which documents exist, when they must be read, which sections are the source of truth, which documents are supporting material, and which must not be used for decisions.

In other words, this is not merely “a folder of instructions.” It is a controlled knowledge-access layer.

Simple Explanation

Imagine that an Ordo program is a cook and the documentation is a large culinary library.

If the cook is simply given the whole library and told “cook,” they may open an old recipe, confuse a dessert with a main course, or treat a note as a mandatory rule.

Documentation Runtime acts as a catalog and dispatcher:

```
only these documents are needed for this step;  
this document is authoritative;  
this document only explains examples;  
this file is deprecated;  
this template may be used only after an approval gate;  
this rendered artifact must be validated separately.
```

Documentation Runtime does not write business logic instead of a Domain Pack. It ensures that the model uses the correct instructions

at the correct moment.

How Documentation Runtime Differs from Ordinary Documentation

Ordinary documentation describes knowledge.

Documentation Runtime controls the use of knowledge.

Ordinary documentation may say:

QA rules are described in file 05_QA_PACKAGE.

Documentation Runtime should be more precise:

NODE package_generation may read 05_QA_PACKAGE_TEMPLATE.

Before creating the QA file, G_QA_SCOPE_CONFIRMED must be checked.

The final rendered QA file must pass RENDER.VALIDATE.

If fixture or source lookup is not confirmed, the QA file cannot be marked ready.

This is no longer merely a description. It is runtime behavior.

Main Responsibilities of Documentation Runtime

In Ordo, Documentation Runtime should solve several tasks.

The first is document cataloging.

Ordo should know which documents belong to the package:

- README.md
- execution contract
- core binding
- profile binding
- domain pack
- compiled IR
- freeform ledger
- validation report
- source virtual docs
- templates
- examples

The second task is selecting the documents needed for the current node.

The model should not reread everything every time. Each NODE may define the documents that are allowed or required.

The third task is identifying the source of truth.

If split files and an all-in-one version both exist, it must be explicit that split files are edited and the all-in-one is a built artifact. Otherwise, a change may be made in the wrong place.

The fourth task is controlling rendered artifacts.

A template does not mean the final file is correct. Ordo must distinguish a template from the actually generated document.

The fifth task is traceability.

When the model creates a result, it should be possible to see which documents it relied on.

Ordo Constructs

Documentation Runtime requires the following language constructs:

DOC.CATALOG
DOC.DEF
DOC.ROLE
DOC.SOURCE_OF_TRUTH
DOC.SELECT
DOC.BIND
DOC.READ
DOC.RENDER
DOC.SPLIT
DOC.MERGE
DOC.VERSION
DOC.DEPRECATE
DOC.TRACE
RENDER.VALIDATE

In simple terms:

DOC.CATALOG describes the list of documents in a package.

DOC.DEF describes a specific document.

DOC.ROLE identifies a document's role: contract, template, example, source, generated artifact, or validation report.

DOC.SOURCE_OF_TRUTH identifies the authoritative file for changing a particular rule.

DOC.SELECT says which documents are needed for the current node.

DOC.BIND binds a document to a node, gate, output, or library.

DOC.TRACE records which documents were actually used during a run.

RENDER.VALIDATE validates the generated final artifact rather than the template.

Small Example

Suppose we have an Ordo program for creating an analytical package.

In Source format, Documentation Runtime may look like this:

```
doc_catalog:
  id: "history_event_docs"

  documents:
    - id: "execution_contract"
      path: "00_ORDO_EXECUTION_CONTRACT.md"
      role: "contract"
      source_of_truth: true

    - id: "domain_pack"
      path: "04_HISTORY_EVENT_DOMAIN_PACK.md"
      role: "domain_rules"
      source_of_truth: true

    - id: "qa_template"
      path: "05_QA_PACKAGE_TEMPLATE.md"
      role: "template"

    - id: "qa_output"
      path: "05_QA_PACKAGE_<ALIAS>.md"
      role: "rendered_artifact"
      generated: true

doc_select:
  node: "generate_qa_package"
  required:
    - "execution_contract"
    - "domain_pack"
    - "qa_template"
  forbidden:
    - "deprecated_all_in_one"

render_validate:
  output: "qa_output"
  rules:
    - "must_not_contain_unresolved_placeholders"
```

- "must_include_manual_test_table"
- "must_match_confirmed_contract"

The important point is that the model does not merely “know about the documents.” It receives rules defining which documents to use, which not to use, and how to validate the result.

Documentation Runtime and All-in-One

Documentation Runtime does not prohibit all-in-one. It changes its role.

All-in-one may be:

- a convenient read-only snapshot;
- an artifact for transfer to a model;
- an assembled representation of split documents;
- a control view of the complete package.

But it should not automatically be the source of truth.

If sectional files are the source of truth, Documentation Runtime should state this explicitly:

```
source_of_truth:
  rules:
    - target: "playbook_logic"
      document: "source_virtual_docs/"
    - target: "rendered_snapshot"
      document: "999_ALL_IN_ONE.md"
      editable: false
```

This protects against a common mistake: the model edits the large all-in-one file while the actual sectional documents remain outdated.

Documentation Runtime and Trace

When an Ordo program works with documents, the trace should show:

```
which documents were selected;
why they were selected;
which documents were rejected;
which sections were used;
which outputs were generated from them;
whether rendered artifacts passed validation.
```

For example:

```

trace_source: "model_self_report"
doc_trace:
  node: "generate_qa_package"

selected:
  - id: "execution_contract"
    reason: "required for confirmed scope"
  - id: "domain_pack"
    reason: "required for domain rules"
  - id: "qa_template"
    reason: "required for output structure"

rejected:
  - id: "deprecated_all_in_one"
    reason: "not source of truth for editing"

rendered_outputs:
  - id: "qa_output"
    validation: "passed"

```

This trace is especially important in debug mode. If the result is wrong, we can see whether the problem lies in the logic or whether the model simply used the wrong document.

Documentation Runtime and Libraries

After Ordo Libraries appear, Documentation Runtime becomes even more important.

A library may include its own documents:

- descriptions of exported gates;
- templates;
- examples;
- compatibility rules;
- changelog;
- tests;
- coverage report.

Ordo should know that these documents belong to the library rather than the main playbook.

For example:

```

library_docs:
  library: "ordo.validation.contract_first"
  version: "0.1"

```

```
documents:
  - id: "contract_gates"
    path: "docs/gates.md"
    role: "library_rule_reference"

  - id: "contract_tests"
    path: "tests/contract_first_tests.yaml"
    role: "test_suite"
```

If a problem originates from a library rule, DOC.TRACE and IMPROVE-
MENT.RECORD should show exactly that.

Documentation Runtime and FREEFORM

FREEFORM often contains explanations, examples, or exceptions. Documentation Runtime should help preserve control over these blocks.

For every FREEFORM block, it is useful to know:

- which document contains it;
- which node/gate/rule it is bound to;
- whether it is normative, an example, or a note;
- whether the model may make decisions based on it;
- whether coverage exists for it.

For example:

```
freeform_doc_binding:
  freeform_id: "FF_EDGE_CASES"
  document: "04_HISTORY_EVENT_DOMAIN_PACK.md"
  section: "Edge cases"
  binding:
    node: "select_path"
    role: "controlled_explanation"
  decision_allowed: true
```

Without such binding, the model may treat any example as a rule or any note as permission to act.

Typical Mistakes

The first mistake is assuming that a file list is already a runtime.

A file list is only a catalog. Runtime begins when selection rules, roles, gates, and trace exist.

The second mistake is failing to define the source of truth.

If two documents contain similar content, the model must know which one is authoritative.

The third mistake is validating the template instead of the final document.

The template may be correct while the generated artifact contains empty sections, unresolved placeholders, or data that did not come from the confirmed contract.

The fourth mistake is allowing the model to decide by itself which documents “seem relevant.”

For a simple question, that may be acceptable. For a production playbook, it is not. There should be `DOC.SELECT`.

The fifth mistake is failing to trace document usage.

Without `DOC.TRACE`, after an error it will be unclear whether the problem was in a rule or in the wrong document being used.

Mini-Exercise

Take any large instruction set you have worked with.

Try to answer:

1. Which documents belong to the package?
2. Which document is the source of truth?
3. Which documents are only examples?
4. Which documents are templates?
5. Which documents are rendered artifacts?
6. Which document is needed for each step?
7. Which documents must not be used for decisions?
8. How can you verify that the final document matches the template and contract?
9. How can the trace show which documents were used?

If these questions have no clear answers, you do not yet have Documentation Runtime. You only have a folder of documents.

Short Summary

Documentation Runtime is the Ordo layer that controls work with a large set of documents.

It defines:

- which documents exist;
- the role of each document;
- which document is the source of truth;
- which documents are needed for a specific node;

- which documents are forbidden or deprecated;
- how rendered artifacts are validated;
- how documentation usage is traced;
- how documents are linked to libraries, FREEFORM, and improvement records.

The main idea is simple:

a large playbook should not merely be read;
it should be executed through a controlled documentation runtime.

Without Documentation Runtime, the model works with documentation as a large pile of text.

With Documentation Runtime, documentation becomes part of the executable Ordo program.

Chapter 28. Rendered Artifact Validation

Why This Is Needed

In many processes, a model creates more than a chat response. It creates a finished artifact: a document, archive, report, instruction, JSON, YAML, Markdown file, PDF, table, Jira package, QA set, or a set of files for handoff to another team.

At first glance, validating the template may seem sufficient. If the template is correct, the result should also be correct. In real work, this is often not true.

The template may be correct while the generated document is not.

For example:

- a mandatory section exists in the template but is empty in the finished file;
- the package structure expects 11 files, but the archive contains 12;
- README says one thing while the QA file says another;
- JSON contains one alias while Markdown contains another;
- the template defines a gate before the final archive, but the archive has already been created before the gate passes;
- a placeholder accidentally remains in the rendered document;
- an unwanted technical fragment appears in the final file.

This is why Ordo must validate not only intent, source program, and template. Ordo must validate the finished result after rendering.

This is Rendered artifact validation.



Figure 79: Nebu — idea: validate the artifact that was actually created

Simple Explanation

Rendered artifact validation means validating the artifact that has already been created, not only the rules that were supposed to create it.

Think of it this way.

A template is a building blueprint.

A rendered artifact is the building after construction.

Checking the blueprint is useful, but insufficient. After construction, the building itself must be inspected: are the doors present, do the stairs lead to the right place, was the roof forgotten, is the electricity connected correctly, and do the rooms match the plan?

The same is true in Ordo:

template validation → validates the plan

rendered artifact validation → validates the actual result

Why This Is a Separate Ordo Layer

In ordinary prompt-based processes, a model often says:

I checked the result.

But this does not always mean that the actual structure of the finished artifact was inspected.

Ordo should make the process more precise:

1. Generate the artifact.
2. Read or analyze the generated artifact itself.
3. Compare it with the output contract.
4. Check mandatory sections.

5. Check consistency between files.
6. Check forbidden elements.
7. Produce a validation report.
8. Only then allow handoff.

Rendered validation is therefore a gate between result creation and delivery to the user.

Main Construct

In Ordo, this mechanism can be described as:

RENDER.VALIDATE

This construct means:

validate the already generated artifact as the actual output, not merely as an expected structure

In a simple form:

```
render_validate:  
  target: "final_package"  
  against:  
    - "output_contract"  
    - "file_manifest"  
    - "mandatory_sections"  
    - "consistency_rules"  
    - "forbidden_content"  
  report: "VALIDATION_REPORT.json"  
  blocking: true
```

The key word is blocking.



Figure 80: Nebu — attention: failed validation blocks handoff

If rendered validation does not pass, the result cannot be considered ready.

What Is Validated

Rendered artifact validation may inspect several levels.

1. File Presence

For example, a package may be required to contain exactly these files:

```
README.md
SUMMARY.json
VALIDATION_REPORT.json
CONSISTENCY_CHECK_REPORT.json
01_HISTORY_EVENT_PASSPORT_<ALIAS>.md
02_JIRA_TASK_<ALIAS>.md
04_IMPLEMENTATION_PROMPT_<ALIAS>.md
05_QA_PACKAGE_<ALIAS>.md
07_PROCESS_IMPROVEMENT_FEEDBACK_<ALIAS>.md
08_QA_AUTOMATION_SPEC_<ALIAS>.yaml
09_QA_AUTOMATION_README_<ALIAS>.md
```

Rendered validation must check not only that these files are described in the instruction, but that they actually exist in the final package.

It should also check for extra files when the package is required to be a compact canonical package.

2. Mandatory Sections

For example, a Jira task may be required to contain:

- a general problem description;
- expected behavior;
- acceptance criteria;
- test scenarios;
- constraints;
- dependencies;
- out-of-scope items.

Rendered validation should open the finished Markdown file and verify that these sections are actually present.

3. Absence of Placeholders

This is a very practical check.

Bad result:

```
<ALIAS>
<TODO>
<CONFIRM_SOURCE_FIELD>
[insert description here]
```

If a placeholder remains in the final artifact, the result is almost always not ready.

Ordo may have a rule:

```
assert_not:
  rendered_artifact_contains:
    - "<TODO>"
    - "<ALIAS>"
    - "TBD"
    - "insert here"
```

4. Consistency Between Files

This is one of the most important parts.

In a complex package, the same information often appears in several places:

- alias;
- event name;
- source field;
- status;
- expected output;
- test case id;
- route/path;
- file list;
- change level;
- no-op behavior;
- rollback rules.

Rendered validation should verify that these values do not contradict one another.

For example:

```
README.md says: alias = LU_CHANGE_STATUS
QA_PACKAGE.md says: alias = LU_CHANGE_CAPITAL
```



Figure 81: Nebu — thinking: consistency often breaks between files

This is not a minor mistake. It is a consistency-gate failure.

5. Compliance with the Output Contract

The output contract defines exactly what must be created.

Rendered artifact validation checks:

whether the result actually matches what was promised

If the contract says:

create only an analytical package, without technical implementation

the rendered artifact must not suddenly contain:

ready Java code
database migrations
configuration changes
real endpoints

If the contract says:

do not include extra files

an archive containing extra payload files must be blocked.

6. Forbidden Actions or Forbidden Content

Rendered validation should check not only what must exist but also what must not exist.

For example:

```
assert_not:  
- "final archive before approval"  
- "invented source row"  
- "unconfirmed alias"
```

- "hidden implementation details"
- "extra YAML with local secrets"

This is especially important when a model works with archives, code, configurations, or documentation for other teams.

Example Ordo Source

```
output:
  id: "history_event_package"
  type: "archive"
  required_files:
    - "README.md"
    - "SUMMARY.json"
    - "VALIDATION_REPORT.json"
    - "CONSISTENCY_CHECK_REPORT.json"
    - "01_HISTORY_EVENT_PASSPORT_<ALIAS>.md"
    - "02_JIRA_TASK_<ALIAS>.md"
    - "04_IMPLEMENTATION_PROMPT_<ALIAS>.md"
    - "05_QA_PACKAGE_<ALIAS>.md"
    - "07_PROCESS_IMPROVEMENT_FEEDBACK_<ALIAS>.md"
    - "08_QA_AUTOMATION_SPEC_<ALIAS>.yaml"
    - "09_QA_AUTOMATION_README_<ALIAS>.md"

render_validation:
  id: "G_RENDERED_PACKAGE_VALID"
  target: "history_event_package"
  blocking: true
  checks:
    - required_files_present
    - no_extra_files
    - no_placeholders
    - alias_consistency
    - required_sections_present
    - validation_report_present
    - consistency_report_present
```

Example Semantic JSON IR

```
{
  "op": "RENDER.VALIDATE",
  "id": "G_RENDERED_PACKAGE_VALID",
  "target": "history_event_package",
```

```

"blocking": true,
"checks": [
  "required_files_present",
  "no_extra_files",
  "no_placeholders",
  "alias_consistency",
  "required_sections_present",
  "validation_report_present",
  "consistency_report_present"
],
"on_fail": {
  "status": "blocked",
  "handoff_allowed": false,
  "required_action": "fix_rendered_artifact"
}
}

```

Relationship with Gates

Rendered validation should almost always be a gate.

Not merely a recommendation:

it is advisable to check the result

But a gate:

the result cannot be handed off until rendered validation has passed

In Ordo, it should therefore be connected to the gate system:

```

gate:
  id: "G_RENDERED_OUTPUT_VALID"
  method: mechanical
  trust_class: deterministic
  type: "rendered_artifact_validation"
  blocking: true
  required_before:
    - "handoff"
    - "archive_delivery"
    - "send_to_developer"

```

Relationship with the Debug Layer

In debug mode, rendered validation should show:

- which files were checked;
- which sections were found;
- which rules passed;
- which rules failed;
- which values were compared;
- exactly where an inconsistency was found;
- which gate blocked handoff.

For example:

```

trace_source: "model_self_report"
render_validation_trace:
  target: "final_archive"
  checks:
    - id: "required_files_present"
      status: "passed"
    - id: "no_extra_files"
      status: "failed"
      evidence:
        extra_files:
          - "payload_update.json"
    - id: "alias_consistency"
      status: "passed"

result:
  gate: "G_RENDERED_PACKAGE_VALID"
  status: "blocked"

```

Relationship with the Test Layer

Rendered artifact validation also needs to be tested.

For example:

```

test:
  id: "TC_RENDER_BLOCKS_EXTRA_FILE"
  method: mechanical
  trust_class: deterministic

fixture:
  package_files:
    - "README.md"
    - "SUMMARY.json"
    - "payload_update.json"

expected:

```

```

gate:
  id: "G_RENDERED_PACKAGE_VALID"
  status: "blocked"

reason:
  - "extra_file_detected"
Or:
test:
  id: "TC_RENDER_BLOCKS_PLACEHOLDER"
  method: mechanical
  trust_class: deterministic

fixture:
  file_content:
    path: "02_JIRA_TASK.md"
    content: "Alias: <ALIAS>"

expected:
  gate:
    id: "G_RENDERED_OUTPUT_VALID"
    status: "blocked"

violation:
  - "placeholder_detected"

```

Relationship with the Feedback & Improvement Loop

If a user receives an artifact and says:

README contains one event name, while the QA file contains another.

Ordo should create an improvement record:

```

improvement_record:
  classification:
    type: "rendered_artifact_inconsistency"
    severity: "high"

  affected_unit:
    kind: "render_validation_rule"
    id: "alias_consistency"

  proposed_patch:

```

- "add consistency check between README.md and QA_PACKAGE.md"
- "add regression test for mismatched event display name"

suggested_tests:

- "TC_RENDER_BLOCKS_DISPLAY_NAME_MISMATCH"

Every defect in a final artifact should therefore become a source of improvement for validation rules.

Typical Mistakes

Mistake 1. Validating Only the Template

The template may be correct while the result is wrong.

Ordo must validate the rendered artifact.

Mistake 2. Treating Self-Check as a Textual Promise

The phrase:

I checked the result

is not a validation report.

A structured gate report is required.

Mistake 3. Not Checking Consistency Between Files

In large packages, major defects often exist not inside one file but between files.

Mistake 4. Allowing Handoff After Failed Validation

If validation failed, handoff must be blocked.

Mistake 5. Not Testing Validation Rules

Validation rules can also be incomplete. They must be tested as part of the Ordo program.

Mini-Exercise

Take any document or file package that a model is expected to create.

Try to answer:

1. Which files or sections are mandatory?
2. Which placeholders must not remain?
3. Which values must be identical in different locations?
4. Which extra files or sections must not exist?
5. Which gate should block delivery of the result?
6. Which validation report should be produced?

If you can answer these questions, you already have the basis for RENDER.VALIDATE.

Short Summary

Rendered artifact validation is the validation of the actual result after it has been created.

In Ordo, this matters because a model may understand the template correctly but still make a mistake in the finished document or package.

The main rule is:

an unvalidated rendered artifact cannot be handed off as a finished result

For complex processes, RENDER.VALIDATE should be a blocking gate before handoff.

Independent validation and atomic-unit coverage

A producer cannot validate its own output by declaring passed. The producer and validator must be separate runtime responsibilities, even when the same underlying model is used in two isolated invocations.

A machine-checkable validation result should identify:

```
validation_result:  
  validator_id: "V_PACKAGE"
```

```

target_id: "package"
target_revision: 3
target_sha256: "..."
validation_contract_id: "package_contract.v1"
mandatory_units:
  - unit_id: "README"
    status: passed
  - unit_id: "runtime_contract"
    status: passed
aggregate_status: passed

```

For a composite artifact, document-level success cannot hide a failed mandatory unit. Aggregate passed is allowed only when every mandatory atomic unit passes. Validation instructions that are missing or ambiguous produce a blocking state rather than inferred criteria.

Chapter 29. The Old Playbook as a Markdown Knowledge Base

Why This Is Needed

Many complex instructions are born not as a language, a program, or a formal workflow. They begin as a large Markdown document: rules, examples, exceptions, tables, reminders, checklists, historical explanations, and decisions once made in chats or tasks.

Such a document can be extremely useful. It preserves team knowledge. It explains context. It shows how an analyst thinks about the process. The problem is that a large Markdown playbook is not an executable instruction.



Figure 82: Nebu — idea: a Markdown playbook is a knowledge base, not a runtime

A model can read it, but that does not mean it will execute it the same way every time.

It may:

- skip an important gate;
- confuse an example with a rule;
- treat an outdated fragment as current;
- use an instruction for the wrong path;
- forget that a decision requires human confirmation;
- read an all-in-one document as ordinary text rather than as a process.

This is why an old Markdown playbook should be treated not as the final format but as a knowledge base from which an Ordo program is gradually extracted.

Simple Explanation

An old playbook is like a large folder of instructions on a desk.

It may contain everything:

- process descriptions;
- business rules;
- technical rules;
- examples;
- exceptions;
- templates;
- checklists;
- decision history;
- notes for the future;
- warnings about mistakes.

But if a person or model is told to “just execute everything in this folder,” a problem appears: it is unclear what is mandatory, what is reference material, what is an example, what is an old note, and what is an active rule.

Ordo proposes a different approach:

Markdown playbook → knowledge source → structured extraction → Ordo Source → compile

The old playbook is not discarded. It becomes a knowledge source. But execution should happen not directly from chaotic text, but through a structured Ordo layer.

What Is Useful in an Old Playbook

In an old Markdown playbook, it is important not only to find ready-made rules. Different types of information must also be classified correctly.

For example:

1. Mandatory rules
2. Decision tree
3. Questions / intake flow
4. Gates
5. Output templates
6. QA rules
7. Validation rules
8. Examples
9. Anti-patterns
10. Domain notes
11. Historical decisions
12. Process improvement feedback

These do not all have the same status.

A mandatory rule should become a gate or assertion.

A decision tree should become a set of `NODE.DEF` and path rules.

A template should become `OUTPUT.DEF` or `TEMPLATE.BIND`.

An example may remain in `FREEFORM`, but with a binding to a specific rule or node.

A warning about a common mistake may become `ASSERT.NOT`.

User feedback may become an `IMPROVEMENT.RECORD` or regression test.

Why All-in-One Markdown Is Dangerous

An all-in-one document appears convenient because everything is in one place. But as an execution format, it is dangerous.



Figure 83: Nebu — attention: all-in-one is dangerous as a runtime source

Reasons include:

- the model may get lost in a long context;
- different parts of the document may contradict one another;
- an old note may look like an active rule;
- an example may be executed incorrectly as a universal template;
- it is difficult to understand which rules cover which path;
- it is difficult to debug which fragment influenced a decision;
- it is difficult to test whether a change broke an old scenario.

In Ordo, all-in-one may be useful as a rendered artifact or archival form, but not as the main runtime source.

The main runtime source should be structured.

How Ordo Views a Markdown Playbook

Ordo does not say:

Markdown is bad.

Ordo says:

Markdown is good for explanation, but insufficient for controlled execution.

The old playbook should therefore be divided into layers:

1. Human explanation layer
Text for people.
2. Ordo Source layer
Human-readable structured program.
3. Semantic JSON IR layer
Machine-oriented execution map.
4. Debug/Test layer
Traces, tests, coverage, improvement records.

Markdown may remain in the first layer. But everything that affects execution must either be formalized or explicitly bound as controlled FREEFORM.

Example of Fragment Classification

Imagine that an old playbook contains this sentence:

Before creating the final archive, a self-check must be performed and the package mu

In Markdown, this is merely a sentence.

In Ordo, it should become:

```
- op: "GATE.DEF"
  id: "G_PACKAGE_SELF_CHECK"
  type: "blocking"
  before: "HANDOFF.FINAL_PACKAGE"
  requires:
    - "validation_report.status == passed"
    - "unexpected_files == []"
```

And also a negative assertion:

```
- op: "ASSERT.NOT"
  id: "A_NO_HANDOFF_WITHOUT_SELF_CHECK"
  condition: "final_package_created == true and self_check_passed != true"
  severity: "blocking"
```

And a test case:

```
test:
  id: "TC_NO_PACKAGE_WITHOUT_SELF_CHECK"
  method: mechanical
  trust_class: deterministic
  expected:
    gate: "G_PACKAGE_SELF_CHECK"
    final_package_created: false
```

This is how one sentence from Markdown becomes an executable part of an Ordo program.

What Should Not Be Formalized Immediately

Not every part of an old playbook needs to be converted into strict opcodes immediately.

Some parts may temporarily remain in controlled FREEFORM:

- long business explanations;
- examples for an analyst;
- historical reasons why a rule exists;
- edge cases that have not yet stabilized;
- term explanations;
- domain commentary.

But every FREEFORM block should have a binding:

```

freeform:
  id: "FF_HISTORY_EVENT_EDGE_CASES"
  bound_to:
    - "NODE.SELECT_PATH"
    - "G_SOURCE_FIELD_CONFIRMED"
  reason: "domain examples are not yet fully formalized"

```

Without binding, FREEFORM becomes chaotic Markdown again.

The Role of Traceability

When migrating an old playbook, it is important not to lose the connection between the old text and the new Ordo constructs.

A traceability matrix is therefore needed:



Figure 84: Nebu — thinking: preserve the connection between old text and Ordo objects

old fragment → Ordo Source object → IR op → test coverage → rendered artifact

For example:

```

traceability:
  source_fragment: "section_12.self_check_before_archive"
  mapped_to:
    - "G_PACKAGE_SELF_CHECK"
    - "ASSERT.NOT.A_NO_HANDOFF_WITHOUT_SELF_CHECK"
    - "TC_NO_PACKAGE_WITHOUT_SELF_CHECK"

```

This makes it possible to answer:

- where does this rule now live in Ordo?
 - is it covered by a test?
 - is it blocking?
 - does it enter compiled IR?
 - is it used during execution?
-

Typical Mistakes

Mistake 1. Simply Renaming Markdown to Ordo

If an old Markdown document is renamed Ordo Source, nothing changes.

Ordo requires structure: nodes, gates, state, outputs, tests, and traceability.

Mistake 2. Moving Everything into FREEFORM

FREEFORM is a controlled escape hatch, not a dumping ground for unformatted rules.

If everything remains in FREEFORM, Ordo gains no controllability.

Mistake 3. Losing Decision History

During formalization, it is easy to discard explanations of why a rule appeared.

But history matters for future improvement. It can be preserved as a note, evidence, or improvement history.

Mistake 4. Not Adding Tests

Migration without tests is a cosmetic change.

An Ordo migration should end with a test layer and coverage report.

Mistake 5. Not Validating Rendered Artifacts

If a playbook generates documents or packages, the finished result must be validated, not only the rules.

Mini-Exercise

Take any large instructional document.

Mark its fragments using five colors or categories:

1. rule
2. gate
3. example
4. explanation
5. warning / anti-pattern

Then, for each rule, try to answer:

- should this become NODE, GATE, ASSERT.NOT, OUTPUT, or FREEFORM?
- does it need a test?
- should this rule be blocking?
- does it always apply, or only to one path?

This is the first step toward migrating an old playbook into Ordo.

Short Summary

An old Markdown playbook is a valuable knowledge base, but not a reliable execution format.

Ordo does not destroy Markdown. It extracts executable structure from it:

rules → gates
questions → nodes
templates → outputs
warnings → ASSERT.NOT
examples → controlled FREEFORM
decisions → state/status semantics
feedback → improvement records

The main idea of this chapter is:

an old playbook is a knowledge source, not the runtime execution layer

Chapter 30. Migrating a Playbook to Ordo

Why This Is Needed

Once we understand that an old Markdown playbook is a knowledge base, the next question appears: how do we turn it into an Ordo program?

This should not be a mechanical rewriting of text. Migrating a playbook to Ordo is a process in which we gradually extract executable structure from the document:

intent → contract → context → state → paths → nodes → gates → outputs → tests → coverage

The goal of migration is not to make the document “prettier.” The goal is to make the process controllable, traceable, testable, and suitable for repeatable execution.



Figure 85: Nebu — idea: migration turns knowledge into executable structure

Simple Explanation

Migrating a playbook to Ordo is similar to turning a large human instruction into a program for controlled execution by a model.

Before:

Read the instruction, understand the context, ask questions, remember to check the p

After:

```
intent:
  id: "create_history_event_package"

contract:
  requires:
    - "alias_confirmed"
    - "source_field_confirmed"
    - "expected_values_confirmed"

nodes:
  - id: "N_SELECT_PATH"
  - id: "N_COLLECT_ALIAS"
  - id: "N_COLLECT_SOURCE_FIELD"

required_gates:
  - "G_CONTRACT_CONFIRMED"
  - "G_PRE_ARCHIVE_APPROVAL"
  - "G_RENDERED_ARTIFACT_VALIDATED"
```

In other words, Ordo makes the hidden logic of a playbook explicit.



Figure 86: Nebu — thinking: hidden logic should become explicit nodes, state, and gates

Main Migration Stages

Migration is best described as a sequence of ten steps.

1. Inventory
 2. Classification
 3. Contract extraction
 4. Decision tree extraction
 5. State model extraction
 6. Gate extraction
 7. Output mapping
 8. FREEFORM binding
 9. Test layer creation
 10. IR compilation and validation
-

Step 1. Inventory

First, we need to understand what the playbook actually contains.

Create a catalog of fragments:

```
source_catalog:
  sections:
    - id: "S01_INTRO"
      type: "explanation"
    - id: "S02_DECISION_TREE"
      type: "decision_logic"
    - id: "S03_OUTPUT_PACKAGE"
      type: "output_rules"
    - id: "S04_QA"
      type: "qa_rules"
```

This step helps prevent knowledge from being lost during migration.

Step 2. Classification

Next, classify every fragment.

Minimum categories include:

- rule
- node
- gate
- status
- output
- template
- example
- anti-pattern
- freeform note
- improvement note

For example:

```
fragment:
  id: "F_SELF_CHECK_BEFORE_ARCHIVE"
  original_type: "markdown paragraph"
  classified_as:
    - "gate"
    - "assert_not"
    - "test_candidate"
```

This matters because different fragment types become different Ordo constructs.

Step 3. Contract Extraction

Every complex playbook has conditions without which the work cannot be considered sufficiently defined.

For a History Event package, such conditions may include:

- confirmed alias;
- confirmed event type;
- confirmed source field;
- understood old/new values;
- understood path;
- confirmed expected outputs;
- confirmed QA expectations.

In Ordo, this becomes a contract:

```
contract:
  required_inputs:
    - "event_alias"
    - "source_field"
    - "path_type"
    - "expected_values"

  missing_input_behavior:
    action: "ask_next_required_question"
    final_output_allowed: false
```

The main rule is:



Figure 87: Nebu — attention: without a complete contract, final output is not allowed

if the contract has not been collected, the playbook must not proceed to final output

Step 4. Decision Tree Extraction

If a playbook contains different scenarios, they should be extracted into a decision tree.

For example:

```
paths:
  - id: "A1"
    name: "direct source field change"
    condition: "change detected in primary source row"

  - id: "A2"
    name: "related entity change"
    condition: "change belongs to entity linked through identification center"

  - id: "A4"
    name: "external history event"
    condition: "input is ExternalHistoryEvent candidate"
```

In debug mode, Ordo should show not only the selected path but also rejected paths:


```
path_explain:
  selected: "A1"
  rejected:
    - path: "A2"
      reason: "related entity was not confirmed"
    - path: "A4"
      reason: "no external event input"
```

Step 5. State Model Extraction

Markdown often hides state inside prose.

For example:

When the analyst confirms the source field, the process may proceed to values.

In Ordo, this should become a state transition:

```
state:
  source_field_confirmed: true
  next_node: "N_COLLECT_VALUES"
```

You need to define:

- which state fields exist;
- who may change them;
- which values mean readiness;
- which values block the process;
- which transitions are allowed;
- which transitions are forbidden.

Without a state model, the model will “keep the process in its head,” which is unreliable.

Step 6. Gate Extraction

A gate is a point where the process must stop or check a condition.

During migration, look for phrases such as:

- before;
- only after;
- must be checked;
- must not be created;
- must be confirmed;
- if undefined, stop;
- do not continue without this.

Such phrases are almost always candidates for GATE.DEF or AS-SERT.NOT.

Example:

```
- op: "GATE.DEF"
  id: "G_PRE_ARCHIVE_APPROVAL"
  type: "blocking"
  before: "OUTPUT.FINAL_ARCHIVE"
  condition: "user_approval == true"
```

Step 7. Output Mapping

Next, define exactly what the playbook must create.

For a simple process, this may be one document. For a complex one, it may be a package of files.

Ordo should describe output explicitly:

```
outputs:
- id: "O_ANALYTICAL_PACKAGE"
  kind: "file_set"
  required_files:
  - "README.md"
  - "SUMMARY.json"
  - "VALIDATION_REPORT.json"
  - "CONSISTENCY_CHECK_REPORT.json"
```

Rendered validation must also be described:

```
render_validate:
  required:
  - "all_required_files_present"
  - "no_unexpected_files"
  - "no_unresolved_placeholders"
  - "cross_file_consistency_passed"
```

Output without validation is a weak point.

Step 8. FREEFORM Binding

After the first seven steps, part of the playbook will still remain unformalized.

That is normal.

But it should be represented as controlled FREEFORM:

```
freeform:
  id: "FF_DOMAIN_EDGE_CASES"
  type: "domain_notes"
  bound_to:
    - "PATH.A4"
    - "G_EXTERNAL_EVENT_NORMALIZED"
  reason: "edge cases are described as examples and not yet stable enough for full f
```

The main rule is:

FREEFORM must be bound to a specific part of execution

Step 9. Test Layer Creation

After migration, create a test layer.

The minimum set includes:

- one happy-path test for each path;
- negative tests for forbidden actions;
- gate tests;
- no-op tests;
- rendered artifact validation tests;
- regression suite.

Example:

```
test:
  id: "TC_A1_HAPPY_PATH"
  fixture:
    user_message: "create a status change event"
  expected:
    path: "A1"
  gates:
    - id: "G_CONTRACT_CONFIRMED"
      status: "passed"
  output:
    package_created: true
```

Testing is what distinguishes an Ordo migration from merely rewriting a document.

Step 10. IR Compilation and Validation

The final step is to compile Ordo Source into Semantic JSON IR and verify that all important parts of the playbook are represented.

The result should include:

- compiled IR;
- traceability report;
- validation report;
- consistency check report;
- coverage report;
- freeform coverage report;
- regression suite summary.

If part of the playbook is not covered, that is not always an error. But it must be visible.

For example:

```
migration_report:
  structured_core: "72%"
  controlled_freeform: "28%"
  uncovered_fragments: 0
  blocking_gates_defined: 14
  tests_defined: 18
  status: "passed_with_notes"
```

Minimum Migration Result

After migration, the result should not be one large file but a set of linked artifacts.

For example:

```
START_HERE_ORDO.md
ORDO_EXECUTION_CONTRACT.md
ORDO_CORE_BINDING.md
ORDO_PROFILE_BINDINGS.md
DOMAIN_PACK.md
PLAYBOOK_ORDO_SOURCE.md
PLAYBOOK_COMPILED_IR.json
FREEFORM_LEDGER.md
FREEFORM_COVERAGE.md
VALIDATION_REPORT.json
CONSISTENCY_CHECK_REPORT.json
```

This is not simply “more files.” These are different runtime views of the same process.

How to Know Whether Migration Succeeded

Migration is successful if you can answer “yes” to the following questions:

1. Is it clear which entry starts the process?
2. Is there a contract?
3. Is there a state model?
4. Are there decision paths?
5. Are there blocking gates?
6. Are outputs defined?
7. Is rendered validation present?
8. Is it clear what remains in FREEFORM?
9. Are there tests?
10. Can you explain why a run followed a particular path?

If the answer is “no” to several items, the playbook has not yet become a complete Ordo program.

Typical Mistakes

Mistake 1. Starting with IR

Do not begin by writing JSON IR manually.

First understand the contract, paths, state, and gates. IR is a compiled representation, not the first document from which work should begin.

Mistake 2. Losing the Link to the Old Playbook

If, after migration, you cannot say where a rule came from, that is a problem.

A traceability matrix is required.

Mistake 3. Formalizing Examples as Rules

An example shows a possible situation. A rule defines mandatory behavior.

These are different things.

Mistake 4. Failing to Identify Blocking Gates

If a gate is not blocking, the model may continue even after a failed condition.

For critical processes, this is unacceptable.

Mistake 5. Not Adding an Improvement Loop

The playbook will not become perfect after the first migration.

A mechanism for collecting problems and improvements should exist from the start.

Mini-Exercise

Take one section of an old playbook and try a mini-migration.

Fill in this table:

Fragment	Type	Ordo object	Gate?	Test needed?	FREEFORM?
----------	------	-------------	-------	--------------	-----------

For example:

“Do not create an archive without self-check”

→ anti-pattern / gate candidate

→ ASSERT.NOT + GATE.DEF

→ yes

→ yes

→ no

This is a simple way to see how text begins to turn into an Ordo program.

Short Summary

Migrating a playbook to Ordo is not rewriting a document in different words.

It is the extraction of executable structure:

knowledge → rules → state → paths → gates → outputs → tests → IR

The main idea of this chapter is:

an Ordo migration succeeds when a playbook can not only be read, but also executed,

The ARF legacy-instruction migration route

ARF treats legacy instruction migration as a dedicated factory mode rather than as informal rewriting. The route is:

- source intake
- clause inventory
- dependency reconstruction
- Ordo mapping
- completeness gate
- shared package-generation tail

source intake records the authoritative source set. clause inventory ensures that every meaningful rule, prohibition, template dependency, approval, and output obligation is accounted for. dependency reconstruction makes hidden ordering and state dependencies explicit. Ordo mapping assigns clauses to contracts, nodes, gates, state, outputs, validators, or bounded freeform. The completeness gate blocks integration while any authoritative clause remains unmapped, duplicated without justification, or weakened.

Chapter 31. Guided Intake as an Ordo Program

Why This Is Needed

In many real processes, a model should not create the final result immediately. It first needs to collect data, clarify context, move through a decision tree, check gates, and only then create a document, archive, report, or other artifact.

This mode of work is called guided intake.

Simply put, guided intake is a controlled interview with the user in which the model does not improvise the order of questions but follows a defined Ordo program.

This is especially important for complex playbooks, where the wrong first question can break the entire process. For example, the model may ask for an event name too early when it should first determine the path, or it may begin generating the final package before the contract is confirmed.

In an ordinary prompt-based approach, guided intake often looks like this:

Ask me several questions to collect the information.

But that is insufficient. The model may:

- ask questions in the wrong order;
- skip a mandatory question;
- confuse a previous answer with a confirmed fact;
- proceed to the result too early;
- fail to record which path was selected;
- fail to explain why some options were rejected and others accepted.

In Ordo, guided intake is not merely a “conversation.” It is a complete Ordo program.

Simple Explanation

Guided intake can be imagined as a form filled in through dialogue rather than on a single page.

The important difference is that this “form” has logic:

```
if the answer is this → go here;
if data is missing → ask a question;
if a gate has not passed → stop;
if the contract is confirmed → proceed to the next stage;
if the user changes a decision → update state and return to the required node.
```

Guided intake is therefore a combination of:

ENTRY → NODE → STATE → QUESTION → ANSWER → GATE → PATH → NEXT NODE

In such a process, the model should not merely “be helpful.” It must act as a controlled process operator.

How Guided Intake Differs from Ordinary Dialogue

Ordinary dialogue may be free-form. The user says something, the model responds, the topic changes, and later the model asks for clarification.

Guided intake has a different nature.

Every question has a purpose:

- the question collects a specific state field;
- the question is bound to a specific NODE;
- the answer must be classified;
- after the answer, a state update is performed;
- then a gate is checked;
- then the next NODE is determined.

For example, if an Ordo program creates a new historical event, the question:

What is the event alias?

is not merely a conversational question. It is an operation that collects a contract field:

```
node:  
  id: "N_COLLECT_ALIAS"  
  asks_for: "event.alias"  
  required: true  
  next_if_answered: "N_COLLECT_SOURCE_FIELD"
```

Basic Guided Intake Elements

Guided intake requires the following Ordo elements:

- ENTRY.DEF
- NODE.DEF
- QUESTION.DEF
- ANSWER.REGISTRY
- STATE.SCHEMA
- STATE.UPDATE
- PATH.SELECT
- GATE.REPORT
- NEXT.NODE
- STATUS.SEMANTICS

ENTRY.DEF

Defines where the process begins.

```
entry:
  id: "history_event_intake"
  purpose: "guided intake for a new History Event"
  start_node: "N_CLASSIFY_INPUT"
```

NODE.DEF

Describes one dialogue node.

```
node:
  id: "N_CLASSIFY_INPUT"
  type: "decision"
  purpose: "understand what kind of event request this is"
  allowed_next:
    - "N_PATH_A1"
    - "N_PATH_A2"
    - "N_PATH_A4"
    - "N_NEED_MORE_CONTEXT"
```

QUESTION.DEF

Describes which question may be asked.

```
question:
  id: "Q_SOURCE_FIELD"
  node: "N_COLLECT_SOURCE_FIELD"
  text: "Which source field is changing?"
  writes_to: "state.contract.source_field"
  required: true
```

ANSWER.REGISTRY

Records user answers not as “text in chat” but as state values.

```
answer:
  question_id: "Q_SOURCE_FIELD"
  raw_text: "status"
  normalized_value: "item.status"
  confidence: "confirmed"
```

STATE.UPDATE

Shows exactly what changed after an answer.

```
state_update:
  field: "contract.source_field"
  before: null
  after: "item.status"
  source: "user_answer"
```

One Main Question at a Time

An important guided-intake rule is:

one NODE – one main question

This does not mean the model can never ask a clarification. But the main movement of the process should remain controlled.

Bad:

What are the alias, source field, old value, new value, display name, path, and QA s

This overloads the user and mixes several state transitions.

Good:

Current step: we need to determine the event alias.

Which alias should we use?

After the answer, the model updates state and proceeds to the next node.

Current Status Must Be Visible

Guided intake must always know where it is.

A minimum service status may look like this:

```
intake_status:
  current_entry: "history_event_intake"
  current_node: "N_COLLECT_ALIAS"
  selected_path: "A1"
  confirmed:
    - "event_type"
    - "source_row"
  pending:
    - "alias"
    - "source_field"
    - "old_new_values"
```

```
blocked_by:
  - "G_CONTRACT_COMPLETE"
```

In normal mode, the model does not necessarily show the complete status to the user. But it must be available in debug mode.

Path Selection in Guided Intake

One of the main functions of guided intake is selecting the correct path.

For example:

A1 – field change in the primary source row;
A2 – field change in a related entity;
A4 – external ExternalHistoryEvent;
A5 – no-op or expected-no-change scenario.

The model must not simply guess. It must execute PATH.SELECT.

```
trace_source: "model_self_report"
path_selection:
  candidate_paths:
    - id: "A1"
      condition: "direct source field change"
    - id: "A2"
      condition: "related entity through identification center"
    - id: "A4"
      condition: "external history event input"

  selected:
    id: "A1"
    reason: "user confirmed direct field change in source row"

  rejected:
    - id: "A2"
      reason: "no related entity context confirmed"
    - id: "A4"
      reason: "input is not ExternalHistoryEvent"
```

This is especially important because an early path-selection error often produces the wrong package at the end.

Gates in Guided Intake

Guided intake should not become an endless interview. It needs control points.

For example:

```
G_PATH_CONFIRMED
G_CONTRACT_COMPLETE
G_SOURCE_ROW_CONFIRMED
G_VALUES_CONFIRMED
G_QA_SCOPE_CONFIRMED
G_PRE_PACKAGE_APPROVAL
```

A gate does not merely “remind.” It blocks further movement.

```
gate:
  id: "G_CONTRACT_COMPLETE"
  method: mechanical
  trust_class: deterministic
  type: "blocking"
  requires:
    - "contract.alias"
    - "contract.source_field"
    - "contract.values"
    - "contract.source_row"
  on_fail:
    action: "ask_missing_question"
```

If the contract is incomplete, the model is not allowed to proceed to final output generation.

How Guided Intake Handles User Clarifications

The user may change a decision.

For example:

```
go back to the previous step
I change my decision to 3
```

Ordo should not ignore this. It should perform a controlled state correction.

```
state_correction:
  reason: "user changed previous decision"
  affected_field: "selected_option"
  before: "2"
  after: "3"
```

```
rollback_to_node: "N_CONFIRM_OPTION"
recheck_gates:
  - "G_PATH_CONFIRMED"
  - "G_CONTRACT_COMPLETE"
```

This is very important. In complex processes, users often clarify or change previous decisions. If guided intake does not support controlled rollback, state quickly becomes unreliable.

Guided Intake and Debug Mode

In debug mode, guided intake should show:

- the current NODE;
- why this specific question was asked;
- which state field it fills;
- which paths remain unresolved;
- which gates block progress;
- which answers are already confirmed;
- which decisions were changed by the user;
- why the model is not proceeding to final output.

Example debug fragment:

```
debug:
  current_node: "N_COLLECT_SOURCE_FIELD"
  question_reason: "contract.source_field is required for Path A1"
  writes_to: "state.contract.source_field"
  blocked_gates:
    - "G_CONTRACT_COMPLETE"
  next_after_answer:
    - "N_COLLECT_VALUES"
```

Guided Intake and the Improvement Loop

Guided intake is one of the main places where improvements emerge.

The user may say:

this question should have been asked earlier

or:

you should not ask for the alias here; determine the source row first

Ordo should create an improvement record:

```

improvement_record:
  type: "intake_order_problem"
  affected_unit:
    kind: "node"
    id: "N_COLLECT_ALIAS"
  proposed_patch:
    - "move N_COLLECT_SOURCE_ROW before N_COLLECT_ALIAS"
  suggested_test:
    id: "TC_SOURCE_ROW_BEFORE_ALIAS"

```

This allows guided intake to improve through a controlled improvement cycle rather than chaotic instruction rewriting.

Guided Intake as Compiled IR

In compiled IR, guided intake may look like a set of opcodes:

```

[
  {
    "op": "ENTRY.DEF",
    "id": "history_event_intake",
    "start_node": "N_CLASSIFY_INPUT"
  },
  {
    "op": "NODE.DEF",
    "id": "N_COLLECT_ALIAS",
    "node_type": "question",
    "writes_to": "contract.alias"
  },
  {
    "op": "QUESTION.DEF",
    "id": "Q_ALIAS",
    "text": "What is the event alias?",
    "required": true
  },
  {
    "op": "GATE.DEF",
    "id": "G_CONTRACT_COMPLETE",
    "type": "blocking"
  },
  {
    "op": "STATE.UPDATE",
    "from": "Q_ALIAS",
    "to": "contract.alias"
  }
]

```

]

This means guided intake can be not only described in words but executed as a structured program.

Typical Mistakes

Mistake 1. Turning Guided Intake into an Ordinary Question List

A list of questions is not guided intake if there is no state, gates, or path logic.

Mistake 2. Asking Everything at Once

When the model asks ten things simultaneously, it loses control of state.

Mistake 3. Failing to Record What Was Confirmed

If the model does not distinguish “the user mentioned” from “the user confirmed,” it may create an incorrect contract.

Mistake 4. Not Supporting Rollback

Users often change decisions. Ordo must be able to update state correctly.

Mistake 5. Allowing Final Output Before Gates

Guided intake must block premature artifact generation.

Mini-Exercise

Take a simple process:

Prepare a response to a customer complaint.

Try to determine:

1. Which ENTRY starts the process?
2. Which 3–5 NODEs are needed?
3. What first question should the model ask?
4. Which state fields must be collected?

5. Which gate should exist before the final text?
 6. What should happen if the user changes the tone from “formal” to “friendly”?
-

Short Summary

Guided intake is not merely a dialogue with a model. It is an Ordo program that controls information collection, path selection, state updates, gates, and transitions between nodes.

Its main value is that a complex process does not turn into chaotic correspondence. The model knows where it is, what has already been confirmed, what still needs to be collected, and why it is not allowed to move forward.

In large playbooks, guided intake is the bridge between human conversation and formal Ordo execution.

Chapter 32. Package Generation

Why This Is Needed

In previous chapters, we examined Ordo as a language that controls dialogue, collects state, selects a path, and passes gates. But in large workflows, the final result is almost never just a single chat response.

Often, the result must be a package:

- a set of Markdown documents;
- a JSON report;
- a YAML specification;
- a README;
- a QA package;
- a Jira description;
- a validation report;
- an archive for handoff to implementation.

This is where Package generation appears.

Package generation is not the moment when a model “simply creates files.” It is a separate stage of Ordo execution with its own contract, gates, output definitions, validation, and handoff.

If this stage is not formalized, the model may:

- create extra files;
- omit mandatory files;
- mix business and technical documentation;

- fail to align the README with the actual package contents;
- create a validation report that does not validate the real package;
- place intermediate files in the archive;
- deliver the package before contract gates are confirmed.

Ordo must make package generation a controlled action.

Simple Explanation

Imagine that an Ordo program is not merely an instruction but a production line.

First it collects requirements. Then it checks the contract. Then it creates documents. Then it validates them. Then it assembles an archive. Then it hands the result to a person.

Package generation is the final production section of this line.

It must answer:

- what exactly is being created;
- which files make up the package;
- which files are mandatory;
- which files are forbidden;
- which formats are allowed;
- which gates must pass before generation;
- how package consistency is checked;
- what exactly is handed to the user.

Package as an Ordo Output

In a simple task, output may look like this:

```
output:
  type: "message"
  format: "text"
```

For package generation, however, the output must be much more precise:

```
output:
  type: "package"
  id: "history_event_analysis_package"
  required_files:
    - "README.md"
    - "SUMMARY.json"
```

```

- "VALIDATION_REPORT.json"
- "CONSISTENCY_CHECK_REPORT.json"
- "01_HISTORY_EVENT_PASSPORT_<ALIAS>.md"
- "02_JIRA_TASK_<ALIAS>.md"
- "04_IMPLEMENTATION_PROMPT_<ALIAS>.md"
- "05_QA_PACKAGE_<ALIAS>.md"
- "07_PROCESS_IMPROVEMENT_FEEDBACK_<ALIAS>.md"
- "08_QA_AUTOMATION_SPEC_<ALIAS>.yaml"
- "09_QA_AUTOMATION_README_<ALIAS>.md"
forbidden_files:
- "drafts/*"
- "tmp/*"
- "raw_notes/*"

```

This structure means the model does not have to guess what belongs in the package.

Package Generation Must Not Start Immediately

One of the main mistakes in large playbooks is creating the final package too early.

Ordo should have a rule:

The final package is not created until all gates that affect its content have passed

For example:

```

gates_before_package_generation:
- "G_INTENT_CONFIRMED"
- "G_CONTRACT_CONFIRMED"
- "G_PATH_SELECTED"
- "G_REQUIRED_VALUES_CONFIRMED"
- "G_OUTPUT_SET_CONFIRMED"
- "G_QA_SCOPE_CONFIRMED"

```

If even one gate has not passed, Ordo must stop:

```

package_generation:
  status: "blocked"
  blocked_by:
    - "G_REQUIRED_VALUES_CONFIRMED"
  message: "The package cannot be generated yet because the required values have not

```

This is an important difference between a prompt and Ordo. A prompt often tries to fulfill a request immediately. Ordo first checks whether it is allowed to do so.

Package Plan

Before creating files, Ordo must build a package plan.

```
package_plan:
  package_id: "PKG_HISTORY_EVENT_LU_CHANGE_STATUS"
  alias: "LU_CHANGE_STATUS"

  files:
    - path: "README.md"
      purpose: "explains package contents and order of use"
      required: true

    - path: "01_HISTORY_EVENT_PASSPORT_LU_CHANGE_STATUS.md"
      purpose: "analytical passport of the history event"
      required: true

    - path: "05_QA_PACKAGE_LU_CHANGE_STATUS.md"
      purpose: "manual testing and QA scenarios"
      required: true

  validation:
    required: true
    reports:
      - "VALIDATION_REPORT.json"
      - "CONSISTENCY_CHECK_REPORT.json"
```

The package plan is a contract between the Ordo program and the final artifact.

If the package plan contains 11 files, the final archive must contain exactly the expected set of files unless a separately confirmed exception exists.

Rendered Artifact Validation

Package generation is closely connected to Rendered artifact validation.

It is not enough to validate the template. The already-created result must be validated.

Bad:

The README template contains a "Package Contents" section, so everything is fine.

Good:

The actual README contains a file list that matches the real archive.

Ordo must check:

- whether all required files were created;
 - whether forbidden files are absent;
 - whether the README describes the actual contents;
 - whether SUMMARY.json matches the package files;
 - whether the validation report refers to real checks;
 - whether all aliases and file names are consistent;
 - whether placeholders remain;
 - whether documents contradict one another.
-

Self-Check Before Handoff

Before the package is handed to the user, a self-check must run.

```
self_check:
  required: true
  before:
    - "handoff"
    - "archive_delivery"

  checks:
    - id: "SC_REQUIRED_FILES"
      description: "all mandatory files are present"

    - id: "SC_NO_FORBIDDEN_FILES"
      description: "no forbidden or intermediate files are present"

    - id: "SC_README_MATCHES_PACKAGE"
      description: "README matches the actual package"

    - id: "SC_VALIDATION_REPORT_PRESENT"
      description: "validation report has been created"

    - id: "SC_CONSISTENCY_REPORT_PRESENT"
      description: "consistency check report has been created"
```

If the self-check does not pass, the package must not be handed off.

```
handoff:
  status: "blocked"
  reason: "self_check_failed"
```

Package Generation in Compiled IR

In compiled IR, this may look like:

```
[
  {
    "op": "OUTPUT.DEF",
    "id": "OUT_ANALYTICAL_PACKAGE",
    "output_type": "package",
    "required_files": [
      "README.md",
      "SUMMARY.json",
      "VALIDATION_REPORT.json"
    ]
  },
  {
    "op": "GATE.REQUIRE",
    "id": "G_BEFORE_PACKAGE",
    "requires": [
      "G_CONTRACT_CONFIRMED",
      "G_OUTPUT_SET_CONFIRMED"
    ]
  },
  {
    "op": "PACKAGE.PLAN",
    "id": "PKG_PLAN_1",
    "from_output": "OUT_ANALYTICAL_PACKAGE"
  },
  {
    "op": "PACKAGE.GENERATE",
    "id": "PKG_GENERATE_1",
    "allowed_after": "G_BEFORE_PACKAGE"
  },
  {
    "op": "RENDER.VALIDATE",
    "id": "VALIDATE_RENDERED_PACKAGE"
  },
  {
    "op": "GATE.REPORT",
    "id": "G_PACKAGE_VALIDATED"
  }
]
```

This again shows that package generation is not simple text genera-

tion but an execution phase.

Package Generation and Debug Mode

In debug mode, Ordo must explain:

- why the package may or may not be created;
- which gates allowed package generation;
- which outputs were expected;
- which files were actually created;
- which checks passed;
- which warnings remain;
- which artifact was handed off.

For example:

```
package_debug:  
  generation_allowed: false  
  blocked_by:  
    - "G_QA_SCOPE_CONFIRMED"  
  reason: "QA scope was not confirmed by user"
```

Or:

```
package_debug:  
  generation_allowed: true  
  required_files_expected: 11  
  required_files_created: 11  
  forbidden_files_found: 0  
  validation_status: "passed"
```

Package Generation and the Improvement Loop

Package generation often produces useful feedback.

A user may say:

the package is missing a runbook for the tester

or:

the README does not explain which file to open first

Ordo should create an improvement record:

```
improvement_record:  
  type: "package_structure_improvement"  
  affected_unit:
```

```
kind: "output_definition"
id: "OUT_ANALYTICAL_PACKAGE"
proposed_patch:
  - "add 10_ANALYST_RUNBOOK.md to required_files"
  - "update README required sections"
suggested_test:
id: "TC_PACKAGE_README_START_HERE"
```

This allows package structures to improve through a controlled improvement cycle.

Typical Mistakes

Mistake 1. Creating a Package Without a Package Plan

Without a plan, the model may create an attractive but incorrect set of files.

Mistake 2. Validating the Template Instead of the Created Artifact

The template may be correct while a specific file is empty or contradictory.

Mistake 3. Failing to Separate Draft and Final

Intermediate notes must not accidentally enter the final package.

Mistake 4. Failing to Block Handoff After Failed Validation

If validation failed, the package is not ready.

Mistake 5. Considering an Archive “Ready” Because It Exists

An archive is ready only after self-check, validation, and consistency check.

Mini-Exercise

Take any process whose result is a set of files.

For example:

A document package for changing a monitoring event.

Try to determine:

1. Which files should be required?
 2. Which files should be forbidden?
 3. What README is needed?
 4. Which gates must pass before generation?
 5. What validation report is needed?
 6. What should block handoff?
-

Short Summary

Package generation is a separate execution phase in Ordo. It must be controlled, verifiable, and blocked until the required gates have passed.

Ordo does not simply create files. It first defines the package contract, builds a package plan, generates required artifacts, validates the rendered result, performs a self-check, and only then hands the package to the user.

This makes it possible to work with large playbooks not as chaotic document generation but as controlled artifact production.

Chapter 33. What Remains in FREEFORM

Why This Is Needed

We have already discussed that Ordo should not try to formalize absolutely everything. Some knowledge needs FREEFORM: explanations, examples, warnings, historical notes, and complex domain wording.

But after migrating a large playbook to Ordo, one question always appears:

What exactly remains in FREEFORM?

This is not a minor detail. The answer shows the maturity level of an Ordo program.

If FREEFORM contains only material that truly should not be formalized, that is normal.

If gates, required decisions, status rules, or output contracts accidentally remain in FREEFORM, that is a problem. The model may miss them, interpret them incorrectly, or execute them inconsistently.

Simple Explanation

FREEFORM is the area where Ordo allows human text to remain human.

But this area must be transparent.

After migration, we need to examine:

- what was converted into structured Ordo;
- what remains in FREEFORM;
- why it remains there;
- whether this is safe;
- whether tests are needed;
- whether some of this text should gradually be formalized.

Otherwise, FREEFORM may become a “dark room” where critical rules are hidden.

What Is Normal to Leave in FREEFORM

It is normal to leave the following in FREEFORM:

- explanations for people;
- examples;
- long domain descriptions;
- decision history;
- edge cases that are not yet stable;
- analytical comments;
- stylistic recommendations;
- temporary notes;
- text templates, if they are not execution rules.

For example:

`freeform:`

`id: "FF_DOMAIN_CONTEXT"`

`purpose: "domain context explanation"`

`content: |`

`In this type of historical event, it is important to distinguish a change to the primary company from a change to a related entity. The analyst must carefully verify the source of the change.`

This is normal FREEFORM if the path-selection rules themselves have already been formalized separately.

What Must Not Be Hidden in FREEFORM

FREEFORM must not contain anything that affects execution as a mandatory rule.

For example, this is bad:

```
freeform:
  content: |
    A self-check must always be performed before creating the archive.
```

If the rule is mandatory, it must be a gate:

```
gate:
  id: "G_PACKAGE_SELF_CHECK"
  method: mechanical
  trust_class: deterministic
  type: "blocking"
  before:
    - "handoff"
    - "archive_delivery"
```

Likewise, the following must not be hidden in FREEFORM:

- blocking gates;
- required fields;
- status semantics;
- output contracts;
- path-selection rules;
- approval requirements;
- negative assertions;
- regression requirements;
- library conflict rules;
- no-op conditions.

FREEFORM After Playbook Migration

When an old Markdown playbook is migrated to Ordo, part of the text becomes structured IR and part remains FREEFORM.

For example:

Before, in Markdown:

"If the event concerns a related entity, the relation must be clarified through the

After migration:

- path rule → structured;
- required question → NODE.DEF;

- state field → STATE.SCHEMA;
- explanation of the Identification Center → FREEFORM.

So FREEFORM does not mean “unimportant.” It means:

this knowledge is needed, but it is not an independent execution instruction.

FREEFORM Ledger

A large Ordo program needs a FREEFORM ledger.

```
freeform_ledger:
- id: "FF_HISTORY_EVENT_CONTEXT"
  source_section: "old_playbook/domain_notes"
  reason: "domain explanation, not executable rule"
  bound_to:
    - "DOMAIN_PACK.history_event"
    - "PATH.A2"
  risk: "low"
  tests_required: false

- id: "FF_EDGE_CASES_EXTERNAL_EVENTS"
  source_section: "old_playbook/external_history_event_notes"
  reason: "edge cases not fully formalized yet"
  bound_to:
    - "PATH.A4"
  risk: "medium"
  tests_required: true
```

The ledger is needed so that residual human text does not escape control.

FREEFORM Coverage

FREEFORM also needs coverage.

Coverage should answer:

- how many FREEFORM blocks exist in the program;
- which are bound to paths/nodes/gates;
- which have tests;
- which have known risks;
- which should be formalized later;
- which have already caused user feedback or problems.

Example:

```

freeform_coverage:
  total_blocks: 5
  bound_blocks: 5
  tested_blocks: 3
  untested_blocks:
    - "FF_EDGE_CASES_EXTERNAL_EVENTS"
    - "FF_LEGACY_STATUS_NOTES"

high_risk_blocks:
  - "FF_LEGACY_STATUS_NOTES"

recommended_actions:
  - "formalize status notes into STATUS.SEMANTICS"
  - "add regression tests for external event edge cases"

```

FREEFORM and the Debug Layer

In debug mode, Ordo must show when a decision relied on FREEFORM.

For example:

```

trace_source: "model_self_report"
knowledge_trace:
  - source_type: "freeform"
    id: "FF_EDGE_CASES_EXTERNAL_EVENTS"
    used_for: "path disambiguation"
    risk: "medium"

```

This matters because if an incorrect decision was based on FREEFORM, we can see exactly which block needs improvement.

Without this, the model may say:

that is how I understood the instruction

Ordo should say more precisely:

the decision used FREEFORM block FF_EDGE_CASES_EXTERNAL_EVENTS, which has medium risk

FREEFORM and the Improvement Loop

If a user identifies a problem connected to FREEFORM, Ordo should create an improvement record.

```

improvement_record:
  type: "freeform_caused_ambiguity"
  affected_unit:
    kind: "freeform"
    id: "FF_LEGACY_STATUS_NOTES"
  problem:
    description: "status rule was interpreted inconsistently"
  proposed_patch:
    - "extract status meanings into STATUS.SEMANTICS"
    - "leave only explanation in FREEFORM"
  suggested_tests:
    - "TC_STATUS_READY_FOR_FIRST_RUN"

```

This makes FREEFORM not merely “residual text,” but a controlled part of the language.

How to Decide What to Formalize Next

After migration, there is no need to formalize everything immediately. But criteria are needed.

A FREEFORM block should be formalized further if it:

- is frequently used for decisions;
- affects gates;
- affects status;
- frequently causes errors;
- has multiple interpretations;
- is needed for regression scenarios;
- is repeated across many Domain Packs;
- could become a reusable library.

A block may remain in FREEFORM if it:

- only explains context;
 - does not change the path;
 - does not block output;
 - does not define required fields;
 - does not conflict with structured rules;
 - is easy for a person to verify.
-

Typical Mistakes

Mistake 1. Treating FREEFORM as a Dumping Ground

FREEFORM is not a place for everything that someone is too lazy to formalize.

Mistake 2. Hiding Gates in Text

If a rule blocks an action, it must be a gate.

Mistake 3. Not Maintaining a FREEFORM Ledger

Without a ledger, it is impossible to understand what remains unformalized.

Mistake 4. Not Testing Risky FREEFORM

If FREEFORM affects decisions, tests are needed.

Mistake 5. Not Revisiting FREEFORM After Feedback

If a user repeatedly identifies a problem, the FREEFORM must be improved or formalized.

Mini-Exercise

Take any long instruction document.

Write down five fragments that are difficult to formalize.

For each one, determine:

1. Is this an explanation or a rule?
 2. Does it affect the path?
 3. Does it block output?
 4. Should it be a gate?
 5. Does it need a test?
 6. Can it remain in FREEFORM?
-

Short Summary

After migrating a playbook to Ordo, it is important not only to show what was formalized. It is equally important to show what remains in FREEFORM.

FREEFORM must be controlled, bound to specific parts of the program, covered by tests where necessary, and visible to the debug and improvement loops.

Good FREEFORM is not chaos. It is an honestly marked area of human knowledge that has not yet become fully formal, but is already governed by Ordo rules.

Chapter 34. Minimum Requirements for an Author

Why This Is Needed

Ordo makes it possible to turn human instructions into a controlled process for an AI model. But this does not mean that any long text automatically becomes a good Ordo program. If the author does not understand what is being described, where the process must stop, which decisions the model may make independently, and which require human confirmation, even the best syntax will not save the instruction.

An Ordo program author does not have to be a programmer in the classical sense. The author may be an analyst, methodologist, product specialist, QA engineer, business expert, or technical writer. But the author must think not only in text, but in processes.

Ordo does not require the author to write Java, Python, or JavaScript. It does require discipline: seeing task structure, distinguishing a rule from an example, an expected result from a preference, a mandatory gate from a recommendation, and a confirmed fact from an assumption.

An Ordo Program Author Designs Model Behavior

When a person writes an ordinary prompt, they often describe only the desired result:

Prepare an analytical package for a new historical event.

For Ordo, this is insufficient. The author must describe not only the result, but also the model's behavior:

- where to begin;
- which data to collect;
- which questions to ask;
- which path options exist;
- what counts as confirmed;

- where the model must stop;
- what is forbidden without approval;
- which outputs to create;
- which gates must pass before handoff;
- which debug/test/improvement records must be supported.

An Ordo program author therefore does not simply write an instruction. The author designs a controlled execution process.

Minimum Requirement 1. Understand the Intent

The author must clearly understand why the Ordo program exists.

Bad:

Help the analyst.

Better:

Guide the analyst through guided intake for a new historical event, collect a confirmation.

The intent must be specific enough to verify whether the program fulfilled its purpose.

If the intent is vague, problems follow: the model asks the wrong questions, enters the wrong path, creates the wrong outputs, and does not understand where it must stop.

Minimum Requirement 2. Distinguish Contract from Context

A common authoring mistake is mixing mandatory conditions with background explanation.

A contract is what the process cannot proceed without.

Context is what helps the model understand the situation better.

For example:

The event concerns a change in company status.

This may be context.

But this is a contract:

Before package generation, the following must be confirmed:

- event alias;
- source row;
- source field;
- old/new values;

- path;
- expected output files.

The author must explicitly mark what is contract and what is merely explanation. Otherwise, the model may interpret an important condition as optional advice.

Minimum Requirement 3. Define State

If a process has multiple steps, the author must describe state.

State is process memory: what is already known, what is confirmed, what still awaits a decision, which gates have passed, and which assumptions remain open.

Without state, the model may repeatedly ask about something already confirmed or, conversely, treat something the user merely discussed as confirmed.

Minimum state for a complex Ordo program should contain:

- confirmed facts;
- assumptions;
- selected path;
- pending decisions;
- passed gates;
- blocked gates;
- selected outputs;
- generated artifacts;
- feedback/improvement records.

The author does not necessarily need to describe a complete state machine immediately. But the author must at least understand which data the process must remember between steps.

Minimum Requirement 4. Design Nodes as Questions, Not Chaotic Dialogue

An Ordo program often works as guided intake. This means the model does not merely “talk”; it moves through process nodes.

Each node should have a clear role:

- collect the alias;
- determine the path;
- confirm the source row;
- collect values;
- check the output contract;
- obtain approval;

- perform validation;
- create the handoff.

Bad node:

Ask the user for everything necessary.

Better node:

```

NODE.DEF collect_source_field:
  goal: "Confirm the source field whose change creates the event"
  ask: "Which source field is the event trigger?"
  required_answer: true
  updates_state:
    - source_field
  next:
    when confirmed: collect_old_new_values

```

The author must be able to divide a large process into logical nodes.

Minimum Requirement 5. Define Gates

A gate is a point where the process checks a condition before moving forward.

The author must be able to answer:

Where is the model not allowed to continue without a check?

For example:

- do not create the final archive without a self-check;
- do not consider the contract confirmed without explicit confirmation;
- do not generate a code prompt without an agreed output contract;
- do not use a source row unless it has been confirmed;
- do not create a HistoryEvent when the scenario is a no-op.

If the author does not define gates, the model acts probabilistically: sometimes correctly, sometimes not.

Ordo exists precisely so that such critical points are described explicitly.

Minimum Requirement 6. Write Negative Rules

People often describe what the model should do but forget to describe what it must not do.

For Ordo, this is critical.

Positive instructions are not enough:

Create a QA package.

Negative instructions are also needed:

ASSERT.NOT:

- do not create final archive before validation passed
- do not invent missing source values
- do not mark assumption as confirmed
- do not hide mandatory gate inside FREEFORM

The author must be able to identify dangerous model actions and prohibit them explicitly.

Minimum Requirement 7. Define Outputs

An output is not merely an “answer.” In complex processes, an output may be a set of documents, JSON structures, prompts, validation reports, QA packages, or changelog entries.

The author must describe:

- which outputs must be created;
- which outputs are mandatory;
- which outputs are auxiliary;
- which fields or sections must be inside them;
- which outputs cannot be created before approval;
- how to verify that an output is ready.

Bad:

Create a package.

Better:

```
OUTPUT.DEF compact_history_event_package:
  required_files:
    - README.md
    - SUMMARY.json
    - VALIDATION_REPORT.json
    - CONSISTENCY_CHECK_REPORT.json
    - 01_HISTORY_EVENT_PASSPORT_<ALIAS>.md
    - 02_JIRA_TASK_<ALIAS>.md
    - 04_IMPLEMENTATION_PROMPT_<ALIAS>.md
    - 05_QA_PACKAGE_<ALIAS>.md
    - 07_PROCESS_IMPROVEMENT_FEEDBACK_<ALIAS>.md
    - 08_QA_AUTOMATION_SPEC_<ALIAS>.yaml
    - 09_QA_AUTOMATION_README_<ALIAS>.md
```

An output must be verifiable.

Minimum Requirement 8. Keep FREEFORM Under Control

The author should not try to formalize everything. But the author must understand that FREEFORM is not a dumping ground for difficult rules.

FREEFORM may be used for:

- explanations;
- examples;
- domain nuances;
- historical notes;
- text templates;
- complex human wording.

But the following must not be hidden in FREEFORM:

- blocking gates;
- required approvals;
- output contracts;
- state transitions;
- status semantics;
- negative assertions.

The author must be able to decide what to formalize and what to leave as controlled FREEFORM.

Minimum Requirement 9. Think About Debugging and Tests from the Beginning

An Ordo program without a debug/test layer quickly becomes difficult to evolve.

At minimum, the author should define:

- which paths need testing;
- which gates are critical;
- which scenarios must be no-op;
- which errors have already occurred;
- which regression tests are needed;
- which debug logs are needed to explain a decision.

This does not mean that every small Ordo program needs a large test suite. But tests should be mandatory for playbooks, Domain Packs, and libraries.

Minimum Requirement 10. Capture Improvement Feedback

An Ordo program author must expect real-world use to reveal problems.

A user may say:

- this question should be asked earlier;
- a gate is needed here;
- this rule did not work;
- this should be moved into a library;
- this needs a regression test;
- this FREEFORM should be formalized.

All such comments should become improvement records instead of disappearing in chat.

The author should maintain the cycle:

feedback → issue record → affected unit → patch suggestion → test suggestion → approval

This is how an Ordo program improves after real use.

What the Author Does Not Need to Know

An Ordo program author does not have to:

- be a backend developer;
- write a compiler;
- know every IR opcode;
- create a perfect structure immediately;
- formalize 100% of domain logic;
- write a complex runner.

But the author must be able to:

- see the process;
- distinguish a rule from an example;
- define stopping points;
- describe expected behavior;
- capture state;
- verify outputs;
- accept feedback and turn it into improvements.

Typical Authoring Mistakes

Mistake 1. Writing Ordo as a Long Prompt

If a document is merely a large block of text without nodes, gates, state, and outputs, it is not yet an Ordo program.

Mistake 2. Failing to Define Who Makes the Decision

The model may help, but it is not allowed to make every decision independently.

The author must explicitly specify:

- model_decision;
- user_approval;
- analyst_decision;
- blocked_until_confirmed.

Mistake 3. Failing to Describe Negative Scenarios

If only successful scenarios are described, the model may behave incorrectly in edge cases.

Mistake 4. Making FREEFORM Too Large

Large FREEFORM without coverage is a return to the large-prompt approach.

Mistake 5. Failing to Plan for Evolution

An Ordo program will almost always change. Without improvement records, a changelog, and regression tests, every change becomes a risk.

Mini-Exercise

Take any complex prompt you have used before and answer these questions:

1. What is the intent of this prompt?
2. Which contract must be confirmed before execution?
3. Which state must be remembered?
4. Which nodes can be identified?
5. Where are gates needed?
6. Which ASSERT.NOT rules are needed?
7. Which output must be created?

8. What can remain in FREEFORM?
9. Which single debug trace would help explain an error?
10. Which single regression test should be added?

If you can answer these questions, you are already thinking like an Ordo program author.

Short Summary

An Ordo program author is not simply someone who writes instructions for AI. The author designs controlled model behavior.

Minimum requirements for an author:

- understand intent;
- distinguish contract from context;
- describe state;
- build nodes;
- define gates;
- write ASSERT.NOT;
- define the output contract;
- control FREEFORM;
- think about debugging and tests;
- capture improvement feedback.

Ordo does not require the author to be a programmer. But it does require thinking in processes, checks, and instruction evolution.

Chapter 35. How Not to Break a Playbook

Why This Is Needed

A large playbook is not merely a long instruction. It is a system of rules, decisions, stops, checks, examples, exceptions, and expected results. As such a system grows, even a good change can easily break it.

Most often, a playbook does not break because someone made an obvious mistake. More often, everything looks harmless:

- a new clarification was added;
- a block was moved higher;
- repetition was removed;
- two rules were merged;
- part of the text was moved into FREEFORM;
- gate wording was changed;

- a new path was added;
- one scenario was fixed without checking the others.

Afterward, the model suddenly starts asking questions in the wrong order, skipping approval, generating the final artifact too early, or confusing an example with a rule.

Ordo exists precisely so that such changes are not made blindly.

Simple Explanation

Breaking a playbook means violating the expected behavior of the process.

You do not have to break the entire document. Breaking one important property is enough:

- the model followed the wrong path;
- a gate stopped being blocking;
- a node started asking unnecessary questions;
- state began updating earlier than required;
- output is created without confirmation;
- FREEFORM started behaving like a hidden rule;
- a library overwrote a local rule;
- a test passes formally, but the real artifact is wrong.

In ordinary instructions, such problems are difficult to see. In Ordo, every important part of a playbook should be connected to a path, node, gate, state, output, trace, or test.

The main rule is therefore simple:

Do not change a playbook as text. Change it as an execution system.

What Counts as a Dangerous Change

Not all changes carry the same risk.

Low risk:

- fixing a textual error;
- clarifying an explanation without changing rules;
- adding an example explicitly marked as an example;
- improving a section title without changing execution flow.

Medium risk:

- adding a new question;
- changing node order;
- clarifying a condition;
- changing an output template;

- moving part of the logic into a library;
- changing a FREEFORM block.

High risk:

- changing path selection;
- changing gate semantics;
- changing status semantics;
- changing a blocking rule;
- changing approval flow;
- changing ASSERT.NOT;
- changing compiler mapping;
- changing a Domain Pack rule;
- changing a reusable library used by several playbooks.

For Ordo, this means: the closer a change is to execution behavior, the stronger the required validation.

Ordo Construct

Safe playbook changes require a dedicated change flow in Ordo.

Minimum flow:

```
CHANGE.PROPOSE
→ IMPACT.ANALYZE
→ AFFECTED.UNIT
→ TEST.SELECT
→ REGRESSION.RUN
→ TRACE.COMPARE
→ HUMAN.APPROVE
→ VERSION.NOTE
```

This means a change should not simply be “inserted into the text.” Its impact must be described.

Example:

```
change:
  id: "CH-001"
  type: "gate_semantics_update"
  summary: "Make package self-check gate blocking"

affected_units:
  - kind: "gate"
    id: "G_PACKAGE_SELF_CHECK"
  - kind: "assertion"
    id: "ASSERT_NO_ARCHIVE_BEFORE_SELF_CHECK"
  - kind: "test"
```

```

    id: "TC_NO_ARCHIVE_WITHOUT_SELF_CHECK"

risk:
  level: "high"
  reason: "Changes final package generation behavior"

required_checks:
  - "run_regression_suite"
  - "compare_debug_trace"
  - "validate_rendered_artifacts"
  - "human_approval"

```

Principle 1. Do Not Change a Rule Without a Test

Every rule change must either use an existing test or create a new one.

Bad:

Added a rule: a self-check is mandatory before creating the archive.

Better:

Added the rule + added a test:
TC_NO_ARCHIVE_WITHOUT_SELF_CHECK

In Ordo, this should be an almost automatic requirement:

RULE.CHANGE requires TEST.DEF or TEST.UPDATE

Otherwise, a change may look correct but remain unprotected against regression.

Principle 2. Do Not Change a Gate Without Status Semantics

A gate is a control point. But a gate only makes sense when the meaning of its statuses is clear.

For example:

```

passed
failed
blocked
pending
not_applicable

```

If a gate changes without checking status semantics, a dangerous situation may arise: the model sees the gate but does not understand whether it must stop.

The rule is therefore:

`GATE.CHANGE` requires `STATUS.SEMANTICS.CHECK`

This is especially important for blocking gates.

Principle 3. Do Not Hide Behavior in FREEFORM

FREEFORM is useful when part of an instruction is not ready for formalization. But FREEFORM must not become a place for hidden gates, path selection, or approval rules.

Bad:

FREEFORM says: “usually the package should be checked before the archive is created.

Better:

Gate:

```
id: G_PACKAGE_SELF_CHECK
blocking: true
```

FREEFORM:

explains how the analyst usually checks the package.

FREEFORM may explain a rule, but it must not replace the rule itself.

Principle 4. Do Not Use Implicit Overrides

When a playbook uses libraries, Profiles, or Domain Packs, behavior can easily be overwritten by accident.

For example, a library has a gate:

`G_CONTRACT_CONFIRMED`

And the local playbook adds a gate with the same name but different logic.

Ordo must not silently accept this.

Correct:

```
override:
  target: "contract_first.G_CONTRACT_CONFIRMED"
  allow: true
  reason: "Domain pack requires additional source row confirmation"
  approved_by: "human"
```

Without an explicit override, the change must be blocked or marked as a conflict.

Principle 5. Validate Not Only the Template, but the Rendered Artifact

A common mistake is checking the template but not the finished document.

For example, the template may contain the correct block, but the final artifact may omit it because of an error in the render step.

Ordo therefore needs the rule:

Artifact is valid only after rendered artifact validation.

It is not enough to say:

the template has a self-check section

You must verify:

the finished file has the self-check section in the correct place and with the corre

Principle 6. Compare the Trace Before and After a Change

For complex playbooks, knowing that tests passed is not enough. You also need to see whether the execution path changed.

Before the change:

input → path A1 → node collect_alias → gate contract_confirmed → output draft

After the change:

input → path A1 → node collect_source_field → node collect_alias → gate contract_con

This may be the correct change. But it must be visible.

Ordo should therefore support:

TRACE.COMPARE

Trace comparison shows:

- which paths changed;
- which nodes were added or removed;
- which gates changed status;
- which state fields changed;
- which outputs changed structure;
- which warnings appeared or disappeared.

Small Example

Imagine that a historical-events playbook sometimes creates the final archive without self-validation.

Bad fix:

Add to the text: “Do not forget to perform a check before creating the archive.”

This wording may be lost again.

Better Ordo fix:

```
assert_not:
  id: "ASSERT_NO_ARCHIVE_BEFORE_VALIDATION"
  method: mechanical
  trust_class: deterministic
  condition: "final_archive_created == true and validation_passed != true"
  severity: "blocking"

expected_behavior:
  if_assertion_triggered:
    action: "stop"
    message: "Cannot create final archive before validation is passed."

test:
  id: "TC_NO_ARCHIVE_BEFORE_VALIDATION"
  fixture:
    user_message: "create the archive immediately"
  expected:
    output:
      final_archive_created: false
  gate:
    id: "G_VALIDATION_BEFORE_ARCHIVE"
    status: "blocked"
```

Now this is not merely advice. It is part of the execution contract.

Typical Mistakes

The first mistake is editing a playbook as an article.

If a playbook is treated as text, the author thinks about elegant wording. If it is treated as an Ordo program, the author thinks about model behavior.

The second mistake is adding rules without tests.

A rule without a test can easily be broken two days later, even if it seems obvious today.

The third mistake is failing to distinguish an example from a rule.

Examples help the model, but if they are not clearly marked, the model may begin executing an example as a mandatory scenario.

The fourth mistake is assuming that a shorter document is automatically better.

Compression may destroy important gates, warnings, or domain-specific exceptions.

The fifth mistake is failing to check old scenarios after a new improvement.

Many changes look correct locally but break a neighboring path.

Mini-Exercise

Take any complex playbook or instruction and choose one change you want to make.

Before making the change, write down:

1. Which behavior should this change improve?
2. Which path or node does it affect?
3. Which gate may change status?
4. Which state field may change?
5. Which output may change?
6. Which test should be added?
7. Which regression test should be run?
8. Is human approval required?

If these questions do not have answers, the change is not ready.

Short Summary

A playbook breaks when it is changed as text rather than as an execution system.

To avoid breaking a playbook, Ordo requires:

- describe the impact of the change;
- bind the change to affected units;
- do not change a rule without a test;
- do not change a gate without status semantics;
- do not hide behavior in FREEFORM;
- do not use implicit overrides;
- validate the rendered artifact;
- compare traces before and after the change;

- run the regression suite;
- record a version note.

Without this, any improvement can accidentally become a new defect.

Chapter 37. Ordo Without a Runtime

Why This Is Needed

When we talk about Ordo, it is easy to imagine a complex system: a compiler, runner, execution engine, test framework, debug console, libraries, coverage reports, and complete infrastructure. In the future, it may indeed look like this. But it is important to understand that Ordo does not begin with a runtime.

Ordo can be useful even before a separate technical execution environment exists.

At the first practical level, Ordo can work as a disciplined instruction format for a model. An author prepares Ordo Source or an Ordo-like document, gives it to the model, and the model executes it as a structured execution contract.

In this mode, there is no external runner that technically blocks incorrect actions. There is no separate engine that automatically calculates coverage or compares state snapshots. But the essential thing is already present: the instruction is organized not as a long prompt, but as a controlled process.

This is an important starting mode because it makes Ordo usable today.

Simple Explanation

Ordo without a runtime means that an Ordo program exists as a structured instruction and the model itself acts as the executor.

The model reads:

- what the intent is;
- what the contract is;
- what state must be maintained;
- which nodes must be traversed;
- which gates must be checked;
- which outputs must be created;
- where execution must stop;
- what must not be done.

It then attempts to execute the process according to these rules.

Ordo without a runtime is therefore not automatic execution in the strict software sense. It is controlled execution through language discipline.

How This Differs from an Ordinary Prompt

An ordinary prompt often looks like this:

Here is a large instruction. Read it carefully and perform the task correctly.

The problem is that the model may not understand which parts are mandatory, which are examples, which are warnings, and which are merely explanations.

Ordo without a runtime formulates the instruction differently:

Here is the execution contract.

First determine the intent.

Then confirm the contract.

Then maintain state.

Then move only along an allowed path.

Before output, check the gates.

If a gate has not passed, stop.

If a rule is not formalized, use controlled FREEFORM.

After completion, provide the handoff.

Even without a technical runner, this structure significantly reduces chaos.

Minimum Format for Ordo Without a Runtime

The simplest Ordo program without a runtime may look like this:

```
ordo:
  version: "0.11"
  mode: "manual_model_execution"

intent:
  goal: "prepare an analytical package for a new historical event"

contract:
  required_confirmations:
    - "alias"
    - "source row"
    - "field mapping"
    - "old/new values"
```

```

state:
  fields:
    alias: null
    source_row_confirmed: false
    values_confirmed: false
    package_ready: false

nodes:
  - id: "N1"
    question: "What is the event alias?"
    writes_to: "state.alias"

  - id: "N2"
    question: "Which source field changes?"
    requires:
      - "state.alias != null"

outputs:
  - id: "O_FINAL_PACKAGE"
    allowed_when:
      - "G_CONTRACT_CONFIRMED == passed"
      - "G_PRE_ARCHIVE_APPROVAL == passed"

gates:
  - id: "G_PRE_ARCHIVE_APPROVAL"
    type: "blocking"
    rule: "do not create the final package without explicit user confirmation"

```

This is not yet a runtime. But it is no longer chaotic text.

The Model's Role in This Mode

Without a runtime, the model must perform several roles at once:

- read Ordo Source;
- maintain state in its response or conversational context;
- not cross gates without grounds;
- ask the question defined by the current node;
- not generate outputs too early;
- explain where it is in the process;
- record problems when the instruction is ambiguous.

This is not ideal because the model can still make mistakes. But it is already much better than hoping it will “understand a long instruction correctly on its own.”

Limitations of the Runtime-Free Mode

Ordo without a runtime has obvious limitations.

First, gates are not technically enforced. They are enforced only by instruction. If the model makes a mistake, it may continue despite a prohibition.

Second, state is not stored by an external system. It exists in the conversation context or in a textual trace.

Third, tests do not run automatically. They can be described, but the result is still checked by the model or a person.

Fourth, the debug trace depends on the model's honesty and discipline. If the model does not expose an operational reason for a decision, there is no external mechanism to force it to do so.

Ordo without a runtime is therefore a good starting point, but not the final form for complex production processes.

When This Is Enough

Runtime-free mode may be sufficient for:

- learning Ordo;
- writing the first playbooks;
- manual guided intake;
- analytical tasks with a human in the loop;
- preparing documentation;
- designing Domain Packs;
- preliminary migration of old instructions;
- validating a concept before automation.

In other words, this mode is suitable for starting, analysis, and manual use.

When a Runtime Is Already Needed

A runtime becomes necessary when requirements for repeatability, control, and evidence become high.

For example:

- the playbook is large and has many paths;
- there are critical blocking gates;
- regression testing must be guaranteed;
- outputs are used in production;
- libraries and versions are involved;
- an audit trail is required;

- different runs must be compared;
- improvement records must be collected automatically;
- expensive mistakes are possible.

In such cases, Ordo without a runtime is no longer sufficient. A helper runner or a full execution engine is needed.

Typical Mistakes

The first mistake is assuming that Ordo without a runtime already provides full determinism.

It does not. This is not yet a deterministic system. It is a structured agreement with the model.

The second mistake is not showing state.

If there is no runtime, state should at least be maintained explicitly in text or in a compact status block.

The third mistake is failing to distinguish blocking gates from recommendations.

Even without a runtime, write:

this gate blocks further progress

rather than merely:

it is advisable to check

The fourth mistake is having no test cases.

Even if they do not run automatically, they should still be described. Otherwise, the author will not know whether a change to the instruction broke something.

The fifth mistake is not collecting feedback.

If users repeatedly point out problems but those problems do not become improvement records, the Ordo program does not evolve systematically.

Mini-Exercise

Take one of your ordinary long instructions for a model and try converting it into runtime-free Ordo.

Identify:

- intent;
- contract;
- state;

- nodes;
- gates;
- outputs;
- ASSERT.NOT;
- debug information;
- expected tests;
- feedback records.

Then ask yourself:

Has the instruction become clearer?

Is it visible where the model must stop?

Is it visible what must be checked before the final result?

Will it be possible to understand why the process went wrong?

Short Summary

Ordo without a runtime is the first practical level of Ordo usage.

In this mode, Ordo is not yet executed by a specialized engine, but it already structures model behavior through contract, state, nodes, gates, outputs, and trace.

It is not a replacement for a runtime, but it is an important transitional format. It makes it possible to start using Ordo today, validate ideas, migrate old playbooks, and gradually prepare the foundation for a helper runner or native model support.

Chapter 37. Ordo Without a Runtime

Why This Is Needed

When we talk about Ordo, it is easy to imagine a complex system: a compiler, runner, execution engine, test framework, debug console, libraries, coverage reports, and complete infrastructure. In the future, it may indeed look like this. But it is important to understand that Ordo does not begin with a runtime.

Ordo can be useful even before a separate technical execution environment exists.

At the first practical level, Ordo can work as a disciplined instruction format for a model. An author prepares Ordo Source or an Ordo-like document, gives it to the model, and the model executes it as a structured execution contract.

In this mode, there is no external runner that technically blocks incorrect actions. There is no separate engine that automatically calcu-

lates coverage or compares state snapshots. But the essential thing is already present: the instruction is organized not as a long prompt, but as a controlled process.

This is an important starting mode because it makes Ordo usable today.

Simple Explanation

Ordo without a runtime means that an Ordo program exists as a structured instruction and the model itself acts as the executor.

The model reads:

- what the intent is;
- what the contract is;
- what state must be maintained;
- which nodes must be traversed;
- which gates must be checked;
- which outputs must be created;
- where execution must stop;
- what must not be done.

It then attempts to execute the process according to these rules.

Ordo without a runtime is therefore not automatic execution in the strict software sense. It is controlled execution through language discipline.

How This Differs from an Ordinary Prompt

An ordinary prompt often looks like this:

Here is a large instruction. Read it carefully and perform the task correctly.

The problem is that the model may not understand which parts are mandatory, which are examples, which are warnings, and which are merely explanations.

Ordo without a runtime formulates the instruction differently:

Here is the execution contract.

First determine the intent.

Then confirm the contract.

Then maintain state.

Then move only along an allowed path.

Before output, check the gates.

If a gate has not passed, stop.

If a rule is not formalized, use controlled FREEFORM.

After completion, provide the handoff.

Even without a technical runner, this structure significantly reduces chaos.

Minimum Format for Ordo Without a Runtime

The simplest Ordo program without a runtime may look like this:

```
ordo:
  version: "0.11"
  mode: "manual_model_execution"

intent:
  goal: "prepare an analytical package for a new historical event"

contract:
  required_confirmations:
    - "alias"
    - "source row"
    - "field mapping"
    - "old/new values"

state:
  fields:
    alias: null
    source_row_confirmed: false
    values_confirmed: false
    package_ready: false

nodes:
  - id: "N1"
    question: "What is the event alias?"
    writes_to: "state.alias"

  - id: "N2"
    question: "Which source field changes?"
    requires:
      - "state.alias != null"

outputs:
  - id: "O_FINAL_PACKAGE"
    allowed_when:
      - "G_CONTRACT_CONFIRMED == passed"
      - "G_PRE_ARCHIVE_APPROVAL == passed"

gates:
  - id: "G_PRE_ARCHIVE_APPROVAL"
```

```
type: "blocking"  
rule: "do not create the final package without explicit user confirmation"
```

This is not yet a runtime. But it is no longer chaotic text.

The Model's Role in This Mode

Without a runtime, the model must perform several roles at once:

- read Ordo Source;
- maintain state in its response or conversational context;
- not cross gates without grounds;
- ask the question defined by the current node;
- not generate outputs too early;
- explain where it is in the process;
- record problems when the instruction is ambiguous.

This is not ideal because the model can still make mistakes. But it is already much better than hoping it will “understand a long instruction correctly on its own.”

Limitations of the Runtime-Free Mode

Ordo without a runtime has obvious limitations.

First, gates are not technically enforced. They are enforced only by instruction. If the model makes a mistake, it may continue despite a prohibition.

Second, state is not stored by an external system. It exists in the conversation context or in a textual trace.

Third, tests do not run automatically. They can be described, but the result is still checked by the model or a person.

Fourth, the debug trace depends on the model's honesty and discipline. If the model does not expose an operational reason for a decision, there is no external mechanism to force it to do so.

Ordo without a runtime is therefore a good starting point, but not the final form for complex production processes.

When This Is Enough

Runtime-free mode may be sufficient for:

- learning Ordo;
- writing the first playbooks;
- manual guided intake;

- analytical tasks with a human in the loop;
- preparing documentation;
- designing Domain Packs;
- preliminary migration of old instructions;
- validating a concept before automation.

In other words, this mode is suitable for starting, analysis, and manual use.

When a Runtime Is Already Needed

A runtime becomes necessary when requirements for repeatability, control, and evidence become high.

For example:

- the playbook is large and has many paths;
- there are critical blocking gates;
- regression testing must be guaranteed;
- outputs are used in production;
- libraries and versions are involved;
- an audit trail is required;
- different runs must be compared;
- improvement records must be collected automatically;
- expensive mistakes are possible.

In such cases, Ordo without a runtime is no longer sufficient. A helper runner or a full execution engine is needed.

Typical Mistakes

The first mistake is assuming that Ordo without a runtime already provides full determinism.

It does not. This is not yet a deterministic system. It is a structured agreement with the model.

The second mistake is not showing state.

If there is no runtime, state should at least be maintained explicitly in text or in a compact status block.

The third mistake is failing to distinguish blocking gates from recommendations.

Even without a runtime, write:

this gate blocks further progress

rather than merely:

it is advisable to check

The fourth mistake is having no test cases.

Even if they do not run automatically, they should still be described. Otherwise, the author will not know whether a change to the instruction broke something.

The fifth mistake is not collecting feedback.

If users repeatedly point out problems but those problems do not become improvement records, the Ordo program does not evolve systematically.

Mini-Exercise

Take one of your ordinary long instructions for a model and try converting it into runtime-free Ordo.

Identify:

- intent;
- contract;
- state;
- nodes;
- gates;
- outputs;
- ASSERT.NOT;
- debug information;
- expected tests;
- feedback records.

Then ask yourself:

Has the instruction become clearer?

Is it visible where the model must stop?

Is it visible what must be checked before the final result?

Will it be possible to understand why the process went wrong?

Short Summary

Ordo without a runtime is the first practical level of Ordo usage.

In this mode, Ordo is not yet executed by a specialized engine, but it already structures model behavior through contract, state, nodes, gates, outputs, and trace.

It is not a replacement for a runtime, but it is an important transitional format. It makes it possible to start using Ordo today, validate

ideas, migrate old playbooks, and gradually prepare the foundation for a helper runner or native model support.

Chapter 38. Ordo with a Helper Runner

Why a Helper Runner Is Needed

In the previous chapter, we examined the simplest way to use Ordo: without a separate runtime, where the model reads Ordo Source or Semantic JSON IR itself and attempts to follow the rules in a disciplined way.

This mode is useful for getting started. But complex processes quickly expose its limits.

The model may forget a gate. It may implicitly change state. It may skip a node. It may say that a check passed even though the trace contains no evidence. It may fail to create an improvement record even when the user explicitly points out a problem.

This is where an intermediate level appears:

Ordo with a helper runner

A helper runner is not a complete “intelligent model” and is not a replacement for an LLM. It is an auxiliary execution layer that takes responsibility for process control.

Its task is simple:

the model thinks and produces content,
the runner controls order, state, gates, tests, and trace.



Figure 88: Nebu — idea

In other words, a helper runner turns Ordo from a structured instruction for a model into an executable process with external control.

Simple Explanation

You can imagine Ordo without a runtime as a cook who reads a recipe and decides independently whether everything was done correctly.

Ordo with a helper runner is more like a kitchen with a process sheet, checkpoints, an action log, and quality control.

The cook still prepares the dish. But the system ensures that the cook:

- does not skip a mandatory step;
- does not serve the dish before inspection;
- does not replace an ingredient without permission;
- records what was done;
- explains why a particular path was selected.

In Ordo, this means:

- the runner stores state;
- the runner determines the current node;
- the runner blocks transitions through blocking gates;
- the runner gives the model only the relevant instruction fragment;
- the runner captures trace;
- the runner runs tests;
- the runner builds the gate report;
- the runner collects feedback records.

The model remains the semantic executor. The runner becomes the process controller.

What Exactly the Helper Runner Does

A helper runner can perform several roles.

1. Loading the Ordo Program

The runner reads Ordo Source or compiled IR.

For example:

- the main Ordo program;
- included libraries;
- the selected Profile;
- the selected Domain Pack;
- templates;
- gates;
- tests;
- the FREEFORM ledger.

It does not rely on the model to read an entire long document correctly. The runner itself determines which parts are needed for the current step.

2. State Control

The runner stores state outside the model.

This is very important. If state exists only in the conversation context, it can easily be lost or changed without notice.

In runner-based mode, state may be a separate JSON object:

```
{
  "run_id": "RUN-001",
  "current_node": "NODE_COLLECT_CONTRACT",
  "confirmed": {
    "alias": true,
    "source_field": false
  },
  "outputs": {
    "final_package_created": false
  },
  "gates": {
    "G_CONTRACT_CONFIRMED": "blocked"
  }
}
```

The model is not allowed simply to declare that state has changed. It must propose a change, and the runner must apply or reject it.

3. Transition Control

The runner checks whether a transition from one node to another is allowed.

For example, the model wants to move to package generation. The runner checks:

- Has the contract been confirmed?
- Have the approval gates passed?
- Is the source row available?
- Has the self-check been completed?
- Are there any ASSERT.NOT violations?

If not, the runner blocks the transition.

This is fundamentally different from an ordinary prompt. In a prompt, the model “must remember” that it cannot proceed. In runner-based Ordo, the transition is technically disallowed.



Figure 89: Nebu — attention

4. Building a Task for the Model

The runner does not necessarily give the model the entire Ordo program.

It may provide only the current execution slice:

```
model_task:
  current_node: "NODE_SOURCE_FIELD_CONFIRMATION"
  allowed_actions:
    - "ask_one_question"
    - "update_state_proposal"
  forbidden_actions:
    - "generate_final_package"
    - "mark_contract_confirmed_without_user_answer"
  context:
    known:
      alias: "LU_CHANGE_STATUS"
    missing:
      - "source_field"
  expected_output:
    type: "question"
```

This reduces the risk that the model becomes confused by a long playbook.

5. Gate Enforcement

The runner can make gates real blockers.

For example:

```
gate:
  id: "G_PRE_ARCHIVE_APPROVAL"
  method: human
  trust_class: human_decision
  type: "blocking"
  condition:
    user_approved_archive_generation: true
```

If the condition is not satisfied, the runner does not allow the action:

`generate_archive`

Even if the model attempts to perform it.

6. Trace and Audit

The runner can capture a complete trace independently of how disciplined the model is in describing it.

The trace should contain:

- input;
- current node;
- selected path;
- rejected paths;
- state before;
- state after;
- gates;
- model proposals;
- runner decisions;
- warnings;
- violations;
- final outputs.

This makes the process auditable.

7. Test Execution

The runner can execute `TEST.DEF`.

For example:

```
take fixture →  
run the Ordo program →  
check expected path →  
check expected gates →  
check forbidden output →  
build report.
```

This is much closer to conventional software testing.

8. Improvement Capture

The runner can automatically collect user comments and turn them into `IMPROVEMENT.RECORD`.

For example, if the user says:

You should have asked for the source field earlier here.

The runner can bind this feedback to:

- run_id;
- node;
- path;
- gate;
- instruction fragment;
- Domain Pack;
- library version.

It can then propose a patch and a regression test.

Ordo Construct

In Source format, helper-runner mode may look like this:

```
execution:
  mode: "normal"
  runtime:
    type: "helper_runner"
    responsibilities:
      - "state_management"
      - "node_routing"
      - "gate_enforcement"
      - "trace_capture"
      - "test_execution"
      - "feedback_capture"

state:
  storage: "external"
  format: "semantic_json"

gates:
  enforcement: "runner_blocking"

trace:
  trace_source: "model_self_report"
  required: true
  level: "decision"

model:
  role: "semantic_executor"
  allowed_to:
    - "interpret_context"
    - "propose_state_update"
```


- "generate_candidate_output"
- "explain_reasoning_summary"

not_allowed_to:

- "override_blocking_gate"
- "silently_change_state"
- "skip_required_node"

In compiled IR, this may be represented as a set of operations:

```
[
  {
    "op": "RUNTIME.DEF",
    "runtime": "helper_runner",
    "responsibilities": [
      "state_management",
      "gate_enforcement",
      "trace_capture"
    ]
  },
  {
    "op": "STATE.EXTERNAL",
    "format": "semantic_json"
  },
  {
    "op": "GATE.ENFORCE",
    "mode": "runner_blocking"
  },
  {
    "op": "MODEL.ROLE",
    "role": "semantic_executor"
  }
]
```

An Important Responsibility Boundary

A helper runner must not pretend that it understands the full meaning of the domain task by itself.

For example, in the History Event Playbook, the runner may know:

- which paths exist;
- which gates must be passed;
- which fields are mandatory;
- which output cannot be created before approval;
- which tests must be run.

But the model is still needed for:

- explaining business meaning;
- producing human-readable text;
- analyzing ambiguous wording;
- proposing names;
- creating documentation;
- summarizing feedback.

The runner does not replace the model. It constrains and organizes the model's work.



Figure 90: Nebu — thinking

Small Example

Without a helper runner, the user writes:

Create a new historical event.

The model must remember the entire playbook, choose a path, ask questions, avoid proceeding too early, and avoid creating the package before the proper time.

With a helper runner, the process looks different.

The runner creates a run:

```
{
  "run_id": "RUN-100",
  "current_node": "ENTRY_START",
  "state": {},
  "allowed_actions": ["classify_input", "ask_entry_question"]
}
```

The model proposes:

```
{
  "proposed_action": "ask_question",
  "question": "What type of change needs to be converted into a historical event?"
}
```

The runner verifies that this is an allowed action and sends the question to the user.

The user then answers. The runner updates state. The model proposes a path. The runner checks the gates. Only then does the process move forward.

As a result, the model does not “hold the entire process in its head.” It works step by step within boundaries controlled by the runner.

How a Helper Runner Differs from Native Ordo

A helper runner is an intermediate level.

The model does not yet support Ordo natively. It has no internal Ordo execution engine. But an external runner helps it execute the Ordo program correctly.

Native Ordo would mean that the model itself can:

- understand Ordo IR;
- execute gates;
- maintain state;
- return trace;
- work with tests;
- support libraries;
- explain decisions in a standard format.

A helper runner performs part of this work externally.

So we can say:

Ordo without a runtime is a disciplined prompt-based mode.

Ordo with a helper runner is a controlled execution-assisted mode.

Native Ordo is a model that supports Ordo as an execution language itself.

When a Helper Runner Is Especially Needed

A helper runner is needed when:

- the process has many nodes;
- gates must be blocking;
- there are many state transitions;
- regression tests exist;
- libraries must be included;
- Domain Packs must be versioned;
- an audit trail is required;
- there are production outputs;
- mistakes can be expensive;
- several people or models work with one playbook.

It is especially useful for playbooks that gradually grow too large to execute reliably through a single long prompt.

Typical Mistakes

The first mistake is assuming that a helper runner must be very complex from the first version.

It does not. A minimal runner may do only a few things:

- store state;
- know the current node;
- block critical gates;
- write trace;
- run simple tests.

That alone is enough to significantly improve controllability.

The second mistake is giving the runner semantic decisions that should belong to the model or a person.

The runner must not invent business meaning. It must control the process.

The third mistake is allowing the model to change state directly.

A better pattern is:

```
the model proposes a state update;
the runner checks it;
the runner applies or rejects it.
```

The fourth mistake is not logging rejected actions.

If the model attempts to cross a gate, the runner should record this in the trace. Otherwise, it will be unclear why the process was blocked.

The fifth mistake is failing to connect the runner to the improvement loop.

If the runner observes a recurring problem, it should become an issue or improvement record.

Mini-Exercise

Imagine that you have an Ordo playbook for preparing an analytical package.

Describe a minimal helper runner for it.

Specify:

- which state it must store;
- which gates it must block;
- which model actions are allowed;
- which model actions are forbidden;

- which trace must be written;
- which tests must be run;
- which user feedback should become improvement records.

Then ask yourself:

Which part of the process should remain with the model?

Which part should be controlled by the runner?

Which part should require human confirmation?

Short Summary

Ordo with a helper runner is a practical middle level between prompt-based usage and native Ordo support in models.

In this mode, the model remains the semantic executor, while the runner controls state, paths, gates, trace, tests, and feedback records.

This sharply improves the reliability of complex Ordo programs, especially where it is important not merely to obtain an answer, but to prove that the process was executed correctly, validated, and left ready for further improvement.

Chapter 39. Ordo as a Native Model Language

Why This Is Needed

Up to this point, we have examined Ordo in two practical modes.

The first is Ordo without a runtime. In this case, Ordo is a strongly structured instruction that the model reads as text and attempts to execute in a disciplined way.

The second is Ordo with a helper runner. Here, part of the control is handled by an external execution layer: it stores state, checks gates, writes trace, runs tests, and prevents the model from skipping critical stops.

But Ordo's strategic goal is larger. The ideal future mode is native model support.

That means the model does not merely read Ordo as text. It understands Ordo as its own execution language.

In this mode, Ordo becomes not an “instruction for the model,” but part of how the model organizes execution, validation, and explanation of the result.

Simple Explanation

An ordinary prompt is a request to the model.

Ordo without a runtime is a structured contract.

Ordo with a helper runner is a contract with an external controller.

Ordo as a native model language is a situation in which the model has internal support for constructs such as:

```
ENTRY.DEF
NODE.DEF
STATE.SCHEMA
GATE.REPORT
ASSERT.NOT
TRACE.LOG
TEST.DEF
IMPROVEMENT.RECORD
FREEFORM.COVERAGE
```

The model does not merely see these words. It knows what they mean at the execution-behavior level.

If it sees `GATE.REPORT`, it understands that a gate must be evaluated.

If it sees `ASSERT.NOT`, it understands that a particular action is explicitly forbidden.

If it sees `STATE.DIFF`, it understands that a state change must be shown.

If it sees `EXPECT.NOOP`, it understands that the test expects no action.

How Native Ordo Differs from Prompt-Based Ordo

In prompt-based mode, the model may read a rule and still miss it if the text is long, ambiguous, or conflicting.

In native mode, Ordo constructs should act as special executable signals for the model.

For example, a prompt-based model sees:

Perform a self-check before the final archive.

And may skip it.

A native Ordo model sees:

```
gate:
  id: G_PACKAGE_SELF_CHECK
  method: mechanical
  trust_class: deterministic
  blocking: true
  required_before:
    - OUTPUT.FINAL_ARCHIVE
```

And must treat this not as advice, but as a blocked transition condition.

Ordo Construct

Native support requires Ordo to have a clear compiled representation.

For example:

```
{
  "ordo_version": "0.11",
  "execution_mode": "normal",
  "state_schema": {
    "contract_confirmed": "boolean",
    "self_check_passed": "boolean",
    "final_archive_created": "boolean"
  },
  "gates": [
    {
      "op": "GATE.DEF",
      "id": "G_PACKAGE_SELF_CHECK",
      "blocking": true,
      "required_before": ["OUTPUT.FINAL_ARCHIVE"],
      "condition": "self_check_passed == true"
    }
  ],
  "assertions": [
    {
      "op": "ASSERT.NOT",
      "id": "A_NO_ARCHIVE_BEFORE_SELF_CHECK",
      "forbid": "final_archive_created == true AND self_check_passed != true"
    }
  ]
}
```

For a person, this may look like technical JSON. But for a model with native Ordo support, it should be an execution map.

Levels of Ordo Support in a Model

We have already discussed possible levels of Ordo support. Here they are in a simpler form.

- L0 – the model does not know Ordo;
- L1 – the model reads Ordo as structured text;
- L2 – the model understands Ordo Source well;
- L3 – the model executes Semantic JSON IR as an execution contract;
- L4 – the model has native Ordo support.

At L0, Ordo can still be useful because it disciplines the prompt.

At L1, the model follows the structure more reliably.

At L2, the model already understands most Ordo patterns.

At L3, the model can execute IR as a process map.

At L4, Ordo becomes the model's own language for behavior control.

Why This Matters

Native Ordo support could reduce the central problem of complex instructions: the model would not have to “guess” what matters most every time.

It should see:

```
this is the goal;  
this is the contract;  
this is state;  
this is a path;  
this is a gate;  
this is a forbidden action;  
this is a test;  
this is a feedback record;  
this is an output;  
this is a handoff.
```

A complex instruction then stops being a long piece of text. It becomes a program for controlling behavior.

What Native Ordo Does Not Solve

Native Ordo does not make a model absolutely deterministic.

The model still works with meaning, language, context, ambiguity, and incomplete data.

Native Ordo does not eliminate human confirmation.

Native Ordo does not guarantee that a domain rule is always correct.
Native Ordo does not replace tests, debug, or the improvement loop.
On the contrary: the stronger Ordo support becomes, the more important tests, trace, and version control become.

Typical Mistakes

The first mistake is assuming that native Ordo means “the model no longer makes mistakes.”

That is incorrect. Native Ordo means that errors become better structured, more visible, and more testable.

The second mistake is waiting for native support instead of using Ordo now.

Ordo is already useful as a structured instruction language. Native support is a strategic goal, not a prerequisite for getting started.

The third mistake is making Ordo too similar to a conventional programming language.

Ordo controls an AI model, not a processor. It must therefore account for semantics, ambiguity, context, human confirmation, and explainability.

The fourth mistake is hiding all complexity inside the compiler.

Even if the compiler becomes very powerful, a person must understand which contracts, gates, tests, and outputs it creates.

Mini-Exercise

Take any complex AI instruction.

Ask yourself:

Which parts of this instruction does the model currently read only as text?

Which of them should be native Ordo constructs?

Where is a gate needed?

Where is ASSERT.NOT needed?

Where is STATE.DIFF needed?

Where is TEST.DEF needed?

Where is IMPROVEMENT.RECORD needed?

Then try to describe what the model should understand not as “text,” but as an execution contract.

Short Summary

Ordo as a native model language is the strategic goal in which a model supports Ordo not as an ordinary prompt, but as its own execution-control language.

In this mode, Ordo constructs become execution signals for the model: state, gates, assertions, tests, traces, outputs, and improvement records.

This does not eliminate human control or make AI infallible. But it moves complex instructions from the level of “long text” to controlled, explainable, and testable behavior.

Chapter 40. Process Rail: How Ordo Keeps AI on Track

Ordo is neither a rigid wizard nor simply a large prompt. Its central idea is to give AI freedom to think and communicate without allowing the process to fall apart.

The **Process Rail** is the supporting structure of a process. It holds state, the route, gates, mandatory decisions, backtracking, and result rules.

In a prompt-only approach, AI can be flexible, but it may forget a step or lose track after a previous answer changes. In a hardcoded wizard, everything is stable, but open-ended answers, clarifications, and complex decisions work poorly. Ordo occupies the middle ground: a living AI dialogue plus a formal Process Rail.

In project creation mode, the PM describes the task in natural language, and the AI Ordo Developer converts it into Ordo YAML, validates it through the CLI, and compiles it into Semantic JSON IR.

In execution mode, the AI Ordo Executor reads Semantic JSON IR, guides a person through the process, uses the CLI as a deterministic helper, and explains the process state in human terms rather than exposing raw tool output.

The short formula is:

AI thinks and communicates.

Process Rail holds the process.

CLI validates deterministic parts.

Semantic JSON IR is the machine-readable form of the Process Rail.

Chapter 41. Contract → Artifact Coverage and Go/No-Go

Ordo is needed not only to guide a person through the right questions. Its value appears when confirmed answers are not lost on the way to the final documents.

This is a common problem in ordinary processes: an analyst confirms the alias, event names, source, fields, normalization, payload, and test strategy, but part of this information never reaches the passport, Jira task, or implementation prompt. Formally, the dialogue succeeded, but the package became incomplete.

Ordo therefore introduces a separate layer:

- confirmed contracts
- expected artifact coverage
- generated artifacts
- deterministic validation
- consistency report
- go/no-go decision

Contract

A Contract is not merely an agreement made in chat. It is a structured object with a status.

For example:

```
{
  "kind": "contract",
  "id": "G_EVENT_IDENTITY_CONTRACT",
  "status": "confirmed",
  "fields": {
    "alias": {
      "value": "LU_CHANGE_CAPITAL",
      "status": "confirmed",
      "required": true
    },
    "name_uk": {
      "value": "Change in company share capital",
      "status": "confirmed",
      "required": true
    }
  }
}
```

A field may be missing, candidate, proposed, confirmed, blocked, or not_applicable. This matters: the model must not present a candidate as a confirmed decision.

Artifact Requirement

An Artifact requirement describes exactly where a confirmed contract must appear.

For example, if the HistoryEvent output contract is confirmed, its key fields must appear not only in the QA package, but also in the passport, Jira task, implementation prompt, and JSON reports.

```
{
  "kind": "artifact_requirement",
  "id": "REQ_HISTORY_EVENT_OUTPUT_IN_PASSPORT_AND_JIRA",
  "when": {
    "contract": "G_HISTORY_EVENT_OUTPUT_CONTRACT",
    "status": "confirmed"
  },
  "requires": [
    {
      "artifact": "01_HISTORY_EVENT_PASSPORT",
      "must_include_fields": ["type", "sub_type", "source", "group", "groupPriority"]
    },
    {
      "artifact": "02_JIRA_TASK",
      "must_include_fields": ["type", "sub_type", "source", "group", "groupPriority"]
    }
  ]
}
```

Why Compile Is Not Enough

compile can verify that an Ordo package contains valid references: the contract exists, the artifact exists, and the requirement does not refer to an unknown ID.

But compile cannot guarantee that a generated Markdown file actually contains the required field. Rendered artifact validation is needed for that.

Rendered Artifact Validation

Rendered artifact validation checks files that have already been created:

01_HISTORY_EVENT_PASSPORT_<ALIAS>.md
02_JIRA_TASK_<ALIAS>.md
04_IMPLEMENTATION_PROMPT_<ALIAS>.md
05_QA_PACKAGE_<ALIAS>.md
SUMMARY.json
VALIDATION_REPORT.json
CONSISTENCY_CHECK_REPORT.json

This layer must answer:

Are all confirmed contracts actually present in the required documents?
Are there contradictions between Passport, Jira, QA, and JSON reports?
Has any candidate/proposed value been recorded as confirmed?
Has the test strategy been omitted?

Go/No-Go

After all checks, Ordo must produce a short machine-readable decision:

```
{
  "kind": "go_no_go",
  "status": "no_go_requires_artifact_fix",
  "blocking_issues": [
    {
      "code": "ORDO-COV-002",
      "message": "Confirmed contract field group is missing from Passport"
    }
  ],
  "warnings": []
}
```

For a person, this can be explained simply:

The package is not ready: the confirmed group field did not reach the historical eve

What This Changes for the AI Ordo Developer

The AI Ordo Developer no longer merely creates documents from templates. It must prove that:

1. all important contracts are confirmed;
2. every confirmed contract has artifact coverage;
3. rendered documents have not lost confirmed values;
4. the consistency report contains no blocking issues;
5. the go/no-go decision allows the package to move forward.

This makes Ordo not merely an instruction language, but a language for the controlled transfer of decisions into final artifacts.

M46.2: The First Executable Validation Layer

At M46.1, we described the new concepts as part of the language. At M46.2, they begin to work in the CLI, but in a limited form: Ordo validates not the generated files themselves, but the declarative route from contract to artifact.

This means:

```
confirmed contract
→ artifact_requirement
→ required artifact
→ required contract fields
```

If a package says that a contract is confirmed but does not specify which artifacts must represent it, `ordo coverage` must fail.

If an `artifact_requirement` refers to a field that does not exist in the contract, `ordo compile` must fail. This is important: an error in the coverage model must be caught before an analyst or PM receives attractive but incomplete documents.

M46.2 does not yet read rendered Markdown or JSON. That is the next layer. The current layer answers a simpler question: does the Ordo package itself know which confirmed contracts must reach which artifacts?

The practical pipeline now looks like this:

```
lint
→ compile          # reference checks for the contract/artifact model
→ coverage          # completeness checks for confirmed contracts
→ validate-state
→ generate-output
→ validate-artifacts # next layer
→ consistency       # next layer
→ go-no-go          # final decision
```

M46.3: Validation of Already Generated Artifacts

At M46.3, the first executable command for rendered artifact validation appears:

```
ordo validate-artifacts <package>
```

It reads not only Ordo Source but also the actual files in `generated_outputs/`. This matters because `compile` and `coverage` can prove that the `contract → artifact` route is described, but they still cannot prove that the finished Markdown or JSON actually contains the confirmed value.

Example problem:

```
G_EVENT_IDENTITY_CONTRACT.event_alias = LU_CHANGE_CAPITAL  
but 02_JIRA_TASK_LU_CHANGE_CAPITAL.md does not contain LU_CHANGE_CAPITAL
```

In this case, `validate-artifacts` must return a blocking issue with code `ORD0-COV-002`.

The current M46.3 layer is not yet a full semantic consistency engine. It performs deterministic checks for the presence of confirmed values in the required rendered files. The next layer, consistency, must check contradictions between Passport, Jira, QA, Prompt, and JSON reports.

M46.4: Consistency Report Between Generated Artifacts

After `validate-artifacts`, Ordo needs another validation layer: consistency.

`validate-artifacts` answers the question: “Is the confirmed value present in the required document?”

consistency answers a different question: “Do all generated documents say the same thing about the same confirmed contract?”

A typical example:

```
alias = LU_CHANGE_CAPITAL
```

If the Passport contains `LU_CHANGE_CAPITAL`, but the Jira task contains another alias or no alias at all, the package cannot be considered ready for developer handoff. In this case, `ordo consistency` must generate `CONSISTENCY_CHECK_REPORT.json` with a blocking issue.

The minimum pipeline for an analytical package is now:

```
lint  
→ compile  
→ coverage  
→ intake/run  
→ generate-output  
→ validate-output  
→ validate-artifacts  
→ consistency
```

This does not replace analyst review, but it removes a class of errors in which a confirmed contract exists in the process yet is represented incompletely or inconsistently across final documents.

M46.5: Final Go/No-Go Helper

After `validate-artifacts` and `consistency`, one short answer is needed: can the generated package be handed off? The following command is added for this purpose:

```
ordo go-no-go <package>
```

It does not replace the individual checks. Instead, it collects them into one pipeline:

```
lint → compile → coverage → validate-state → validate-artifacts → consistency → go/no-go
```

The result is `reports/GO_NO_GO_REPORT.json`. If there is at least one blocking issue, the command returns a no-go status and a non-zero exit code.

Importantly, this is a deterministic helper, not AI model execution or a business runtime. The command answers only one question: are the Ordo source, confirmed contracts, generated artifacts, and consistency report aligned?

M46.6: Pre-Release Audit and State Reuse

M46.6 does not add another major language layer. It is a pre-release audit after M46.1–M46.5. The main check is whether the entire new `contract → artifact → consistency → go/no-go` line works as one helper pipeline.

A practical M46.6 clarification is that if guided intake has already been executed and the package contains `reports/intake_report.json`, the `ordo go-no-go <package>` command may reuse that state without requiring `--answers` again. This better matches the real process:

```
intake → generate-output → validate-artifacts → consistency → go-no-go
```

This does not change the principle: `go-no-go` remains a deterministic helper and does not execute an AI model or business runtime. It only verifies whether confirmed contracts reached generated artifacts and whether the artifacts contradict one another.

M46.7: Clean Pre-Release Candidate

M46.7 introduces no new language semantics. It is a consolidation step before pre-release: the source archive must contain source files, documentation, tests, and package definitions, but it must not carry old results from local runs.

The practical M46.7 rule is:

compiled/	generated by ordo compile
reports/	generated by helper commands
runtime/	generated by intake/run flows
generated_outputs/	generated by ordo generate-output

In the source archive, these directories may contain only .gitkeep. Real reports, compiled IR, runtime snapshots, and generated documents must be created by the current CLI run. This supports Ordo’s central principle: do not trust an old self-report; reproduce evidence from the current source state.

Chapter 42. External Audit of the Pre-release Package

Ordo is built around a simple idea: it is not enough to say that a process is valid; you must show exactly how to verify it.

That is why, after contract/artifact coverage, rendered artifact validation, the consistency report, and the go/no-go pipeline appeared, one more layer was needed: an external package audit. This is not a new language feature or a new runtime. It is a practical way to hand the package to another person or another AI session and say: “do not trust our reports; verify them yourself.”

What an external audit package is

An external audit package is a set of instructions for independently checking an Ordo workspace.

It answers:

- what exactly must be in the archive;
- which old directories must no longer exist;
- which CLI commands must be run;
- which reference packages must pass checks;
- how generated artifacts must be verified;
- how consistency and go/no-go must be checked;
- which package limitations must not be hidden.

This matters because Ordo must not turn a validation report into an article of faith. A report must be reproducible evidence.

Why manual review is not enough

If a reviewer is simply asked to “look at the package,” everyone will review it differently. One person will read the README. Another will

run only lint. A third will inspect the book source but not check the CLI.

M46.8 therefore introduces not a new runtime, but a standard verification route:

```
archive structure
→ CLI install
→ repo-check
→ unit tests
→ active package checks
→ generated artifact validation
→ consistency
→ go-no-go
→ audit verdict
```

This makes external verification repeatable.

What the reviewer must check

The reviewer must verify not only that the code does not crash, but that promises match behavior.

For example:

- if the README says the legacy site/catalog/playbook root was removed, the archive must not contain those directories;
- if the CLI says `ordo test` is static mode, the output must make that visible;
- if a package has confirmed contracts, they must reach Passport, Jira, QA Package, Implementation Prompt, and JSON reports;
- if `go-no-go` returns `go`, it must be clear which deterministic checks support that result.

Honest boundary of the Ordo preview

The external audit must also verify honest limitations.

The current Ordo preview does not execute live AI reasoning, REST calls, Mongo checks, or production business runtime. The CLI is a deterministic helper layer. It checks structure, references, coverage, rendered artifacts, consistency, and the `go/no-go` report.

That is not a weakness when stated clearly. The weakness would be to call static validation a complete production runtime.

Practical result

After M46.8, an Ordo package can be handed to another session with a ready audit prompt. The reviewer must return a short verdict:

```
go
no_go
go_with_warnings
```

and explain which commands were run, which artifacts were inspected, and which blocking issues or warnings were found.

This turns pre-release verification from a one-time author action into a reproducible process.

Important command order

`ordo repo-check` verifies source archive cleanliness. It must therefore run on a freshly unpacked package before `pip install -e .` and before tests, preferably as `PYTHONDONTWRITEBYTECODE=1 ordo repo-check .`, so Python itself does not create `__pycache__`. After installation or testing, `__pycache__` and `egg-info` may appear and the source-hygiene check may correctly fail. This is not a blocker if generated metadata is removed again before final packaging.

M46.9: self-running the external audit checklist

M46.9 adds no new language logic. Its task is to take the M46.8 checklist and run the package as a pre-release candidate: archive structure, CLI, active packages, generated artifact validation, consistency, go/no-go, documentation, and book source.

The important M46.9 result is that an audit must not trust `M*_VALIDATION_REPORT.json` by itself. It must either execute the commands or explicitly record that a command was not run. For Ordo, this is fundamental: self-report is not evidence without verification.

M46.9 found a minor documentation-hygiene mismatch: the checklist expected `cli/docs/GO_NO_GO_M46_5.md`, while the canonical CLI document was already named `cli/docs/GO_NO_GO.md`. This was not a runtime blocker, but pre-release audits must fix such issues because a reviewer should not have to guess which document is correct.

The practical rule after M46.9 is:

`repo-check` → `install CLI` → `tests` → `active packages` → `generated artifact flow` → `go-no`

If all stages pass, the package may receive a go verdict as a source-available pre-release candidate. This still does not mean production

runtime: the Ordo CLI checks deterministic structure, contracts, artifacts, and consistency; it does not execute an AI model, REST, Mongo, or a real business backend.

Chapter 43. Final Handoff: How to Transfer Ordo for External Review

Final handoff is not a new language feature. It is the point at which the package is stable enough for another developer, analyst, or AI session to review without the author's participation.

This is especially important for Ordo because the language is built around trust in verification. If a package claims to be a release candidate, an external reviewer must quickly understand three things: what to check, how to run it, and where to see the machine-readable result.

In M48, handoff adds no new CLI commands and does not change the Process Rail. It only assembles a verification route around existing layers: lint, compile, coverage, validate-state, validate-artifacts, consistency, and go-no-go.

The main final-handoff rule is simple: the reviewer should not reconstruct the history of every milestone. They need a short route:

README → final_handoff → external_audit → active packages → go-no-go report

The handoff package should therefore contain separate documents explaining:

- where to start;
- what is in scope;
- which commands to run;
- which results are expected;
- which questions to ask after review.

Final handoff also records honesty boundaries. If `ordo test` is a static runner, that must be visible. If the package is source-available rather than open-source, that must be visible. If the book PDF was not regenerated, that must also be stated explicitly.

Ordo must not look more complete than it is. The strength of the Process Rail is not in hiding incompleteness, but in showing exactly what has passed verification and what still requires a decision.

Chapter 44. Receiving External Feedback After Review

After an Ordo package is handed over for external review, the most dangerous mistake is to start fixing everything immediately.

In the Process Rail, that is the wrong route. Feedback must first become a structured fact and only then a decision.

review finding → feedback item → triage → decision → milestone

Why you should not fix things immediately

An external reviewer may find a real blocker, a useful improvement, a documentation misunderstanding, or an idea outside the current release scope.

If all of this is mixed into the code immediately, the release candidate stops being stable. M49 therefore adds not new runtime logic, but a feedback-intake layer.

Feedback item

Every comment must be recorded as a separate item:

```
id
area
severity
evidence
recommended_action
decision
target_milestone
status
```

This separates the fact from the decision.

Decisions

Feedback uses simple statuses:

```
accepted
accepted_with_scope_limit
needs_more_evidence
deferred
rejected
not_applicable
```

For example, if a reviewer says the CLI crashes in CI, that may be a blocker and accepted. If the reviewer proposes rewriting the entire language, that may be deferred or `accepted_with_scope_limit`.

Role of AI Ordo Developer

AI Ordo Developer must not immediately modify the package. It must first:

1. split feedback into separate findings;
2. classify each finding;
3. record evidence;
4. propose a decision;
5. name a target milestone only for accepted items.

This continues Ordo's central idea: the model may reason flexibly, but the process must remain controlled.

Chapter 45. Feedback Decision Planning

After external review, it is important not to rush into rewriting code. For Ordo, something else matters more: every comment must pass through a short decision process.

`comment` → `classification` → `decision` → `milestone` → `acceptance criterion` → `verification`

Why this is a separate layer

Feedback intake collects comments. But the existence of a comment does not mean it must immediately be added to the release candidate.

M50 adds a simple rule: decision first, change second.

Decision types

`accepted_now`
`accepted_next_milestone`
`deferred`
`rejected`
`needs_reproduction`
`needs_owner_decision`

This prevents blockers, useful ideas, documentation clarifications, and strategic owner decisions from being mixed together.

What a decision must contain

Every decision must contain:

- a short description;
- severity;
- affected layer;
- target milestone;
- rationale;
- acceptance criteria;
- validation commands.

How this works for Ordo

Ordo does not merely accept feedback. Ordo converts feedback into a controlled change plan. This continues the Process Rail idea: the model may reason flexibly, but the process must not lose its trace.

Important limitation

M50 does not change the CLI, runtime, or package business logic. It is a decision layer for subsequent changes.

Chapter 46. Publication Readiness Decision

M51 is not a language feature. It is a release-readiness checkpoint.

By this point Ordo has moved from a broad source workspace into a cleaner Process Rail preview with deterministic validation layers: contract coverage, rendered artifact validation, cross-artifact consistency, and go/no-go reports.

The publication-readiness question is therefore not “does the idea exist?” but “is the current package coherent enough to hand to another person or AI session for preview evaluation?”

M51 answers: yes, as a source-available preview candidate, but not as an open-source release.

The owner still decides where to publish, whether to publish the full source archive or only the Developer Bundle, whether to regenerate PDF book artifacts, and whether to keep the current source-available preview posture or later choose a formal open-source license.

This keeps Ordo honest: readiness is recorded, but publication is not implied.

Chapter 47. Two-tier Rendering Model

By this point, Ordo can already verify whether confirmed contracts reached generated artifacts. But another problem appears: not all output templates are the same.

Some templates are simple. They contain only substitutions such as `{{ state.alias }}` or `{{ state.list | bullets }}`. The CLI can safely render them.

Other templates are complex. They contain loops, conditions, tables, traceability matrices, nested YAML blocks, or manual QA/automation scenarios. A simple deterministic renderer should not render such templates.

Ordo therefore introduces a two-tier rendering model.

Tier one: deterministic rendering

A deterministic template is fully supported by the `ordo.simple` CLI renderer.

```
render_mode: deterministic
renderer: ordo.simple
requires_model_rendering: false
```

It allows simple substitutions:

```
{{ state.scalar }}
{{ state.list | bullets }}
{{ state.value | safe_name }}
{{ state.object | json }}
```

But constructs such as these are forbidden:

```
{% for %}
{% if %}
loop.index
.items()
| default("...")
```

If such a construct appears in a deterministic template, `ordo lint` or `ordo generate-output` must fail. The CLI must not pretend to understand complex AI templates.

Tier two: model-assisted rendering

A model-assisted template is rendered by an AI model.


```
render_mode: model_assisted
renderer: ai.markdown
requires_model_rendering: true
validation: strict_confirmed_state_only
tbd_policy: preserve_tbd_until_confirmed
```

The CLI does not render such a template directly. Instead, it creates a handoff packet:

```
runtime/model_assisted_render_handoff/<ARTIFACT_ID>.json
```

The packet contains:

- template content;
- confirmed state;
- expected output path;
- TBD policy;
- forbidden inference rules;
- post-validation requirements.

The AI may fill a complex Markdown/YAML/JSON artifact, but it may not invent missing values.

Main rule

Model-assisted rendering may use only confirmed state and explicit TBD defaults.

If a value is not confirmed, it must remain:

△ TBD

The AI may not remove TBD merely because a value “seems obvious.”

Post-validation

After AI rendering, Ordo returns to deterministic mode:

validate-artifacts → consistency → go-no-go

The system checks:

- whether unresolved placeholders remain;
- whether YAML/JSON is valid;
- whether confirmed values match state;
- whether inferred values appeared;
- whether TBD was removed without confirmation.

The two-tier rendering model creates an honest boundary: the model may help with a complex document, but final verification becomes deterministic again.

Standard rendering-layer errors

ORDO-RENDER-001 deterministic template contains unsupported syntax
ORDO-RENDER-002 model-assisted template rendered by simple renderer
ORDO-RENDER-003 model-assisted output contains inferred unconfirmed value
ORDO-RENDER-004 TBD marker removed without confirmed state
ORDO-RENDER-005 model-assisted YAML output is invalid
ORDO-RENDER-006 model-assisted output not validated after rendering

Chapter 48. Runtime Source of Truth and CLI Honesty

After contract → artifact coverage and two-tier rendering appeared, Ordo gained another important layer: the runtime source of truth.

The idea is simple: an Ordo package cannot be executed as a random collection of files. It must be loaded as a consistent runtime:

`ordo.yml` → `source/program.ordo.yml` → `compiled/program.ir.json` → `run_state.json` → `ge`

`ordo.yml` is the package entry point. `source/program.ordo.yml` is the editable source of truth. `compiled/program.ir.json` is the runtime artifact from which helper commands obtain the Process Rail, nodes, gates, and outputs. `run_state.json`, or a report with embedded state, describes the current execution state. Generated artifacts are the rendering result.

Why this is needed

Without this rule, AI Ordo Developer may accidentally conduct guided intake “from memory”: start with the wrong question, skip a gate, or use stale instructions after YAML changes.

M53 closes this class of errors. If YAML is newer than compiled IR, a runtime helper must block execution and request recompilation.

ORDO-RUNTIME-004: IR is stale. Run `ordo compile` before guided execution.

Standard Developer workflow

`runtime-status`
`lint`
`compile`
`test`
`coverage`
`validate-state`

check-gate / next-step
generate-output
validate-output
validate-artifacts
consistency
go-no-go
package

Importantly, compile does not mean the final package is valid. It only creates Semantic JSON IR. Final readiness is determined after validate-artifacts, consistency, and go-no-go.

CLI honesty

Ordo also introduces a truthfulness rule: the model must not write “CLI validation passed” if the CLI was not actually run.

Allowed statuses are:

executed_cli_passed
executed_cli_failed
logical_self_check_only
not_run_cli_unavailable
not_run_user_skipped

This separates real verification from a logical assumption. If the CLI is unavailable, that is not itself an error, but it must be stated honestly.

What this provides

M53 brings the Developer Bundle closer to a real runtime package: AI works flexibly, but every critical transition is checked deterministically.

Chapter 49. Runtime Guided Intake Entry Protocol

The Runtime Guided Intake Entry Protocol prevents the model from starting guided intake “from memory” or through free-form file reading.

In early versions, runtime started through `START_HERE_RUNTIME_MODE.md`, `ordo.yml`, and compiled IR. After M59/M60, enforced Runtime Mode became stricter: a runtime package does not need source YAML, and the model does not read `compiled/*` files directly.

Current route

```
START_HERE_RUNTIME_MODE.md
→ cli_embedded/ordo runtime-entry .
→ cli_embedded/ordo next-step . --format auto
→ user answer
→ cli_embedded/ordo intake . --submit <NODE_ID> --answer-file <answer_file>
→ evidence + snapshot + session-trace
→ next CLI-rendered step
```

This means the model does not decide which node comes next. It receives the next step from the CLI.

Why compiled IR must not be read directly

compiled/program.ir.json is the canonical machine target. In Runtime Mode, however, it belongs to the CLI, not to the model.

The model must not:

```
open compiled/program.ir.json;
read compiled/program.ordo.view directly;
form questions from compiled/*;
quote compiled/* in chat.
```

The only legal sources are CLI commands:

```
runtime-entry
next-step
next-step --format auto
next-step --format ordo-code
render-runtime-view
intake --submit
verify-targets
verify-session
```

Short protocol after an answer

After every user answer, the AI must show a short runtime protocol:

```
Step: <submitted node>
Action: intake --submit
Result: accepted / rejected / blocked
Evidence: <path> sha256=<digest>
Trace: runtime/session.ordo.trace sha256=<digest>
Decision: ask next / clarify / stop
```

Without evidence and a trace digest, the next question must not be asked.

Runtime view

M60.3 adds runtime_view:

```
json
ordo-code
json,ordo-code
```

In json mode, the AI sees a standard report. In ordo-code mode, next-step --format auto adds the current contract fragment. In mixed mode, both formats are allowed.

Main formula

```
JSON IR decides.
Ordo-code explains.
Session-trace proves.
```

This protocol does not replace validate-state, validate-output, validate-artifacts, consistency, go-no-go, or verify-session. It defines the correct start and cycle for guided intake.

Chapter 50. Runtime Mode Start Files Standard

After M55, detailed runtime rules no longer need to be inserted into a large prompt every time.

Every runtime-ready Ordo package has two start files:

```
START_HERE_RUNTIME_MODE.md
START_PROMPT_RUNTIME_MODE.md
```

START_HERE_RUNTIME_MODE.md contains the full protocol: how to read ordo.yml, how to check source/IR, how to work with run_state, how not to conduct guided intake “from memory,” how to record CLI status, how not to bypass gates, and how to run artifact validation.

START_PROMPT_RUNTIME_MODE.md is minimal. It only tells the AI to read START_HERE_RUNTIME_MODE.md and begin the runtime loading protocol.

The source-of-truth chain remains:

```
ordo.yml = manifest / entrypoint
source/program.ordo.yml = editable source of truth
compiled/program.ir.json = runtime source for guided execution
run_state.json = current execution state
generated artifacts = rendered output
```

The book PDF is not regenerated at this step.

Chapter 51. Package Build Profiles: dev / runtime / evidence

M56 adds a standard division of an Ordo package into three profiles.

Why this is needed

During development, one subject package may contain everything at once: source YAML, compiled IR, tests, run inputs, generated outputs, runtime snapshots, and reports. This is useful for a developer, but dangerous for guided runtime: the executor may accidentally inspect YAML or stale intermediate files instead of the current compiled/program.ir.json.

Three profiles are therefore introduced:

- dev – complete source package for development and audit
- runtime – clean executable package for guided intake
- evidence – compilation, validation, hash/provenance/status evidence

Runtime profile

A runtime package must work without editable YAML:

- README.md
- START_HERE_RUNTIME_MODE.md
- START_PROMPT_RUNTIME_MODE.md
- ordo.runtime.json
- compiled/program.ir.json
- output_templates/
- reports/CLI_VALIDATION_SUMMARY.md
- reports/BUILD_MANIFEST.json
- reports/SHA256SUMS.txt

It must not contain:

- source/program.ordo.yaml
- tests/
- run_inputs/
- domain/
- runtime/state_snapshots/
- generated_outputs/
- release/*.zip

The main rule is:

Runtime package must not require source YAML for execution.
Runtime package must use compiled/program.ir.json as primary runtime source.

CLI

Packaging is explicit:

```
ordo package <package> --profile dev --out <zip>
ordo package <package> --profile runtime --out <zip>
ordo package <package> --profile evidence --out <zip>
```

For the runtime profile, the CLI checks compiled IR availability, IR freshness relative to YAML, runtime start files, output templates, and CLI evidence.

Evidence

Every build profile generates or uses:

```
reports/BUILD_MANIFEST.json
reports/SHA256SUMS.txt
reports/package_report.json
```

The runtime profile also generates:

```
ordo.runtime.json
```

This gives the executor a clean runtime package and the reviewer a separate evidence package.

The book PDF was not regenerated in M56.

Standard package-profile errors

```
ORD0-PACKAGE-001 unknown package profile
ORD0-PACKAGE-002 runtime profile includes source YAML
ORD0-PACKAGE-003 runtime profile missing compiled IR
ORD0-PACKAGE-004 runtime profile missing output templates
ORD0-PACKAGE-005 runtime profile missing START_HERE_RUNTIME_MODE.md
ORD0-PACKAGE-006 runtime profile missing ordo.runtime.json
ORD0-PACKAGE-007 runtime profile missing BUILD_MANIFEST.json
ORD0-PACKAGE-008 runtime profile missing SHA256SUMS.txt
ORD0-PACKAGE-009 evidence profile includes editable source files
ORD0-PACKAGE-010 package claims executed_cli_passed without CLI evidence
```

Chapter 52. Runtime Checkpoint Discipline

M57 adds a checkpoint rule to Ordo Runtime Mode.

The idea is simple:

- one node at a time
- one contract at a time
- one decision at a time
- earliest incomplete node wins

This prevents AI Ordo Developer from compressing several runtime nodes into one response. If business fields, ChangeRecord, trigger/no-op, and naming decisions are all closed at once, it becomes unclear which contract was actually confirmed and where gaps remain.

Checkpoint table

run_state must now contain, or receive from helper commands, a checkpoint_table:

```
{
  "current_node": "",
  "last_closed_node": "",
  "earliest_incomplete_node": "",
  "checkpoint_table": {},
  "forward_allowed": false,
  "open_required_fields": [],
  "node_merge_attempt_detected": false
}
```

Runtime behavior

If a gap is found, Ordo does not move forward. It returns to the earliest incomplete node, asks one focused question, and continues only after that gap is closed.

next-step must prioritize earliest_incomplete_node, not the last node the model wanted to reach.

Why this matters

Checkpoint discipline makes guided intake audit-friendly: at every moment it is visible which node is open, which required fields are

confirmed, which remain open, and whether forward movement is allowed.

The book PDF was not regenerated at this step.

Standard checkpoint-discipline errors

```
ORD0-CHECKPOINT-001 node advanced before current contract closed
ORD0-CHECKPOINT-002 earlier mandatory node incomplete
ORD0-CHECKPOINT-003 multiple node contracts merged without allow_batch_confirmation
ORD0-CHECKPOINT-004 missing checkpoint table in run_state
ORD0-CHECKPOINT-005 next-step ignored earliest_incomplete_node
ORD0-CHECKPOINT-006 generated output requested while checkpoint gaps remain
```

Chapter 53. CLI-enforced Runtime Package

After the runtime profile appeared, it became clear that a clean runtime package containing compiled/program.ir.json but no CLI does not provide full enforcement. The model could read the IR directly and pass through guided intake without deterministic helper reports.

M59 therefore introduces a runtime trust layer: a runtime package must contain not only compiled IR, but also an embedded CLI that is the only legal execution interface.

```
runtime package = compiled IR + start files + output templates + embedded runtime CL
```

Main rule

A runtime package without a runnable CLI is not an enforced runtime package.

```
ordo package --profile runtime adds:
```

```
cli_embedded/ordo
cli_embedded/README.md
cli_embedded/ordo_pkg/ordo/...
```

The embedded CLI exposes runtime commands only. Authoring, compile, release, and package commands are blocked in the runtime wrapper.

Hard stop instead of a soft fallback

The old approach allowed the system to say CLI status: not_run_cli_unavailable and continue. That was an honest self-report, but it did not change behavior.

The new approach is:

```
if cli_embedded/ordo cannot run → stop
```

Continuation is allowed only after explicit user approval of a non-deterministic fallback. In that mode, every generated artifact must contain:

```
DETERMINISM_NOT_ENFORCED
```

Incremental intake

M59.2 makes guided intake step-by-step. The model should no longer walk through the entire scenario on its own. For every user answer, it calls:

```
cli_embedded/ordo intake . --submit <NODE_ID> --answer-file <tmp_answer.yaml>
```

The CLI accepts or blocks the transition, writes an evidence report, and returns a digest. The model may not ask the next question until the submit has been executed.

Evidence, snapshots, and live state

Every accepted submit writes:

```
reports/intake_submit_report.json
runtime/evidence/*_evidence.json
runtime/state_snapshots/SESSION-*.json
runtime/live_session_state.json
```

live_session_state.json is a UX cache for automatically continuing a session. It does not replace evidence and is not proof by itself.

Tamper-evident verification

M59.3 adds hash-chain snapshots, verify-session, canary detection, and a human final verification gate.

The command:

```
cli_embedded/ordo verify-session .
```

checks that the runtime session has not drifted silently. Expected clean lines are:

```
session-chain: intact
canary-scan: clean
```

A canary does not physically prevent direct IR reading, but it makes leakage provable: if service canary text appears in outputs or trace, the session receives a failure.

Trust level

M59 defines Level 1 without MCP and without a sandbox:

level_1_cli_in_package_hard_stop_hash_chain_human_verify

This is not cryptographic protection against an attacker with full filesystem access. It protects against silent drift and accidental CLI bypass.

Formula

CLI available → Runtime Mode enforced.

CLI unavailable → hard stop.

CLI bypassed → session invalid / evidence missing / canary or chain failure.

Chapter 54. Multi-target Runtime Compilation Layer

M60 adds a horizontal compilation-target model to Ordo. The idea is simple: Ordo does not replace JSON with another format. It generates several derived representations from one canonical model.

```
source/program.ordo.yaml
      ↓
canonical normalized model
      ↓
target emitters
├─ compiled/program.ir.json
├─ compiled/program.ordo.view
└─ runtime/session.ordo.trace
```

Why not replace JSON

Semantic JSON IR works well as a machine contract:

- stable
- hashable
- machine-readable
- convenient for the CLI
- convenient for verify-session

For a model, however, JSON often looks like “data” that can be slightly rearranged or extended. Ordo therefore adds an AI-facing projection that looks like a code-like contract without becoming a second source of truth.

The first three targets

M60 uses only three formats:

```
json-ir
ordo-code-view
session-trace
```

Python and Java targets are not part of this stage.

The main M60 formula

```
JSON IR decides.
Ordo-code explains.
Session-trace proves.
```

json-ir

compiled/program.ir.json remains the canonical runtime target. The CLI executes runtime logic from it.

A runtime package is never created without json-ir, even when --runtime-view ordo-code is selected.

ordo-code-view

compiled/program.ordo.view is a code-like projection for the model. It presents a node contract in a form that explicitly shows:

```
kind
question
allowed answers
transition
reject rules
evidence requirements
```

The model does not read this file directly. It receives fragments through the CLI:

```
ordo next-step . --format ordo-code
ordo render-runtime-view . --format ordo-code --node <NODE_ID>
```

session-trace

runtime/session.ordo.trace is an append-only proof program. The CLI writes it during an actual intake run. The model may not write or repair the trace itself.

Target manifest

To prevent targets from drifting apart, M60 adds:

compiled/targets.manifest.json

It records:

canonical_ir_hash
target paths
target roles
target hashes
derived_from_ir_hash
mutable session-trace metadata

The command:

ordo verify-targets .

must report:

target-set: consistent

Main danger

A multi-target system is dangerous when each target starts living a separate life.

The rule is therefore strict:

one canonical IR;
all other targets are derived;
all targets are verified by hashes;
Runtime Mode works only through the CLI.

Chapter 55. Ordo-code Runtime View

ordo-code-view is an AI-facing projection that presents the runtime contract as a code-like fragment. It is not designed for a human developer. Its purpose is to help the model see the process as a strict contract rather than as free-form JSON.

Example

Instead of exposing raw JSON to the model, the CLI can return:

```
node N_PATH_SELECT {
  kind: branch
  answer_type: enum

  allowed {
    A -> N_EVENT_ALIAS
    B -> N_EVENT_ALIAS
    C -> N_EVENT_ALIAS
    D -> N_EVENT_ALIAS
  }

  reject unless answer in [A, B, C, D]

  evidence required:
    next_step_report
    intake_submit_report
    runtime_evidence
}
```

This fragment is harder for the model to “interpret freely”: it is clear that only A/B/C/D are allowed, any other answer must be rejected, and progression without evidence is not allowed.

Why this is not Java or Python

Ordo-code is intentionally not a general-purpose language. It has no imports, loops, side effects, or arbitrary logic.

Its role is to be:

- code-like
- strict
- readable by the model
- limited to the Ordo state machine
- derived from JSON IR

How the model should receive it

Legal:

```
cli_embedded/ordo next-step . --format auto
cli_embedded/ordo next-step . --format ordo-code
cli_embedded/ordo render-runtime-view . --format ordo-code --node <NODE_ID>
```

Illegal:

open compiled/program.ordo.view directly
read compiled/program.ir.json directly
reproduce compiled/* in the chat

In Runtime Mode, all compiled/* files belong to the CLI.

Runtime view modes

M60.3 allows a runtime package to be created in three modes:

```
ordo package . --profile runtime --runtime-view json
ordo package . --profile runtime --runtime-view ordo-code
ordo package . --profile runtime --runtime-view json,ordo-code
```

In json mode, next-step --format auto returns a normal report without a code-like block.

In ordo-code mode, next-step --format auto automatically returns current_contract.

In json,ordo-code mode, either format can be selected explicitly.

Why this matters

The purpose of Ordo-code view is not to create another runtime. The purpose is to improve model behavior:

- fewer invented transitions;
- less free interpretation of allowed answers;
- better correlation between the compiled project and model responses;
- a visible contract fragment at every step.

The source of truth does not change:

- JSON IR decides.
- Ordo-code explains.

Chapter 56. Session-trace as a Proof Program

session-trace is not a description of the decision tree. It is a record of the actual path taken through a runtime session.

The idea is that every user decision becomes not merely a field in state, but a line in a proof program that can be verified.

Trace file

The runtime package contains:

```
runtime/session.ordo.trace
```

At the beginning, the trace is initialized but contains no user decisions. Every accepted intake --submit appends a new step.

Example trace fragment

```
step 001:
  accept N_EVENT_GOAL with answer "Fall of Constantinople" -> N_PATH_SELECT
  evidence sha256:...
  snapshot sha256:...
```

The CLI writes the trace. The model may not create, edit, or “fix” it manually.

How trace is connected to evidence

After every submit, the CLI updates:

```
reports/intake_submit_report.json
runtime/evidence/*_evidence.json
runtime/state_snapshots/SESSION-*.json
runtime/session.ordo.trace
runtime/live_session_state.json
```

The evidence report contains trace metadata: path, digest, step, and fragment. This lets the model show proof to the user without directly reading internal compiled targets.

Why trace does not replace a snapshot

A snapshot shows state. A trace shows the path that led to that state.

```
snapshot = what is known now
trace = which decisions led here
```

They are therefore verified together.

Verification

The command:

```
cli_embedded/ordo verify-session .
```

checks:

target-set
session-chain
session-trace
canary-scan

A clean session must report:

target-set: consistent
session-chain: intact
session-trace: intact
canary-scan: clean

If the trace is changed manually, verify-session must report failure.
If the model bypasses the CLI and creates no trace or evidence, that also becomes visible.

Why the model needs this

In a long session, a model may forget what was actually confirmed.
Trace reduces the risk of that drift:

every step has a node;
every answer has evidence;
every transition has a next node;
every digest can be verified;
the whole session can be logically replayed and verified.

Session-trace therefore turns runtime from a mere conversation into a verifiable execution history.

Chapter 57. Scenario Testing and Path-Walk

In the previous chapters, we added several verification layers to the Ordo runtime package:

CLI-enforced runtime
per-node evidence
hash-chain snapshots
verify-session
multi-target compilation
Ordo-code runtime view
session-trace proof program

These answer the question:

Does a specific runtime run have evidence, and can it be verified?

But there is another question:

Does the model behave correctly and consistently across many different scenarios?

That requires a separate testing layer. One possible approach is **PathWalk**.

What scenario testing is

A normal CLI unit test checks a command:

```
next-step works
intake --submit accepts a correct answer
verify-session detects a damaged trace
```

Scenario testing checks not a command, but the behavior of the model as a whole:

```
the model receives a runtime package
the model must call next-step
the model asks the user a question
the user or test scenario provides an answer
the model submits the answer through the CLI
the CLI writes evidence, snapshot, and trace
the model moves forward only after an accepted submit
```

Scenario testing therefore answers a practical question:

Is the model actually following the Process Rail, rather than merely saying that it

What PathWalk is

PathWalk is a companion utility or benchmark approach for checking traversal of Ordo scenarios.

It can:

```
create a test decision tree
generate a ground-truth path
let the model walk that path
add noise, clarifications, or incorrect answers
inspect actual runtime artifacts after the run
calculate a score
```

PathWalk is not part of the Ordo core. It is an external or companion layer.

Its purpose is not to replace the runtime CLI, but to test whether the model uses it.

Why model self-report cannot be trusted

A model can write:

CLI executed and passed.

By itself, this proves nothing.

After M59/M60, the evidence is in files:

```
reports/next_step_report.json
reports/intake_submit_report.json
runtime/evidence/*_evidence.json
runtime/state_snapshots/*.json
runtime/session.ordo.trace
reports/target_verification_report.json
reports/session_verification_report.json
```

PathWalk must inspect these files.

Otherwise, we return to the old problem:

the model said it completed everything correctly
but nobody verified that the CLI was actually invoked

How PathWalk should work with an M60 runtime package

In enforced mode, PathWalk should give the model only the runtime CLI protocol.

A typical cycle is:

```
./cli_embedded/ordo runtime-status .
./cli_embedded/ordo verify-targets .
./cli_embedded/ordo next-step . --format auto
./cli_embedded/ordo intake . --submit <NODE_ID> --answer-file <answer-file>
./cli_embedded/ordo verify-session .
```

The model must not directly read:

```
compiled/program.ir.json
compiled/program.ordo.view
compiled/targets.manifest.json
```

Even program.ordo.view is an AI-facing projection, but it must be delivered through the CLI rather than by direct file reading.

How PathWalk relates to the three M60 targets

M60 has a simple formula:

JSON IR decides.
Ordo-code explains.
Session-trace proves.

PathWalk should use it this way:

JSON IR

is not read directly by the model, but is the canonical runtime contract for the C

Ordo-code view

can be shown to the model through next-step --format auto or render-runtime-view.

Session-trace

is verified after the run as the proof program of actual accepted decisions.

PathWalk does not create a new source of truth. It verifies whether the model works correctly with the existing sources of evidence.

Modes worth comparing

PathWalk is especially useful when several modes are compared:

enforced + json
enforced + ordo-code
enforced + json,ordo-code
ir_readable baseline
freeform baseline

For example, we can check:

whether the model makes more mistakes in json-only mode
whether Ordo-code view helps it keep allowed answers more accurately
whether mixed mode provides better stability
whether freeform mode drifts from the decision tree faster

This is no longer merely a test of Ordo. It is a measurement of model behavior.

What PathWalk should evaluate

One overall score is not enough. It is better to separate evaluation:

path correctness

whether the final path matched the expected path

protocol compliance

whether the model invoked the CLI every time

runtime integrity

whether verify-targets and verify-session passed

compiled-read violations
whether the model attempted to read compiled/* directly

robustness
whether the model handled clarifications, noise, incorrect answers, or corrections

This matters because a model may reach the correct final node in the wrong way.

For Ordo, the correct method matters too.

Backtracking and restore-session

In guided intake, real users often change their minds:

no, let's go back
I made a mistake in the previous answer
choose another branch

This is a natural scenario. But as of M60.3, restore-session is not yet a mandatory runtime command.

PathWalk can therefore test simple corrections when the tree itself supports them. Full rollback is better implemented as a separate future layer:

restore-session must be append-only
it must write evidence
it must write a trace event
it must be checked by verify-session
it must not silently delete history

Backtracking matters, but it should not become chaotic manual state editing.

What PathWalk must not do

PathWalk must not:

replace the embedded CLI
create evidence reports itself on behalf of the CLI
encourage the model to read compiled/*
calculate success only from the model's response text
treat protocol bypass as acceptable when the final answer looks correct

A test utility that does these things may be useful as a baseline, but not for enforced-runtime verification.

Summary

The runtime CLI verifies one specific run.

PathWalk verifies whether the model consistently completes many different scenarios.

In a correct architecture, they do not compete:

The Ordo runtime package provides rules and evidence.

PathWalk provides an experiment on model behavior.

That is why PathWalk should be documented as a separate companion testing layer rather than as part of the language core.

M60.3.2: why bare intake is forbidden in benchmark automation

A scenario runner must not invoke the CLI like this:

```
./cli_embedded/ordo intake .
```

That invocation means: “run guided intake and, if there is no answer, ask through `input()`.” This is normal for a human in a terminal. It is dangerous for a subprocess or benchmark worker because the process can hang.

Starting with M60.3.2, the Ordo runtime CLI must behave as follows:

```
no --submit
no --answers
no --non-interactive
stdin is not a TTY
      ↓
fail fast
reason: no_answers_and_not_interactive_and_no_tty
```

For PathWalk, this means one simple rule:

in benchmark automation, always use an explicit mode:

```
- intake --submit ...
- or intake --answers ... --non-interactive
```

Likewise, next-step should give the model a short current fragment rather than the full internal checkpoint table. Full details may exist in the report file, but the model should not receive unnecessary runtime noise in stdout.

Chapter 58. Restore-session: Safe Backtracking

Sometimes, during guided intake, the user wants to change a previous answer:

```
Let's return to the previous step.  
I want to choose a different path.  
No, that answer was wrong.
```

Before M60.4, the Ordo runtime had no native command for this kind of return. That created a temptation for external utilities or the model itself to edit session state manually. This is dangerous because it can hide the fact that history was changed.

M60.4 adds:

```
ordo restore-session <package> --to-seq <N>
```

Inside a runtime package:

```
./cli_embedded/ordo restore-session . --to-seq <N>
```

Restore does not delete history

The main rule is:

```
restore-session does not erase earlier snapshots.  
restore-session appends a new event to history.
```

Ordo does not rewind the session as if later steps had never happened. The CLI takes state from an earlier snapshot and creates a new restore event.

This matters for trust: both a human and a verification utility can see that a return occurred.

What restore-session writes

A successful restore creates or updates:

```
reports/restore_session_report.json  
runtime/evidence/*RESTORE_TO_SEQ_<N>*_evidence.json  
runtime/state_snapshots/SESSION-*_RESTORE_TO_SEQ_<N>.json  
runtime/session.ordo.trace  
runtime/live_session_state.json
```

The trace receives a step such as:

```
action: restore_session  
node: RESTORE_TO_SEQ_<N>
```

Backtracking is therefore part of the evidence history, not a hidden file operation.

How the model must behave after restore

After restore, the model must not decide on its own which question comes next. It must call the CLI again:

```
./cli_embedded/ordo next-step . --format auto
```

Only then may it ask the next question.

How to verify restore

The final verification remains:

```
./cli_embedded/ordo verify-session .
```

A session is healthy only if the restore event preserves:

- target-set
- session-chain
- session-trace
- evidence digest
- snapshot hash
- canary scan

Why this matters for PathWalk

Scenario-testing utilities such as PathWalk often contain correction and backtrack scenarios. After M60.4, PathWalk must not rewrite runtime state itself. It must call the embedded CLI `restore-session`.

This preserves the central property of the Ordo runtime: every important transition is visible in evidence artifacts.

M60.4 formula

- JSON IR decides.
- Ordo-code explains.
- Session-trace proves.
- Restore-session goes back without erasing history.

Chapter 59. PathWalk Benchmark Readiness

The previous chapter added safe backtracking through restore-session. This made PathWalk scenarios with backtrack and correction technically possible in the current Ordo runtime protocol.

Before expensive benchmark runs against external models, however, another question must be answered: is the testing utility itself correctly connected to the current runtime package?

That is the purpose of PathWalk Benchmark Readiness.

Why a readiness smoke test is needed

PathWalk is not part of the runtime core. It is a companion utility for checking how a model traverses Ordo scenarios.

Before real API runs, the infrastructure itself must be checked:

- PathWalk builds an M60 runtime package;
- it uses `./cli_embedded/ordo`, not the old `ordo_run.py`;
- all supported runtime views work;
- the correct runtime artifacts are inspected;
- score files are generated;
- aggregate summaries work.

This is done with a cheap no-API smoke test.

Matrix smoke

Example:

```
PYTHONPATH=cli:. python3 -m ordo_pathwalk.cli matrix-smoke \
  --out /tmp/pathwalk_matrix_smoke \
  --depth 2 \
  --branching 2 2 \
  --force
```

The matrix covers:

```
enforced + json
enforced + ordo-code
enforced + json,ordo-code
```

This is not a real model-quality test. PathWalk follows ground truth through the embedded CLI. The purpose is to prove that protocol, scorer, and aggregator are connected correctly.

What readiness does and does not prove

A passing smoke test proves wiring readiness. It does not prove that a model behaves well.

The distinction is essential:

```
wiring readiness != model quality
perfect dry-run != calibrated benchmark
```

A ground-truth driver is expected to produce perfect or near-perfect metrics. That means the infrastructure can execute the experiment, not that the metric weights are correct.

Calibration remains blocked without variance

Before real calibration, PathWalk needs evidence containing:

- non-perfect cases;
- failed cases;
- per-noise-type labels;
- repeated seeds or model runs;
- a clear distinction between protocol violations and path-quality mistakes.

If every component metric equals 1.0, there is no useful variance for calibrating score weights.

Practical conclusion

PathWalk Benchmark Readiness is a preflight gate:

```
first prove the harness is wired correctly;
then run real model or transcript evidence;
only then discuss calibration.
```

This keeps benchmark claims honest and prevents infrastructure readiness from being mislabeled as model-quality evidence.

Chapter 60. Model Benchmark Protocol

Why this chapter exists

After M60.6, Ordo has a stable no-API PathWalk dry-run baseline. But a dry-run baseline is not a real model benchmark: the ground-truth driver traverses the tree ideally, so component metrics can all equal 1.0.

M60.6.3 defines the next layer: the real model benchmark protocol.

The main rule is:

Dry-run proves wiring.
Model benchmark measures behavior.
Calibration requires variance.

Why weights cannot be changed immediately

M60.6.2 confirmed a perfect dry-run baseline:

60/60 cases passed
all component metrics = 1.0
metric variance = 0

This proves infrastructure readiness. It does not show which
path_quality_score weights are better.

Therefore:

weights remain locked until real model or transcript evidence passes calibration gate

Two allowed benchmark modes

API-driven benchmark

A model actually traverses scenarios through the PathWalk harness.

In enforced mode, it must interact only through the runtime CLI:

```
./cli_embedded/ordo next-step . --format auto  
./cli_embedded/ordo intake . --submit <NODE_ID> --answer-file <file>  
./cli_embedded/ordo restore-session . --to-seq <N> --reason "..."  
./cli_embedded/ordo verify-session .
```

Direct reading of compiled/* is forbidden.

Transcript-replay benchmark

Instead of a live API, the benchmark consumes a previously recorded
transcript of model behavior.

This is a safer first pilot because scoring and failure buckets can be
validated without external-provider cost and nondeterminism.

Required artifacts

A minimum model benchmark produces:

MODEL_BENCHMARK_PLAN.json
jobs/<job_id>.json

```
transcripts/<job_id>_transcript.json
scores/<job_id>_score.json
RAW_MODEL_METRICS.csv
SUMMARY.json
SUMMARY.md
MODEL_RUN_MANIFEST.json
CALIBRATION_DECISION.md
CALIBRATION_DECISION.json
```

The benchmark must retain transcript evidence, not only a final score.

Failure categories matter

A model can reach the correct terminal node while violating the runtime protocol. Therefore benchmark analysis must separate:

```
path-quality mistakes
protocol violations
runtime-integrity failures
compiled-read violations
noise-recovery failures
```

An aggregate score must not hide these categories.

Calibration gate

Calibration requires real variation. Evidence should include failed and non-perfect cases, multiple noise types, and repeated runs where nondeterminism matters.

Until those conditions are met:

```
calibration = blocked
weights = unchanged
```

Protocol boundary

The benchmark harness measures model behavior. It does not replace the embedded runtime CLI, create runtime evidence on behalf of the CLI, or silently repair model mistakes.

The benchmark must observe the process as it actually happened.

Summary

Dry-run proves the experiment can run.

Transcript replay proves scoring can interpret behavior.

Live model runs measure actual behavior.
Calibration changes weights only after sufficient variance exists.

Chapter 61. Real Module Testcase Generation

Why this step is needed

After M60.6, the dry-run baseline verifies wiring but does not provide enough diverse data for score calibration.

The next useful step is not immediately a large model benchmark. It is to teach PathWalk to build test cases from a real Ordo module.

The M60.7 idea is:

```
real source/program.ordo.yaml
      ↓
real decision tree / graph
      ↓
terminal paths
      ↓
testcase artifacts
      ↓
controlled noise
```

Input

The basic input is:

```
source/program.ordo.yaml
```

PathWalk may read source YAML for testcase generation because this belongs to the authoring and testing layer.

During enforced runtime execution, however, the model still must not read compiled/* directly. It must use the embedded CLI.

Required noise types

M60.7 must generate more than ideal happy paths. Controlled confusion patterns are needed:

Type	Meaning
clean_path	correct branch traversal without noise

Type	Meaning
distraction	unrelated question during intake
backtrack	return to a previous node
skip_ahead	attempt to answer a future step too early
invalid_branch	answer not allowed by the current branch
clarification_without_submit	clarification without a submitted answer
correction_backtrack	correction of an earlier submitted answer

Generated artifacts

The initial generation contract includes:

```

REAL_MODULE_TESTCASE_PLAN.json
REAL_MODULE_GRAPH_SUMMARY.json
cases/<case_id>.json
cases/<case_id>.md
RAW_TESTCASE_MATRIX.csv
SUMMARY.json
SUMMARY.md
VALIDATION_REPORT.json

```

This is a testcase-generation contract, not yet a model-benchmark contract.

What must not be mixed

M60.7 must not silently become runtime execution, scoring, or calibration.

The layers are different:

```

generate cases
review cases
execute cases
score execution
calibrate benchmark

```

Each needs its own evidence boundary.

M60.7.1 and M60.7.2

The first steps establish real-module graph extraction and terminal-path enumeration. The generator derives a graph summary and explicit terminal paths from the source module.

Later steps generate clean-path cases and bounded-noise variants.

Supported bounded-noise line

The completed artifact-only line supports:

```
distraction
invalid_branch
clarification_without_submit
skip_ahead
```

More complex conversational recovery patterns, especially back-track and `correction_backtrack`, are not automatically promoted in this line. They remain future improvements so M60.7 does not become an endless block of incremental behavior work.

Correct sequence

The stable sequence is:

```
source/program.ordo.yaml
  ↓
REAL_MODULE_GRAPH_SUMMARY
  ↓
REAL_MODULE_TERMINAL_PATHS
  ↓
clean-path testcase artifacts
  ↓
bounded-noise testcase artifacts
```

Runtime execution, scoring, and benchmark orchestration remain separate milestones.

M60.7 closure

M60.7 closes at a stable artifact-only boundary. The generator can produce structured, reviewable cases from a real module without claiming that those cases were executed by a model.

That distinction is important:

```
generated testcase != executed testcase
review-ready != runtime-verified
```

Chapter 62. Human Review Scenario Cards

M61.0 adds an intermediate PathWalk layer between generated real-module testcase artifacts and future runtime execution.

Its purpose is simple: QA, a developer, or a reviewer should be able to read a generated case as a scenario card instead of reverse-engineering raw JSON.

What existed before

After M60.7, the artifact-only chain is:

```
source/program.ordo.yaml
→ REAL_MODULE_GRAPH_SUMMARY.json
→ REAL_MODULE_TERMINAL_PATHS.json
→ clean path cases
→ bounded noise cases
```

The clean and noise cases are structured, but they are primarily machine artifacts.

M61.0 adds a human layer:

```
clean path cases + bounded noise cases
→ human review scenario cards
```

Command

```
PYTHONPATH=cli:. python3 -m ordo_pathwalk.cli real-module-review-cards \
  --summary runs/real_module_clean_cases/SUMMARY.json \
  --summary runs/real_module_noise_cases/SUMMARY.json \
  --out runs/real_module_review_cards \
  --force
```

Artifacts

The command creates:

```
cards/<card_id>.json
cards/<card_id>.md
REVIEW_CARDS.json
REVIEW_CARDS.md
RAW_REVIEW_CARD_MATRIX.csv
VALIDATION_REPORT.json
```

cards/*.md is the primary human-facing format. A card shows:

- case ID;

- path ID;
- noise pattern;
- scripted steps;
- expected behavior;
- expected terminal;
- expected outputs;
- review checklist.

What this does not do

M61.0 does not execute the runtime.

It also does not perform:

- model/API benchmark runs;
- scoring;
- calibration;
- watchdog or process-boundary hardening;
- runtime-harness matrix execution.

This boundary is deliberate. It avoids reopening the blocked M60.6.5 / M60.6.4.1 runtime-execution branch.

Readiness is explicit

Review-card artifacts separate readiness states:

```
review_cards_ready      # target of M61.0
runtime_execution_ready # false
scoring_ready           # false
calibration_ready       # false
```

The cards are ready for human review. They are not evidence that runtime execution passed or that a model is good.

Why this is useful

Human review cards provide practical value without heavy execution infrastructure:

- terminal-path coverage can be reviewed manually;
- distraction, invalid-branch, clarification, and skip-ahead cases become readable;
- the cards can serve as QA checklists;
- future runtime-execution work can be prepared on top of reviewed scenarios.

Stop boundary

M61.0 is the correct layer after the M60.8 handoff because it improves usability without opening runtime orchestration.

Future work remains separate:

M62.0 Runtime Execution of Generated Testcases
backtrack
correction_backtrack
scoring generated cases
calibration generated cases
watchdog/process-boundary hardening

The main principle is:

first make generated cases understandable to people;
then execute them safely;
only after that score and calibrate.

Chapter 63. Visual Graph Generator as a Companion Utility

Visual Graph Generator is a utility for visually reviewing Ordo programs. It reads `source/program.ordo.yaml` or compatible YAML/IR and creates a graph in Mermaid, SVG, or PNG format.

This utility is not runtime core. It does not execute a session, call a model, modify YAML, or claim that business logic has passed runtime validation. Its role is simpler and very useful: show the process structure so an author, reviewer, or developer can understand it quickly.

Where it lives

Starting with M61.2, the utility is included in the package at:

`utilities/ordo_visual_graph_generator/`

PathWalk remains alongside it:

`ordo_pathwalk/`

These two utilities should not be merged. PathWalk is responsible for test and review artifacts. Visual Graph Generator is responsible for visually explaining the tree.

Basic usage

Mermaid graph:

```
python3 utilities/ordo_visual_graph_generator/ordo_graph.py \
  source/program.ordo.yaml \
  --format mmd \
  --out runs/visual_graph/program.mmd
```

SVG graph, if Graphviz is installed:

```
python3 utilities/ordo_visual_graph_generator/ordo_graph.py \
  source/program.ordo.yaml \
  --format svg \
  --out runs/visual_graph/program.svg
```

Typical author workflow

1. The author writes or receives source/program.ordo.yaml.
2. Visual Graph Generator shows the process tree.
3. The author reviews nodes, transitions, gates, artifacts, and terminal branches.
4. PathWalk generates terminal paths, clean cases, bounded-noise cases, and review cases.
5. Visual Graph annotation overlay can highlight problematic or new graph elements.

Annotation overlay

A separate annotation-overlay mode can highlight graph elements and add comments. This is useful during review: it can show not only a path, but any node, gate, state field, output, or edge that needs attention.

Responsibility boundary

Visual Graph Generator must remain a read-only utility:

Ordo YAML/IR → graph artifacts

It must not become a runtime runner, scorer, or benchmark harness. If automatic execution of generated cases is needed, that must be a separate milestone rather than a hidden expansion of the graph utility.

Chapter 64. Shared Workflow for Companion Utilities

M61.3 defines a simple practical workflow for utilities that accompany Ordo. This is not a runtime layer or benchmark runner. It is an author/reviewer layer: first inspect the tree, then generate path/case/card artifacts, and then, when needed, highlight comments on the graph.

Why a separate workflow is needed

After M61.2, the package contains two different companion utilities:

Visual Graph Generator

→ shows an Ordo YAML/IR tree as Mermaid/SVG/PNG

PathWalk

→ creates graph summaries, terminal paths, clean/noise cases, and review cards

If they are merely placed side by side without a workflow, a package user may not immediately understand where to start. M61.3 answers that question.

Recommended order

source/program.ordo.yaml

- Visual Graph Generator: inspect the tree
- PathWalk real-module-graph: obtain a structural summary
- PathWalk real-module-paths: inspect terminal paths
- PathWalk real-module-clean-cases: create clean-path cases
- PathWalk real-module-noise-cases: create bounded-noise cases
- PathWalk real-module-review-cards: create readable scenario cards
- Visual Graph annotation overlay: highlight comments or problematic areas

What to inspect first

For an author or reviewer, the best first artifact is the graph:

```
python3 utilities/ordo_visual_graph_generator/ordo_graph.py \  
  source/program.ordo.yaml \  
  --format svg \  
  --out runs/companion_review/full_graph.svg
```

If Graphviz is unavailable, create Mermaid:

```
python3 utilities/ordo_visual_graph_generator/ordo_graph.py \  
  source/program.ordo.yaml \  
  --format mmd \  
  --out runs/companion_review/full_graph.mmd
```

Next: PathWalk artifacts

After visual review, structural review artifacts can be created:

```
PYTHONPATH=cli:. python3 -m ordo_pathwalk.cli real-module-graph \  
  --source source/program.ordo.yaml \  
  --out runs/companion_review/full_graph.mmd
```

```
--out runs/companion_review/graph \  
--force
```

Then terminal paths:

```
PYTHONPATH=cli:. python3 -m ordo_pathwalk.cli real-module-paths \  
--summary runs/companion_review/graph/REAL_MODULE_GRAPH_SUMMARY.json \  
--out runs/companion_review/paths \  
--force
```

Then clean/noise cases and human review cards.

What this does not prove

This workflow does not prove that a model executed a testcase. It does not run the runtime or score model behavior. It gives a human understandable artifacts for analysis.

This is a fundamental boundary:

visual/review artifacts $\neq$ runtime execution results

Stable boundary

At M61.3, the companion-utility layer has a complete practical form:

Visual Graph Generator + PathWalk = author/reviewer toolkit

Runtime execution of generated testcases remains a separate future milestone, not part of this workflow.

Chapter 65. Closing M61: A Stable Companion Utilities Layer

M61 completes the layer of utilities that accompany Ordo without becoming runtime core.

This layer supports practical work with the language: inspect the tree, understand paths, generate clean/noise test cases, and prepare human-readable review cards.

What belongs to the stable layer

At the M61 boundary, two companion utilities are considered stable.

PathWalk is responsible for the artifact-only testing/review flow:

```
source/program.ordo.yaml
→ graph summary
→ terminal paths
→ clean cases
→ bounded noise cases
→ review cards
```

Visual Graph Generator is responsible for the read-only visualization flow:

```
source/program.ordo.yaml
→ Mermaid / SVG / PNG graph
→ subtree / context / path views
→ optional annotation overlays
```

What this layer does not do

Companion utilities do not execute a runtime session and do not prove that a model traversed a process correctly. They help authors and reviewers see structure, prepare scenarios, and inspect logic manually.

The important boundary is:

visual/review artifacts $\neq$ runtime execution evidence

Why M61 should close

After M61.3, a complete practical route exists:

YAML → visual graph → paths → cases → review cards

Adding more small noise variants or partial integrations without a strong new milestone would reopen an endless improvement block. M61 therefore closes as a stable utility layer.

What remains for the future

The backlog retains:

- runtime execution of generated testcases;
- scoring and calibration for executed cases;
- process-boundary/watchdog hardening;
- backtrack and correction_backtrack patterns;
- possible future utility unification only when it has a separate justification.

These are not blockers for M61. They should become separate future milestones.

Chapter 66. APF as a Standard Applied Module

APF, or `ordo.applied_project_factory`, is not a utility and not run-time core. It is a standard applied module distributed with the Ordo language package that helps create or correct other Ordo processes and playbooks.

Its place in the package is:

```
packages/ordo_applied_project_factory/
```

Why APF is needed

An ordinary user or PM should not have to write `source/program.ordo.yaml` by hand. APF guides them through human review of the process:

```
process goal
→ process type
→ roles
→ input policy
→ output catalog
→ decision tree
→ node/branch review
→ terminal output binding
→ source YAML generation
→ validation / handoff
```

The main idea is that the user confirms a human understanding of the process, and the model converts that understanding into Ordo source / IR.

How APF differs from utilities

Visual Graph Generator and PathWalk help inspect a process from the outside. APF is the process for creating processes.

```
APF = standard applied module
Visual Graph = read-only renderer
PathWalk = testcase/review artifact generator
Ordo CLI = deterministic validation/runtime tooling
```

How to review APF

After import, APF can be analyzed through the same companion route already present in the package:

```
packages/ordo_applied_project_factory/source/program.ordo.yaml
→ Visual Graph Generator
→ PathWalk real-module-graph
→ PathWalk real-module-paths
→ PathWalk clean/noise cases
→ PathWalk review cards
```

This does not replace source YAML or JSON IR. These are review aids that help expose structure, terminal paths, and scenarios.

M62.2 boundary

M62.2 only documents APF as a standard module. It does not rewrite APF branches, introduce new opcodes, or execute/score generated testcases.

The next logical step is to classify APF language-pattern candidates: what remains a documentation pattern, what becomes an APF sub-flow, what needs a schema convention, and what may eventually become an IR/runtime construct.

Important PathWalk clarification

The full graph → paths → clean/noise cases → review cards route is stable for processes whose terminal paths can be enumerated without unresolved cycles. APF itself is a self-hosted authoring process with review loops, so for imported v0.1.0-alpha.14, the PathWalk graph summary works but terminal-path/testcase generation for APF itself may be blocked by cycle edges. This is not an M62.2 error: adapting APF to cycle-aware testcase generation must be a separate future step.

Chapter 67. APF Language Pattern Extraction

M62.3 does not add a new runtime or rewrite APF. Its task is different: take the ideas discovered during review of `ordo.applied_project_factory` and classify them by maturity level.

The main rule is that APF needing a pattern does not mean the pattern should immediately become an opcode or IR object.

The correct ladder is:

```
APF need
→ documented APF pattern
```


- reusable APF subflow or state convention
- use in multiple packages
- lint/check candidate
- formal language construct
- IR/runtime object only if truly necessary

Why this matters

When creating applied processes, it is easy to want to formalize everything immediately:

```
INPUT.POLICY
TERMINAL.OUTPUT.BIND
TREE.AUTHOR.PROGRESSIVE
NODE.REVIEW
FLOW.JOIN
```

But some of these are user-facing authoring workflow. Some are schema conventions. Some are future lint rules. Only a very small subset may eventually become IR-level constructs.

Current classification

M62.3 classifies APF candidates as:

```
Documentation pattern
APF reusable subflow
Schema convention
Artifact standard
Rendering standard
Package authoring policy
Lint candidate
Future IR candidate
```

The nearest practical candidates for an APF patch are input policy, output candidate catalog, progressive tree authoring, node/branch review, terminal output binding, and terminal readiness check.

The strongest future IR candidates are `FLOW.JOIN` and `SHARED.TAIL.REFERENCE`, but they require a separate design milestone.

M62.3 boundary

M62.3 does not perform:

```
APF YAML rewrite
new opcodes
runtime-core changes
```

execution/scoring/calibration
watchdog/process-boundary hardening

It is only a plan for extracting language patterns from APF experience.

Next healthy boundary

After M62.3, M62 should close and record:

M62.1 – APF imported
M62.2 – APF documented
M62.3 – APF patterns classified

Only then should a separate M63 line open for continued branch review and a scoped YAML patch.

Chapter 68. Closing M62: Stable APF Integration

M62 closes the first integration line for `ordo.applied_project_factory` in the Ordo language package.

What was done

- APF was mapped against the current Ordo package.
- APF was imported as a standard applied module.
- Documentation was added for APF as a standard module.
- The APF route with companion utilities was documented.
- Candidate APF language patterns were classified without immediately promoting them to IR/opcode.

Current architectural boundary

Ordo language core
→ runtime / CLI / IR / validation semantics

Companion utilities
→ PathWalk
→ Visual Graph Generator

Standard applied modules
→ `ordo_applied_project_factory`

APF is neither runtime core nor a utility. It is a standard applied module demonstrating self-hosted creation of processes and playbooks in Ordo.

What is not part of M62

M62 does not rewrite APF branches, implement terminal output binding, or add new IR objects. These belong in a separate M63+ plan.

Important PathWalk note

APF has review-loop cycles, so PathWalk terminal-path enumeration for APF itself is not a current gate. The stable checks for APF are:

Visual Graph rendering
PathWalk graph summary
APF lint / compile / test

Next logical step

After M62, M63 can open:

M63.0 – APF Branch Review Continuation Plan

Its starting point is branch 1 Node review; branch 1 and branch 2 can then be closed before a scoped YAML patch is made.

Chapter 69. M63: APF Release-candidate Integration

M63 opens a separate integration line for `ordo.applied_project_factory` as a release-candidate standard applied module.

Importantly, APF does not become part of runtime core. It is a standard applied module that uses Ordo to create other playbook/process packages.

What M63.0 records

- M62 contained APF v0.1.0-alpha.14 as a historical import point.
- The M63 target is APF v0.1.0-rc.1, source base alpha.21.
- APF patterns are classified cautiously: not all become IR/opcode.

- `FLOW.JOIN` and `SHARED.TAIL.REFERENCE` remain future IR candidates.
- `validate-factory-output` remains APF-local or optional until stable parent-CLI semantics exist.

Why this is a separate line

M62 closed package-level APF integration as a standard module. M63 must perform release-candidate acceptance: update the package, metadata, validation profile, known limitations, and classification matrix.

Boundary

M63.0 does not rewrite APF YAML, change runtime core, or add new opcodes. It is a planning/delta-review gate before importing rc.1.

Chapter 70. APF rc.1 as a Release-candidate Standard Module

In M63.1, `ordo.applied_project_factory` was updated from the historical `alpha.14` import point to `v0.1.0-rc.1`.

APF is now recorded as a standard applied module rather than an incidental example package:

```
module_id: ordo.applied_project_factory
version: 0.1.0-rc.1
lifecycle: release-candidate
is_standard_applied_module: true
```

Importantly, APF patterns did not automatically become core opcodes or IR objects. In this line, they remain module-local workflows, schema/documentation patterns, or future candidates.

Known limitations:

- `FLOW.JOIN` and `SHARED.TAIL.REFERENCE` remain future IR candidates.
- `validate-factory-output` remains APF-local / optional.
- consistency: `passed_with_warnings` does not block rc.1, but warnings are not hidden.

Chapter 71. APF rc.1 Validation Profile

At this step, APF v0.1.0-rc.1 remains a standard applied module in the Ordo language package.

The main decision is that APF does not fail because `validate-factory-output` is absent from the parent CLI. This check is currently APF-local / optional until it is formally promoted to the parent CLI.

The mandatory rc.1 profile is:

```
lint
compile
test
coverage
validate-state
next-step
validate-output
validate-artifacts
consistency
go-no-go
repo-check clean source
```

`consistency: passed_with_warnings` is not hidden and does not become a blocker when `go/no-go = go`. The warnings concern `contract/default-value coverage` and must remain visible in release notes and validation reports.

This step does not change APF logic, add new IR/opcodes, or move APF-local patterns into core runtime.

Chapter 72. APF rc.1: Language Pattern Classification

M63.3 records an important boundary: APF v0.1.0-rc.1 is already a standard applied module, but its internal patterns do not automatically become part of the language core.

APF demonstrates many useful ideas: input policy, progressive tree authoring, node/branch/subtree review, terminal output binding, template recipe, mock render, and the validation handoff tail. For release-candidate integration, however, they remain APF-local, documentation/schema patterns, or future tooling candidates.

The strongest candidates for a future IR decision are `FLOW.JOIN` and `SHARED.TAIL.REFERENCE`. They are kept in the backlog because they

need separate stable semantics for source YAML and Semantic JSON IR.

This allows APF rc.1 to be accepted without a breaking migration: the module is already useful as an applied process, while core runtime is not changed in haste.

Chapter 73. How APF Began Creating Itself

This chapter is a practical story from the development of `ordo.applied_project_factory`. It shows not only the finished result, but also how a complex Ordo process can gradually emerge through its own use.

1. Why this project could not simply be “written”

Applied Project Factory did not begin as a finished module or as an ordinary set of instructions. The initial task was simpler, but deeper: understand whether we could create a process that helps build other processes.

The conventional approach would be to sit down and immediately describe the complete workflow: which questions to ask the user, how to build a decision tree, how to create YAML, how to validate the result, and how to assemble the package. But this approach has a hidden problem. To describe such a process well, you already need a way of thinking about it. You need to know which branches will exist, where the terminal points are, and how results, templates, validation, and final package handoff will work. At the beginning, that knowledge did not yet exist.

APF therefore could not simply be written top-down as a finished specification. We gradually discovered its shape. At first, only one thing was clear: the future module must not merely generate text. It must guide a user through the creation of a new applied process so that the result can actually be executed and verified.

It had to be not a reference guide or advice, but a process that could be traversed:

input → questions → decisions → branches → results → templates → validation → package

But how do you build such a process while the process itself is still being created? This is where the main turning point appeared.

2. The main turning point: we created part of the process and started using it

The most important decision in APF's history was methodological rather than technical. We realized that we did not have to finish the entire Applied Project Factory before using it. We could create one viable part of the process and then use that part as a working tool to build the rest.

That part became the free-form branch — the future **branch 3: free dialogue**.

Its purpose was simple: the user did not have to provide a finished tree, YAML, or formal structure. They could explain the idea in natural language: what they wanted to create, which options they saw, which documents should be produced, and which decisions the process should make. In this mode, the model must not lose those ideas. It should extract from the dialogue:

- possible questions;
- possible branches;
- possible results;
- hints for future templates;
- open questions;
- assumptions;
- a draft of the future tree.

Once this branch became clear enough, we used it not as a future part of a finished APF, but as a current working mechanism. We literally began creating APF through an APF-like mode: we discussed freely what needed to be added, the model extracted structure, we confirmed decisions, and those decisions then became part of the process itself.

It resembled the familiar image of “pulling yourself out of the water by your own hair.” At first, the whole mechanism does not exist. But one small working part does. It is not perfect, yet it is already sufficient to help build the next part. The next part then strengthens the first. Gradually, the process begins to stabilize itself.

At this point, APF stopped being merely an object of design. It became a tool for designing itself.

This matters for the whole book because it demonstrates one of the strongest properties of a well-organized process language: it can be useful before final release. You do not have to wait for total completion before gaining value. A stable fragment that can be executed, reviewed, refined, and used again may be enough.

3. How free dialogue became the process skeleton

At first, free dialogue could seem too soft for a strict process. If the user simply talks, what guarantees a structured result? Where is the boundary between an idea, an assumption, and an approved tree node?

We solved this not by forbidding free form, but by introducing intermediate states.

Free dialogue does not immediately become approved YAML. It passes through a sequence:

```
free dialogue
→ structure extraction
→ structure draft
→ review
→ normalization
→ approved process fragment
```

This gave us a very practical working model. The user could think naturally without switching to YAML or formal tables. The model, in turn, was not allowed to silently convert what had been said into a final contract. It first showed the extracted structure, and we approved or corrected it.

Gradually, the complete logic of branch 3 emerged:

```
the user speaks freely
→ the model extracts a possible structure
→ separates confirmed from unconfirmed information
→ shows a process draft
→ routes the result into the shared review path
```

This branch gave us a practical way to continue. It prevented us from getting stuck trying to describe the perfect process immediately. We could start with a vague vision, while each step transformed that vision into a more precise structure.

From that point on, APF grew not as one large document, but as a living process in which every new fragment was reviewed.

4. The other branches appeared around this core

Once it became clear that free dialogue could be a working entry into the process, we also saw that one entry was not enough. Real users arrive with different levels of readiness.

Some have only an idea. Some already have an approximate scheme. Some want to build a tree from scratch. Others are not creating a new process at all, but correcting an existing one. APF therefore gained four starting branches:

1. Domain model + decision tree
2. Manual decision tree
3. Free dialogue
4. Correction of an existing process

Importantly, these were not four independent processes. We gradually arrived at an architecture where branches have different entry paths but converge into shared parts.

Branch 1 became the main path for gradually creating a process: the model helps build the domain model, questions, nodes, branches, and terminal paths.

Branch 2 became an adapter for a manual tree: if the user already has a tree in any human-readable form, APF does not force them to rewrite everything in YAML. The tree is first normalized and then reviewed.

Branch 3 remained free form, but no longer as chaotic brainstorming. It became a controlled path for extracting structure.

Branch 4 became the path for improving an existing process. This was necessary because, as soon as APF itself began to exist, we could see that the real life of a module is not only creation. It also includes correction, targeted changes, adaptation, and compatibility checks.

Together, these branches gave APF flexibility. But stability appeared only when shared logic was extracted into shared blocks.

5. A terminal point stopped being merely an ending

One of the next major steps concerned terminal points.

At an early stage, it is easy to think of a terminal point as simply the end of a tree path. The user answered the questions, we reached the final node, so the branch is complete. For APF, that was not enough.

A process that creates an applied module or working instruction cannot end with “we reached the end.” It must answer:

- what is created on this terminal path?
- is it an existing result, a new artifact, no result, or a deferred decision?
- is there a template?
- is it clear which state fields are required?

- can a filled example be shown?
- has the user confirmed the binding between the result and this path?

This is how the shared subprocess for results and templates appeared.

We did not implement result logic separately in every starting branch. That would have been quick, but poor for long-term stability. Instead, after their branch-specific steps, all branches enter one shared mechanism:

```
terminal point detected
→ choose result policy
→ check template state
→ review a filled example
→ user confirmation or correction
→ bind result to terminal point
→ check terminal-path readiness
```

This decision greatly improved process quality. Before it, a terminal path could be half-finished: the tree appeared complete, but nobody knew which document should come out. After it, the terminal path became a real contract: the path is not ready until its result is understood.

The distinction between “no result is needed” and “the result decision is deferred” became especially important. If no result is needed, that is an explicit decision. If the decision is deferred, it does not disappear. It becomes an incomplete item and returns during final review.

This removed one of the most dangerous forms of ambiguity: when “not decided yet” accidentally starts looking like “nothing is needed.”

6. Template review reduced late-stage findings

Another difficulty involved templates. We could agree that a document is created on a terminal path, but without a template different executors would create different documents. For APF, that was unacceptable.

We therefore established a rule: an active artifact to be created must have a template or an explicitly recorded temporary limitation. But even that was not enough. A template must not only exist; it must be seen in use.

This led to the idea of a filled example. The model does not merely show the template structure. It generates an example of the future

document using test or placeholder values. The user therefore sees not an abstract scheme, but something close to a real result.

For large documents, we moved toward file-based review: it is better to give the user a file or review package than to force them to assess a large document directly in chat. This made review closer to real work with a book, documentation, or package files.

After this, findings decreased not because we reviewed less, but because errors appeared earlier. If a template is incomplete, that is visible before package handoff. If a required section has no data source, that is visible before package generation. If an artifact is unnecessary, it can be removed before final binding.

At this point, APF became much more practical. It stopped relying on the promise that “we will generate the document later” and began requiring: show the document form now, review it now, approve it now, or explicitly record that it is not yet complete.

7. Final package handoff became a separate discipline

After the starting branches and the result/template path stabilized, one more danger remained: each branch could have its own final path. That would quickly cause the rules to diverge.

We therefore extracted the final part into a shared validation and package-handoff path:

```
approve source YAML creation
→ create source YAML
→ minimal validation
→ decide on full validation
→ review validation result
→ correction cycle
→ final review of incomplete items
→ assemble handoff package
→ final handoff
```

This final path made process completion the same for all starting branches.

The honest separation of validation states was especially important. Minimal validation means the structure is not broken at a basic level. It does not mean the module is release-ready. Full validation looks much wider: tests, coverage, state, results, artifacts, consistency, and the decision whether the module may proceed.

We explicitly established the principle:

skipped full validation = limitation, not a successful pass

This short rule protects the process from a false green status. If validation was not run, that is neither success nor failure. It is a visible limitation.

Likewise, incomplete artifacts, deferred result decisions, open corrections, or failed validation cannot silently pass into final handoff. They are corrected, removed from active scope, or explicitly block handoff.

8. How the process matured through iterations

APF evolved not in one large leap, but through a series of iterations. At each iteration, we closed not an abstract “quality” concern, but a concrete problem that appeared while creating the process itself.

First, we stabilized the individual starting branches. Then we discovered that result and template logic should not be duplicated, so we extracted it into a shared subprocess. We did the same for validation and final package handoff. After that, we reviewed the whole tree to check whether all branches converged correctly and whether orphan nodes, dead ends, or duplicated logic remained.

The key stages looked like this:

- create the basic APF logic;
- add correction mode for an existing process;
- stabilize gradual tree creation;
- formalize the manual-tree adapter;
- stabilize structure extraction from free dialogue;
- extract the result/template path into a shared block;
- extract validation and package handoff into a shared final path;
- verify the integrity of the entire tree;
- perform full validation;
- adapt the module to release-candidate state.

At first, findings were conceptual: the process could still be unclear, branches could duplicate logic, and terminal points could be insufficiently defined. Toward the end, findings became mostly technical: service data, reports, validation profiles, package consistency, and version references.

This is an important sign of stabilization. When the main logic is still unstable, every improvement may change the shape of the entire process. As the process matures, improvements become targeted.

We were no longer redesigning APF; we were bringing it to release-candidate quality as a stable module.

9. Why almost no substantive findings remained at the end

The most interesting part of this work is not that we created many files or passed validation. It is how the nature of errors changed.

At the beginning, we repeatedly clarified the nature of the process itself:

- what is a terminal point?
- when is a result considered approved?
- can a branch exist without a template?
- how does free dialogue become structure?
- what should happen to an incomplete decision?
- where does a starting branch end and a shared path begin?

These were architectural questions.

After introducing the shared result/template subprocess and the shared final validation path, most of these questions disappeared. The process gained clear “rails.” New branches could no longer end arbitrarily because all of them had to pass through the same checks. A result could not get lost because binding it to a terminal point required an explicit decision. A template could not remain implicit because the review package had to show it. Validation could not be decorative because final handoff inspected real blockers.

By the end, we were no longer asking “what should APF be?” We were asking “is everything in the package synchronized with the APF we have already approved?”

That is the normal transition from design to release-candidate state.

10. The main lesson for this book

The APF story matters not only as the story of one module. It demonstrates a way to create complex processes within the language package.

The most valuable thing was not the final archive or a specific set of nodes. It was the way we reached a stable result:

1. do not try to write the whole process perfectly at once;
2. create one viable branch;
3. start using it to create the process itself;
4. turn free-form ideas into a structure draft;
5. approve each fragment;
6. extract repeated logic into shared subprocesses;
7. distinguish incomplete from approved;
8. do not call handoff ready without validation;
9. mature the process through targeted iterations rather than constant redesign.

The free-dialogue branch became more than one APF mode. It became the starting engine that allowed APF to grow. We first created a mechanism for turning an unclear conversation into structure, and then used that mechanism to structure APF itself.

It is like building a bridge where the first small section already lets you carry materials for the next one. The bridge is not finished, but it is already helping to build itself.

That is why APF emerged not as a static instruction, but as a living, executable process. It was born through its own application. And this may be the most important knowledge to preserve in the book: sometimes a complex process does not need to be fully invented in advance. It is enough to create the first working fragment, begin using it, and allow it to build the rest in a disciplined way.

Chapter 74. EXECUTION_TRACE — the Complete History of Process Execution

EXECUTION_TRACE is a first-class Ordo language element that preserves the factual history of one process run. It is not an ordinary log and not hidden model reasoning. It is a structured, verifiable record of which nodes, paths, actions, decisions, gates, state changes, and outputs actually occurred during execution.

The main rule is:

one RUN → one primary EXECUTION_TRACE

The trace is stored as an append-only sequence of events. An event already recorded is not rewritten; a correction or clarification is added as a new event. This allows the trace to support audit, debug, replay, regression testing, analyst-behavior analysis, and playbook improvement.

Main structure

```
execution_trace:
  id: trace.history_event.001
  version: "1.0"
  run:
    run_id: run.history_event.001
    process_id: history_event.guided_intake
    process_version: "1.42"
    execution_mode: normal
    runtime_mode: chat_internal
    trace_source: hybrid
  status: running
  started_at: "2026-07-10T14:00:00+03:00"
  actor:
    actor_type: analyst
  source:
    entry_point: start
    input_refs: []
  capture_level: standard
  events: []
  replay:
    replayable: false
    replay_mode: deterministic
    required_inputs_preserved: false
  integrity:
    event_count: 0
    sequence_complete: true
```

What one event records

Each event has a sequence number, stable ID, type, time, actor, payload, and outcome. When needed, it also references the active node/path/phase, state before and after, a decision, gate, output, or parent event.

Detail levels

- minimal — run lifecycle, path, gates, and outputs;
- standard — also inputs, decisions, and state diffs;
- full — also actions, validations, warnings, and checkpoints;
- audit — full evidentiary detail, actor attribution, and integrity chain.

The default is standard.

Replay

A trace can support four modes: `deterministic`, `re_evaluate`, `simulation`, and `audit_only`. `replayable: true` is allowed only when required inputs are preserved or safely referenced and a strategy is defined for external dependencies.

Security

The trace does not store passwords, tokens, secrets, complete confidential documents, or private chain of thought. Sensitive values use redaction, secure references, or hashes. Model decisions retain a short reason code and evidence references.

Difference from TRACE.LOG

TRACE.LOG is an individual diagnostic message. EXECUTION_TRACE is the complete canonical artifact for an entire run. Many TRACE.LOG events may be part of one EXECUTION_TRACE, but they do not replace it.

Legacy compatibility

The old trace: block remains a compatible representation. New Ordo programs should use `execution_trace:.` Legacy fields are converted by an adapter into canonical trace events.

How the compiler and runtime work with EXECUTION_TRACE

The compiler does not write execution history. It transforms the `execution_trace: block` into a normalized EXECUTION_TRACE.DEF instruction: whether tracing is enabled, which detail level to use, where to write the file, and which replay mode is allowed.

Before the first step, the runtime creates `runtime/execution_trace.json`. After every meaningful action, it appends an immutable event. At completion, it adds a terminal event, final state, and checksum. After terminal status, new events are forbidden.

The minimal level stores only the traversal skeleton; standard adds answers, decisions, and state changes; full adds technical actions and warnings; audit includes all permitted event types. The file format remains the same in every case.

Replay can be deterministic, re-evaluate rules, simulate without side effects, or be audit-only. Secrets are automatically redacted, and the model's internal chain of thought never enters the trace.

Chapter 75. Validation, Replay, and Practical Use of EXECUTION_TRACE

Once EXECUTION_TRACE became a language element and gained runtime semantics, it also had to be tested as strictly as gates, state, and generated outputs.

What must be tested

The minimum validation set has six layers:

1. canonical YAML examples parse and conform to the closed catalog of values;
2. the compiler creates exactly one EXECUTION_TRACE.DEF;
3. the runtime creates an append-only trace, numbers events correctly, and forbids writes after terminal status;
4. integrity validation detects sequence gaps, incorrect event_count, and checksum changes;
5. replay modes correctly control re-evaluation and side effects;
6. secrets are redacted before being written to disk.

How to read a trace

A human reads the trace from top to bottom as an execution chronology. For each event, inspect:

- where it occurred (location);
- who performed it (actor);
- which data was used (payload);
- whether state changed (state_effect);
- which gate, decision, or output it is connected to (correlation);
- how it ended (outcome).

Four replay modes

deterministic repeats preserved inputs and decisions. re_evaluate keeps the inputs but recalculates decisions and gates. simulation forbids external side effects. audit_only executes nothing and only reconstructs the history for inspection.

For analyst-behavior analysis or training, `audit_only` is often sufficient. For regression testing, `deterministic` and `simulation` are usually more useful.

How trace modification is detected

After the terminal event, the runtime calculates a checksum of the canonical trace representation. If someone changes a payload, sequence, event count, or another part of the history, the recalculated checksum will not match. This is not a digital signature and does not replace protected storage, but it is a reliable control against accidental or unapproved artifact modification.

What is not included in the trace

`EXECUTION_TRACE` does not store the model's private chain of thought. It stores only an observable decision summary, reason code, evidence references, and factual process transitions. Passwords, tokens, and other secrets must be redacted or replaced with a secure reference before serialization.

Practical rule

A trace is useful only when it can be verified, stored safely, and replayed under a

Chapter 76. Backlog, Maturity-state Synchronization, and the Purpose of PathWalk

After many milestones, Ordo encountered an organizational problem: implementation could move forward while backlog and maturity-state documents still described an earlier state.

M76 introduces a synchronization rule for governance artifacts.

```
implementation state
→ validation evidence
→ backlog status
→ maturity-state
```

The backlog is not a wish list detached from code. A closed item must have evidence. If implementation is incomplete, maturity-state must not imply full maturity.

M76 also clarifies the primary purpose of PathWalk. Its main role is not to produce one universal "model quality score," but to measure process traversal through separate components:

- path correctness
- protocol compliance
- runtime integrity
- compiled-read violations
- noise recovery

An aggregate score may be a useful summary, but it must not hide the cause of a failure.

M76 also records graph cycles and dead-end paths as a separate backlog line. A graph cycle is not always an error: a review loop or correction loop may be intentional. Tooling must therefore distinguish an allowed cycle from terminal-path enumeration that cannot be resolved.

The main M76 rule is:

- maturity claims follow evidence;
- benchmark purpose follows observable behavior;
- graph warnings must preserve process semantics.

Chapter 77. Optional Flow Reuse

M77 adds a mechanism for reusing flow fragments in Ordo. The important boundary is that this is a **recommended mechanism**, not a rule forcing every playbook to split its process into reusable flows.

Reuse is appropriate when the same logic genuinely repeats and has a stable contract.

- duplicate stable logic
 - reuse candidate
 - namespace/state contract
 - reference resolution
 - runtime provenance

Namespace and state merge

A reusable flow has its own namespace. Its state must not silently overwrite parent-process state.

Merge rules explicitly define:

- input mapping
- local state
- exported state
- conflict policy

A conflict without a defined rule is a blocking validation error.

Compiler lowering

The compiler resolves flow references and creates a runtime representation without introducing a second source of truth. Reuse syntax belongs to the authoring layer; compiled IR receives unambiguous resolved references and provenance.

An unresolved reference, namespace collision, or incompatible contract blocks compile or validation.

Runtime semantics

When runtime enters a reusable flow, it preserves:

- caller
- callee
- entry transition
- state mapping
- return transition
- provenance

The trace must show that logic was executed through a reused flow rather than looking like an unexplained jump between nodes.

Advisory reuse detection

The CLI may detect similar flow fragments and recommend reuse. The recommendation is not an automatic rewrite.

reuse candidate detected → advisory

The author decides whether the logic is semantically shared.

The main M77 principle is:

- reuse is optional;
- conflicts are explicit;
- compiler resolves;
- runtime preserves provenance.

Chapter 78. APF Replay, Prompt Registry, and Internal Mini-prompts

M78 evaluates APF not only with static tests but through a real process-creation scenario. The goal is to observe the analyst experience: where the process is clear, where unnecessary friction appears, and where the runtime or prompt layer encourages undesirable model behavior.

Real-case replay

APF replay takes a real case through the existing process rails and records:

- active step
- analyst input
- classification
- state mutation
- transition
- friction finding

This is not a replacement for a benchmark. Replay is a product/process review of a specific applied module.

Prompt Registry reconciliation

As APF evolved, different prompts appeared: startup prompts, runtime prompts, handoff prompts, and internal task-specific prompts.

The Prompt Registry makes the following visible for every prompt:

- prompt id
- purpose
- owner layer
- version
- entry condition
- expected output
- replacement/deprecation relation

Duplicate or obsolete prompts must not remain silently active.

Internal mini-prompts

M78 also reviews where an internal mini-prompt is genuinely useful. A mini-prompt is appropriate for a bounded model-assisted task with a clear input/output contract.

It must not replace:

- runtime transition rules
- gates
- state contracts
- compiler validation
- deterministic CLI checks

The main principle is:

- use mini-prompts for bounded model work;
- use APF/runtime contracts for process control.

Chapter 79. Generic Template Tooling

M79 moves template handling from playbook-specific solutions into a generic tooling layer.

This is a separate Python utility in the package, similar in role to Visual Graph Generator or PathWalk. It is not runtime core.

Template Registry

Every managed template receives a registry record:

```
template id
version
format
render mode
input contract
output contract
owner
compatibility metadata
```

A consistency gate checks that template references and the registry do not drift apart.

Generic renderer

The renderer has one interface independent of a specific playbook.

It receives:

```
template reference
confirmed input/state
render context
output destination
```

and returns an output plus rendering evidence.

The renderer must not invent missing business values.

Generic review engine

The review engine checks a generated artifact against the template contract and creates an evidence format suitable for machine and human review.

Checks include:

```
required sections
unresolved placeholders
```

confirmed-state consistency
format validity
TBD policy

Version diff and breaking-change gate

Template version diff identifies changes between versions. The breaking-change gate blocks an incompatible change unless it has an explicit migration decision.

Real playbooks

M79 validates the tooling against at least two real playbooks. This matters: generic tooling is not considered generic merely because its interface is named that way.

The main M79 rule is:

registry identifies;
renderer produces;
review engine verifies;
version diff protects compatibility.

Chapter 80. Session Package Load and Cache

A practical problem appeared in long APF/playbook sessions: the model could unpack the archive and reread package files at every step.

unpack → read → execute one step
→ unpack → read → execute next step

This is slow, creates noise, and increases the risk that different steps rely on different reread fragments.

M80 introduces a session package load-and-cache contract.

Load once

After the first successful package-read pass, runtime records:

package_loaded
package_version
package_fingerprint
unpacked_location

manifest_validated
source_of_truth_loaded

If the package version and fingerprint have not changed, the validated package is reused within the current session.

Reload conditions

A repeated unpack/read is required only when:

new package version supplied
fingerprint changed
runtime cache lost
required file missing
manifest validation failed
explicit full reload requested

Cache hit

The correct runtime flow is:

MESSAGE_RECEIVED
→ PACKAGE_CACHE_CHECK
→ cache valid: use loaded baseline
→ cache invalid: unpack + validate + load
→ ACTIVE_NODE_EXECUTION

After a valid cache hit, another full unpack/read is forbidden.

Trace and diagnostics

Runtime records events such as:

PACKAGE_CACHE_HIT
PACKAGE_RELOAD_REQUIRED
PACKAGE_LOADED
PACKAGE_CACHE_INVALID

Metrics and diagnostics make unnecessary reloads and invalidation reasons visible.

APF and playbook boundary

Fingerprinting, cache validation, reload conditions, and trace events belong to the APF/runtime contract.

A specific playbook may contain a short instruction not to reread the factory package at every step, but it must not invent its own cache semantics.

The main M80 principle is:

```
validate once;  
reuse while fingerprint is stable;  
reload only for an explicit reason;  
record cache behavior in trace.
```

Chapter 81. Deterministic ARF Runtime Control Model

Purpose

ARF-generated playbooks are executable contracts, not advisory prose. Their runtime must know which mode is active, which node is active, which actions are allowed, which evidence is authoritative, and what blocks progress.

Default control profile

A strict working playbook declares:

```
runtime_control:  
  runtime_mode: PROCESS_EXECUTOR_ONLY  
  decision_model: closed_world  
  default_role: executor  
  undefined_action: blocked_missing_instruction
```

Closed-world execution means that an action is permitted only when the active contract explicitly allows it.

Mutually exclusive modes

ARF uses three explicit modes:

EXECUTION_MODE

Execute the active node contract without redesigning the process or mutating the playbook.

DESIGN_MODE

Discuss and propose changes. Repository, package, and runtime mutation are forbidden.

AUTHORIZED_MAINTENANCE_MODE

Change the playbook or factory only after explicit user authorization and only within the authorized scope.

A mode change is itself an explicit governed transition. Conversation drift or a helpful suggestion cannot switch modes.

Instruction precedence

The effective instruction order is:

- confirmed user decision
- active node contract
- authoritative node inputs
- confirmed process state
- authorization state

A lower level cannot weaken or override a higher level. Conflicting instructions at the same level produce `blocked_ambiguous_instruction`. Missing authoritative instructions produce `blocked_missing_instruction`. Execution mode does not permit invented or custom criteria.

Runtime enforcement

Before executing an action, the runtime checks:

1. current mode;
2. active node profile;
3. prerequisites;
4. allowed inputs;
5. allowed and forbidden actions;
6. exact authorization scope;
7. explicit destination transition;
8. required validation and evidence.

Failure is fail-closed. The runtime does not substitute best judgement.

Independent validation

Artifact production and artifact validation are separate responsibilities. A producer cannot self-declare success. Validation results are

bound to the target identifier, revision, hash, and validation contract. Composite outputs are validated at mandatory atomic-unit level.

State and invalidation

Confirmations, validations, and authorizations are revision-bound evidence. When an upstream source changes, dependent downstream evidence is invalidated. Completion status is determined by the weakest mandatory gate, not by the most optimistic label.

Relationship to anti-patterns

These rules enforce existing fundamental anti-pattern classes, especially:

- `CONTROL_FLOW_INTEGRITY_VIOLATION`;
- `AUTHORIZATION_BOUNDARY_VIOLATION`;
- `RESPONSIBILITY_CONFLATION`;
- `STATUS_EVIDENCE_MISMATCH`;
- `STATE_COHERENCE_VIOLATION`;
- `POLICY_ENFORCEMENT_GAP`.

They do not introduce new fundamental anti-patterns.

Practical outcome

The model remains useful for reasoning and discussion, but execution remains governed. It always has an explicit active node and mode, and it cannot silently skip, merge, redesign, validate, authorize, or complete work outside the declared contract.

Appendix A. Ordo Glossary

This glossary defines the terms used by the current Ordo language, compiler, runtime, package, and governance layers. Definitions are normative unless a referenced specification narrows them further.

Active node

The single node that currently owns execution focus in a deterministic run.

Artifact

A declared output produced by a program, playbook, compiler, run-time, or validation process.

Assertion

A machine-checkable statement that must hold for a run, package, or artifact.

Canonical source

The authoritative representation from which derived files are generated or validated.

Contract

A structured declaration of required inputs, fields, outputs, statuses, and acceptance conditions.

Control level

The degree of formal control applied to execution: light, standard, or strict.

Derived artifact

A generated representation that must never silently become the source of truth.

Entry mode

An explicitly authorized way to enter a node, such as root, resume, retry, recovery, or migration.

Execution trace

An append-only record of node entries, decisions, state changes, gates, and outputs.

Fail closed

A policy in which missing or invalid evidence blocks progress rather than being treated as success.

Flow reuse

A first-class reference to a shared continuation or join whose state mapping and return behavior are explicit.

Gate

A deterministic control point that permits or blocks progress based on declared evidence.

Graph contract

The declaration of entry, terminal, cycle, provenance, and reachability expectations for a process graph.

Node context

The bounded state, knowledge, tools, and output contract visible to one active node.

Opcode

A semantic operation in Ordo IR, such as `NODE.DEF`, `GATE.CHECK`, or `TRACE.LOG`.

Pattern

A governed reusable process structure with applicability rules, evidence, lifecycle, and composition constraints.

Process Rail

A structured execution rail that preserves progress, current focus, and controlled deviation handling.

Prompt registry

A governed registry of prompt assets, stable IDs, checksums, lifecycle, and authority boundaries.

Provenance

Evidence of where a transition, value, artifact, prompt, or decision came from.

Runtime checkpoint

A restorable state snapshot with identity, integrity, and continuation information.

Source construct

A top-level YAML construct accepted by the source language and lowered by the compiler.

State projection

An explicit subset of state imported into or exported from a node or shared flow.

Terminal outcome

A declared successful, blocked, cancelled, or otherwise controlled end state.

Validation profile

A named collection of checks and severities applied to a source or package.

Canonical capability names

The following capability names are copied from the current capability catalog and should not be replaced with improvised aliases.

Ordo Capability Catalog

The machine-readable registry is `capability_catalog.yaml`.

Conversation Scope Guard

- Contract: ORD0-CAP-CSG-001
- Source construct: `conversation_scope_guard`
- Core default: disabled
- Status: language-integrated optional specification
- Opcodes: `CONVERSATION.SCOPE.DEF`, `DEVIATION.CLASSIFY`, `DEVIATION.HANDLE`, `DEVIATION.ESCALATE`, `STATE.PROTECT`, `PROCESS.PAUSE`, `PROCESS.RESUME`, `PROCESS.EXIT`

- Trace events: `conversation.deviation.detected`, `conversation.deviation.classified`, `conversation.redirect.emitted`, `conversation.escalation.changed`, `conversation.scope_guard.bypassed_for_control_in`, `conversation.scope_guard.bypassed_for_safety`, `process.pause.requested`, `process.paused`, `process.resume.requested`, `process.resumed`, `process.exit.requested`, `process.exited`

Appendix B. Opcode and Source-Construct Catalog

This appendix is generated from the canonical language registries. The catalog is descriptive; schemas and compiler validation remain authoritative.

Ordo v0.12 Opcode / Construct Catalog

Program and metadata

Construct	Type	Description
<code>PROGRAM.DEF</code>	op	Defines program metadata.
<code>GRAPH.CONTRACT</code>	op	Defines graph entry, terminal, cycle and transition-provenance contracts.
<code>CONTROL_LEVEL.DEF</code>	field/construct	Defines light, standard, or strict.
<code>EXECUTION.MODE</code>	field/construct	Defines <code>full_runtime</code> , <code>chat_internal</code> , or <code>freeform_only</code> .

Core execution

Construct	Type	Description
<code>INTERACTION.MODEL</code>	op	Defines human/AI interaction roles and raw tool-output policy.

Construct	Type	Description
PROCESS_RAIL.DEF	op	Defines the Process Rail contract for guided AI execution.
CONVERSATION.SEMANTICS	op	Defines classification and routing rules for conversational input.
HYBRID_EXECUTION.MODE	op	Defines AI-led execution with CLI as deterministic helper layer.
INTENT.DEF	op	Defines user goal.
CONTRACT.DEF	op	Defines expected result contract.
CONTEXT.DEF	op	Defines available context.
STATE.SCHEMA	op	Defines state structure.
ENTRY.DEF	op	Defines entry point.
NODE.DEF	op	Defines question/decision node.
NODE.ASK	op	Asks node question.
CLARIFY.REQUEST	op	Controlled clarification for unmatched input.
PATH.DEF	op	Defines execution path.
STEP.RUN	op	Runs a step.
OUTPUT.DEF	op	Defines allowed output.
HANDOFF.DEF	op	Defines handoff behavior.

Gates and assertions

Construct	Type	Description
GATE.DEF	op	Defines gate.
GATE.CHECK	op	Evaluates gate.
GATE.METHOD	field/construct	Required method classification.

Construct	Type	Description
ASSERTION.DEF	op	Canonical assertion primitive.
ASSERTION.PROJECT	compiler action	Projects assertion to runtime/test/debug.
ASSERT.NOT	shortcut	Runtime projection of negative assertion.
EXPECT.NOT	test projection	Test projection of negative assertion.

Contract and artifact coverage

Construct	Type	Description
CONTRACT.INSTANCE	op	Declares a first-class process contract instance with field statuses.
CONTRACT.FIELD	field/construct	Declares a typed field inside a contract.
CONTRACT.STATUS	field/construct	Declares field/contract status: missing, candidate, proposed, confirmed, blocked, not_applicable.
ARTIFACT.DEF	op	Declares a generated artifact target.
ARTIFACT.REQUIREMENT	op	Maps confirmed contract fields to required artifacts.
COVERAGE.RULE	op	Defines deterministic coverage policy.
RENDERED_ARTIFACT.ASSERT	op	Asserts that rendered artifact content contains required contract fields/sections.

Construct	Type	Description
CONSISTENCY.REPORT	op	Records cross-artifact consistency validation.
GO_NO_GO.DECISION	op	Records machine-readable readiness decision.

Debug/Test/Improvement

Construct	Type	Description
TRACE.LOG	op	Records execution trace.
TRACE.SOURCE	field/construct	Declares trace source.
DECISION.LOG	op	Records decision.
PATH.EXPLAIN	op	Explains path selection.
STATE.SNAPSHOT	op	Records state snapshot.
STATE.DIFF	op	Records state diff.
GATE.REPORT	op	Records gate result.
TEST.DEF	op	Defines test.
FIXTURE.DEF	op	Defines test fixture.
EXPECT.PATH	op	Expected path.
EXPECT.STATE	op	Expected state.
EXPECT.OUTPUT	op	Expected output.
EXPECT.GATE	op	Expected gate result.
REGRESSION.SUITE	op	Regression suite.
COVERAGE.REPORT	op	Coverage report.
IMPROVEMENT.RECORD	op	Structured improvement record.

Libraries and layers

Construct	Type	Description
LIB.INCLUDE	op	Includes library.
LIB.USE	op	Uses exports from library.
VERSION.REQUIRE	op/field	Requires version.
NAMESPACE.RESOLVE	compiler action	Resolves local IDs into namespaced IDs.
LAYER.CONFLICT.CHECK	op	Checks unresolved layer conflicts.

Construct	Type	Description
OVERRIDE.DEF	op	Defines explicit override.

FREEFORM

Construct	Type	Description
FREEFORM.DEF	op	Defines controlled freeform block.
FREEFORM.COVERAGE	op	Coverage metadata for freeform.
FREEFORM.MATURITY	field/construct	Maturity lifecycle.
FREEFORM.INCIDENT_COUNTER	field/construct	Incident counter.
FREEFORM.FORMALIZATION_WARNING	field/construct	Suggested formalization.
TEMPLATE.RENDER_POLICY	concept	Describes deterministic vs model-assisted rendering policy for output templates.
MODEL_RENDER.HANDOFF	runtime evidence	Records an AI rendering handoff packet for a model-assisted template.

Execution trace

- EXECUTION_TRACE.DEF — declares the canonical trace artifact contract for a run.
- TRACE.EVENT.APPEND — appends an immutable canonical event. Reserved for runtime/compiler integration in Part 2.

Conversation Scope Guard

Construct	Type	Description
CONVERSATION.SCOPE.DEF	op	Defines the optional conversation scope guard policy in Semantic JSON IR.

Construct	Type	Description
DEVIATION.CLASSIFY	op	Classifies a user message against the canonical deviation taxonomy.
DEVIATION.HANDLE	op	Selects the response and process-preservation action for a classified deviation.
DEVIATION.ESCALATE	op	Advances the explicit deviation escalation policy.
STATE.PROTECT	op	Protects process state from unauthorized mutation during deviation handling.
PROCESS.PAUSE	op	Pauses a process while preserving resumable state.
PROCESS.RESUME	op	Resumes a previously paused process.
PROCESS.EXIT	op	Exits a process with controlled incomplete terminal semantics.

Ordo Source Construct Catalog

Canonical top-level Ordo Source constructs. The machine-readable source is `source_construct_catalog.yaml`.

Construct	Status
ordo	supported
module	supported
includes	supported
interaction_model	supported
process_rail	supported
conversation_semantics	supported
conversation_scope_guard	supported
hybrid_execution	supported

Construct	Status
intent	supported
contract	supported
contracts	supported
state	supported
execution_trace	supported
nodes	supported
gates	supported
assertions	supported
artifacts	supported
artifact_requirements	supported
coverage_rules	supported
rendered_artifact_assertions	supported
go_no_go	supported
outputs	supported
freeform	supported
prompt_registry	supported
startup_package_profile	supported

flow_reuse | supported |
runtime_capabilities | supported |

Appendix C. Verified Program Examples

Each example is taken from a canonical package that passes the current package linter. Long sources are excerpted to keep the appendix readable; the complete source remains in the package path shown for each example.

Guided intake

Canonical source: `packages/history_event_guided_intake/source/program.ordo.yaml`

The excerpt demonstrates program metadata, graph or node declarations, contracts, and package-specific governance.

```
ordo:
  version: '0.12'
  package: history_event.guided_intake
  control_level: standard
  execution_mode: chat_internal
includes:
- library: ordo.validation.contract_first
  version: ^0.2.0
```

```

    as: contract_first
- library: ordo.qa.release_validation
  version: ^0.1.0
  as: qa_release
intent:
  id: INTENT_HISTORY_EVENT_GUIDED_INTAKE
  description: Керовано зібрати мінімальний контракт для створення нового History Event
    фінального архіву.
contract:
  id: CONTRACT_HISTORY_EVENT_GUIDED_INTAKE
  required:
    - event_goal
    - selected_path
    - event_alias
    - display_name_uk
    - display_name_en
    - source_field
    - value_semantics
    - qa_scope
    - test_coverage_level
    - test_strategy_contract
    - approval_received
state:
  id: STATE_HISTORY_EVENT_GUIDED_INTAKE
  schema:
    event_goal: null
    selected_path: null
    event_alias: null
    display_name_uk: null
    display_name_en: null
    source_field: null
    value_semantics: []
    qa_scope: []
    approval_received: false
    output_allowed: false
    final_package_created: false
    test_coverage_level: null
    test_strategy_contract: []
    manual_qa_coverage: []
    functional_test_coverage: []
    unit_test_coverage: []
    test_documentation_requirement: null
    test_propagation_required: true
graph_contract:
  entry_node: N_EVENT_GOAL
  external_terminal_targets:

```

- G_APPROVAL_CONFIRMED
- STOP_NEEDS_APPROVAL

nodes:

- id: N_EVENT_GOAL
 - question: Яку бізнесову зміну має відображати нова історична подія?
 - answer_type: free_text
 - on_answer:
 - update_state:
 - event_goal: \$answer
 - next: N_PATH_SELECT
 - on_unmatched_input:
 - action: CLARIFY.REQUEST
 - strategy: rephrase_and_narrow
 - max_attempts: 2
 - on_exhausted:
 - action: escalate_to_human
 - reason: event goal remains unclear
- id: N_PATH_SELECT
 - question: 'Питання: оберіть Ordo path для цієї History Event.

Варіанти:

A – зміна у source row / ChangeRecord flow, далі без додаткових legacy A-subpath

B – derived або агрегований стан

C – зовнішній ExternalHistoryEvent

D – інший або нестандартний шлях

Відповідайте: A, B, C або D. Якщо вибрано A, runtime переходить одразу в A-flow: contract → business fields contract → ChangeRecord contract → trigger/no-op → no coverage.'

answer_type: enum

allowed_answers:

- A
- B
- C
- D

on_answer:

A:

- update_state:
 - selected_path: A
- next: N_EVENT_ALIAS

```

B:
  update_state:
    selected_path: B
    next: N_EVENT_ALIAS
C:
  update_state:
    selected_path: C
    next: N_EVENT_ALIAS
D:
  update_state:
    selected_path: D
    next: N_EVENT_ALIAS
on_unmatched_input:
  action: CLARIFY.REQUEST
  strategy: show_allowed_answers
  max_attempts: 2
  on_exhausted:
    action: escalate_to_human
    reason: path selection did not match allowed paths
prompt_refs:
- prompt_id: hp.source_type.clarification.v1
  use: during_clarification
- id: N_EVENT_ALIAS
  question: Який технічний alias події?
  answer_type: free_text
  on_answer:
    update_state:
      event_alias: $answer
      next: N_DISPLAY_NAME_UK
  on_unmatched_input:
    action: CLARIFY.REQUEST
    strategy: rephrase_and_narrow
    max_attempts: 2
    on_exhausted:
      action: escalate_to_human
      reason: event alias remains unclear
- id: N_DISPLAY_NAME_UK
  question: Яка українська назва події для користувача?
  answer_type: free_text
  on_answer:
    update_state:
      display_name_uk: $answer
      next: N_DISPLAY_NAME_EN
  on_unmatched_input:
    action: CLARIFY.REQUEST
    strategy: rephrase_and_narrow

```



```

    max_attempts: 2
    on_exhausted:
      action: escalate_to_human
      reason: Ukrainian display name remains unclear
  prompt_refs:
  - prompt_id: hp.localization.bilingual_texts.v1
    use: before_question
- id: N_DISPLAY_NAME_EN
  question: Яка англійська назва події для технічної документації?
  answer_type: free_text
  on_answer:
    update_state:
      display_name_en: $answer
      next: N_SOURCE_FIELD
  on_unmatched_input:
    action: CLARIFY.REQUEST
    strategy: rephrase_and_narrow
    max_attempts: 2
    on_exhausted:
      action: escalate_to_human
      reason: English display name remains unclear
  prompt_refs:
  - prompt_id: hp.localization.bilingual_texts.v1
    use: before_question
- id: N_SOURCE_FIELD
  question: 'Питання: підтвердіть source field або набір полів для бізнесової зміни.

```

Варіанти формату відповіді:

1. одне поле, наприклад `item.status`
2. група вкладених полів, наприклад `item.capital.value + item.capital.currency`

Applied Project Factory

Canonical source: `packages/ordo_applied_project_factory/source/program.ordo.yaml`

The excerpt demonstrates program metadata, graph or node declarations, contracts, and package-specific governance.

```

ordo:
  version: '0.12'
  package: ordo.applied_project_factory
  control_level: standard
  execution_mode: chat_internal

```

```

package_version: 0.1.0-rc.1
language_package_version: 0.12.0-preview-rc1
module:
  id: ordo.applied_project_factory
  version: 0.1.0-rc.1
  versioning_scope: module_local
  language_compatibility: ordo >= 0.12.0-preview-rc1, M62 line closure compatible
  parent_language_package_version: 0.12.0-preview-rc1
  inclusion_model: language_package_includes_module_at_explicit_module_version
  release_channel: release_candidate
  previous_internal_labels:
    - M61.1
    - M61.2
    - M61.3
    - M61.4
    - M61.5
    - M61.6
  whole_tree_integration_review:
    version: 0.1.0-alpha.20
    status: closed_human_review
    source_version: 0.1.0-alpha.19
    scope:
      - startup_branches_integration
      - shared_output_template_joins
      - shared_validation_handoff_joins
      - terminal_deferred_unfinished_gates
      - orphans_dead_ends_duplication_scan
    human_review_blocks:
      startup_branches_integration: approved
      shared_output_template_joins: approved
      shared_validation_handoff_joins: approved
      terminal_deferred_unfinished_gates: approved
      orphans_dead_ends_duplication: approved_human_review
    real_structural_scan_required: true
    real_structural_scan_status: passed
  full_validation_fix:
    version: 0.1.0-alpha.21
    source_version: 0.1.0-alpha.20
    scope:
      - manifest_version_alignment
      - test_coverage_backfill
      - contract_artifact_marker_backfill
      - full_validation_state_fixture
      - artifact_consistency_report
    process_logic_changed: false
    status: applied_scoped_technical_fix

```

```

m62_release_candidate_adaptation:
  version: 0.1.0-rc.1
  source_version: 0.1.0-alpha.21
  parent_language_line: M62 line closure
  status: applied
  scope:
    - adopt current M62 language package as parent base
    - preserve APF alpha.21 process logic
    - sync module packaging/docs to standard applied module layer
    - validate with M62 parent CLI available commands
    - keep future IR candidates FLOW.JOIN and SHARED.TAIL.REFERENCE as documented
      candidates, not runtime core changes
  process_logic_changed: false
includes:
- library: ordo.process_rail.core
  version: ^0.1.0
  as: process_rail
- library: ordo.project_authoring.validation
  version: ^0.1.0
  as: project_validation
- library: ordo.output_template.contracts
  version: ^0.1.0
  as: template_contracts
interaction_model:
  human_role: pm_or_analyst_without_ordo_yaml_knowledge
  ai_role: ai_ordo_project_factory_developer
  proactive_ai_behavior: required
  raw_tool_output_policy: ai_interprets_before_user
  yaml_visibility_policy: hidden_from_pm_by_default
intent:
  id: INTENT_APPLIED_PROJECT_FACTORY
  description: 'Допомогти РМ/аналітику створити новий прикладний Ordo-проект будь-як
    типу без написання YAML напряду; AI обирає режим, у free-dialogue накопичує raw
    notes, структурує їх у candidate nodes/gates/outputs/templates, формує draft tree
    проводить depth-first review і генерує source/program.ordo.yaml. APF v0.1.0-alpha
    фіксує user-facing формат показу процесу під час design/review, щоб режим traversal
    branch selection, node review decision і validation gate не змішувалися. APF v0.
    додає четвертий режим старту – коригування існуючого процесу – з batch та target
    improvement subflows. APF v0.1.0-alpha.16 adds progressive branch-1 authoring,
    input policy, terminal output binding, and template/mock-example verification
    before YAML generation. APF v0.1.0-alpha.17 treats manual-tree authoring as an
    adapter into the progressive review path rather than a separate downstream process
    APF v0.1.0-alpha.18 closes the shared terminal output/template subflow as a reusable
    block across all startup branches: terminal points explicitly choose an output
    policy; new artifacts are described by free-form artifact intent rather than fixed
    type lists; the runtime shows available terminal state fields, derives an output
  
```

recipe with direct-insert vs generated sections, and uses file-first review pack for document-like artifacts. Template/mock/mapping review is simplified to four user decisions: confirm, revise, defer, or remove artifact. Deferred templates are recorded as unfinished and cannot be used by final gates until completed or removed. APF v0.1.0-alpha.19 closes the shared validation/handoff tail as the reusable final path for all startup branches and shared subflows: source YAML generation approval, minimal validation, full-validation decision, validation result review, scoped correction loop, final unfinished-items gate, handoff package generation, and final handoff. Validation failure UX is simplified: failed checks show a short issue list and route either into correction mode or a blocked/defer state; details live inside correction mode. Skipped full validation for alpha is recorded as a limitation, never as passed. Handoff package generation is file and blocked by failed minimal validation or active unfinished artifacts/template. APF v0.1.0-alpha.20 closes the whole-tree integration review after the branch and shared-tail closures: all four startup branches join the shared output/template subflow and shared validation/handoff tail; terminal/deferred/unfinished gates are aligned; legacy unreachable nodes are explicitly marked as deprecated compat nodes rather than active orphans; structural scan results are recorded in the alpha.20 validation report.'

purpose: ' APF v0.1.0-alpha.12 separates unreviewed sibling branches from not-selected control actions and blocked control actions; review state must show which alternates were intentionally not selected and which actions are blocked until readiness conditions are met.'

contract:

id: CONTRACT_APPLIED_PROJECT_FACTORY

required:

- factory_authoring_mode
- applied_project_goal
- applied_process_type
- runtime_human_role
- runtime_ai_role
- runtime_entry_point
- decision_tree_blueprint
- state_schema_blueprint
- output_artifact_catalog
- output_template_catalog
- validation_gate_catalog
- first_release_output_scope
- approval_to_generate_source_yaml
- domain_model_notes
- open_questions
- draft_tree_review_status
- basic_test_case_catalog
- domain_input_sources
- domain_output_template_sources
- approved_decision_tree

- tree_review_depth_first_complete
- free_dialogue_raw_notes
- free_dialogue_structured_notes
- candidate_decision_nodes
- candidate_gates
- candidate_outputs
- candidate_templates
- free_dialogue_draft_tree_ready
- stabilized_tree_branch
- self_hosted_authoring_loop_status
- focused_svg_policy
- graph_rendering_policy_status
- language_improvement_proposals_status
- module_versioning_policy
- graph_annotation_overlay_policy
- auto_svg_generation_policy
- user_facing_node_description_policy
- user_facing_extraction_policy
- process_feedback_policy
- process_feedback_policy_status
- node_review_display_contract
- node_review_display_policy_status
- confirmed_review_path
- current_review_branch
- unreviewed_sibling_branches
- deferred_return_points
- current_node_review_record
- node_review_control_gate_status
- node_decision_gate_policy
- current_node_review_decision
- node_review_ordo_layer_visibility
- incremental_yaml_update_policy
- incremental_yaml_validation_profile
- incremental_yaml_validation_status
- last_incremental_yaml_patch_summary
- full_project_validation_policy
- full_project_validation_status

Hybrid Executor

Canonical source: `packages/ordo_hybrid_executor/source/program.ordo.yaml`

The excerpt demonstrates program metadata, graph or node declarations, contracts, and package-specific governance.

`ordo:`

```

    version: '0.12'
    package: ordo.hybrid_executor
    control_level: standard
    execution_mode: chat_internal
includes:
- library: ordo.process_rail.core
  version: ^0.1.0
  as: process_rail
- library: ordo.hybrid_execution.helpers
  version: ^0.1.0
  as: hybrid_helpers
interaction_model:
  human_role: user
  ai_role: ordo_executor
  proactive_ai_behavior: required
  raw_tool_output_policy: ai_interprets_before_user
process_rail:
  rail_id: ORDO_HYBRID_EXECUTION_RAIL
  purpose: Виконати готовий Semantic JSON IR через AI Ordo Executor без перетворення
    CLI на головний діалоговий runtime.
  state_tracking: required
  allow_deviation: true
  require_resume_after_deviation: true
  backtracking: enabled
hybrid_execution:
  ai_role: ordo_executor
  semantic_ir_role: process_rail
  cli_role: deterministic_helper
  raw_tool_output_policy: ai_interprets_before_user
  allow_deviation: true
  require_resume_after_deviation: true
  backtracking: enabled
conversation_semantics:
  on_unmatched_input:
    strategy: classify_and_route
    allowed_classes:
      - answer_current_question
      - answer_future_question
      - correction_previous_answer
      - clarification_request
      - domain_context
      - approval
      - refusal
      - out_of_scope
    require_state_validation: true
intent:

```

```

    id: INTENT_HYBRID_EXECUTION
    description: Допомогти AI Ordo Executor виконати готовий Semantic JSON IR, тримаючи
        Process Rail і використовуючи CLI/helper checks лише для детермінованих частин.
contract:
    id: CONTRACT_HYBRID_EXECUTION
    required:
        - semantic_ir_loaded
        - current_rail_node
        - rail_state_validated
        - human_input_classified
        - helper_result_interpreted
        - resume_after_deviation
        - output_allowed_by_gate
state:
    id: STATE_HYBRID_EXECUTION
    schema:
        semantic_ir_loaded: false
        current_rail_node: null
        completed_nodes: []
        invalidated_nodes: []
        human_input_classified: null
        rail_state_validated: false
        helper_result_interpreted: false
        deviation_detected: false
        resume_after_deviation: false
        output_allowed_by_gate: false
        human_visible_summary: null
graph_contract:
    entry_node: N_LOAD_IR
    allowed_cycle_regions:
        - id: CR_EXECUTION_LOOP
          nodes:
            - N_CLASSIFY_INPUT
            - N_VALIDATE_STATE
            - N_INTERPRET_HELPER
            - N_HANDLE_DEVIATION
            - N_OUTPUT_GATE
            - N_NEXT_MOVE
nodes:
    - id: N_LOAD_IR
      question: Завантажити готовий Semantic JSON IR і визначити стартовий Process Rail
          node.
      answer_type: system_action
      on_answer:
          update_state:
              semantic_ir_loaded: true

```

```

    next: N_CLASSIFY_INPUT
  on_unmatched_input:
    action: CLARIFY.REQUEST
    strategy: explain_that_execution_needs_compiled_ir
    max_attempts: 1
    on_exhausted:
      action: block_execution
      reason: semantic json ir is missing
- id: N_CLASSIFY_INPUT
  question: Класифікувати повідомлення людини відносно поточного Process Rail node.
  answer_type: ai_classification
  on_answer:
    update_state:
      human_input_classified: $answer.class
    next: N_VALIDATE_STATE
  on_unmatched_input:
    action: CLARIFY.REQUEST
    strategy: classify_or_ask_for_clarification
    max_attempts: 2
    on_exhausted:
      action: keep_current_node
      reason: input class remains unclear
- id: N_VALIDATE_STATE
  question: Викликати deterministic helper для перевірки запропонованого стану.
  answer_type: tool_result
  on_answer:
    update_state:
      rail_state_validated: true
    next: N_INTERPRET_HELPER
  on_unmatched_input:
    action: CLARIFY.REQUEST
    strategy: retry_or_explain_validation_problem
    max_attempts: 1
    on_exhausted:
      action: block_next_step
      reason: state validation unavailable
- id: N_INTERPRET_HELPER
  question: Перетворити helper result у людське пояснення без raw technical dump.
  answer_type: ai_explanation
  on_answer:
    update_state:
      helper_result_interpreted: true
      human_visible_summary: $answer
    next: N_HANDLE_DEVIATION
  on_unmatched_input:
    action: CLARIFY.REQUEST

```



```

    strategy: produce_plain_language_summary
    max_attempts: 1
    on_exhausted:
      action: block_handoff
      reason: helper result was not interpreted
- id: N_HANDLE_DEVIATION
  question: Якщо є correction/deviation, виконати rail resume або backtracking.
  answer_type: ai_process_decision
  on_answer:
    update_state:
      resume_after_deviation: true
    next: N_OUTPUT_GATE
  on_unmatched_input:
    action: CLARIFY.REQUEST
    strategy: ask_whether_user_changes_previous_decision
    max_attempts: 1
    on_exhausted:
      action: continue_without_deviation
      reason: no confirmed deviation
- id: N_OUTPUT_GATE
  question: Перевірити, чи дозволено генерувати output на поточному стані.
  answer_type: gate_check
  on_answer:
    update_state:
      output_allowed_by_gate: $answer.allowed
    next: N_NEXT_MOVE
  on_unmatched_input:
    action: CLARIFY.REQUEST
    strategy: explain_output_gate
    max_attempts: 1
    on_exhausted:
      action: block_output
      reason: output gate unresolved
- id: N_NEXT_MOVE
  terminal: true
  question: 'Обрати наступний conversational move: питання, пояснення, повернення
    назад або output.'
  answer_type: ai_decision
  on_answer:
    update_state:
      current_rail_node: $answer.next_node

```

Project Builder

Canonical source: packages/ordo_project_builder/source/program.ordo.yaml

The excerpt demonstrates program metadata, graph or node declarations, contracts, and package-specific governance.

```
ordo:
  version: '0.12'
  package: ordo.project_builder
  control_level: standard
  execution_mode: chat_internal
includes:
- library: ordo.process_rail.core
  version: ^0.1.0
  as: process_rail
- library: ordo.project_authoring.validation
  version: ^0.1.0
  as: project_validation
interaction_model:
  human_role: pm
  ai_role: ordo_developer
  proactive_ai_behavior: required
  raw_tool_output_policy: ai_interprets_before_user
process_rail:
  rail_id: ORDO_PROJECT_AUTHORIZING_RAIL
  purpose: Створити або модернізувати Ordo-проект через PM dialogue без вимоги, щоб
  state_tracking: required
  allow_deviation: true
  require_resume_after_deviation: true
  backtracking: enabled
conversation_semantics:
  on_unmatched_input:
    strategy: classify_and_route
    allowed_classes:
      - answer_current_question
      - answer_future_question
      - correction_previous_answer
      - clarification_request
      - domain_context
      - design_suggestion_request
      - approval
      - refusal
      - out_of_scope
    require_state_validation: true
intent:
  id: INTENT_ORDO_PROJECT_BUILDER
  description: Допомогти PM створити новий Ordo-проект через діалог із AI Ordo Devel
  IR.
contract:
```

```

id: CONTRACT_ORDO_PROJECT_BUILDER
required:
- project_goal
- project_domain
- human_roles
- ai_roles
- process_rail_goal
- state_model_summary
- intake_or_execution_mode
- required_outputs
- deterministic_helper_scope
- approval_before_compile
state:
  id: STATE_ORDO_PROJECT_BUILDER
  schema:
    project_goal: null
    project_domain: null
    human_roles: []
    ai_roles: []
    process_rail_goal: null
    state_model_summary: null
    intake_or_execution_mode: null
    required_outputs: []
    deterministic_helper_scope: []
    yaml_project_created: false
    syntax_checked: false
    semantic_ir_compiled: false
    approval_before_compile: false
    pm_visible_summary: null
  graph_contract:
    entry_node: N_PROJECT_GOAL
    external_terminal_targets:
    - G_APPROVAL_BEFORE_COMPILE
    - STOP_NEEDS_APPROVAL
  nodes:
  - id: N_PROJECT_GOAL
    question: Який Ordo-проект ви хочете створити і яку проблему він має вирішувати?
    answer_type: free_text
    on_answer:
      update_state:
        project_goal: $answer
        next: N_PROJECT_DOMAIN
    on_unmatched_input:
      action: CLARIFY.REQUEST
      strategy: ask_for_project_goal_in_plain_language
      max_attempts: 2

```

```

    on_exhausted:
      action: escalate_to_human
      reason: project goal remains unclear
- id: N_PROJECT_DOMAIN
  question: Яку доменну область або тип процесу має описувати цей Ordo-проект?
  answer_type: free_text
  on_answer:
    update_state:
      project_domain: $answer
    next: N_ROLES
  on_unmatched_input:
    action: CLARIFY.REQUEST
    strategy: ask_for_domain_examples
    max_attempts: 2
    on_exhausted:
      action: escalate_to_human
      reason: project domain remains unclear
- id: N_ROLES
  question: Хто буде людиною в цьому процесі і яку роль має виконувати ШІ?
  answer_type: free_text
  on_answer:
    update_state:
      human_roles: $answer.human_roles
      ai_roles: $answer.ai_roles
    next: N_PROCESS_RAIL_GOAL
  on_unmatched_input:
    action: CLARIFY.REQUEST
    strategy: ask_for_pm_user_ai_roles
    max_attempts: 2
    on_exhausted:
      action: escalate_to_human
      reason: roles remain unclear
- id: N_PROCESS_RAIL_GOAL
  question: 'Що саме Process Rail має утримувати: питання, рішення, gates, повернення?'
  answer_type: free_text
  on_answer:
    update_state:
      process_rail_goal: $answer
    next: N_STATE_MODEL
  on_unmatched_input:
    action: CLARIFY.REQUEST
    strategy: show_process_rail_examples
    max_attempts: 2
    on_exhausted:
      action: escalate_to_human
      reason: process rail goal remains unclear

```

- id: N_STATE_MODEL
 - question: Який мінімальний state треба вести, щоб процес не губився?
 - answer_type: free_text
 - on_answer:
 - update_state:
 - state_model_summary: \$answer
 - next: N_MODE_SELECT
 - on_unmatched_input:
 - action: CLARIFY.REQUEST
 - strategy: ask_for_decisions_and_fields_to_track
 - max_attempts: 2
 - on_exhausted:
 - action: escalate_to_human
 - reason: state model remains unclear
- id: N_MODE_SELECT
 - question: Це Ordo-проект для створення інших проектів, для виконання готового проц
 - answer_type: enum
 - allowed_answers:
 - authoring
 - execution
 - mixed
 - on_answer:
 - authoring:
 - update_state:
 - intake_or_execution_mode: authoring
 - next: N_REQUIRED_OUTPUTS
 - execution:
 - update_state:
 - intake_or_execution_mode: execution
 - next: N_REQUIRED_OUTPUTS
 - mixed:
 - update_state:
 - intake_or_execution_mode: mixed
 - next: N_REQUIRED_OUTPUTS
 - on_unmatched_input:
 - action: CLARIFY.REQUEST
 - strategy: show_allowed_answers
 - max_attempts: 2
 - on_exhausted:
 - action: escalate_to_human
 - reason: mode selection did not match allowed modes
- id: N_REQUIRED_OUTPUTS
 - question: Які output artifacts має створювати цей Ordo-проект?
 - answer_type: list
 - on_answer:

Appendix D. Checklists

This appendix contains practical checklists for working with Ordo programs. They can be used when creating a new playbook, reviewing an existing instruction, preparing a Domain Pack, connecting libraries, debugging, testing, or performing the final check before handoff.

A checklist in Ordo is not a substitute for gates or tests. It is a supporting tool for an author, analyst, or reviewer that helps prevent important parts of the process from being missed.

D.1. Checklist: Is the Idea Ready for Ordo?

Before turning an instruction into an Ordo program, determine whether it actually contains a controllable process.

- ☐ There is a clear intent.
- ☐ There is an expected result.
- ☐ The model's responsibility boundaries are defined.
- ☐ There are actions the model must not perform.
- ☐ There are points that require human confirmation.
- ☐ There are alternative execution paths.
- ☐ There is data that must be collected before action.
- ☐ There are rules that must not be skipped.
- ☐ There is a final handoff or output.
- ☐ There is an error risk that justifies formalization.

If most items are empty, this may not yet be an Ordo program; it may simply be a short request to a model.

If most items are satisfied, the process is a good candidate for Ordo.

D.2. Intent Checklist

Intent should answer the question: what does the user actually want to obtain?

- ☐ Intent is expressed in one or more simple sentences.
- ☐ Intent is not mixed with implementation details.
- ☐ Intent contains no unconfirmed assumptions.
- ☐ Intent is separated from the output format.
- ☐ Intent is not replaced by a task title.
- ☐ Intent does not describe a random example instead of the general goal.
- ☐ Intent is understandable without reading the entire program.

Poor intent:

Do it like in the previous example.

Better intent:

Guide the user through controlled data collection for creating an analytical package

D.3. Contract Checklist

The contract defines what must be confirmed before execution begins or before critical actions are performed.

- [] Required fields are known.
- [] Decisions requiring user confirmation are known.
- [] It is clear what counts as confirmed.
- [] It is clear what must not be treated as confirmed.
- [] Rules for an incomplete contract are defined.
- [] A gate blocks further execution without a valid contract.
- [] The contract is not silently constructed from model assumptions.
- [] The contract is stored in state.
- [] Contract changes are recorded as a state diff.

Critical rule:

If the contract is not confirmed, the model must not behave as though it were confirmed

D.4. Context Checklist

Context includes the data, documents, rules, examples, and constraints required for correct execution.

- [] Documents that act as rule sources are identified.
- [] Input data is identified.
- [] Examples that are only examples are marked as such.
- [] Context priority is defined.
- [] Conflict-resolution behavior is defined.
- [] Data that must not be invented is identified.
- [] Sources that may be cited or used as evidence are identified.
- [] Context is not mixed with state.

D.5. State Checklist

State shows what is known, what is confirmed, what is awaiting a decision, and what is prohibited.

- [] State has an explicit schema.
- [] State contains confirmed and pending values.
- [] State does not mix facts with assumptions.
- [] State is updated after every important step.
- [] A state snapshot is available for debug mode.
- [] A state diff records value changes.

- [] Hidden conversational memory is not used instead of state.
- [] Critical decisions are not lost inside response text.

Example of useful state:

```
state:
  contract:
    alias:
      value: "LU_CHANGE_STATUS"
      status: "confirmed"

  approvals:
    pre_archive:
      status: "pending"

  execution:
    selected_path: "A1"
```

D.6. Entry and Node Checklist

Entry and Node control the dialogue.

- [] There is a starting ENTRY.
- [] Every NODE has a clear purpose.
- [] Every NODE asks only necessary questions.
- [] A NODE does not jump ahead.
- [] A NODE has entry conditions.
- [] A NODE has exit conditions.
- [] A NODE updates state.
- [] A NODE knows where to transition next.
- [] Rules exist for ambiguous user answers.
- [] Rules exist for revising a previous decision.

Poor:

Ask the user everything that is needed.

Better:

```
node:
  id: "N_COLLECT_ALIAS"
  asks:
    - "Confirm the event alias."
  writes:
    - "state.contract.alias"
  next:
    when_confirmed: "N_COLLECT_SOURCE_FIELD"
```


D.7. Path Checklist

A Path is needed when the same intent can be fulfilled through different routes.

- [] All primary paths are listed.
- [] Selection conditions are defined for each path.
- [] Each path has a reason.
- [] Debug mode records rejected paths and reasons.
- [] Ambiguity handling is defined.
- [] Risky paths require a gate or user approval.
- [] A path is not selected based on model intuition alone.
- [] Path selection can be tested.

D.8. Gate Checklist

A Gate is a control point, not advice.

- [] The gate has an id.
- [] The gate has a pass condition.
- [] The gate has a status.
- [] The gate has evidence.
- [] The consequence of fail/block is defined.
- [] It is clear whether the gate is blocking.
- [] The gate cannot pass without required data.
- [] The gate is not hidden inside FREEFORM.
- [] The gate appears in the debug trace.
- [] The gate has a test or coverage.

Critical rule:

If a gate is blocking, the model must not continue the action that the gate blocks.

D.9. ASSERT.NOT Checklist

Negative assertions prevent unsafe actions.

- [] Actions the model must not perform are listed.
- [] Every prohibited action has an ASSERT.NOT.
- [] ASSERT.NOT is linked to a specific risk.
- [] ASSERT.NOT is checked before output.
- [] ASSERT.NOT has a test.
- [] ASSERT.NOT is not replaced by a soft recommendation.

Examples:

- [] Do not create the final archive before approval.
- [] Do not invent a source row.

- [] Do not treat an example as a confirmed rule.
- [] Do not change the contract without confirmation.

D.10. Output Checklist

Output must be explicit and verifiable.

- [] The output type is defined.
- [] The output format is defined.
- [] Mandatory output parts are defined.
- [] Excluded output content is defined.
- [] Required pre-output gates are defined.
- [] Output can be checked through validation.
- [] Output includes a handoff note or next action.
- [] Output is not created before the permitted point.

D.11. FREEFORM Checklist

FREEFORM is useful for complex semantic content, but it must remain controlled.

- [] FREEFORM has an id.
- [] There is a reason why this part is not formalized.
- [] Purpose is defined.
- [] Binding is defined.
- [] Usage locations are identified.
- [] It is clear what FREEFORM cannot override.
- [] Coverage or a coverage plan exists.
- [] No hidden gates exist inside FREEFORM.
- [] No hidden statuses exist inside FREEFORM.
- [] No rules remain that should be moved into Core, Profile, or Domain Pack.

D.12. Debug Checklist

Debug mode should explain execution rather than merely repeat the final response.

- [] There is a run_id.
- [] There is an input snapshot.
- [] The selected path is recorded.
- [] Rejected paths and reasons are recorded.
- [] There is a decision log.
- [] There are state snapshots.
- [] There are state diffs.
- [] There is a gate report.
- [] There is a knowledge trace.

- [] Warnings are recorded.
- [] Violations are recorded.
- [] A failure explanation exists when the process fails.

The debug trace should answer:

Why did the model do this?

D.13. Test Checklist

Tests should verify Ordo-program behavior, not only final wording.

- [] TEST.DEF exists.
- [] FIXTURE.DEF exists.
- [] Expected path is defined.
- [] Expected state is defined.
- [] Expected gates are defined.
- [] Expected output is defined.
- [] EXPECT.NOT covers prohibited actions.
- [] No-op tests exist.
- [] Negative tests exist.
- [] Edge-case tests exist.
- [] Tests exist for FREEFORM-backed decisions.

D.14. Regression Suite Checklist

A regression suite is required before changes are released.

- [] A baseline scenario set exists.
- [] Scenarios exist for every path.
- [] Scenarios exist for blocking gates.
- [] No-op scenarios exist.
- [] Invalid or incomplete input scenarios exist.
- [] Library scenarios exist.
- [] Domain Pack scenarios exist.
- [] The regression suite runs before release.
- [] Regression results are recorded in a report.

D.15. Coverage Checklist

Coverage shows how much of the Ordo program is actually controlled and verified.

- [] All primary paths are covered.
- [] All blocking gates are covered.
- [] All critical ASSERT.NOT rules are covered.
- [] All output types are covered.

- [] All status transitions are covered.
- [] No-op scenarios are covered.
- [] Critical FREEFORM blocks are covered.
- [] Imported libraries are covered.
- [] Uncovered areas are listed.
- [] A coverage improvement plan exists.

D.16. Feedback and Improvement Loop Checklist

When a user reports a problem, it should become a structured record.

- [] Feedback is captured.
- [] The original user message is recorded.
- [] The problem is classified.
- [] Severity is recorded.
- [] The affected unit is identified.
- [] A root-cause hypothesis exists.
- [] A proposed patch exists.
- [] A suggested test exists.
- [] Required approval is identified.
- [] A version note or changelog item exists.
- [] A regression test is added after the fix.

Feedback must not disappear inside chat history.

D.17. Ordo Library Checklist

Connected libraries require control over version, namespace, and conflicts.

- [] The library is included explicitly.
- [] The version is pinned.
- [] An alias or namespace is defined.
- [] Used exports are identified.
- [] A compatibility check has been performed.
- [] Conflicts have been checked.
- [] Override is allowed only explicitly.
- [] There are no implicit imports.
- [] A trust level is defined.
- [] The library is covered by tests or has its own test pack.

Poor:

```
include:
- "ordo.qa"
```

Better:

```
include:
- library: "ordo.qa.manual_runbook"
  version: "0.1"
  as: "manual_qa"
```

D.18. Domain Pack Checklist

A Domain Pack should describe domain logic, not a random collection of examples.

- [] Domain vocabulary exists.
- [] Domain-specific paths exist.
- [] Domain-specific gates exist.
- [] Domain-specific statuses exist.
- [] Domain-specific output templates exist.
- [] Mapping to Ordo Core exists.
- [] Controlled FREEFORM is used for complex edge cases.
- [] Domain tests exist.
- [] A coverage report exists.
- [] An improvement loop exists.

D.19. Legacy Playbook Migration Checklist

- [] The old playbook has been split into logical parts.
- [] Intent has been identified.
- [] Contract has been identified.
- [] The decision tree has been identified.
- [] Paths have been identified.
- [] Nodes have been identified.
- [] Gates have been identified.
- [] Statuses have been identified.
- [] Outputs have been identified.
- [] Handoff has been identified.
- [] FREEFORM blocks have been identified.
- [] Reusable libraries have been identified.
- [] Semantic JSON IR has been built.
- [] Test cases have been created.
- [] A coverage report has been created.
- [] A consistency check has been completed.

D.20. Final Handoff Checklist

Before handing an Ordo program or package over for use, verify:

- [] All mandatory sections are present.
- [] No blocking gates remain unresolved.

- [] No contract fields remain pending.
- [] No conflicts remain unresolved.
- [] No implicit assumptions remain.
- [] No hidden rules remain in FREEFORM.
- [] A debug trace can be obtained.
- [] The regression suite passed, or its absence is stated honestly.
- [] A coverage report has been produced.
- [] Improvement records were addressed or moved to backlog.
- [] The version note was updated.
- [] The handoff note was written.

Brief Summary

Checklists do not automatically make an Ordo program correct. They help authors avoid missing the core elements:

intent
 contract
 context
 state
 path
 node
 gate
 assertion
 output
 debug
 test
 coverage
 feedback
 library
 domain pack
 handoff

If an Ordo program passes these checklists, it is not only readable; it can also be maintained, tested, improved, and handed to other people or systems.

D.22. Ordo v0.12 Reliability Checklist

Use this checklist after upgrading a program or playbook to Ordo v0.12.

- [] Every gate has a mandatory method field.
- [] Every gate defines `trust_class`.
- [] Mechanical gates are not mixed with semantic model-judgment gates.

- [] Gates with method: self_verification have an evidence protocol.
- [] Critical semantic gates use a generator/critic or self_consistency pattern.
- [] Gate reports show method and trust_class.
- [] Execution traces include trace_source.
- [] The Ordo program defines execution_mode.
- [] Documentation does not imply that model_self_report equals a runtime_enforced lo
- [] ASSERT.NOT is described as a shortcut/projection of ASSERTION.
- [] Critical prohibitions are modeled as ASSERTION with phase: [runtime, test].
- [] EXPECT.NOT is derived from ASSERTION projection rather than manually duplicating
- [] NODE defines on_unmatched_input or an explicit fallback.
- [] Unmatched input is handled through CLARIFY.REQUEST.
- [] The node_coverage_gap feedback class is used for recurring uncovered answers.
- [] Program defines control_level: light / standard / strict.
- [] A strict program has regression coverage for mandatory gates and assertions.
- [] Trace, gate_report, and improvement_record use namespaced IDs.
- [] Includes define a version.
- [] Layer priority is defined and respected.
- [] Override is explicit and includes a reason.
- [] No unresolved layer conflicts remain.
- [] FREEFORM blocks define maturity.
- [] FREEFORM incident_count and incident_threshold are defined for risky blocks.
- [] FREEFORM blocks exceeding the threshold produce a formalization warning or impro

D.23. gate.method Checklist

- [] method: mechanical is used only for deterministic checks.
- [] method: self_verification is used for semantic model judgments supported by evid
- [] method: self_consistency is used for critical repeated model checks.
- [] method: human is used for human decisions.
- [] Examples contain no GATE.CHECK without method.
- [] gate_report contains no status: passed without method/trust_class explanation.

D.24. execution_mode Checklist

- [] full_runtime is used only where a runner actually controls transitions.
- [] chat_internal is honestly described as an intermediate mode.
- [] freeform_only is not presented as fully controlled execution.
- [] In chat_internal, documentation states that gate invocation is not enforceable w
- [] chat_internal defines state_backing or another mechanism for reducing state drif
- [] Strict processes do not use freeform_only without a dedicated warning.

D.25. ASSERTION Checklist

- [] Prohibitions are modeled through ASSERTION.
- [] Every ASSERTION defines polarity.

- [] Every ASSERTION defines phase.
- [] A blocking assertion uses severity: block.
- [] Runtime projection creates ASSERT.NOT or a gate.
- [] Test projection creates EXPECT.NOT.
- [] Debug projection creates a violation record.
- [] The regression suite verifies critical assertions.

D.26. Namespace, Version, and Layer-Priority Checklist

- [] Source may use local IDs, but IR uses full namespaced IDs.
- [] All improvement records reference namespaced IDs.
- [] All libraries define a version.
- [] Floating versions are not used for strict processes.
- [] Core is not overridden by a lower layer without explicit override.
- [] Domain Pack does not silently change Profile.
- [] A Library does not silently change Domain Pack.
- [] FREEFORM cannot override a formal rule.
- [] G_NO_UNRESOLVED_LAYER_CONFLICT is present for complex programs.

Appendix E. Anti-patterns

An anti-pattern is an approach that appears convenient or fast but makes an Ordo program less controllable, less verifiable, or less safe in real use.

This appendix does not replace a full playbook review. It is useful as a quick list of common mistakes before an Ordo program is handed over for use, a library is published, or a final package is assembled.

E.1. One Large Prompt Instead of an Ordo Program

Symptom

All logic is described in one large block of text without explicit intent, contract, state, node, gate, output, and handoff elements.

Why This Is a Problem

The model may execute only part of the instruction, skip an important gate, confuse an example with a rule, or move to the final result before the process is actually ready.

How to Fix It

Split the instruction into explicit parts:

intent → contract → context → state → path → steps → gates → result → handoff

E.2. Contract Hidden in Prose

Symptom

The text contains phrases such as “obviously,” “when needed,” or “if everything is ready,” but does not define who confirms readiness or how confirmation is recorded.

Why This Is a Problem

The model begins deciding by itself what is already confirmed, even when the user has not confirmed it.

How to Fix It

Model the contract explicitly through `CONTRACT.DEF`, `ANSWER.REGISTRY`, `APPROVAL.REQUIRE`, and blocking gates.

E.3. Gate Used as Advice

Symptom

A gate is described as “it is recommended to check,” but the process can continue after the gate fails.

Why This Is a Problem

Such a gate does not control execution. It only decorates the documentation.

How to Fix It

Use a blocking gate for critical checks:

if gate.failed → stop / ask / repair / no handoff

E.4. Missing `ASSERT.NOT`

Symptom

The Ordo program describes what must be done but does not describe what must not be done.

Why This Is a Problem

The model may create an unnecessary file, invent a missing value, skip approval, or generate a final package in a situation where it should stop.

How to Fix It

Add negative assertions:

```
ASSERT.NOT final_output_before_approval  
ASSERT.NOT invented_source_row  
ASSERT.NOT hidden_required_gate_inside_freeform
```

E.5. FREEFORM Used as a Dumping Ground

Symptom

Everything that was difficult to formalize is placed in FREEFORM: gates, rules, status semantics, approval logic, templates, and prohibitions.

Why This Is a Problem

FREEFORM stops being a controlled escape hatch and becomes the place where the real execution logic is hidden.

How to Fix It

Keep in FREEFORM only what genuinely should not be formalized at the current stage. Move critical rules into GATE.DEF, ASSERT.NOT, STATUS.SEMANTICS, OUTPUT.DEF, or a library.

E.6. Invisible Library Imports

Symptom

The Ordo program uses rules or templates from a library, but this is not declared through include, import, or use.

Why This Is a Problem

It becomes unclear where a rule came from, which library version was used, and who owns a behavior change.

How to Fix It

Include every library explicitly:

```
include:
  - library: "ordo.validation.contract_first"
    version: "0.1"
    as: "contract_first"
```

E.7. Override Without Permission

Symptom

A Profile, Domain Pack, or Library silently rewrites a gate, status, or output rule defined in another layer.

Why This Is a Problem

The behavior change becomes invisible. Existing tests may fail without an obvious cause.

How to Fix It

Every override must be explicit and include a reason and trace:

```
override:
  allow:
    - target: "G_PRE_ARCHIVE_APPROVAL"
      reason: "domain pack requires stricter approval gate"
```

E.8. Testing Only the Final Text

Symptom

Tests verify only the final response but do not verify path, state, gates, no-op behavior, or prohibited actions.

Why This Is a Problem

The model may reach the correct result through an incorrect process. In production playbooks, this is unsafe.

How to Fix It

Test behavior:

EXPECT.PATH
EXPECT.STATE
EXPECT.GATE
EXPECT.OUTPUT
EXPECT.NOOP
EXPECT.NOT

E.9. Missing Debug Trace

Symptom

When the Ordo program behaves incorrectly, there is no record of the selected path, rejected paths, passed gates, or knowledge sources used.

Why This Is a Problem

It is impossible to determine what failed: the instruction, domain rule, library, fixture, compiler, or model behavior.

How to Fix It

For complex Ordo programs, run debug or test mode and retain:

TRACE.LOG
DECISION.LOG
PATH.EXPLAIN
STATE.DIFF
GATE.REPORT
KNOWLEDGE.TRACE

E.10. Feedback Lost in Chat

Symptom

The user reports a problem or improvement, but it remains only in the conversation and is not converted into a structured improvement record.

Why This Is a Problem

The same issue repeats. Instructions improve accidentally, without backlog, tests, or changelog history.

How to Fix It

Use the Feedback & Improvement Loop:

```
FEEDBACK.CAPTURE  
→ ISSUE.RECORD  
→ ROOT_CAUSE.LINK  
→ PATCH.SUGGEST  
→ TEST.SUGGEST  
→ human approval  
→ regression run
```

E.11. All-in-One as the Source of Truth

Symptom

A large assembled Markdown file is edited directly while the individual source files remain outdated.

Why This Is a Problem

The build stops being reproducible. Different documentation versions begin contradicting one another.

How to Fix It

Treat individual documents as the source of truth:

section files / chapter files / domain pack files → all-in-one → PDF / archive

E.12. Validating the Template Instead of the Rendered Artifact

Symptom

The template or Markdown source is checked, but the actual PDF, archive, Jira task, or other assembled output is not checked.

Why This Is a Problem

The failure may appear only during rendering or package generation: a broken link, missing file, wrong order, unclosed code block, or incorrect numbering.

How to Fix It

Add `RENDER.VALIDATE` and inspect the actual result before handoff.

E.13. “The Model Will Figure It Out”

Symptom

The author does not explicitly define path selection, gates, outputs, or exceptions because the model is expected to “understand anyway.”

Why This Is a Problem

The more complex the process, the more expensive incorrect model assumptions become.

How to Fix It

Everything that matters for execution must be represented explicitly in Source or IR. If something is not formalized, it should be controlled FREEFORM, not a silent assumption.

E.14. Overgrown Core

Symptom

Domain-specific rules, templates for particular documents, or product-specific behavior are added to Core.

Why This Is a Problem

Core loses universality. The language becomes harder to maintain and less portable across domains.

How to Fix It

Keep Core minimal and move specialization into Profiles, Domain Packs, and Libraries.

E.15. No Responsibility Boundary Between Model and Runner

Symptom

It is unclear what the model decides, what the helper runner controls, and what requires human confirmation.

Why This Is a Problem

A gate may be skipped, output may be created without approval, and the trace may not show who authorized a transition.

How to Fix It

Separate responsibilities explicitly:

model → semantic work

runner → process control / gates / state / tests

human → approval / governance / final decisions

E.16. Quick Pre-handoff Check

Before handing over an Ordo program or package, quickly verify:

- Is intent explicit?
- Is there a contract?
- Are gates hidden inside FREEFORM?
- Are blocking gates present before final output?
- Are ASSERT.NOT rules defined for prohibited actions?
- Are all libraries included explicitly?
- Is there a debug/test mode for complex logic?
- Is there a regression suite?
- Is feedback converted into improvement records?
- Was the actual rendered artifact checked?

Main rule:

If behavior cannot be explained, verified, and reproduced, it is not yet an Ordo pro

E.17. False Deterministic Gate

Symptom

A gate looks like a normal control point but has no method, or a semantic judgment is presented as if it were a mechanical check.

Why This Is a Problem

The user sees status: passed but cannot tell whether the gate was checked by code or assessed by a model.

How to Fix It

Every gate should define method and trust_class:

```
gate:
  id: G_NO_UNSUPPORTED_FACTS
  method: self_verification
  trust_class: model_judgment
```

E.18. Gate Without method

Symptom

The program contains GATE.DEF or GATE.CHECK, but the verification method is not specified.

Why This Is a Problem

The playbook author has not made an explicit choice between a mechanical check, model judgment, and human decision.

How to Fix It

In v0.12, missing gate.method should be a compilation error.

E.19. Trace Without trace_source

Symptom

The debug trace shows selected path, state diff, and gate report but does not state whether the trace is a runtime log, hybrid trace, or model self-report.

Why This Is a Problem

The trace may create unjustified confidence in the model's explanation.

How to Fix It

Add:

```
trace_source: model_self_report | runtime_enforced | hybrid
```


E.20. Chat-only Mode Presented as Full Runtime

Symptom

The process runs in chat without an external runner, but the documentation describes gates as though code enforced them.

Why This Is a Problem

In `chat_internal`, code may check a gate, but the model still decides when to invoke that check.

How to Fix It

Declare explicitly:

```
execution_mode: chat_internal
```

and do not confuse it with:

```
execution_mode: full_runtime
```

E.21. One Prohibition Duplicated Manually in Three Places

Symptom

The same prohibition is independently described as `ASSERT.NOT`, a negative gate, and `EXPECT.NOT`.

Why This Is a Problem

Sooner or later, one of the three records will be updated while the others are not.

How to Fix It

Describe the prohibition once as `ASSERTION`, then derive its runtime, test, and debug projections.

E.22. Missing Fallback for Unmatched Input

Symptom

`NODE.DEF` has `allowed_answers` but no `on_unmatched_input`.

Why This Is a Problem

When the user gives an unexpected answer, the model returns to improvisation.

How to Fix It

Add:

```
on_unmatched_input:  
  action: CLARIFY.REQUEST  
  strategy: rephrase_and_narrow  
  max_attempts: 2
```

E.23. Local IDs in Feedback Records

Symptom

An improvement record references G1 or N2 without identifying the layer or library that owns the ID.

Why This Is a Problem

In a complex system, G1 may exist in Core, Profile, Domain Pack, and Library at the same time.

How to Fix It

Use only full namespaced IDs in feedback, traces, and reports.

E.24. Floating Library Version

Symptom

A library is included without a version, or with an overly broad version range in a strict process.

Why This Is a Problem

Program behavior may change even when the source file does not.

How to Fix It

Define a version policy and do not use floating versions in critical or strict workflows.

E.25. FREEFORM Never Matures

Symptom

The same FREEFORM block repeatedly produces feedback incidents but is never formalized.

Why This Is a Problem

FREEFORM becomes a permanent gray area.

How to Fix It

Add a maturity lifecycle:

```
freeform:
  maturity: candidate_for_formalization
  incident_count: 3
  incident_threshold: 3
```

and create FREEFORM_FORMALIZATION_RECOMMENDED.

E. Fundamental anti-pattern taxonomy

Individual detector cases are organized under a small stable set of fundamental failure classes. This prevents the registry from growing into hundreds of unrelated top-level rules.

1. RESPONSIBILITY_CONFLATION
2. CONTROL_FLOW_INTEGRITY_VIOLATION
3. AUTHORIZATION_BOUNDARY_VIOLATION
4. EVIDENCE_REALITY_MISMATCH
5. STATUS_EVIDENCE_MISMATCH
6. INCOMPLETE_VALIDATION_MODEL
7. TRACEABILITY_AND_COVERAGE_LOSS
8. STATE_COHERENCE_VIOLATION
9. PROVENANCE_AND_AUTHORITY_VIOLATION
10. BOUNDARY_AND_OWNERSHIP_VIOLATION
11. IDENTITY_AND_UNIQUENESS_MODEL_FAILURE
12. POLICY_ENFORCEMENT_GAP

A fundamental rule describes the broad invariant. A subpattern describes a recurring concrete failure. A detector case is a machine-checkable manifestation used by tests or runtime validation.

Adding another detector case does not create a new fundamental rule. A new fundamental-level anti-pattern requires separate owner

approval and evidence that the existing taxonomy cannot represent the failure without distortion.

Appendix F. Practical YAML and Schema Reference

This appendix is generated from the current machine-readable schemas. It explains field types, enumerations, required fields, and descriptions without treating prose as a substitute for validation.

F.1 Reading field types

- **enum**: choose only one of the listed values.
- **reference/id**: the referenced object must exist and satisfy provenance rules.
- **path**: the path must stay inside the allowed package or output root.
- **free text**: human-readable guidance; it must not be used as an implicit deterministic condition.
- **object/array**: validate nested members against the referenced schema.

F.2 Ordo ANTIPATTERN.DEF

Canonical schema: `language/schemas/antipattern_def.schema.json`

Field	Type	Required	Allowed values / description
schema_version	unspecified	yes	
id	string	yes	
name	string	yes	
summary	string	yes	
severity	unspecified; enum	yes	info, warning, error, critical
enforcement	unspecified; enum	yes	advisory, blocking
scope	array	yes	
symptoms	array	yes	
detection	object	yes	
recovery	object	yes	
remediation	object	yes	

Field	Type	Required	Allowed values / description
evidence	object	yes	
tags	array	no	
version	string	no	
status	unspecified; enum	no	draft, active, deprecated
classification_level	unspecified; enum	no	fundamental, subpattern, detector_case
fundamental_id	string	no	
generalization_string	string	no	

F.3 Ordo anti-pattern evidence reference

Canonical schema: language/schemas/antipattern_evidence_ref.schema.json

Field	Type	Required	Allowed values / description
schema_version	unspecified	yes	
evidence_type	unspecified	yes	
evidence_id	string	yes	
hook_id	string	yes	
source_id	string	yes	
gate_id	string / null	no	
context_type	unspecified; enum	yes	conversation, process_trace, repository_state, package_state, evidence_state, run-time_state
decision	unspecified; enum	yes	allow, allow_with_advisory, block, inconclusive

Field	Type	Required	Allowed values / description
finding_ids	array	yes	
report_digest	string	yes	
recorded_at	string	yes	

F.4 Ordo ANTIPATTERN.FINDING

Canonical schema: language/schemas/antipattern_finding.schema.json

Field	Type	Required	Allowed values / description
schema_version	unspecified	yes	
finding_type	unspecified	yes	
finding_id	string	yes	
rule_id	string	yes	
antipattern_id	string	yes	
matched	boolean	yes	
severity	unspecified; enum	yes	info, warning, error, critical
enforcement	unspecified; enum	yes	advisory, blocking
decision	unspecified; enum	yes	allow, allow_with_advisory, block, inconclusive
message	string	yes	
evidence	array	yes	
recovery	object	yes	
remediation	object	yes	
source	object	yes	
timestamps	object	yes	
resolution	object	no	

F.5 Ordo anti-pattern gate report

Canonical schema: language/schemas/antipattern_gate_report.schema.json

Field	Type	Required	Allowed values / description
schema_version	unspecified	yes	
report_type	unspecified	yes	
decision	unspecified; enum	yes	allow, allow_with_advisory, block, inconclusive
summary	object	yes	
blocking_findings	array	yes	
advisory_findings	array	yes	
findings	array	yes	
gate_id	string	yes	
context_type	unspecified; enum	yes	conversation, process_trace, repository_state, package_state, evidence_state, run-time_state
source_id	string	yes	
enabled_antipatterns	array	no	
runtime_error	string	no	
inconclusive_error_to_block	boolean	no	

F.6 Ordo Anti-pattern Severity and Enforcement Policy

Canonical schema: language/schemas/antipattern_policy.schema.json

Field	Type	Required	Allowed values / description
schema_version	unspecified	yes	
policy_version	string	yes	
severity_order	array	yes	
decision_matrix	object	yes	
critical_must_block	boolean	no	

Field	Type	Required	Allowed values / description
error_default_enforced	enum	no	advisory, blocking
warning_default_enforced	enum	no	
info_default_enforced	enum	no	

F.7 Ordo anti-pattern wiring hook

Canonical schema: language/schemas/antipattern_wiring_hook.schema.json

Field	Type	Required	Allowed values / description
schema_version	unspecified	yes	
hook_id	string	yes	
phase	unspecified; enum	yes	before_node_execution, after_state_update_before_transition, before_repository_mutation, before_package_finalization, before_final_status_claim
adapter	object	yes	
context_type	unspecified; enum	yes	conversation, process_trace, repository_state, package_state, evidence_state, run_time_state

Field	Type	Required	Allowed values / description
source_id	string	yes	Stable node, transition, gate, package, evidence, or repository mutation identifier.
input	object	yes	
output	object	yes	
decision_policy	object	yes	
routing	object	yes	
enabled_antipath	boolean	no	
deny_overrides	boolean	no	

F.8 ARF Control Model Contract

Canonical schema: `language/schemas/arf_control_model_contract.schema.json`

Field	Type	Required	Allowed values / description
schema_version	unspecified	yes	
contract_id	string	yes	
default_control_profile	unspecified	yes	
decision_model	unspecified	yes	
default_role	unspecified	yes	
undefined_action	unspecified	yes	
modes	object	yes	
mode_switching	object	yes	
ambiguity_policy	object	yes	
transition_policy	object	yes	
node_contract_profiles	object	yes	

F.9 ARF Node Contract Profiles

Canonical schema: `language/schemas/arf_node_contract.schema.json`

This schema does not expose top-level properties; inspect its composition (allOf, oneOf, or referenced definitions) with the validator.

F.10 Artifact Requirement Schema

Canonical schema: `language/schemas/artifact_requirement_schema.yaml`

This schema does not expose top-level properties; inspect its composition (allOf, oneOf, or referenced definitions) with the validator.

F.11 Artifact Schema

Canonical schema: language/schemas/artifact_schema.yaml

This schema does not expose top-level properties; inspect its composition (allOf, oneOf, or referenced definitions) with the validator.

F.12 Assertion Schema

Canonical schema: language/schemas/assertion_schema.yaml

This schema does not expose top-level properties; inspect its composition (allOf, oneOf, or referenced definitions) with the validator.

F.13 Ordo Capability Maturity

Canonical schema: language/schemas/capability_maturity.schema.json

Field	Type	Required	Allowed values / description
specification	unspecified	yes	
schema_support	unspecified	yes	
toolchain_support	unspecified	yes	
runtime_enforcement	unspecified	yes	
model_benchmark	unspecified	yes	
production_recommendation	unspecified	yes	

F.14 Ci Release Clean Gate Policy Schema

Canonical schema: language/schemas/ci_release_clean_gate_policy_schema.yaml

Field	Type	Required	Allowed values / description
ci_release_clean_gates	object	yes	

F.15 Ordo Clean Package Gate Schema

Canonical schema: language/schemas/clean_package_gate_schema.yaml

Field	Type	Required	Allowed values / description
clean_package_gate	object	yes	

F.16 Contract Schema

Canonical schema: language/schemas/contract_schema.yaml

This schema does not expose top-level properties; inspect its composition (allOf, oneOf, or referenced definitions) with the validator.

F.17 Ordo Conversation Scope Guard Source Declaration

Canonical schema: language/schemas/conversation_scope_guard_source.schema.json

Field	Type	Required	Allowed values / description
supported	boolean	yes	
enabled	boolean	yes	
mode	unspecified; enum	no	advisory, guided_redirect, strict_redirect, locked_process
state_change_on_unspecified	unspecified	no	
scope	object / array / string	no	
out_of_scope_behavior	object / string	no	
escalation	object	no	
trace	object	no	

F.18 Conversation Semantics Schema

Canonical schema: language/schemas/conversation_semantics_schema.yaml

Field	Type	Required	Allowed values / description
input_classes	array	no	
routing_rules	object	no	
unmatched_input_policy	string; enum	no	clarify_before_state_change, reject, log_and_continue, route_to_human_review
clarification_policy	string	no	
resume_policy	string	no	

F.19 Coverage Rule Schema

Canonical schema: language/schemas/coverage_rule_schema.yaml

This schema does not expose top-level properties; inspect its composition (allOf, oneOf, or referenced definitions) with the validator.

F.20 Ordo CSG Package Binding

Canonical schema: language/schemas/csg_package_binding.schema.json

Field	Type	Required	Allowed values / description
supported	boolean	yes	
enabled	boolean	yes	
decision_status	unspecified; enum	yes	confirmed_enabled, confirmed_not_required
contract	string	no	
policy	string	no	
tests	string	no	
trace_events	string	no	

F.21 Ordo Delta Backlog Schema

Canonical schema: language/schemas/delta_backlog_schema.yaml

Field	Type	Required	Allowed values / description
delta_backlog	object	yes	

F.22 Ordo Derived Artifact Sync Schema

Canonical schema: language/schemas/derived_artifact_sync_schema.yaml

Field	Type	Required	Allowed values / description
artifact_sync	object	yes	

F.23 Ordo Derived Artifact Sync Validation Profile Schema

Canonical schema: language/schemas/derived_artifact_sync_validation_profile_schema.yaml

Field	Type	Required	Allowed values / description
derived_artifact_sync_validation_profile	object	yes	

F.24 Ordo DETECT.RULE

Canonical schema: language/schemas/detect_rule.schema.json

Field	Type	Required	Allowed values / description
schema_version	unspecified	yes	
id	string	yes	
antipattern_id	string	yes	
name	string	yes	
description	string	yes	
input_contract	object	yes	
condition	object	yes	
output_contract	object	yes	
evaluation	object	yes	
version	string	no	
status	unspecified; enum	no	draft, active, deprecated
tags	array	no	

F.25 Ordo Deviation Classification Record

Canonical schema: language/schemas/deviation_classification.schema.json

Field	Type	Required	Allowed values / description
op	unspecified	yes	
message_ref	string	yes	
active_node_ref	string / null	yes	
active_question_ref	string / null	no	

Field	Type	Required	Allowed values / description
classification	unspecified; enum	yes	answer_to_active_question, clarification, correction, back-track_request, requirement_change, pause_request, resume_request, exit_request, process_meta_question, related_context, unrelated_topic, unsafe_or_emergency_message, unclassified_input
confidence	unspecified; enum	no	confirmed, high, medium, low, unknown
matched_scope_evidence	boolean	no	
classification_reason	string	no	
state_mutation_allowed	boolean	yes	

Field	Type	Required	Allowed values / description
action	unspecified; enum	yes	accept_answer, clarify_active_step, apply_correction, backtrack, reopen_contract, pause_process, resume_process, exit_process, answer_process_meta, register_related_context, redirect, bypass_for_safety, request_classification_clarification
trace_required	boolean	no	

F.26 Execution Trace Schema

Canonical schema: `language/schemas/execution_trace_schema.yaml`

This schema does not expose top-level properties; inspect its composition (allOf, oneOf, or referenced definitions) with the validator.

F.27 Flow Join Schema

Canonical schema: `language/schemas/flow_join_schema.yaml`

This schema does not expose top-level properties; inspect its composition (allOf, oneOf, or referenced definitions) with the validator.

F.28 Freeform Schema

Canonical schema: `language/schemas/freeform_schema.yaml`

This schema does not expose top-level properties; inspect its composition (allOf, oneOf, or referenced definitions) with the validator.

F.29 Gate Schema

Canonical schema: language/schemas/gate_schema.yaml

This schema does not expose top-level properties; inspect its composition (allOf, oneOf, or referenced definitions) with the validator.

F.30 Go No Go Schema

Canonical schema: language/schemas/go_no_go_schema.yaml

This schema does not expose top-level properties; inspect its composition (allOf, oneOf, or referenced definitions) with the validator.

F.31 Improvement Record Schema

Canonical schema: language/schemas/improvement_record_schema.yaml

This schema does not expose top-level properties; inspect its composition (allOf, oneOf, or referenced definitions) with the validator.

F.32 Interaction Model Schema

Canonical schema: language/schemas/interaction_model_schema.yaml

Field	Type	Required	Allowed values / description
human_role	string	no	
ai_role	string	no	
cli_role	string	no	
raw_tool_outputs	string, enum	no	summarize_before_user, show_on_request, show_full, never_show_raw
decision_authority	object	no	
review_points	array	no	

F.33 Library Include Schema

Canonical schema: language/schemas/library_include_schema.yaml

This schema does not expose top-level properties; inspect its composition (allOf, oneOf, or referenced definitions) with the validator.

F.34 Migration Ambiguity.Schema

Canonical schema: language/schemas/migration_ambiguity.schema.json

This schema does not expose top-level properties; inspect its composition (allOf, oneOf, or referenced definitions) with the validator.

F.35 Migration Clause Inventory.Schema

Canonical schema: language/schemas/migration_clause_inventory.schema.json

Field	Type	Required	Allowed values / description
schema_version	unspecified	yes	
source_ref	string	yes	
clauses	array	yes	

F.36 Migration Dependency Graph.Schema

Canonical schema: language/schemas/migration_dependency_graph.schema.json

Field	Type	Required	Allowed values / description
schema_version	unspecified	yes	
graph_id	string	yes	
nodes	array	yes	
edges	array	yes	

F.37 Migration Loss Finding.Schema

Canonical schema: language/schemas/migration_loss_finding.schema.json

Field	Type	Required	Allowed values / description
schema_version	unspecified	yes	
finding_id	string	yes	
clause_id	string	yes	

Field	Type	Required	Allowed values / description
loss_type	unspecified; enum	yes	unmapped_clause, partial_coverage, mandatory_strength_downgrade, authorization_boundary_loss, decision_semantic_loss, evidence_requirement_loss, unsupported_exclusion, unit_merge_without_equivalence, construct_mapping_missing
severity	unspecified; enum	yes	warning, error, critical
message	string	yes	
blocking	boolean	yes	
mapped_unit_ids	array	no	

F.38 Migration Ordo Mapping.Schema

Canonical schema: language/schemas/migration_ordo_mapping.schema.json

Field	Type	Required	Allowed values / description
schema_version	unspecified	yes	
mapping_id	string	yes	
entries	array	yes	

F.39 Migration Traceability Matrix.Schema

Canonical schema: language/schemas/migration_traceability_matrix.schema.json

Field	Type	Required	Allowed values / description
schema_version	unspecified	yes	
matrix_id	string	yes	

Field	Type	Required	Allowed values / description
source_ref	string	yes	
rows	array	yes	

F.40 Node Schema

Canonical schema: `language/schemas/node_schema.yaml`

This schema does not expose top-level properties; inspect its composition (allOf, oneOf, or referenced definitions) with the validator.

F.41 Process Instruction Migration Intake.Schema

Canonical schema: `language/schemas/process_instruction_migration_intake.schema.json`

Field	Type	Required	Allowed values / description
schema_version	unspecified	yes	
intake_id	string	yes	
source	object	yes	
migration_goal	string	yes	
preservation_contract	object	yes	
decomposition_policy	object	yes	
validation_policy	object	yes	

F.42 Ordo Process Instruction Migration Package

Canonical schema: `language/schemas/process_instruction_migration_package.schema.json`

Field	Type	Required	Allowed values / description
schema_version	unspecified	yes	
package_id	string	yes	
source	object	yes	
intake	object	yes	
clause_inventory	object	yes	
ambiguities	array	yes	
dependency_graph	object	yes	
ordo_mapping	object	yes	
traceability_matrix	object	yes	
gate_report	object	yes	
playbook	object	yes	

F.43 Process Rail Schema

Canonical schema: language/schemas/process_rail_schema.yaml

Field	Type	Required	Allowed values / description
rail_id	string	no	
state_tracking	string; enum	no	required, recommended, none
allow_deviation	boolean	no	
require_resume_after_deviation	boolean	no	
resume_policy	string	no	
backtracking	string; enum	no	disabled, restricted, enabled
backtracking_policy	object	no	
skip_ahead_policy	string	no	
stale_answer_policy	string	no	

F.44 Program Level Approval Gate Schema

Canonical schema: language/schemas/program_level_approval_gate_schema.yaml

Field	Type	Required	Allowed values / description
gate_id	string	no	Stable local identifier for the program-level approval gate.
applies_to	array	no	
profile	string; enum	no	light, standard, strict
severity_policy	object	no	
required_checks	array	no	
approval_decision	object	no	

F.45 Program Level Contract Schema

Canonical schema: language/schemas/program_level_contract_schema.yaml

Field	Type	Required	Allowed values / description
program_id	string	yes	Local stable program identifier.
module_id	string	yes	Canonical module/package identifier, for example ordo.applied_project_factory.
version	string	no	Module/package version.
ordo_version	string	no	Compatible Ordo language version or line.
lifecycle	string; enum	no	draft, alpha, beta, release-candidate, stable, deprecated
control_level	string; enum	no	light, standard, strict
execution_mode	string; enum	no	full_runtime, chat_internal, freeform_only, dry_run, test
contract_profile	string	no	
compatibility	object	no	
runtime_profile	object	no	
required_review	array	no	
required_validation	array	no	

F.46 Prompt Ref Schema

Canonical schema: language/schemas/prompt_ref_schema.yaml

Field	Type	Required	Allowed values / description
prompt_id	string	no	Must resolve to prompt_registry.prompts[*].prompt_id.
use	string	no	
required_for_profile	boolean	no	
state_change_allowed	boolean	no	Optional local override must not contradict registry-level prompt metadata.
notes	string	no	

F.47 Ordo Prompt Registry Packaging Checks Schema

Canonical schema: language/schemas/prompt_registry_packaging_checks_schema.yaml

Field	Type	Required	Allowed values / description
prompt_registry_packaging_checks	object	yes	

F.48 Prompt Registry Schema

Canonical schema: language/schemas/prompt_registry_schema.yaml

Field	Type	Required	Allowed values / description
registry_id	string	no	Stable identifier for the package prompt registry.
version	string	no	Registry version local to the package.

Field	Type	Required	Allowed values / description
default_language	string	no	Default BCP-47-like or package-local language code for prompt files.
prompt_root	string	no	Package-relative prompt root folder, for example prompts/.
prompts	array	no	

F.49 Prompt Registry Validation Profile Schema

Canonical schema: language/schemas/prompt_registry_validation_profile_schema.yaml

This schema does not expose top-level properties; inspect its composition (allOf, oneOf, or referenced definitions) with the validator.

F.50 Release Clean Gate Provenance Schema

Canonical schema: language/schemas/release_clean_gate_provenance_schema.yaml

Field	Type	Required	Allowed values / description
schema_version	unspecified	yes	
gate_id	string	yes	
gate_class	unspecified	yes	
repository	string	yes	
revision	string	yes	
ref	string	yes	
run_id	string	yes	
profile	unspecified	yes	
fail_on_warning	unspecified	yes	
linkage	object	yes	

F.51 Rendered Artifact Assertion Schema

Canonical schema: language/schemas/rendered_artifact_assertion_schema.yaml

This schema does not expose top-level properties; inspect its composition (allOf, oneOf, or referenced definitions) with the validator.

F.52 Rendering Template Schema

Canonical schema: language/schemas/rendering_template_schema.yaml

This schema does not expose top-level properties; inspect its composition (allOf, oneOf, or referenced definitions) with the validator.

F.53 Ordo Conversation Scope Guard Strictness and Escalation Policy

Canonical schema: language/schemas/scope_guard_policy.schema.json

Field	Type	Required	Allowed values / description
mode	unspecified; enum	yes	advisory, guided_redirect, strict_redirect, locked_process
resolved_policy	object	yes	
escalation	object	yes	

F.54 Semantic Json Ir Schema

Canonical schema: language/schemas/semantic_json_ir_schema.yaml

This schema does not expose top-level properties; inspect its composition (allOf, oneOf, or referenced definitions) with the validator.

F.55 Shared Tail Reference Schema

Canonical schema: language/schemas/shared_tail_reference_schema.yaml

This schema does not expose top-level properties; inspect its composition (allOf, oneOf, or referenced definitions) with the validator.

F.56 Startup Package Profile Schema

Canonical schema: language/schemas/startup_package_profile_schema.yaml

Field	Type	Required	Allowed values / description
startup_package_profile	object	yes	

F.57 Startup Profile Validation Profile Schema Convention

Canonical schema: language/schemas/startup_profile_validation_profile_schema.yaml

Field	Type	Required	Allowed values / description
startup_profile_validation	object	yes	

F.58 Ordo Conversation Scope Guard State Protection

Canonical schema: language/schemas/state_protection.schema.json

Field	Type	Required	Allowed values / description
trigger_classification	array	yes	
protected_state	array	yes	
allowed_mutations	array	yes	
rollback_on_violation	unspecified	yes	
blocking	unspecified	yes	

F.59 Trace Schema

Canonical schema: language/schemas/trace_schema.yaml

This schema does not expose top-level properties; inspect its composition (allOf, oneOf, or referenced definitions) with the validator.

Appendix G. APF Session Package Cache

The APF session package cache is a derived runtime optimization. It must be invalidated when the source fingerprint, package version, runtime profile, or authority boundary changes. A cache entry never becomes canonical source and must be reproducible from the verified package.